

ECR Assistant

AI-powered Engineering Change Request analysis - complete project handbook

Three agents: Retrieval, Correlation, Summarization

Deterministic scoring engines with an LLM for plain-language answers

All data in CSV files - 45 ECRs, 64 requirements, 251 test cases, 68 defects

Version of 17 September 2026

Prepared as a single reference document for questions about this project.

Contents

1. About this document	3
2. Executive summary	4
3. The problem and the outcome	5
4. How to run the system	6
5. Code structure at a glance	7
6. Architecture overview	9
7. The three agents and five steps	10
8. Planning, routing and human approval	12
9. Impacted components	13
10. Regression test selection engine	14
11. Defect analysis	15
12. Retrieval (RAG) and embeddings	16
13. Dependency graph and code analysis	17
14. LLM setup, resilience and cost control	18
15. Performance	19
16. Data storage: CSV files	20
17. Sample data: what it is and is not	22
18. API reference	23
19. User interface	24
20. Testing and quality	25
21. Security and privacy	26
22. Limitations and roadmap	27
23. Repository map	28
24. Glossary	29
25. Questions a manager may ask	30
26. Five-minute demo script	32
26. Complete source code	33

1. About this document

This handbook describes the ECR Assistant exactly as the code stands today. It is written so that a reader who has never seen the repository - or an assistant answering questions about it - can explain what the system does, how it works, what the numbers mean and how to run it.

Everything here is drawn from the working code: the agent definitions, the scoring engines, the data files and the test suite. Where a number appears (a test count, a duration) it was produced by running the system, not estimated.

What this document does not contain

- No credentials. The LLM API key lives only in backend/.env, which is excluded from version control. The key value is not reproduced anywhere in this document.
- No customer data. All sample data in the project is synthetic, written for this project.
- No internal Capgemini or client information beyond the name of the LLM gateway that the backend is configured to call.

2. Executive summary

The ECR Assistant answers one question: "An Engineering Change Request has landed - what does it touch, which regression tests should we run, and has anything like it broken before?"

Today an engineer answers that by hand, reading five systems: the change record, the requirements tool, the test management tool, the defect tracker, and the review threads. The assistant does that retrieval, joins the pieces together and writes the answer in plain language.

What it produces for one ECR number

- A plain-language answer with the identifiers it relied on.
- The components the change touches directly and the ones it reaches indirectly, each with its reason, plus recommended actions.
- A recommended regression suite: which tests, in which order, why each was chosen, and how much execution time that saves against the full suite.
- A defect analysis: which past defects could come back, why, what to check, which defects are open right now in the affected area, and the defect history of the affected components.
- Review comments, evidence artefacts and any missing evidence.

Headline numbers

Measure	Value
Agents	3 (Retrieval, Correlation, Summarization)
Traced steps per analysis	5
Time for one analysis	Usually 10 to 30 seconds with the live LLM (three requests); under 1 second offline
Time before the performance work	About 230 seconds
Regression reduction (example ECR-2026-001)	67 of 170 tests, 60.6% fewer, 179 min instead of 424 min
Sample data	45 ECRs, 64 requirements, 251 tests, 2,510 test runs, 68 defects, 55 components
Automated tests	111, all passing
Backend size	127 Python files, about 12,700 lines
Frontend size	54 React files

3. The problem and the outcome

Before

For each change request an engineer opens the change record, searches the requirements tool for anything related, opens the test management tool and guesses which regression tests matter, checks the defect tracker for similar past failures, and reads the review thread. The work is slow, it is repeated for every change, and the result depends on who did it and how much time they had.

Two failure modes follow. Run everything, and the regression suite costs hours of machine and people time for a one-line copy change. Run too little, and a change ships next to an old defect nobody remembered.

After

The user types an ECR number. Three agents gather the evidence, join it up and explain it. The answer is on screen within seconds, with every claim traceable to an identifier, and the recommended test suite is sized to the change rather than to habit.

Where the value is

- Time saved per change: the manual gathering is the expensive part, not the judgement.
- Consistency: the same change always produces the same components and the same suite, because the numbers come from deterministic code, not from a model.
- Explainability: every requirement match, every selected test and every impacted component carries a written reason, so a reviewer can disagree with a specific line rather than with a black box.
- Institutional memory: the defect analysis surfaces failures that nobody on the current team would remember.

4. How to run the system

Requirements

- Python 3.12 or newer, with the virtual environment in .venv at the repository root.
- Node.js with yarn or npm for the React frontend.
- No database server, no Docker requirement, no network requirement: the system runs fully offline in demo mode when no LLM key is configured.

From VS Code (the usual way here)

- Open the Run and Debug panel (Ctrl+Shift+D).
- Choose "Backend" and press the green play button; or choose "Full stack" to start the backend and frontend together.
- Wait for this line in the terminal, which confirms the CSV data loaded:
startup.data_loaded ... ecrs=45 requirements=64 test_cases=251
- The frontend opens on http://localhost:3000 and calls the backend on http://localhost:8000.

From a terminal

```
cd backend
..\venv\Scripts\python.exe -m uvicorn app.main:app --port 8000 --reload
cd frontend
yarn start
```

With Docker

```
docker compose up --build
```

The compose file starts the backend and the frontend only; there is no database service, because the data is the CSV folder, which is mounted into the backend container.

First start after a data change

At startup the backend builds the search index in the background, so the API answers immediately. By default the vectors are made locally, which takes a few seconds. With `REMOTE_EMBEDDINGS=true` they come from the gateway instead: the first build after a data change is slower, and the result is cached in `backend/.cache/embeddings.json` so later starts only embed rows whose text changed.

Checking the data without starting the backend

```
cd backend
..\venv\Scripts\python.exe -m app.seed
```

This loads every CSV file into a private in-memory copy and prints the row counts. If a value is malformed it reports the file, line and column, for example:

```
defects.csv line 12, column 'detected_at': Invalid isoformat string
```

5. Code structure at a glance

The project has two halves: a Python backend that does the analysis and a React frontend that shows it. All data sits in CSV files next to the backend.

Top-level folders

Folder or file	What it holds
backend/	The FastAPI app, the three agents, the selection logic, the data and the tests
backend/data/	The CSV data files and the saved reports - the only data store
frontend/	The React app: Dashboard, ECR Analysis and Report pages
docs/	Architecture, agent design, API, deployment and demo guides, and this handbook
scripts/	Helper scripts, including the one that builds this PDF
.vscode/	Run configurations: Backend, Frontend, Full stack, Backend tests
docker-compose.yml	Runs the backend and frontend in containers

Inside backend/app (127 Python files, about 12,700 lines)

File or folder	What it does
main.py	Starts the app: loads the CSV data, builds the search index in the background, registers the API routes
config.py	Every setting (LLM, embeddings, test-selection weights), read from backend/.env
prompts.py	Every instruction sent to the LLM, one block per agent
database.py	Loads the CSV files into a working SQL copy at startup
data/csv_store.py	Reads, checks and writes back the CSV files
models/	One class per CSV table: ECR, Requirement, TestCase, Defect, Component and so on
schemas/	The shapes of the data passed between steps and returned by the API
repositories/	Reusable database queries
agents/state.py	The shared workflow state, the three agents and their steps
agents/orchestrator.py	The workflow graph: Retrieval, Correlation, optional human approval, Summarization
agents/runtime.py	Run registry, live progress events, step timing and the step log
agents/nodes/agents.py	The three agents
agents/nodes/steps.py	The five steps the agents run
agents/nodes/ (other files)	The parts inside the steps: understanding the ECR, planning the search, requirements, code impact, past defects, dependencies, comments, impacted components, tests, links, the correlation review, defect insights and the summary
agents/tools/	Small reusable functions the steps call: searches, graph walks, code parsing
services/	Bigger building blocks: test selection, search (RAG), code analysis, reports, chat, ECR handling, confidence
llm/	LLM connectors (OpenAI-compatible gateway, Anthropic, offline mock) and the retry and circuit-breaker wrapper
vectorstore/	Where search vectors are kept (in memory by default)
graph/	The component dependency graph (NetworkX)
integrations/	Source-system adapters: the local CSV data today, Azure DevOps and Jira as stubs
api/routes/	The REST endpoints

seed/	python -m app.seed (checks the CSV files) and the small sample code repository the analyzer reads
tests/	111 automated tests

Inside frontend/src

File or folder	What it does
App.js	The page routes
pages/AnalyzePage.jsx	The ECR Analysis page - most of the UI
pages/DashboardPage.jsx, pages/ReportPage.jsx	The dashboard and the full report
components/agents/AgentPipeline.jsx	The live checklist and timer while the agents run
components/common/Sections.jsx	Collapsible sections and plain tables
context/AnalysisContext.jsx	Keeps the current ECR, the live events and the result
services/api.js	Every call to the backend
components/ui/	Generic UI building blocks (shadcn/ui)

How one analysis moves through the code

- The page calls POST `/api/ecr/{id}/analyze` (`api/routes/analysis.py`), and `services/ecr_service.py` starts a run.
- `agents/orchestrator.py` runs the graph on a background thread. The page follows the progress through GET `/api/workflows/{id}/stream`.
- The Retrieval Agent runs `understand_ecr` and `gather_evidence`: it classifies the change, LLM request 1 plans the search, then requirements, code impact, past defects, dependencies, comments and evidence are gathered.
- The Correlation Agent runs `assess_impact` and `select_tests`: the impacted components, the test selection and ordering (`services/test_selection_service.py`), the links between items, the defect insights, and LLM request 2, which reviews everything for conflicts and gaps.
- If "Require approval" was ticked, the approval gate waits for Approve or Reject.
- The Summarization Agent runs `summarize`: LLM request 3 writes the answer and the defect summary, and `services/report_service.py` builds the report.
- The report and the step log are written back to the CSV files (`impact_reports.csv`, `agent_runs.csv` and `reports/`). The page then loads the result from GET `/api/ecr/{id}/analysis`.

Where to change common things

To change	Edit
The wording sent to the LLM	<code>backend/app/prompts.py</code>
The API key, models and timeouts	<code>backend/.env</code> and <code>backend/app/config.py</code>
The data	<code>backend/data/*.csv</code>
Test selection weights and threshold	<code>backend/app/config.py</code> (<code>TEST_WEIGHT_*</code> , <code>TEST_SELECTION_THRESHOLD</code>)
What the Analysis page shows	<code>frontend/src/pages/AnalyzePage.jsx</code>

6. Architecture overview

Layers

Layer	Responsibility
React frontend	Three pages; live progress over Server-Sent Events; text and tables only
FastAPI layer	REST endpoints, validation, optional API key, rate limit placeholder
Service layer	ECR service, report service, test selection, RAG, code analysis, chat
Agent runtime	LangGraph state machine, run registry, event stream, step tracing
Repository layer	SQLAlchemy 2.0 queries over the working copy of the data
Data	CSV files in backend/data, loaded at startup, written back on every change

Request flow for one analysis

- The user enters an ECR number and presses Analyze.
- The API creates a workflow run, returns its id immediately and starts the agents on a worker thread, so the HTTP request never blocks.
- The browser opens an SSE stream for that workflow id and receives an event for every step start and finish, which is what fills the live checklist and the timer.
- The three agents run in order, each writing into one shared state object.
- If "Require approval" is ticked, the graph takes a conditional edge to a checkpoint and waits for a decision.
- The Summarization agent writes the answer and assembles the report; the report is persisted and the stream closes with a completion event carrying the duration.

Why three agents and not ten

The first version had ten specialist agents. Three matches what a person actually does: find it, join it up, explain it. Inside the three agents the fine-grained work still exists as steps and parts, each traced and each able to fail without stopping the analysis, so the progress view stays detailed without a sprawling graph. A guardrail test fails the build if a fourth agent appears.

7. The three agents and five steps

Agent	Step	What it does
Retrieval	1. Understand the ECR	Classifies the change, extracts features and keywords, plans the investigation
Retrieval	2. Gather evidence	Requirements, code impact, related defects, dependencies, comments and evidence
Correlation	3. Assess impact	Directly and indirectly impacted components, recommended actions
Correlation	4. Select tests	Discovers, scores, selects and orders tests; links artefacts; analyses defects
Summarization	5. Summarize	Writes the grounded answer and the defect summary, assembles the report

Agent 1 - Retrieval

Purpose: given an ECR number, gather everything that exists about it.

- The LLM plans the search first (request 1): search words and things that could break, which widen the requirement and defect searches.
- Change classification is deterministic: each change type has a signal vocabulary, and the paths of the changed files add weight. If the top two types are within 0.08 of each other the result is a mixed change.
- Entity extraction pulls affected features, technical keywords, candidate components and risk indicators (for example financial transaction, schema migration, authentication change, shared library change, concurrency change).
- Requirement tracing combines three channels: explicit links on the ECR, semantic search and keyword search, merged and re-ranked, each with a written reason.
- Code impact parses the declared changed files with Python's ast module, walks the import graph and resolves which components the change reaches.
- Defect retrieval merges semantic similarity over incident write-ups with defects recorded against the impacted components.
- Collaboration retrieval pulls review comments and evidence, and extracts decisions, raised risks, scope statements and evidence gaps.

Agent 2 - Correlation

Purpose: connect every artefact to the ECR, find the impacted components and decide which tests matter.

- Impact assessment lists the components the change touches (section 9).
- Test discovery, scoring, selection and ordering produce the recommended suite (section 10).
- The correlation graph links requirements, tests, defects, components, comments and evidence to the ECR with typed, explained edges.
- Defect insight checks each related defect against this change (section 11).
- The LLM reviews the whole correlated bundle (request 2) for conflicts and gaps, which the page lists under Conflicts and Gaps.

Agent 3 - Summarization

Purpose: answer the question from the correlated bundle, and publish the report.

- The answer is generated only from the correlated bundle, and it cites the identifiers it used. The model is instructed never to invent an identifier or a number.
- The defect summary explains the defect findings in three or four plain sentences.

- One LLM request (request 3) writes both the answer and the defect summary.
- The report assembles fifteen sections, the confidence score and the execution trace.

Each ECR is analysed within its own scope

Every ECR names its own scope: its linked requirements and its affected test cases (ecrs.csv).

The analysis shows only data that belongs to it (backend/app/agents/nodes/scope.py):

- Requirements: only the linked ones (plus the requirement in an "L2R26:TC1" style test id).
- Tests: the listed affected test cases and the tests traced to those requirements. Similarity search still scores and orders them, but never adds a test.
- Past defects: only those raised against the linked requirements.
- Impacted components: the component of each linked requirement changes directly; the other components those requirements name are listed as reached.
- Comments and evidence: only those attached to the ECR itself. For ECR-1 and ECR-2 the export's attachments are their evidence.

For the sample ECRs, the affected test cases are the tests traced to their linked requirements.

An ECR created without any linked requirement or test falls back to the wider search.

Steps and parts

Each step calls smaller parts. A part that fails is recorded, marks its evidence unavailable, lowers confidence and lets the analysis finish. Only two steps are critical: understanding the ECR and summarizing. Everything else degrades.

8. Planning, routing and human approval

The plan

After classification the planner decides what evidence is worth gathering for this specific change, and writes a reason for every decision. Requirements, code impact, related defects and review comments are always gathered. The dependency walk is skipped for a presentation-only change with no risk indicators, because it cannot change the answer for such a change.

Routing

The routing reason is recorded with each run (it is not shown on the page), for example: "UI change with 0 risk indicator(s) detected. Not needed: component dependencies." Dependency-heavy change types are database, API, backend, integration, security, infrastructure and mixed changes.

Human in the loop

When "Require approval" is ticked for a run, the workflow stops at a checkpoint before the report is published. The UI shows how many tests are recommended and offers Approve or Reject. The workflow blocks until the decision arrives, then continues or withholds the recommendation. This is a real conditional edge in the graph, not a UI trick.

9. Impacted components

The Correlation Agent lists which components a change touches. No score is computed - the output is the list itself, with a reason on every entry.

- Changed directly: components resolved from the changed files by the code analyzer. When no file resolves, the components named in the ECR are used, then the components of its linked requirements.
- Reached through dependencies: components the dependency walk shows the change propagates to.
- Each component carries its reasons: changed directly, how many dependency hops away, whether it has related past defects, and its business criticality.
- Recommended actions come from the change's risk indicators (for example schema migration or shared library change) and from open review concerns.

Two design rules worth knowing

- Direct impact rests on the strongest available evidence: components resolved from the actual changed files beat components guessed from keywords.
- Components below a propagation floor (0.25) are described as reachable, not impacted, and left out. Without that rule a one-line copy change pulled in services two hops away from it.

Examples from the current data

ECR	Changed directly
ECR-2026-001 (multi-currency validation)	Payment Service, Payment Validator, Currency Service - and 13 more reached
ECR-2026-002 (profile page button label)	Frontend Portal only; the dependency walk is not needed

10. Regression test selection engine

How a test is scored

Every candidate test is scored from 0 to 100 on five weighted factors.

Factor	Weight	What it measures
Requirement match	0.30	The test verifies a requirement the change affects
Component match	0.25	The test covers an impacted component
Dependency relevance	0.15	The test covers something downstream of the change
Historical failure	0.15	This test has caught real failures before
Semantic similarity	0.15	The test text resembles the change description

Selection and ordering

- Tests scoring below the relevance threshold (60) are dropped. The threshold, not the cap, is what decides suite size; the cap of 120 tests is a safety valve.
- Selected tests are banded into P0 to P3 and ordered by band, then component criticality, then relevance, then shortest runtime - so the fastest failure signal inside a band comes first.
- The baseline is scoped to the ECR's own product domain, so a payments change is measured against the payments suite and never against the rail suite.
- Every selected test carries a written reason, and the excluded ones are counted.

Worked examples from the current data

ECR	Selected	Effect
ECR-2026-001	67 of 170	60.6% fewer tests, 179 min instead of 424 min, 9 P0 and 23 P1
ECR-2026-020	39 of 170	77.1% fewer tests, 101 min instead of 424 min
ECR-2026-002	7 of 170	95.9% fewer tests, 11 min instead of 424 min, all P1

The reduction figure answers the question a test manager asks first: what do we not have to run?

11. Defect analysis

Defect analysis runs inside the same three agents - the Retrieval agent finds the defects, the Correlation agent checks them against the change, and the Summarization agent explains them.

What it answers for each related defect

- What went wrong, and the root cause.
- Why it matters for this change: the change touches the component where it happened, or it broke a requirement this ECR also affects, or it reached production last time.
- Can it happen again: High, Medium or Low.
- What to check: the recommended tests that cover it, or a manual check when no test covers it.

How the recurrence rating is decided

A weighted score combines severity, similarity, whether this change touches the defect's component (directly or indirectly), whether the defect broke a shared requirement, whether it escaped to production, and how often it was reopened. One rule overrides the score: a defect can only be rated Medium or High if this change actually touches its code. Without that rule almost every defect in the sample rated High, because most of them are severe and reached production.

What the section shows

- A plain-language summary of three or four sentences.
- Related past defects, with the four answers above.
- Defects that are open right now on the affected components, with how long they have been open.
- The defect history of each affected component: how many past defects, how many were high or critical, how many are still open, how many reached production, when the last one happened and the usual cause.

Because the last two parts come from component history rather than from text similarity, the section is never empty: even a change with no similar past defect shows how defect-prone the area is. Every sample ECR produces a populated section.

Example

For ECR-2026-014 (reserve stock at add-to-cart) the summary reads: 10 past defects are related; BUG-401 (stock oversold during a flash sale) and BUG-421 (checkout failed when the inventory service timed out) could happen again; the recommended tests cover every defect that could come back; BUG-421 is still open in the affected area. The history table shows the Order Service with 4 past defects, 1 still open, the most recent 25 days ago, usually caused by a resilience gap.

12. Retrieval (RAG) and embeddings

Hybrid retrieval

Requirements, defects, test cases and ECR descriptions are indexed. A query is answered by blending two signals: dense vector similarity (weight 0.65) and lexical keyword overlap (weight 0.35). Enterprise artefacts are short and full of identifiers, so pure vector search is brittle on them; the lexical half keeps identifier matches strong.

Embeddings

- By default the vectors are made locally (hashed, 1024 dimensions), so searching costs no gateway requests at all.
- Set `REMOTE_EMBEDDINGS=true` to use the gateway's embedding model (amazon.titan-embed-text-v2:0) instead. Vectors from two different embedders are not comparable, so the index records which embedder built it and rebuilds when that changes.
- Remote vectors are cached in `backend/.cache/embeddings.json`, keyed by a hash of each row's text, so only rows whose text changed are embedded again.

13. Dependency graph and code analysis

Component graph

The components and their dependencies are loaded into a NetworkX graph: 55 nodes and 72 edges in the current data. Each edge carries a type (synchronous API, library, database access, asynchronous event) and a criticality, and the propagation weight is the type weight multiplied by a criticality factor. Walking downstream from the changed components produces the blast radius, decayed by distance, plus the critical paths through it.

Code analysis

A real Python analyzer parses the sample repository (18 modules) with the ast module: functions, classes, imports, call targets, route constants. From the declared changed files it resolves the directly impacted modules, walks the import graph for dependents, and maps files to components. Java and C# analyzers exist as stubs behind the same interface.

14. LLM setup, resilience and cost control

Provider

The backend talks to an OpenAI-compatible enterprise gateway (the Capgemini Generative Engine Platform). Configuration lives in backend/.env, which is excluded from version control: OPENAI_API_KEY, OPENAI_BASE_URL, OPENAI_MODEL, OPENAI_FAST_MODEL and OPENAI_EMBEDDING_MODEL. The key value is not part of this document.

Role	Model configured
The three agent requests (OPENAI_FAST_MODEL)	anthropic.claude-haiku-4-5
Ask a Question answers (OPENAI_MODEL)	openai.gpt-5-mini
Embeddings, only with REMOTE_EMBEDDINGS=true	amazon.titan-embed-text-v2:0 (1024 dimensions)

The rule that keeps it trustworthy

Deterministic engines compute every number; the LLM only writes prose about numbers it was given. A model failure therefore cannot change the impacted components or a test list - it can only cost you the nicer wording, because every call carries a deterministic fallback text.

Resilience

- There is no LLM call at startup: every gateway request counts against the quota, and each agent falls back to its rules when the gateway is down.
- Only a rate limit (HTTP 429) or a failed connection is retried - twice, honouring the gateway's Retry-After and waiting at most 4 seconds. A timeout (60 seconds) is not retried, because the gateway may already have processed, and billed, that request.
- After three consecutive failures a circuit breaker opens and every later call is served locally; it half-opens after a 120 second cooldown and tries the real model again.
- A refused call returns the caller's grounded fallback text. This matters: an earlier version returned the offline provider's echo of the prompt, which briefly put raw prompt text on the page.

Cost control

Three LLM requests are made per analysis, one per agent: plan the search, review the correlated bundle, and write the answer with the defect summary. Step narration is off by default (LLM_NARRATION=false). Turning it on adds about seven more calls per analysis and makes runs several times slower, for wording only.

Offline mode

With no key configured the system runs end to end on deterministic code and rule-based text. Demo mode is a first-class path, not a stub: the same components, the same suites, the same sections.

15. Performance

Situation	Time	Note
Live LLM, current code	Usually 10 to 30 s	Three short requests, one per agent
Offline (no LLM)	Under 1 s	All deterministic work
Live LLM, before the optimisation	About 230 s	Nine sequential calls on a throttled gateway
Search index at startup	A few seconds	Local vectors; the remote option is slower the first time, then cached

What made it slow, and what changed

- The old code made about nine LLM calls per analysis, one after another, seven of them only to reword text that already existed in a deterministic form. Those are off by default now.
- Each throttled call was waiting roughly 25 seconds through its retries. Retry waits are now capped, and a call that still fails uses its fallback text.
- What is left is three short requests, one per agent.
- A fixed 220 ms pause per step, added originally to make demos watchable, is now 0 by default.
- Search uses local vectors by default, so it needs no gateway requests at all.

What the user sees

The page shows a live timer while the agents run, each step's duration in the checklist, and the total time in the execution trace once the run completes.

16. Data storage: CSV files

All data lives in CSV files under backend/data. There is no database server and no .db file in the project.

File	Rows	Contents
ecrs.csv	45	Change requests, including the columns of the change-tracking export (see below)
requirements.csv	64	Requirements traced to components and features
test_cases.csv	251	Test cases with folder, technique, polarity, automation status, failure rate
test_executions.csv	2,510	Ten past runs per test, consistent with its failure rate
defects.csv	68	Past and open defects with root cause, component and linked requirements
components.csv	55	Services, modules, databases and interfaces with business criticality
dependencies.csv	72	Which component depends on which, with type and criticality
comments.csv	55	Review comments on ECRs, with author, category and sentiment
evidence.csv	46	Test runs, reviews and documents attached to ECRs
impact_reports.csv	grows	One row per completed analysis
agent_runs.csv	grows	Step log: status, reasoning, confidence, duration
feedback.csv	grows	User feedback on recommendations
reports/ (folder)	grows	The full report of each analysis, one JSON file per run

ECRs from the change-tracking export

ECR-1 and ECR-2 were added in the column layout of the change-tracking tool export, and the app returns those values unchanged: Type, Id, Summary, Status, Severity, Assigned Testers, Actual and Planned Modified Software Objects, Build Resolved In, Test Estimate (Archived), Estimate, Test Estimate, Creation Date, Modified Date, Software Integration Test Assigned Testers, Test Actual Time, Description, Affected Test Cases, Steps to Reproduce, Observed Behavior, Expected Behavior and Attachments. They are separate columns in ecrs.csv (lifecycle_status holds the export's Status, so an analysis never overwrites it - the app's own NEW / ANALYSED state stays in status).

- The affected test cases (L2R26:TC1 ...) exist in test_cases.csv, and their requirements (L2R26, L2R35, L2R01, L2R05) in requirements.csv. The requirement and test wording was written from the ECR descriptions as sample text and should be replaced with the approved text.
- Every test the ECR lists is always recommended, marked P0 and run first, with the reason "Listed as an affected test case on the ECR".
- More export rows can be added the same way: one row per ECR in ecrs.csv, list columns as JSON text, for example ["Deepak Yadav"].

Format conventions

- The header row is the column names, which match the model fields.
- Lists and objects are JSON text inside the cell; booleans are true or false; times are ISO-8601.
- An empty cell means "use the default".
- dependencies.csv refers to components by component_id, not by row number.
- A full report is too large for a cell, so impact_reports.csv stores a path and the JSON lives in data/reports/<workflow_id>.json.

How the app uses them

At startup the files are loaded into a working SQL copy - a temporary SQLite file rebuilt on every start - so the repositories, agents and routes keep using ordinary SQLAlchemy queries. Every committed change is written back to the matching CSV file by a background writer that coalesces changes, writes to a temporary file and swaps it in, so a crash cannot leave a half-written file. Pending changes are also flushed on shutdown.

Editing the data

Edit the files with the backend stopped, or restart it afterwards, then run `python -m app.seed` to check they load. If a file is open in Excel when the app tries to write it, the write is retried and logged; the next change rewrites it.

17. Sample data: what it is and is not

All data in this project is synthetic. It was written for the project to exercise the engines; it contains no customer data and no data exported from a real programme.

It models two domains that share one schema:

- Commerce and payments: portal, gateway, payment service and validator, currency, orders, refunds, ledger, invoicing, fraud screening, plus the newer inventory, shipping, catalog, promotions, loyalty, subscriptions, wallet, chargebacks, payouts, tax, KYC, consent, audit, search, mobile backend, event broker, support portal, warehouse, gift cards and returns.
- Rail (I-ETMS / PTC onboard segment): requirements written as "the on-board segment shall ..." statements with TBC parameters, and test cases exported in the enterprise format with Folder, Optimization Technique, Test Type (Positive or Negative), Test Technique, Retired and Scorable columns. This mirrors the artefact shapes used on rail signalling programmes.

The domain field is the only discriminator; both domains use the same tables, the same engines and the same agents. That is the point: the platform is domain-agnostic, and only the content differs.

18. API reference

All endpoints are under the /api prefix. Interactive documentation is served at /docs.

Endpoint	Purpose
GET /api/health	Status of data, LLM, vector store and code analysis
GET /api/health/data	Row counts of the loaded data
GET /api/ecr	List or search change requests
GET /api/ecr/{id}	One change request
POST /api/ecr/{id}/analyze	Start an analysis; returns a workflow id and stream URL
GET /api/ecr/{id}/analysis	Full analysis payload (live run, else the saved report)
GET /api/ecr/{id}/impact	Impact, dependency and defect view
GET /api/ecr/{id}/tests	Recommended tests, filterable by priority
GET /api/ecr/{id}/sources	The raw multi-source view behind the analysis
GET /api/ecr/{id}/history	Previous analyses of this ECR
GET /api/ecr/{id}/report	The full report
POST /api/ecr/{id}/approve	Approve or reject at the human checkpoint
POST /api/ecr/{id}/ask	Ask a follow-up question about this ECR
GET /api/workflows/{id}	Status of one workflow run
GET /api/workflows/{id}/stream	Server-Sent Events stream of agent progress
GET /api/agents	The agent catalogue
GET /api/agents/tools	Every registered tool
GET /api/agents/runs	Persisted step log
GET /api/reports	Saved reports
GET /api/dashboard/stats	Dashboard figures
GET /api/components	Components
GET /api/components/graph	Dependency graph as nodes and edges
POST /api/chat/query	Conversational query across ECRs
POST /api/feedback	Record feedback on a recommendation

19. User interface

React 19 with Tailwind and shadcn/ui. Three pages, deliberately plain: text and tables, no charts and no graph visualisations.

Dashboard

Every change request and the latest analysis of each, as plain tables.

ECR Analysis

- An ECR lookup box with an approval switch and an Analyze button.
- While the agents run: a live checklist of the three agents and their steps, each with its own duration, and a running timer.
- Execution Trace: one row per step with the agent that ran it, its status and duration, and the total time.
- ECR Summary: the headline facts plus a Copy for Teams button that copies the summary as formatted text.
- ECR Details, Requirements, Recommended Test Cases, Impacted Components (with recommended actions), Defect Analysis, Conflicts and Gaps, Review Comments and Evidence.
- Ask a Question: follow-up questions answered from the same correlated bundle.

Report

The full report section by section, including the confidence score and its penalties.

20. Testing and quality

111 automated tests, all passing. They run in parallel across two workers, each with a private copy of the data files, so a test that writes data cannot disturb the real CSVs.

Area	What the tests guard
Three-agent guardrail	Exactly three agents; five steps; no module posing as an agent
Workflow	Every stage produces output; the expected payment impact; defect analysis is explained
Impacted components	Direct and indirect components are found; no score is produced
Test selection	Suite is smaller than the baseline; every test has a reason; bands are counted
Retrieval and graph	Hybrid search finds the right artefacts; blast radius is correct
CSV store	Files load; changes are written back; a bad value names file, line and column
LLM resilience	429 is retried; a refused call returns the fallback, never the prompt
API	Endpoint contracts and row counts

21. Security and privacy

- The LLM API key lives in backend/.env, which is gitignored. It is not in the repository and not in this document.
- Logs redact anything that looks like a secret. Workflow and ECR identifiers appear on every agent event for traceability.
- An optional API key can be required on every /api call, with CORS origins and a per-IP rate limit available.
- What leaves the building: with a remote LLM configured, the ECR text, requirements, comments and the correlated bundle are sent to that gateway. Setting LLM_PROVIDER=mock keeps everything local, and the system still produces the full analysis.
- The data folder is the whole data set; restricting who can read and write it is the main data control.

22. Limitations and roadmap

Known limitations

- The sample data is synthetic. Connecting real sources is the next step, not a rewrite: the provider interfaces exist and the local providers implement them.
- Enterprise connectors (Azure DevOps, Jira, TestRail, GitHub) are defined as interfaces with adapters that fail loudly until configured.
- Workflow runs are held in memory and are lost on restart; the saved report survives.
- The CSV writer assumes a single backend process per data folder.
- Only Python code analysis is implemented; Java and C# analyzers are stubs.
- The gateway currently rate-limits this key, which is what makes the three LLM requests take seconds rather than under a second.

Natural next steps

- Connect one real source system (requirements or defects) behind the existing interface.
- Move workflow runs and the event stream to Redis so several backend processes can serve them.
- Feed real execution results back in, so the historical-failure factor learns from actual runs.
- Add a per-team threshold configuration so different products can size suites differently.

23. Repository map

Path	Contents
backend/app/main.py	FastAPI application, startup, warmup
backend/app/config.py	All settings and engine weights
backend/app/database.py	Working copy engine, session factory, CSV loading
backend/app/data/csv_store.py	CSV table specs, loading, validation, write-back
backend/app/agents/state.py	Agent catalogue, shared workflow state
backend/app/agents/orchestrator.py	LangGraph graph, approval edge, report persistence
backend/app/agents/runtime.py	Run registry, event stream, step tracing
backend/app/agents/nodes/	The five steps and their parts
backend/app/agents/tools/	Registered tools used by the steps
backend/app/services/	Test selection, RAG, code analysis, report, chat, confidence
backend/app/models/	SQLAlchemy models - one per CSV table
backend/app/api/routes/	REST endpoints
backend/app/seed/sample_repo/	The small Python repository the code analyzer parses
backend/data/	The CSV data files and the reports folder
backend/app/tests/	111 tests
frontend/src/pages/	Dashboard, ECR Analysis, Report
frontend/src/components/agents/	Live agent checklist
docs/	Architecture, agent design, API, deployment, demo guide

24. Glossary

Term	Meaning
ECR	Engineering Change Request: a proposed change to the product
Agent	One of the three autonomous workers in the graph
Step	A traced unit of work owned by one agent; there are five
Part	A smaller function inside a step; may fail without stopping the analysis
P0 to P3	Priority bands of the recommended regression suite
Reduction	How much smaller the recommended suite is than the full domain suite
RAG	Retrieval Augmented Generation: search that grounds the model's answer
Embedding	A numeric vector representing text, used for similarity search
Circuit breaker	Protection that stops calling a failing endpoint for a cooldown period
Demo mode	Running with no LLM key: the same analysis with rule-based text
Human in the loop	The approval checkpoint, used when "Require approval" is ticked

25. Questions a manager may ask

Scope and value

Q: What does this actually do?

A: You give it a change request number. It finds the related requirements, code impact, past defects, review comments and evidence, works out which components it affects, recommends which regression tests to run, and explains all of it in plain language within seconds.

Q: What problem does it solve?

A: The manual gathering across five systems before every change. That work is slow, repetitive and inconsistent between people.

Q: How much time does it save?

A: Two ways. The gathering itself, which took an engineer a significant part of an hour per change, now takes seconds. And the test run: for a payments change the recommendation was 67 tests instead of 170, which is 179 minutes instead of 424. For a copy change it was 7 tests, 11 minutes instead of 424.

Q: Is it accurate?

A: Every number is computed by deterministic code with weights you can see and change, and every recommendation carries a written reason, so it can be checked line by line. The model writes only the prose.

Q: What if the AI is wrong?

A: The AI cannot change a score or a test list; it only writes the summary. If the gateway is down the system falls back to rule-based text and the analysis is unchanged.

Technology

Q: What is it built with?

A: Python with FastAPI on the backend, LangGraph for the agent workflow, SQLAlchemy over CSV data, React with Tailwind on the frontend. The LLM is reached through an OpenAI-compatible enterprise gateway.

Q: How is the code organised?

A: A Python FastAPI backend and a React frontend. In the backend, agents/ holds the workflow (orchestrator.py), the three agents (nodes/agents.py) and their five steps (nodes/steps.py); services/ holds test selection, search and reports; prompts.py holds every LLM instruction; the data is the CSV files in backend/data. Section 5 has the full map.

Q: How many agents are there, and why three?

A: Three - Retrieval, Correlation, Summarization - running five traced steps. Three maps onto what a person does: find it, join it up, explain it. A test fails the build if a fourth appears.

Q: Where is the data?

A: In CSV files under backend/data, one per table. No database server. They are loaded at startup and every change is written back to them.

Q: Is any of this real production data?

A: No. All sample data is synthetic and written for this project. The rail artefacts follow the shape of real programme exports, but the content is invented.

Q: Does it send our data to an AI provider?

A: With the gateway configured, the ECR text and the correlated bundle are sent there for the two summary calls. Set the provider to mock and nothing leaves the machine; the analysis still runs.

Q: Where is the API key?

A: In backend/.env, which is excluded from version control. It is not in the repository, not in this document, and not in any shared file.

Operations

Q: How long does an analysis take?

A: Usually 10 to 30 seconds with the live model, under a second offline. It was around 230 seconds before the performance work; the fix was to stop making LLM calls that only reworded existing text, cap the retry waiting, and keep it to three short requests, one per agent.

Q: What happens when the gateway is rate-limited?

A: The call is retried briefly, then the deterministic text is used. After three consecutive failures the circuit breaker opens for two minutes and everything is served locally. The analysis still completes.

Q: How do we add our own data?

A: Edit the CSV files, or replace them with exports from the real systems in the same column format, then restart. `python -m app.seed` validates them and names the file, line and column of any bad value.

Q: How do we connect the real tools?

A: The provider interfaces are already there - requirements, defects, tests, repository - with local implementations that work and enterprise adapters that fail loudly until configured. Wiring one real source is an adapter, not a redesign.

Q: How do we know it works?

A: 111 automated tests cover the agents, both engines, retrieval, the data layer and the API. They run in seconds.

Q: Can it run without internet?

A: Yes, completely. That is demo mode and it is a supported path, not a stub.

Q: What would you do next?

A: Connect one real source system, move workflow runs out of process memory so it can scale horizontally, and feed real test results back in so the historical-failure factor learns.

Review and control

Q: Can a person override it?

A: Yes. Tick "Require approval" and the analysis stops at a checkpoint before the report is published, until someone approves or rejects it. Tests can also be excluded from the recommendation.

Q: How do we audit what happened?

A: Every step writes a row to `agent_runs.csv` with its status, reasoning, confidence and duration, and every completed analysis writes to `impact_reports.csv` with the full report JSON beside it.

Q: What does the confidence score mean?

A: It blends the confidence of each step and applies penalties for failures and missing evidence, so a run that lost a source is visibly less certain rather than silently wrong.

26. Five-minute demo script

-
- Open the ECR Analysis page. Point out the line "Data source: CSV files - 45 ECRs available".
 - Enter ECR-2026-001 and press Analyze. While it runs, point at the live checklist: three agents, five steps, each with its own timer.
 - When it finishes (usually well under a minute) read the ECR Summary aloud: 3 components changed directly, 67 of 170 tests recommended, 60.6% fewer, 179 minutes instead of 424.
 - Open Impacted Components: what changes directly, what is reached through dependencies, and the recommended actions.
 - Open Recommended Test Cases and show that every row has a reason.
 - Open Defect Analysis: BUG-101 currency conversion, why it matters here, whether it can come back and which test covers it - plus what is open right now in that area.
 - Open Execution Trace and show the five steps, the agent behind each and the total time.
 - Now enter ECR-2026-002, the profile page button label, and run it: 7 tests, 11 minutes, and only the Frontend Portal changed directly.
 - Close with the contrast: same system, same engines, two very different recommendations, each explained.

26. Complete source code

Every source file of the project follows, grouped by area and shown with line numbers. A line too long for the page continues on the next line, marked with a plus sign. Generated files, dependencies, build output and the data files themselves are not included; data samples are in the section after the code.

Backend - entry point and configuration

backend/app/main.py

```
1  """FastAPI application entrypoint."""
2  from __future__ import annotations
3
4  import threading
5  from contextlib import asynccontextmanager
6
7  from fastapi import Depends, FastAPI
8  from fastapi.middleware.cors import CORSMiddleware
9  from fastapi.responses import JSONResponse
10 from starlette.requests import Request
11
12 from app.api.routes import agents, analysis, chat, ecr, health, reports
13 from app.config import settings
14 from app.data.csv_store import writer as csv_writer
15 from app.database import init_db, session_scope
16 from app.dependencies import rate_limit, require_api_key
17 from app.logging import configure_logging, get_logger
18 from app.services.rag_service import get_rag_service
19
20 configure_logging()
21 logger = get_logger("ecr.api")
22
23 DESCRIPTION = """
24 **ECR Intelligence Agent** - AI-powered Engineering Change Request analysis.
25
26 Give it an ECR number and exactly three agents do the manual work:
27
28 1. **Retrieval Agent** - pulls the ECR record, traced and semantically matched
29 requirements, historical defects, code impact, dependency reach, review
30 comments and evidence.
31 2. **Correlation Agent** - links every artefact to the ECR, works out which
32 components are impacted and selects the regression tests that matter.
33 3. **Summarization Agent** - answers in plain language with citations and
34 publishes the report.
35
36 Runs fully offline in demo mode when no LLM credentials are configured.
37 """
38
39
40 def _warmup() -> None:
41     """Index in the background so startup is never blocked.
42
43     There is no LLM probe here: every gateway request counts against the quota,
44     and each agent's call already falls back to its rules when the gateway is down.
45     """
46     try:
47         with session_scope() as db:
48             get_rag_service().index_all(db)
49     except Exception as exc: # pragma: no cover - warmup must never kill the app
50         logger.warning("startup.warmup_failed", error=str(exc))
51
52
53 @asynccontextmanager
54 async def lifespan(app: FastAPI):
55     counts = init_db() # load backend/data/*.csv
56     logger.info("startup.data_loaded", data_dir=settings.data_dir, **counts)
57     threading.Thread(target=_warmup, name="warmup", daemon=True).start()
58     logger.info("startup.complete", env=settings.app_env, provider=settings.resolved_llm_provider())
59     yield
60     csv_writer.flush() # write any pending changes back to the CSV files
61     logger.info("shutdown.complete")
62
63
64 app = FastAPI(
65     title=settings.app_name,
66     description=DESCRIPTION,
67     version=settings.app_version,
68     lifespan=lifespan,
69     docs_url="/docs",
```

```

70     redoc_url="/redoc",
71 )
72
73 app.add_middleware(
74     CORSMiddleware,
75     allow_origins=settings.cors_origins,
76     allow_credentials=True,
77     allow_methods=["*"],
78     allow_headers=["*"],
79     expose_headers=["*"],
80 )
81
82 GUARDS = [Depends(require_api_key), Depends(rate_limit)]
83
84 app.include_router(health.router, prefix=settings.api_prefix)
85 app.include_router(ecr.router, prefix=settings.api_prefix, dependencies=GUARDS)
86 app.include_router(analysis.router, prefix=settings.api_prefix, dependencies=GUARDS)
87 app.include_router(analysis.workflow_router, prefix=settings.api_prefix, dependencies=GUARDS)
88 app.include_router(agents.router, prefix=settings.api_prefix, dependencies=GUARDS)
89 app.include_router(reports.router, prefix=settings.api_prefix, dependencies=GUARDS)
90 app.include_router(chat.router, prefix=settings.api_prefix, dependencies=GUARDS)
91
92
93 @app.get("/")
94 def root() -> dict[str, str]:
95     return {
96         "name": settings.app_name,
97         "version": settings.app_version,
98         "docs": "/docs",
99         "health": f"{settings.api_prefix}/health",
100     }
101
102
103 @app.exception_handler(Exception)
104 async def unhandled_exception(request: Request, exc: Exception) -> JSONResponse: # pragma: no cover
105     logger.exception("api.unhandled", path=str(request.url.path))
106     return JSONResponse(
107         status_code=500,
108         content={"detail": "internal error", "path": str(request.url.path)},
109     )

```

backend/app/config.py

```

1  """Application configuration.
2
3  Everything a deployment may want to change lives here and is overridable
4  through environment variables (or a ``.env`` file at the repo root / backend/).
5  """
6  from __future__ import annotations
7
8  from functools import lru_cache
9  from pathlib import Path
10 from typing import List, Literal
11
12 from pydantic import Field, field_validator
13 from pydantic_settings import BaseSettings, SettingsConfigDict
14
15 BACKEND_DIR = Path(__file__).resolve().parent.parent
16 REPO_ROOT = BACKEND_DIR.parent
17
18
19 class TestSelectionWeights(BaseSettings):
20     """Weights of the deterministic test-selection algorithm (sum to 1.0)."""
21
22     model_config = SettingsConfigDict(env_prefix="TEST_WEIGHT_", extra="ignore")
23
24     requirement_match: float = 0.30
25     component_match: float = 0.25
26     dependency_relevance: float = 0.15
27     historical_failure: float = 0.15
28     semantic_similarity: float = 0.15
29
30
31 class Settings(BaseSettings):
32     model_config = SettingsConfigDict(
33         env_file=(REPO_ROOT / ".env", BACKEND_DIR / ".env"),
34         env_file_encoding="utf-8",
35         extra="ignore",
36         case_sensitive=False,
37     )
38
39     # -- application -----
40     app_env: Literal["development", "test", "production"] = "development"
41     app_name: str = "ECR Intelligence Agent"

```

```

42 app_version: str = "1.0.0"
43 log_level: str = "INFO"
44 api_prefix: str = "/api"
45
46 # -- security -----
47 api_key: str | None = None # when set, clients must send X-API-Key
48 cors_origins: List[str] = Field(default_factory=lambda: ["*"])
49 rate_limit_per_minute: int = 0 # 0 disables the rate-limit placeholder
50
51 # -- data -----
52 # CSV files are the only data store: one file per table in DATA_DIR.
53 data_dir: str = str(BACKEND_DIR / "data")
54 # Working copy the CSVs are loaded into for querying. Empty = a temporary
55 # SQLite file rebuilt from the CSVs on every start.
56 database_url: str = ""
57 embedding_cache_path: str = str(BACKEND_DIR / ".cache" / "embeddings.json")
58 db_echo: bool = False
59
60 # -- LLM -----
61 llm_provider: Literal["mock", "openai", "anthropic", "azure_openai", "auto"] = "auto"
62 openai_api_key: str | None = None
63 openai_model: str = "gpt-4o"
64 # A cheaper/faster model used for the many short narration calls.
65 openai_fast_model: str = ""
66 openai_base_url: str = "https://api.openai.com/v1"
67 openai_embedding_model: str = "text-embedding-3-small"
68 # Off: search embeddings are computed locally and cost no gateway requests.
69 # On: every search query (and any re-index) is also sent to the embedding
70 # endpoint, on top of the three agent requests.
71 remote_embeddings: bool = False
72 embedding_batch_size: int = 16
73 anthropic_api_key: str | None = None
74 anthropic_model: str = "claude-sonnet-5"
75 anthropic_base_url: str = "https://api.anthropic.com/v1"
76 azure_openai_endpoint: str | None = None
77 azure_openai_api_key: str | None = None
78 azure_openai_deployment: str = "gpt-4o"
79 azure_openai_api_version: str = "2024-08-01-preview"
80 # Long enough for the largest agent reply: a request that times out has
81 # probably still been billed by the gateway, and its answer is thrown away.
82 llm_timeout_seconds: float = 60.0
83 # Only throttled (429) or never-connected requests are retried - never one
84 # the gateway may already have processed, so each agent costs one request.
85 llm_max_retries: int = 2
86 llm_rate_limit_max_wait_seconds: float = 4.0
87 # An analysis makes exactly three LLM requests, one per agent: Retrieval plans
88 # the search, Correlation reviews the bundle, Summarization writes the answer
89 # and the defect summary in one reply. On adds a request per intermediate step
90 # to reword it (about 7 more per analysis) - leave it off to stay at three.
91 llm_narration: bool = False
92 narration_max_tokens: int = 320
93 llm_max_tokens: int = 1200
94
95 # -- vector store -----
96 vector_store: Literal["memory", "pgvector", "chroma"] = "memory"
97 embedding_dim: int = 1024
98 chroma_path: str = str(BACKEND_DIR / ".chroma")
99
100 # -- agent runtime -----
101 agent_step_delay_ms: int = 0 # >0 slows each step down for demos
102 human_in_the_loop: bool = False # can be overridden per analysis request
103 workflow_timeout_seconds: float = 180.0
104 max_agent_retries: int = 2
105
106 # -- engines -----
107 test_weights: TestSelectionWeights = Field(default_factory=TestSelectionWeights)
108 test_selection_threshold: float = 60.0 # tests scoring below this are dropped
109 # Safety valve only - the relevance threshold should decide the suite size,
110 # not the cap. Raise/lower per execution budget.
111 max_selected_tests: int = 120
112
113 @field_validator("cors_origins", mode="before")
114 @classmethod
115 def _split_origins(cls, value: object) -> object:
116     if isinstance(value, str):
117         return [item.strip() for item in value.split(",") if item.strip()]
118     return value
119
120 @property
121 def is_sqlite(self) -> bool:
122     return not self.database_url or self.database_url.startswith("sqlite")
123
124 def resolved_llm_provider(self) -> str:
125     """Resolve ``auto`` into a concrete provider based on available keys."""

```

```

126         if self.llm_provider != "auto":
127             return self.llm_provider
128         if self.azure_openai_api_key and self.azure_openai_endpoint:
129             return "azure_openai"
130         if self.openai_api_key:
131             return "openai"
132         if self.anthropic_api_key:
133             return "anthropic"
134         return "mock"
135
136
137 @lru_cache
138 def get_settings() -> Settings:
139     return Settings()
140
141
142 settings = get_settings()

```

backend/app/database.py

```

1  """Database engine and session factory over the CSV data files.
2
3  The CSV files in ``DATA_DIR`` (``backend/data``) are the only place data lives.
4  ``init_db()`` loads them into a working SQL copy at startup, so repositories,
5  agents and routes keep using ordinary SQLAlchemy queries, and every committed
6  change is written back to the matching CSV file (see :mod:`app.data.csv_store`).
7
8  The working copy is a throwaway SQLite file in the system temp folder, rebuilt
9  from the CSVs on every start. ``DATABASE_URL`` can point it somewhere else.
10 """
11 from __future__ import annotations
12
13 import hashlib
14 import tempfile
15 from collections.abc import Iterator
16 from contextlib import contextmanager
17 from pathlib import Path
18
19 from sqlalchemy import create_engine, event
20 from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker
21 from sqlalchemy.pool import StaticPool
22
23 from app.config import settings
24 from app.logging import get_logger
25
26 logger = get_logger("ecr.db")
27
28
29 class Base(DeclarativeBase):
30     """Declarative base shared by every ORM model."""
31
32
33 def _working_copy_url() -> str:
34     if settings.database_url:
35         return settings.database_url
36     # One working copy per data folder, so two checkouts never share a file.
37     digest = hashlib.sha1(str(Path(settings.data_dir).resolve()).encode("utf-8")).hexdigest()[:10]
38     return f"sqlite:///{{(Path(tempfile.gettempdir()) / f'ecr_assistant_{{digest}}.db').as_posix())}"
39
40
41 DATABASE_URL = _working_copy_url()
42 IS_SQLITE = DATABASE_URL.startswith("sqlite")
43
44
45 def _make_engine():
46     kwargs: dict = {"echo": settings.db_echo, "future": True, "pool_pre_ping": True}
47     if IS_SQLITE:
48         # check_same_thread=False: LangGraph nodes run in a worker thread pool.
49         kwargs["connect_args"] = {"check_same_thread": False}
50         if DATABASE_URL in ("sqlite://", "sqlite:///memory:"):
51             kwargs["poolclass"] = StaticPool # one shared in-memory database
52     return create_engine(DATABASE_URL, **kwargs)
53
54
55 engine = _make_engine()
56
57 if IS_SQLITE: # keep FK constraints honest on SQLite too
58
59     @event.listens_for(engine, "connect")
60     def _fk_pragma(dbapi_connection, _record): # pragma: no cover - trivial
61         cursor = dbapi_connection.cursor()
62         cursor.execute("PRAGMA foreign_keys=ON")
63         cursor.close()
64

```

```

65
66 SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)
67
68
69 def get_db() -> Iterator[Session]:
70     """FastAPI dependency yielding a request-scoped session."""
71     db = SessionLocal()
72     try:
73         yield db
74     finally:
75         db.close()
76
77
78 @contextmanager
79 def session_scope() -> Iterator[Session]:
80     """Context manager used by agents/background workers."""
81     db = SessionLocal()
82     try:
83         yield db
84         db.commit()
85     except Exception:
86         db.rollback()
87         raise
88     finally:
89         db.close()
90
91
92 def enable_pgvector() -> bool:
93     """Create the pgvector extension when the working copy is PostgreSQL."""
94     if IS_SQLITE:
95         return False
96     from sqlalchemy import text
97
98     try:
99         with engine.begin() as conn:
100             conn.execute(text("CREATE EXTENSION IF NOT EXISTS vector"))
101             logger.info("pgvector.enabled")
102             return True
103     except Exception as exc: # pragma: no cover - depends on server privileges
104         logger.warning("pgvector.unavailable", error=str(exc))
105         return False
106
107
108 _loaded_counts: dict[str, int] | None = None
109
110
111 def init_db(*, reload: bool = False) -> dict[str, int]:
112     """Rebuild the working copy from the CSV files (once per process).
113
114     Raises :class:`app.data.csv_store.CsvDataError` naming the file, line and
115     column when a CSV cannot be loaded.
116     """
117     global _loaded_counts
118     from app import models # noqa: F401 (registers mappers)
119     from app.data import csv_store
120
121     if _loaded_counts is not None and not reload:
122         return _loaded_counts
123     csv_store.writer.flush() # never drop tables with changes still unwritten
124     Base.metadata.drop_all(bind=engine)
125     Base.metadata.create_all(bind=engine)
126     with SessionLocal() as db:
127         _loaded_counts = csv_store.load_all(db)
128     csv_store.install_write_back(SessionLocal, engine)
129     logger.info("database.loaded_from_csv", data_dir=settings.data_dir, working_copy=DATABASE_URL)
130     return _loaded_counts

```

backend/app/logging.py

```

1 """Structured logging for the platform.
2
3 Uses ``structlog`` when available and degrades to the stdlib logger otherwise.
4 Secrets are never logged: :func:`redact` scrubs anything that looks like a key.
5 """
6 from __future__ import annotations
7
8 import logging
9 import sys
10 from typing import Any, Mapping
11
12 from app.config import settings
13
14 _SENSITIVE_HINTS = ("key", "token", "secret", "password", "authorization")
15

```

```

16
17 def redact(payload: Mapping[str, Any]) -> dict[str, Any]:
18     """Return ``payload`` with sensitive looking values masked."""
19     clean: dict[str, Any] = {}
20     for key, value in payload.items():
21         if any(hint in key.lower() for hint in _SENSITIVE_HINTS):
22             clean[key] = "***redacted***"
23         elif isinstance(value, Mapping):
24             clean[key] = redact(value)
25         else:
26             clean[key] = value
27     return clean
28
29
30 def configure_logging() -> None:
31     level = getattr(logging, settings.log_level.upper(), logging.INFO)
32     logging.basicConfig(
33         level=level,
34         stream=sys.stdout,
35         format="%(asctime)s | %(levelname)-7s | %(name)s | %(message)s",
36     )
37     logging.getLogger("uvicorn.access").setLevel(max(level, logging.INFO))
38     try: # pragma: no cover - structlog is optional
39         import structlog
40
41         structlog.configure(
42             processors=[
43                 structlog.contextvars.merge_contextvars,
44                 structlog.processors.add_log_level,
45                 structlog.processors.TimeStamper(fmt="iso"),
46                 structlog.dev.ConsoleRenderer(colors=False),
47             ],
48             wrapper_class=structlog.make_filtering_bound_logger(level),
49             cache_logger_on_first_use=True,
50         )
51     except Exception: # pragma: no cover
52         pass
53
54
55 class _StdlibAdapter:
56     """Minimal key/value logger used when structlog is unavailable."""
57
58     def __init__(self, name: str) -> None:
59         self._log = logging.getLogger(name)
60
61     def _emit(self, level: int, event: str, **kw: Any) -> None:
62         if kw:
63             extras = " ".join(f"{k}={v}" for k, v in redact(kw).items())
64             self._log.log(level, "%s | %s" % (event, extras))
65         else:
66             self._log.log(level, event)
67
68     def debug(self, event: str, **kw: Any) -> None:
69         self._emit(logging.DEBUG, event, **kw)
70
71     def info(self, event: str, **kw: Any) -> None:
72         self._emit(logging.INFO, event, **kw)
73
74     def warning(self, event: str, **kw: Any) -> None:
75         self._emit(logging.WARNING, event, **kw)
76
77     def error(self, event: str, **kw: Any) -> None:
78         self._emit(logging.ERROR, event, **kw)
79
80     def exception(self, event: str, **kw: Any) -> None:
81         self._log.exception(event, extra=redact(kw) or None)
82
83
84 def get_logger(name: str = "ecr") -> Any:
85     try: # pragma: no cover - structlog is optional
86         import structlog
87
88         return structlog.get_logger(name)
89     except Exception: # pragma: no cover
90         return _StdlibAdapter(name)

```

backend/app/dependencies.py

```

1 """Shared FastAPI dependencies: auth, rate limiting placeholder, services."""
2 from __future__ import annotations
3
4 import time
5 from collections import defaultdict, deque
6 from typing import Annotated

```

```

7
8 from fastapi import Depends, Header, HTTPException, Request, status
9 from sqlalchemy.orm import Session
10
11 from app.config import settings
12 from app.database import get_db
13
14 DbSession = Annotated[Session, Depends(get_db)]
15
16 _BUCKETS: dict[str, deque[float]] = defaultdict(deque)
17
18
19 async def require_api_key(x_api_key: str | None = Header(default=None)) -> None:
20     """Optional API-key gate. Disabled unless `API_KEY` is configured."""
21     if not settings.api_key:
22         return
23     if x_api_key != settings.api_key:
24         raise HTTPException(
25             status_code=status.HTTP_401_UNAUTHORIZED, detail="invalid or missing X-API-Key"
26         )
27
28
29 async def rate_limit(request: Request) -> None:
30     """In-process fixed-window limiter.
31
32     A placeholder for a real gateway limiter: it is per-process and per-client
33     IP, which is enough to protect a demo deployment. Disabled when
34     `RATE_LIMIT_PER_MINUTE` is 0.
35     """
36     limit = settings.rate_limit_per_minute
37     if not limit:
38         return
39     client = request.client.host if request.client else "unknown"
40     now = time.time()
41     bucket = _BUCKETS[client]
42     while bucket and now - bucket[0] > 60:
43         bucket.popleft()
44     if len(bucket) >= limit:
45         raise HTTPException(
46             status_code=status.HTTP_429_TOO_MANY_REQUESTS, detail="rate limit exceeded"
47         )
48     bucket.append(now)
49
50
51 CommonGuards = [Depends(require_api_key), Depends(rate_limit)]

```

backend/app/prompts.py

```

1 """Everything the ECR Assistant says to the AI model, in one place.
2
3 Each agent has a SYSTEM prompt (who it is, what we need) and a TASK template
4 (the request, with details filled into the `{placeholders}`). Every prompt is
5 restricted to the one ECR being analysed: its linked requirements, its listed
6 test cases, the defects linked to those requirements, and the comments and
7 evidence attached to it (see `app/agents/nodes/scope.py`). Wording changes here
8 never change the impacted parts, the test list or the data.
9
10 When editing: keep every `{placeholder}` and every reply field name
11 (`summary`, `answer` ...), write a literal brace twice (`{{` `}}`),
12 and restart the backend. `app/tests/test_services/test_prompts.py` checks this.
13 """
14 from __future__ import annotations
15
16 # Shared by all three agents: stay inside this ECR.
17 GROUND_RULES = (
18     "\n\nRules:\n"
19     "- Work only on this change request (ECR). Use only the information given to you about it: "
20     "its description, its linked requirements, its listed test cases, the past bugs linked to "
21     "those requirements, and the comments and evidence attached to it.\n"
22     "- Don't mention any requirement, test, bug, component, comment or evidence that isn't in "
23     "that information, and don't make up IDs, numbers or decisions.\n"
24     "- Refer to items by the IDs you were given.\n"
25     "- If something isn't there, say it isn't recorded - don't guess.\n"
26     "- No scores or ratings of your own."
27 )
28
29 RAILWAY_CHECKS = (
30     "\n\nFor a railway change, use the terms the record itself uses (for example onboard, crew "
31     "display, departure test, braking, horn, control type) and don't widen it to other systems."
32 )
33
34 JSON_ONLY = "\n\nReply with JSON only."
35
36

```

```

37 # 1. Retrieval Agent - goes first and describes the change.
38 RETRIEVAL_SYSTEM_PROMPT = (
39     "You're the Retrieval Agent. Read this one change request and describe it in its own terms: "
40     "what's changing, what could break, the words that describe it, and whether the change "
41     "record itself points to other parts of the system."
42     + GROUND_RULES
43     + RAILWAY_CHECKS
44     + JSON_ONLY
45 )
46
47 RETRIEVAL_TASK = ""Change request {ecr_id}: {title}
48 {description}
49 Files: {changed_files}
50 Linked requirements: {linked_requirements}
51 Rule-based check: {classification}
52
53 Send back:
54 - summary: two short sentences - what's changing and what could break, based only on this record.
55 - search_terms: up to {max_terms} words or phrases taken from this record (no IDs).
56 - could_break: up to {max_behaviours} things this record says or implies could stop working.
57 - needs_dependency_check: true only if this record says other parts of the system are affected, \
58 otherwise false.
59 - dependency_reason: one sentence saying why, from this record.""
60
61
62 # 2. Correlation Agent - checks this ECR's own items against each other.
63 CORRELATION_SYSTEM_PROMPT = (
64     "You're the Correlation Agent. Check this change request's own items against each other - "
65     "its requirements, listed tests, linked past bugs, comments and evidence - and tell us what "
66     "fits, what really disagrees, and what is really missing. Keep each point to one sentence; "
67     "the team sees them under \"Conflicts and Gaps\".\n\n"
68     "Only call something a conflict when two items truly contradict each other. A test that "
69     "checks another configuration still works is a regression check, not a conflict. Evidence "
70     "attached to the change request covers the change request as a whole. Don't change the "
71     "impacted parts or the test list."
72     + GROUND_RULES
73     + RAILWAY_CHECKS
74     + JSON_ONLY
75 )
76
77 # Kept short on purpose: a long reply gets cut off, and cut-off JSON can't be read.
78 CORRELATION_TASK = ""This change request and everything that belongs to it:
79 {bundle}
80
81 Our rule-based findings (confirm, correct or drop them):
82 {rule_findings}
83
84 Send back:
85 - summary: up to three short sentences about this change request only.
86 - key_links, conflicts and gaps: up to {max_findings} each, written as \
87 {{\"ids\": [...], \"text\": \"one sentence under 30 words\"}}, using only IDs from above. \
88 Use an empty list if there's nothing.
89
90 Keep it short, on one line.""
91
92
93 # 3. Summarization Agent - answers the question (also used by "Ask a Question").
94 ANSWER_SYSTEM_PROMPT = (
95     "You're the Summarization Agent. Answer the question about this one change request using "
96     "only its own information: the answer first, then the two or three facts behind it, then "
97     "anything that needs attention. If someone approved or rejected the analysis, say who - a "
98     "rejection means the test recommendation is on hold. Don't repeat lists the page already "
99     "shows. If the question is about something outside this change request, say you can only "
100     "answer about this one."
101     + GROUND_RULES
102     + RAILWAY_CHECKS
103 )
104
105 # Added for the main analysis, which also explains past bugs.
106 SUMMARY_STYLE = "\n\nUse everyday English and normal sentences - no headings, bullets or markdown."
107
108 SUMMARY_TASK = ""Question: {question}
109
110 This change request and everything that belongs to it:
111 {bundle}
112
113 Send back:
114 - answer: up to 8 sentences, about this change request only.
115 {defect_instruction}""
116
117 # Only used when past bugs were linked to the change.
118 SUMMARY_DEFECT_INSTRUCTION = (
119     "- defect_summary: 3 or 4 short sentences about the past bugs linked to {ecr_id} (only those) - "
120     "which could come back, why, and what to check."

```

```

121 )
122
123 SUMMARY_NO_DEFECT_INSTRUCTION = "- defect_summary: an empty string."
124
125
126 # Optional step explanations - only used when LLM_NARRATION=true.
127 NARRATION_SYSTEM_PROMPT = (
128     "Make this explanation of one change request clearer for engineering and test leads. The facts "
129     "are already correct - keep it short and don't add anything that isn't there."
130 )
131
132 # The offline model copies back the text between "DETERMINISTIC BASELINE NARRATIVE:"
133 # and the blank line before "Rewrite" - keep both exactly as they are.
134 NARRATION_TASK = ""What to explain: {task}
135
136 Facts (correct - don't contradict them):
137 {context}
138
139 DETERMINISTIC BASELINE NARRATIVE:
140 {fallback}
141
142 Rewrite the draft above in plain words. Keep every ID and number the same. At most 6 sentences.""

```

Backend - CSV data store

backend/app/data/__init__.py

```

1 """CSV-backed data storage."""

```

backend/app/data/csv_store.py

```

1 """CSV files are the system of record.
2
3 One CSV per table lives in ``DATA_DIR`` (``backend/data`` by default). At startup
4 the files are loaded into an in-memory SQL engine, so every repository, agent
5 tool and API route keeps using ordinary SQLAlchemy queries. Whenever a session
6 commits a change to a table, that table is written back to its CSV file by a
7 background writer, so nothing is lost on restart and the files can be edited
8 by hand.
9
10 Conventions
11 * header row = the model's column names; ``id`` is never stored (rows are
12   identified by their business key, e.g. ``ecr_id``)
13 * lists and dicts are JSON text, booleans are ``true``/``false``,
14   timestamps are ISO-8601
15 * ``dependencies.csv`` refers to components by ``component_id``
16 * a full analysis report is too large for a cell, so ``impact_reports.csv``
17   stores a path and the JSON lives in ``reports/<workflow_id>.json``
18 """
19 from __future__ import annotations
20
21 import csv
22 import json
23 import os
24 import tempfile
25 import threading
26 import time
27 from dataclasses import dataclass, field
28 from datetime import datetime, timezone
29 from pathlib import Path
30 from typing import Any
31
32 from sqlalchemy import JSON, Boolean, DateTime, Float, Integer, event, select
33 from sqlalchemy.engine import Engine
34 from sqlalchemy.exc import IntegrityError
35 from sqlalchemy.orm import Session
36
37 from app.config import settings
38 from app.logging import get_logger
39 from app.models import (
40     AgentRun,
41     Comment,
42     Component,
43     Defect,
44     Dependency,
45     ECR,
46     Evidence,
47     Feedback,
48     ImpactReport,
49     Requirement,
50     TestCase,
51     TestExecution,
52 )

```

```

53
54 logger = get_logger("ecr.csv")
55
56 SKIP_COLUMNS = {"id"}
57 SUPPRESS = "csv_suppress_write_back"
58 DIRTY = "csv_dirty_tables"
59 _MISSING = object()
60
61
62 class CsvDataError(ValueError):
63     """A CSV file could not be loaded; the message names the file, line and column."""
64
65
66 @dataclass(frozen=True)
67 class TableSpec:
68     file: str
69     model: type
70     required: bool = True
71     # column -> relative path pattern; the cell holds the path, the file holds the JSON
72     file_columns: dict[str, str] = field(default_factory=dict)
73
74     @property
75     def table(self) -> str:
76         return self.model.__tablename__
77
78
79 # Load order matters: dependencies need components, executions need test cases.
80 TABLES: list[TableSpec] = [
81     TableSpec("components.csv", Component),
82     TableSpec("dependencies.csv", Dependency),
83     TableSpec("requirements.csv", Requirement),
84     TableSpec("defects.csv", Defect),
85     TableSpec("test_cases.csv", TestCase),
86     TableSpec("test_executions.csv", TestExecution),
87     TableSpec("ecrs.csv", ECR),
88     TableSpec("comments.csv", Comment),
89     TableSpec("evidence.csv", Evidence),
90     TableSpec(
91         "impact_reports.csv",
92         ImpactReport,
93         required=False,
94         file_columns={"report_json": "reports/{workflow_id}.json"},
95     ),
96     TableSpec("agent_runs.csv", AgentRun, required=False),
97     TableSpec("feedback.csv", Feedback, required=False),
98 ]
99 TABLE_BY_NAME = {spec.table: spec for spec in TABLES}
100
101
102 def data_dir() -> Path:
103     return Path(settings.data_dir)
104
105
106 # -----
107 # value conversion
108 # -----
109 def _encode(column, value: Any) -> str:
110     if value is None:
111         return ""
112     if isinstance(column.type, JSON):
113         return json.dumps(value, ensure_ascii=False)
114     if isinstance(column.type, Boolean):
115         return "true" if value else "false"
116     if isinstance(column.type, DateTime):
117         return value.isoformat()
118     return str(value)
119
120
121 def _decode(column, raw: str | None) -> Any:
122     if raw is None or raw.strip() == "":
123         return _MISSING
124     kind = column.type
125     if isinstance(kind, JSON):
126         return json.loads(raw)
127     if isinstance(kind, Boolean):
128         return raw.strip().lower() in ("true", "1", "yes", "y")
129     if isinstance(kind, Integer):
130         return int(float(raw))
131     if isinstance(kind, Float):
132         return float(raw)
133     if isinstance(kind, DateTime):
134         parsed = datetime.fromisoformat(raw.strip().replace("Z", "+00:00"))
135         return parsed if parsed.tzinfo else parsed.replace(tzinfo=timezone.utc)
136     return raw

```

```

137
138
139 def headers(spec: TableSpec) -> list[str]:
140     names = [c.name for c in spec.model.__table__.columns if c.name not in SKIP_COLUMNS]
141     if spec.model is Dependency:
142         names = ["source_component", "target_component"] + [
143             n for n in names if n not in ("source_component_id", "target_component_id")
144         ]
145     if spec.model is TestExecution:
146         names = [n for n in names if n != "test_case_pk"]
147     return names
148
149
150 # -----
151 # load
152 # -----
153 def load_all(session: Session, directory: Path | None = None) -> dict[str, int]:
154     """Load every CSV into the (empty) database. Raises CsvDataError on bad data."""
155     directory = Path(directory or data_dir())
156     session.info[SUPPRESS] = True
157     counts: dict[str, int] = {}
158     component_pk: dict[str, int] = {}
159     test_pk: dict[str, int] = {}
160     try:
161         for spec in TABLES:
162             path = directory / spec.file
163             if not path.exists():
164                 if spec.required:
165                     logger.warning("csv.missing_file", file=str(path))
166                     counts[spec.table] = 0
167                     continue
168             columns = {c.name: c for c in spec.model.__table__.columns}
169             objects = []
170             with path.open(newline="", encoding="utf-8-sig") as handle:
171                 reader = csv.DictReader(handle)
172                 for row in reader:
173                     line = reader.line_num
174                     if not any((value or "").strip() for value in row.values() if isinstance(value, str)):
175                         continue # blank line
176                     values: dict[str, Any] = {}
177                     for name, raw in row.items():
178                         if name is None:
179                             continue
180                         name = name.strip()
181                         if spec.model is Dependency and name in ("source_component", "target_component"):
182                             key = (raw or "").strip()
183                             if key not in component_pk:
184                                 raise CsvDataError(
185                                     f"{spec.file} line {line}: unknown component '{key}' in {name}"
186                                 )
187                             values[f"{name}_id"] = component_pk[key]
188                             continue
189                         column = columns.get(name)
190                         if column is None or name in SKIP_COLUMNS:
191                             continue
192                         if name in spec.file_columns:
193                             if raw and (directory / raw).exists():
194                                 values[name] = json.loads((directory / raw).read_text(encoding="utf-8"))
195                                 continue
196                         try:
197                             decoded = _decode(column, raw)
198                         except (ValueError, TypeError) as exc:
199                             raise CsvDataError(
200                                 f"{spec.file} line {line}, column '{name}': {exc}"
201                             ) from exc
202                         if decoded is not _MISSING:
203                             values[name] = decoded
204                     if spec.model is TestExecution:
205                         test_id = values.get("test_case_id")
206                         if test_id not in test_pk:
207                             raise CsvDataError(f"{spec.file} line {line}: unknown test case '{test_id}'")
208                         values["test_case_pk"] = test_pk[test_id]
209                     objects.append(spec.model(**values))
210     session.add_all(objects)
211     try:
212         session.flush()
213     except IntegrityError as exc:
214         raise CsvDataError(f"{spec.file}: {exc.orig}") from exc
215     if spec.model is Component:
216         component_pk = {obj.component_id: obj.id for obj in objects}
217     if spec.model is TestCase:
218         test_pk = {obj.test_case_id: obj.id for obj in objects}
219     counts[spec.table] = len(objects)
220     session.commit()

```

```

221     except Exception:
222         session.rollback()
223         raise
224     finally:
225         session.info.pop(SUPPRESS, None)
226         session.info.pop(DIRTY, None)
227     logger.info("csv.loaded", directory=str(directory), **counts)
228     return counts
229
230
231 # -----
232 # write back
233 # -----
234 def export_table(spec: TableSpec, engine: Engine, directory: Path | None = None) -> int:
235     """Write one table to its CSV file (atomically)."""
236     directory = Path(directory or data_dir())
237     table = spec.model.__table__
238     columns = {c.name: c for c in table.columns}
239     with engine.connect() as conn:
240         rows = conn.execute(select(table).order_by(table.c.id)).mappings().all()
241         component_ids: dict[int, str] = {}
242         if spec.model is Dependency:
243             component_table = Component.__table__
244             component_ids = dict(
245                 conn.execute(select(component_table.c.id, component_table.c.component_id)).all()
246             )
247         names = headers(spec)
248         out: list[dict[str, str]] = []
249         for row in rows:
250             record: dict[str, str] = {}
251             for name in names:
252                 if spec.model is Dependency and name in ("source_component", "target_component"):
253                     record[name] = component_ids.get(row[f"{name}_id"], "")
254                 elif name in spec.file_columns:
255                     relative = spec.file_columns[name].format(**dict(row))
256                     target = directory / relative
257                     if row[name] is not None and not target.exists():
258                         target.parent.mkdir(parents=True, exist_ok=True)
259                         target.write_text(json.dumps(row[name], ensure_ascii=False, default=str), encoding="utf
+ -8")
260                     record[name] = relative if row[name] is not None else ""
261             else:
262                 record[name] = _encode(columns[name], row[name])
263             out.append(record)
264         _atomic_write(directory / spec.file, names, out)
265     return len(out)
266
267
268 def export_all(engine: Engine, directory: Path | None = None) -> dict[str, int]:
269     return {spec.table: export_table(spec, engine, directory) for spec in TABLES}
270
271
272 def _atomic_write(path: Path, names: list[str], rows: list[dict[str, str]]) -> None:
273     path.parent.mkdir(parents=True, exist_ok=True)
274     fd, tmp = tempfile.mkstemp(prefix=f"{path.stem}.", suffix=".tmp", dir=path.parent)
275     try:
276         with os.fdopen(fd, "w", newline="", encoding="utf-8") as handle:
277             writer = csv.DictWriter(handle, fieldnames=names)
278             writer.writeheader()
279             writer.writerows(rows)
280         for attempt in range(5):
281             try:
282                 os.replace(tmp, path)
283                 return
284             except PermissionError: # e.g. the file is open in Excel
285                 if attempt == 4:
286                     raise
287                 time.sleep(0.2 * (attempt + 1))
288     finally:
289         if os.path.exists(tmp):
290             os.remove(tmp)
291
292
293 class CsvWriter:
294     """Coalesces commits and writes changed tables back from a background thread."""
295
296     def __init__(self, delay_seconds: float = 0.4) -> None:
297         self.delay = delay_seconds
298         self.engine: Engine | None = None
299         self._pending: set[str] = set()
300         self._lock = threading.Lock()
301         self._flush_lock = threading.Lock()
302         self._wake = threading.Event()
303         self._thread: threading.Thread | None = None

```

```

304
305 def schedule(self, tables: set[str]) -> None:
306     with self._lock:
307         self._pending |= tables
308         if self._thread is None or not self._thread.is_alive():
309             self._thread = threading.Thread(target=self._loop, name="csv-writer", daemon=True)
310             self._thread.start()
311         self._wake.set()
312
313 def _loop(self) -> None:
314     while True:
315         self._wake.wait()
316         time.sleep(self.delay)
317         self._wake.clear()
318         self.flush()
319
320 def flush(self) -> None:
321     """Write every pending table now (called on shutdown and by tests)."""
322     with self._lock:
323         tables, self._pending = self._pending, set()
324         if not tables or self.engine is None:
325             return
326         with self._flush_lock:
327             for spec in TABLES:
328                 if spec.table not in tables:
329                     continue
330                 try:
331                     export_table(spec, self.engine)
332                 except Exception as exc:
333                     logger.warning("csv.write_failed", file=spec.file, error=str(exc))
334                     with self._lock:
335                         self._pending.add(spec.table)
336
337
338 writer = CsvWriter()
339
340
341 _write_back_installed = False
342
343
344 def install_write_back(session_factory, engine: Engine) -> None:
345     """Track which tables each commit changes and hand them to the writer."""
346     global _write_back_installed
347     writer.engine = engine
348     if _write_back_installed:
349         return
350     _write_back_installed = True
351
352 @event.listens_for(session_factory, "after_flush")
353 def _track(session: Session, _context) -> None:
354     if session.info.get(SUPPRESS):
355         return
356     changed = session.info.setdefault(DIRTY, set())
357     for obj in (*session.new, *session.deleted):
358         if getattr(obj, "__tablename__", None) in TABLE_BY_NAME:
359             changed.add(obj.__tablename__)
360     for obj in session.dirty:
361         if getattr(obj, "__tablename__", None) in TABLE_BY_NAME and session.is_modified(obj):
362             changed.add(obj.__tablename__)
363
364 @event.listens_for(session_factory, "after_commit")
365 def _commit(session: Session) -> None:
366     tables = session.info.pop(DIRTY, set())
367     if tables and not session.info.get(SUPPRESS):
368         writer.schedule(tables)
369
370 @event.listens_for(session_factory, "after_rollback")
371 def _rollback(session: Session) -> None:
372     session.info.pop(DIRTY, None)
373
374
375 # -----
376 # inspection
377 # -----
378 def current_counts(db: Session) -> dict[str, int]:
379     return {
380         "components": db.query(Component).count(),
381         "dependencies": db.query(Dependency).count(),
382         "requirements": db.query(Requirement).count(),
383         "defects": db.query(Defect).count(),
384         "test_cases": db.query(TestCase).count(),
385         "test_executions": db.query(TestExecution).count(),
386         "ecrs": db.query(ECR).count(),
387         "comments": db.query(Comment).count(),

```

```

388         "evidence": db.query(Evidence).count(),
389     }
390
391
392 def files_status(directory: Path | None = None) -> dict[str, Any]:
393     directory = Path(directory or data_dir())
394     return {
395         "data_dir": str(directory),
396         "files": {spec.file: (directory / spec.file).exists() for spec in TABLES},
397     }

```

Backend - models (one per CSV table)

backend/app/models/__init__.py

```

1  """SQLAlchemy ORM models."""
2  from app.models.agent_run import AgentRun
3  from app.models.collaboration import Comment, Evidence
4  from app.models.component import Component
5  from app.models.defect import Defect
6  from app.models.dependency import Dependency
7  from app.models.ecr import ECR
8  from app.models.impact_report import Feedback, ImpactReport
9  from app.models.requirement import Requirement
10 from app.models.test_case import TestCase, TestExecution
11
12 __all__ = [
13     "AgentRun",
14     "Comment",
15     "Component",
16     "Evidence",
17     "Defect",
18     "Dependency",
19     "ECR",
20     "Feedback",
21     "ImpactReport",
22     "Requirement",
23     "TestCase",
24     "TestExecution",
25 ]

```

backend/app/models/agent_run.py

```

1  """Persisted agent/workflow telemetry."""
2  from __future__ import annotations
3
4  from datetime import datetime
5  from typing import Any
6
7  from sqlalchemy import JSON, DateTime, Float, String, Text
8  from sqlalchemy.orm import Mapped, mapped_column
9
10 from app.database import Base
11 from app.models.base import IdMixin, utcnow
12
13
14 class AgentRun(IdMixin, Base):
15     __tablename__ = "agent_runs"
16
17     workflow_id: Mapped[str] = mapped_column(String(64), index=True)
18     ecr_id: Mapped[str] = mapped_column(String(64), index=True, default="")
19     agent_name: Mapped[str] = mapped_column(String(80))
20     agent_key: Mapped[str] = mapped_column(String(64), default="")
21     status: Mapped[str] = mapped_column(String(24), default="COMPLETED")
22     action: Mapped[str] = mapped_column(String(240), default="")
23     input: Mapped[dict[str, Any]] = mapped_column(JSON, default=dict)
24     output: Mapped[dict[str, Any]] = mapped_column(JSON, default=dict)
25     reasoning: Mapped[str] = mapped_column(Text, default="")
26     confidence: Mapped[float] = mapped_column(Float, default=0.0)
27     execution_time: Mapped[float] = mapped_column(Float, default=0.0)
28     error: Mapped[str] = mapped_column(Text, default="")
29     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
30
31     def as_dict(self) -> dict[str, Any]:
32         return {
33             "workflow_id": self.workflow_id,
34             "ecr_id": self.ecr_id,
35             "agent": self.agent_name,
36             "agent_key": self.agent_key,
37             "status": self.status,
38             "action": self.action,
39             "input": dict(self.input or {}),
40             "output": dict(self.output or {}),

```

```

41         "reasoning": self.reasoning,
42         "confidence": self.confidence,
43         "execution_time": self.execution_time,
44         "error": self.error,
45         "timestamp": self.created_at.isoformat() if self.created_at else None,
46     }

```

backend/app/models/base.py

```

1  """Shared column helpers for ORM models."""
2  from __future__ import annotations
3
4  from datetime import datetime, timezone
5
6  from sqlalchemy import DateTime, Integer
7  from sqlalchemy.orm import Mapped, mapped_column
8
9
10 def utcnow() -> datetime:
11     return datetime.now(timezone.utc)
12
13
14 class TimestampMixin:
15     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
16     updated_at: Mapped[datetime] = mapped_column(
17         DateTime(timezone=True), default=utcnow, onupdate=utcnow
18     )
19
20
21 class IdMixin:
22     id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)

```

backend/app/models/collaboration.py

```

1  """Human artefacts attached to a change: review comments and evidence.
2
3  These are the sources engineers currently read by hand across several systems
4  (ADO work item discussions, review threads, test evidence attachments), which
5  is exactly the manual effort the assistant removes.
6  """
7  from __future__ import annotations
8
9  from datetime import datetime
10 from typing import Any
11
12 from sqlalchemy import JSON, DateTime, Float, String, Text
13 from sqlalchemy.orm import Mapped, mapped_column
14
15 from app.database import Base
16 from app.models.base import IdMixin, utcnow
17
18
19 class Comment(IdMixin, Base):
20     """A discussion entry on an ECR, requirement, defect or test case."""
21
22     __tablename__ = "comments"
23
24     comment_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
25     ecr_id: Mapped[str] = mapped_column(String(64), index=True, default="")
26     target_type: Mapped[str] = mapped_column(String(32), default="ECR")
27     target_id: Mapped[str] = mapped_column(String(64), default="", index=True)
28     author: Mapped[str] = mapped_column(String(120), default="")
29     author_role: Mapped[str] = mapped_column(String(80), default="Engineer")
30     body: Mapped[str] = mapped_column(Text, default="")
31     sentiment: Mapped[str] = mapped_column(String(16), default="NEUTRAL")
32     category: Mapped[str] = mapped_column(String(48), default="DISCUSSION")
33     source_system: Mapped[str] = mapped_column(String(48), default="AZURE_DEVOPS")
34     references: Mapped[list[str]] = mapped_column(JSON, default=list)
35     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
36
37     def as_dict(self) -> dict[str, Any]:
38         return {
39             "comment_id": self.comment_id,
40             "ecr_id": self.ecr_id,
41             "target_type": self.target_type,
42             "target_id": self.target_id,
43             "author": self.author,
44             "author_role": self.author_role,
45             "body": self.body,
46             "sentiment": self.sentiment,
47             "category": self.category,
48             "source_system": self.source_system,
49             "references": list(self.references or []),
50             "created_at": self.created_at.isoformat() if self.created_at else None,

```

```

51         }
52
53
54     class Evidence(IdMixin, Base):
55         """An artefact proving something about the change (run, log, review, doc)."""
56
57         __tablename__ = "evidence"
58
59         evidence_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
60         ecr_id: Mapped[str] = mapped_column(String(64), index=True, default="")
61         target_type: Mapped[str] = mapped_column(String(32), default="TEST_CASE")
62         target_id: Mapped[str] = mapped_column(String(64), default="", index=True)
63         title: Mapped[str] = mapped_column(String(240), default="")
64         evidence_type: Mapped[str] = mapped_column(String(48), default="TEST_RUN")
65         summary: Mapped[str] = mapped_column(Text, default="")
66         outcome: Mapped[str] = mapped_column(String(32), default="PASS")
67         uri: Mapped[str] = mapped_column(String(300), default="")
68         source_system: Mapped[str] = mapped_column(String(48), default="AZURE_DEVOPS")
69         confidence: Mapped[float] = mapped_column(Float, default=1.0)
70         captured_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
71
72         def as_dict(self) -> dict[str, Any]:
73             return {
74                 "evidence_id": self.evidence_id,
75                 "ecr_id": self.ecr_id,
76                 "target_type": self.target_type,
77                 "target_id": self.target_id,
78                 "title": self.title,
79                 "evidence_type": self.evidence_type,
80                 "summary": self.summary,
81                 "outcome": self.outcome,
82                 "uri": self.uri,
83                 "source_system": self.source_system,
84                 "confidence": self.confidence,
85                 "captured_at": self.captured_at.isoformat() if self.captured_at else None,
86             }

```

backend/app/models/component.py

```

1  """Components (services, modules, databases) of the sample enterprise landscape."""
2  from __future__ import annotations
3
4  from sqlalchemy import Float, String, Text
5  from sqlalchemy.orm import Mapped, mapped_column
6
7  from app.database import Base
8  from app.models.base import IdMixin, TimestampMixin
9
10
11     class Component(IdMixin, TimestampMixin, Base):
12         __tablename__ = "components"
13
14         component_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
15         name: Mapped[str] = mapped_column(String(160))
16         type: Mapped[str] = mapped_column(String(48))
17         business_criticality: Mapped[str] = mapped_column(String(16), default="MEDIUM")
18         criticality_score: Mapped[float] = mapped_column(Float, default=0.5)
19         owner: Mapped[str] = mapped_column(String(120), default="Unassigned")
20         repository: Mapped[str] = mapped_column(String(200), default="")
21         description: Mapped[str] = mapped_column(Text, default="")
22         business_domain: Mapped[str] = mapped_column(String(80), default="General")
23         domain: Mapped[str] = mapped_column(String(48), default="commerce")
24
25         def as_dict(self) -> dict:
26             return {
27                 "component_id": self.component_id,
28                 "name": self.name,
29                 "type": self.type,
30                 "business_criticality": self.business_criticality,
31                 "criticality_score": self.criticality_score,
32                 "owner": self.owner,
33                 "repository": self.repository,
34                 "description": self.description,
35                 "business_domain": self.business_domain,
36                 "domain": self.domain,
37             }

```

backend/app/models/defect.py

```

1  """Historical defects and production incidents."""
2  from __future__ import annotations
3
4  from datetime import datetime
5  from typing import Any

```

```

6
7 from sqlalchemy import JSON, DateTime, Float, String, Text
8 from sqlalchemy.orm import Mapped, mapped_column
9
10 from app.database import Base
11 from app.models.base import IdMixin, TimestampMixin, utcnow
12
13
14 class Defect(IdMixin, TimestampMixin, Base):
15     __tablename__ = "defects"
16
17     defect_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
18     title: Mapped[str] = mapped_column(String(240))
19     description: Mapped[str] = mapped_column(Text, default="")
20     severity: Mapped[str] = mapped_column(String(16), default="MEDIUM")
21     status: Mapped[str] = mapped_column(String(32), default="CLOSED")
22     root_cause: Mapped[str] = mapped_column(Text, default="")
23     root_cause_category: Mapped[str] = mapped_column(String(80), default="")
24     affected_component: Mapped[str] = mapped_column(String(64), default="")
25     related_components: Mapped[list[str]] = mapped_column(JSON, default=list)
26     linked_requirements: Mapped[list[str]] = mapped_column(JSON, default=list)
27     escaped_to_production: Mapped[bool] = mapped_column(default=False)
28     reopen_count: Mapped[int] = mapped_column(default=0)
29     detected_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
30     severity_weight: Mapped[float] = mapped_column(Float, default=0.5)
31
32     def as_dict(self) -> dict[str, Any]:
33         return {
34             "defect_id": self.defect_id,
35             "title": self.title,
36             "description": self.description,
37             "severity": self.severity,
38             "status": self.status,
39             "root_cause": self.root_cause,
40             "root_cause_category": self.root_cause_category,
41             "affected_component": self.affected_component,
42             "related_components": list(self.related_components or []),
43             "linked_requirements": list(self.linked_requirements or []),
44             "escaped_to_production": self.escaped_to_production,
45             "reopen_count": self.reopen_count,
46             "detected_at": self.detected_at.isoformat() if self.detected_at else None,
47         }

```

backend/app/models/dependency.py

```

1 """Directed dependency edges between components."""
2 from __future__ import annotations
3
4 from sqlalchemy import Float, ForeignKey, String, UniqueConstraint
5 from sqlalchemy.orm import Mapped, mapped_column, relationship
6
7 from app.database import Base
8 from app.models.base import IdMixin, TimestampMixin
9 from app.models.component import Component
10
11
12 class Dependency(IdMixin, TimestampMixin, Base):
13     __tablename__ = "dependencies"
14     __table_args__ = (
15         UniqueConstraint("source_component_id", "target_component_id", name="uq_dependency_edge"),
16     )
17
18     source_component_id: Mapped[int] = mapped_column(ForeignKey("components.id", ondelete="CASCADE"))
19     target_component_id: Mapped[int] = mapped_column(ForeignKey("components.id", ondelete="CASCADE"))
20     dependency_type: Mapped[str] = mapped_column(String(32), default="SYNC_API")
21     criticality: Mapped[str] = mapped_column(String(16), default="MEDIUM")
22     weight: Mapped[float] = mapped_column(Float, default=0.5)
23     description: Mapped[str] = mapped_column(String(255), default="")
24
25     source: Mapped[Component] = relationship(foreign_keys=[source_component_id], lazy="joined")
26     target: Mapped[Component] = relationship(foreign_keys=[target_component_id], lazy="joined")
27
28     def as_dict(self) -> dict:
29         return {
30             "source": self.source.component_id if self.source else None,
31             "source_name": self.source.name if self.source else None,
32             "target": self.target.component_id if self.target else None,
33             "target_name": self.target.name if self.target else None,
34             "dependency_type": self.dependency_type,
35             "criticality": self.criticality,
36             "weight": self.weight,
37             "description": self.description,
38         }

```

backend/app/models/ecr.py

```
1  """Engineering Change Requests."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  from sqlalchemy import JSON, String, Text
7  from sqlalchemy.orm import Mapped, mapped_column
8
9  from app.database import Base
10 from app.models.base import IdMixin, TimestampMixin
11
12
13 class ECR(IdMixin, TimestampMixin, Base):
14     __tablename__ = "ecrs"
15
16     ecr_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
17     title: Mapped[str] = mapped_column(String(240))
18     description: Mapped[str] = mapped_column(Text, default="")
19     # Analysis state set by the app (NEW, ANALYSING, ANALYSED).
20     status: Mapped[str] = mapped_column(String(32), default="NEW")
21     change_type: Mapped[str] = mapped_column(String(48), default="UNCLASSIFIED")
22     business_domain: Mapped[str] = mapped_column(String(80), default="General")
23     requested_by: Mapped[str] = mapped_column(String(120), default="Change Board")
24     priority: Mapped[str] = mapped_column(String(16), default="MEDIUM")
25     target_release: Mapped[str] = mapped_column(String(48), default="")
26     linked_requirements: Mapped[list[str]] = mapped_column(JSON, default=list)
27     changed_files: Mapped[list[str]] = mapped_column(JSON, default=list)
28     change_metadata: Mapped[dict[str, Any]] = mapped_column(JSON, default=dict)
29
30     # Columns of the change-tracking tool export (Type, Status, Severity ...).
31     # They are kept exactly as exported and never changed by an analysis.
32     record_type: Mapped[str] = mapped_column(String(32), default="ECR")
33     lifecycle_status: Mapped[str] = mapped_column(String(48), default="")
34     severity: Mapped[str] = mapped_column(String(32), default="")
35     assigned_testers: Mapped[list[str]] = mapped_column(JSON, default=list)
36     actual_modified_objects: Mapped[list[str]] = mapped_column(JSON, default=list)
37     planned_modified_objects: Mapped[list[str]] = mapped_column(JSON, default=list)
38     build_resolved_in: Mapped[str] = mapped_column(String(64), default="")
39     test_estimate_archived: Mapped[str] = mapped_column(String(32), default="")
40     estimate: Mapped[str] = mapped_column(String(32), default="")
41     test_estimate: Mapped[str] = mapped_column(String(32), default="")
42     creation_date: Mapped[str] = mapped_column(String(40), default="")
43     modified_date: Mapped[str] = mapped_column(String(40), default="")
44     sit_assigned_testers: Mapped[list[str]] = mapped_column(JSON, default=list)
45     test_actual_time: Mapped[str] = mapped_column(String(32), default="")
46     affected_test_cases: Mapped[list[str]] = mapped_column(JSON, default=list)
47     steps_to_reproduce: Mapped[str] = mapped_column(Text, default="")
48     observed_behavior: Mapped[str] = mapped_column(Text, default="")
49     expected_behavior: Mapped[str] = mapped_column(Text, default="")
50     attachments: Mapped[list[str]] = mapped_column(JSON, default=list)
51
52     def as_dict(self) -> dict[str, Any]:
53         return {
54             "ecr_id": self.ecr_id,
55             "title": self.title,
56             "description": self.description,
57             "status": self.status,
58             "change_type": self.change_type,
59             "business_domain": self.business_domain,
60             "requested_by": self.requested_by,
61             "priority": self.priority,
62             "target_release": self.target_release,
63             "linked_requirements": list(self.linked_requirements or []),
64             "changed_files": list(self.changed_files or []),
65             "change_metadata": dict(self.change_metadata or {}),
66             "record_type": self.record_type or "ECR",
67             "lifecycle_status": self.lifecycle_status or "",
68             "severity": self.severity or "",
69             "assigned_testers": list(self.assigned_testers or []),
70             "actual_modified_objects": list(self.actual_modified_objects or []),
71             "planned_modified_objects": list(self.planned_modified_objects or []),
72             "build_resolved_in": self.build_resolved_in or "",
73             "test_estimate_archived": self.test_estimate_archived or "",
74             "estimate": self.estimate or "",
75             "test_estimate": self.test_estimate or "",
76             "creation_date": self.creation_date or "",
77             "modified_date": self.modified_date or "",
78             "sit_assigned_testers": list(self.sit_assigned_testers or []),
79             "test_actual_time": self.test_actual_time or "",
80             "affected_test_cases": list(self.affected_test_cases or []),
81             "steps_to_reproduce": self.steps_to_reproduce or "",
82             "observed_behavior": self.observed_behavior or "",
```

```

83         "expected_behavior": self.expected_behavior or "",
84         "attachments": list(self.attachments or []),
85         "created_at": self.created_at.isoformat() if self.created_at else None,
86         "updated_at": self.updated_at.isoformat() if self.updated_at else None,
87     }

```

backend/app/models/impact_report.py

```

1  """Persisted impact analysis reports and user feedback."""
2  from __future__ import annotations
3
4  from datetime import datetime
5  from typing import Any
6
7  from sqlalchemy import JSON, DateTime, Float, Integer, String, Text
8  from sqlalchemy.orm import Mapped, mapped_column
9
10 from app.database import Base
11 from app.models.base import IdMixin, utcnow
12
13 class ImpactReport(IdMixin, Base):
14     __tablename__ = "impact_reports"
15
16     workflow_id: Mapped[str] = mapped_column(String(64), index=True)
17     ecr_id: Mapped[str] = mapped_column(String(64), index=True)
18     confidence: Mapped[float] = mapped_column(Float, default=0.0)
19     total_tests: Mapped[int] = mapped_column(Integer, default=0)
20     selected_tests: Mapped[int] = mapped_column(Integer, default=0)
21     reduction_percentage: Mapped[float] = mapped_column(Float, default=0.0)
22     execution_time: Mapped[float] = mapped_column(Float, default=0.0)
23     approval_status: Mapped[str] = mapped_column(String(24), default="NOT_REQUIRED")
24     report_json: Mapped[dict[str, Any]] = mapped_column(JSON, default=dict)
25     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
26
27     def summary(self) -> dict[str, Any]:
28         return {
29             "workflow_id": self.workflow_id,
30             "ecr_id": self.ecr_id,
31             "confidence": self.confidence,
32             "total_tests": self.total_tests,
33             "selected_tests": self.selected_tests,
34             "reduction_percentage": self.reduction_percentage,
35             "execution_time": self.execution_time,
36             "approval_status": self.approval_status,
37             "created_at": self.created_at.isoformat() if self.created_at else None,
38         }
39
40
41 class Feedback(IdMixin, Base):
42     __tablename__ = "feedback"
43
44     ecr_id: Mapped[str] = mapped_column(String(64), index=True)
45     workflow_id: Mapped[str] = mapped_column(String(64), default="")
46     target_type: Mapped[str] = mapped_column(String(32), default="RECOMMENDATION")
47     target_id: Mapped[str] = mapped_column(String(64), default="")
48     useful: Mapped[bool] = mapped_column(default=True)
49     comment: Mapped[str] = mapped_column(Text, default="")
50     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
51
52     def as_dict(self) -> dict[str, Any]:
53         return {
54             "ecr_id": self.ecr_id,
55             "workflow_id": self.workflow_id,
56             "target_type": self.target_type,
57             "target_id": self.target_id,
58             "useful": self.useful,
59             "comment": self.comment,
60             "created_at": self.created_at.isoformat() if self.created_at else None,
61         }
62

```

backend/app/models/requirement.py

```

1  """Business/system requirements traced to components and features."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  from sqlalchemy import JSON, String, Text
7  from sqlalchemy.orm import Mapped, mapped_column
8
9  from app.database import Base
10 from app.models.base import IdMixin, TimestampMixin
11

```

```

12
13 class Requirement(IdMixin, TimestampMixin, Base):
14     __tablename__ = "requirements"
15
16     requirement_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
17     title: Mapped[str] = mapped_column(String(240))
18     description: Mapped[str] = mapped_column(Text, default="")
19     business_domain: Mapped[str] = mapped_column(String(80), default="General")
20     priority: Mapped[str] = mapped_column(String(16), default="MEDIUM")
21     status: Mapped[str] = mapped_column(String(32), default="APPROVED")
22     component: Mapped[str] = mapped_column(String(64), default="")
23     linked_components: Mapped[list[str]] = mapped_column(JSON, default=list)
24     features: Mapped[list[str]] = mapped_column(JSON, default=list)
25     source_system: Mapped[str] = mapped_column(String(48), default="LOCAL")
26     domain: Mapped[str] = mapped_column(String(48), default="commerce")
27
28     def as_dict(self) -> dict[str, Any]:
29         return {
30             "requirement_id": self.requirement_id,
31             "title": self.title,
32             "description": self.description,
33             "business_domain": self.business_domain,
34             "priority": self.priority,
35             "status": self.status,
36             "component": self.component,
37             "linked_components": list(self.linked_components or []),
38             "features": list(self.features or []),
39             "source_system": self.source_system,
40             "domain": self.domain,
41         }

```

backend/app/models/test_case.py

```

1  """Test cases and their execution history."""
2  from __future__ import annotations
3
4  from datetime import datetime
5  from typing import Any
6
7  from sqlalchemy import JSON, DateTime, Float, ForeignKey, String, Text
8  from sqlalchemy.orm import Mapped, mapped_column
9
10 from app.database import Base
11 from app.models.base import IdMixin, TimestampMixin, utcnow
12
13 class TestCase(IdMixin, TimestampMixin, Base):
14     __tablename__ = "test_cases"
15
16     test_case_id: Mapped[str] = mapped_column(String(64), unique=True, index=True)
17     title: Mapped[str] = mapped_column(String(240))
18     description: Mapped[str] = mapped_column(Text, default="")
19     feature: Mapped[str] = mapped_column(String(120), default="")
20     suite: Mapped[str] = mapped_column(String(120), default="Regression")
21     component: Mapped[str] = mapped_column(String(64), default="")
22     linked_components: Mapped[list[str]] = mapped_column(JSON, default=list)
23     requirement_id: Mapped[str] = mapped_column(String(64), default="")
24     test_type: Mapped[str] = mapped_column(String(32), default="FUNCTIONAL")
25     test_level: Mapped[str] = mapped_column(String(32), default="SYSTEM")
26     automation_status: Mapped[str] = mapped_column(String(32), default="AUTOMATED")
27     # Fields carried over from enterprise test-management exports
28     # (folder / optimization technique / positive-negative / design technique).
29     folder: Mapped[str] = mapped_column(String(120), default="")
30     optimization_technique: Mapped[str] = mapped_column(String(64), default="Default")
31     test_polarity: Mapped[str] = mapped_column(String(16), default="Positive")
32     test_technique: Mapped[str] = mapped_column(String(80), default="")
33     retired: Mapped[bool] = mapped_column(default=False)
34     scorable: Mapped[bool] = mapped_column(default=True)
35     domain: Mapped[str] = mapped_column(String(48), default="commerce")
36     average_execution_time: Mapped[float] = mapped_column(Float, default=60.0)
37     historical_failure_rate: Mapped[float] = mapped_column(Float, default=0.05)
38     last_executed_at: Mapped[datetime | None] = mapped_column(
39         DateTime(timezone=True), nullable=True
40     )
41
42     tags: Mapped[list[str]] = mapped_column(JSON, default=list)
43     source_system: Mapped[str] = mapped_column(String(48), default="LOCAL")
44
45     def as_dict(self) -> dict[str, Any]:
46         return {
47             "test_case_id": self.test_case_id,
48             "title": self.title,
49             "description": self.description,
50             "feature": self.feature,
51             "suite": self.suite,

```

```

52         "component": self.component,
53         "linked_components": list(self.linked_components or []),
54         "requirement_id": self.requirement_id,
55         "test_type": self.test_type,
56         "test_level": self.test_level,
57         "automation_status": self.automation_status,
58         "folder": self.folder,
59         "optimization_technique": self.optimization_technique,
60         "test_polarity": self.test_polarity,
61         "test_technique": self.test_technique,
62         "retired": self.retired,
63         "scorable": self.scorable,
64         "domain": self.domain,
65         "average_execution_time": self.average_execution_time,
66         "historical_failure_rate": self.historical_failure_rate,
67         "tags": list(self.tags or []),
68         "source_system": self.source_system,
69     }
70
71
72 class TestExecution(IdMixin, Base):
73     __tablename__ = "test_executions"
74
75     test_case_pk: Mapped[int] = mapped_column(ForeignKey("test_cases.id", ondelete="CASCADE"))
76     test_case_id: Mapped[str] = mapped_column(String(64), index=True)
77     result: Mapped[str] = mapped_column(String(16), default="PASS")
78     duration_seconds: Mapped[float] = mapped_column(Float, default=60.0)
79     build_id: Mapped[str] = mapped_column(String(64), default="")
80     defect_id: Mapped[str] = mapped_column(String(64), default="")
81     executed_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utcnow)
82
83     def as_dict(self) -> dict[str, Any]:
84         return {
85             "test_case_id": self.test_case_id,
86             "result": self.result,
87             "duration_seconds": self.duration_seconds,
88             "build_id": self.build_id,
89             "defect_id": self.defect_id,
90             "executed_at": self.executed_at.isoformat() if self.executed_at else None,
91         }

```

Backend - agent state and orchestration

backend/app/agents/state.py

```

1  """Shared LangGraph workflow state and the agent catalogue.
2
3  The platform has exactly three agents - Retrieval, Correlation, Summarization.
4  Everything else in the workflow is a *step* owned by one of them. Agents only
5  exchange validated Pydantic payloads (serialised to dicts so the state stays
6  checkpointable). Keys that several steps write carry reducers.
7  """
8  from __future__ import annotations
9
10 import operator
11 from typing import Annotated, Any, TypedDict
12
13 from app.schemas.agent import AgentDescriptor
14
15
16 def merge_dicts(left: dict[str, Any], right: dict[str, Any]) -> dict[str, Any]:
17     merged = dict(left or {})
18     merged.update(right or {})
19     return merged
20
21
22 def merge_unique(left: list[str], right: list[str]) -> list[str]:
23     seen = list(left or [])
24     for item in right or []:
25         if item not in seen:
26             seen.append(item)
27     return seen
28
29
30 class ECRWorkflowState(TypedDict, total=False):
31     # -- run context -----
32     workflow_id: str
33     ecr_id: str
34     ecr_input: dict[str, Any]
35     options: dict[str, Any]
36
37     # -- step outputs -----
38     ecr_analysis: dict[str, Any]

```

```

39     retrieval_plan: dict[str, Any]
40     plan: list[str]
41     plan_detail: dict[str, Any]
42     skipped_steps: Annotated[list[str], merge_unique]
43     requirements: list[dict[str, Any]]
44     requirement_analysis: dict[str, Any]
45     defects: list[dict[str, Any]]
46     defect_analysis: dict[str, Any]
47     defect_insights: dict[str, Any]
48     defect_summary: str
49     comments: list[dict[str, Any]]
50     evidence: list[dict[str, Any]]
51     collaboration_analysis: dict[str, Any]
52     correlation: dict[str, Any]
53     correlation_review: dict[str, Any]
54     code_impact: dict[str, Any]
55     dependency_analysis: dict[str, Any]
56     impact_analysis: dict[str, Any]
57     discovered_tests: list[dict[str, Any]]
58     test_discovery: dict[str, Any]
59     selected_tests: list[dict[str, Any]]
60     test_selection: dict[str, Any]
61     prioritized_tests: list[dict[str, Any]]
62     test_prioritization: dict[str, Any]
63     approval: dict[str, Any]
64     question: str
65     final_answer: str
66     answer_context: dict[str, Any]
67     answer_citations: list[str]
68     final_report: dict[str, Any]
69
70     # -- telemetry (written by every step -> reducers) -----
71     execution_trace: Annotated[list[dict[str, Any]], operator.add]
72     errors: Annotated[list[dict[str, Any]], operator.add]
73     step_confidence: Annotated[dict[str, float], merge_dicts]
74     tools_used: Annotated[dict[str, Any], merge_dicts]
75
76
77 # The only agents in the system.
78 AGENT_CATALOG: list[AgentDescriptor] = [
79     AgentDescriptor(
80         key="retrieval_agent",
81         name="Retrieval Agent",
82         purpose=(
83             "Given an ECR number, use the LLM to plan what to search for, then gather everything "
84             "that exists about it: the change record, traced and semantically matched "
85             "requirements, historical defects, code impact, dependency reach, review comments "
86             "and evidence artefacts."
87         ),
88         tools=[
89             "classify_change",
90             "extract_entities",
91             "llm.plan_retrieval",
92             "build_plan",
93             "trace_requirements",
94             "semantic_requirement_search",
95             "keyword_requirement_search",
96             "analyze_changed_files",
97             "walk_import_graph",
98             "semantic_defect_search",
99             "defects_by_component",
100             "build_graph",
101             "downstream_walk",
102             "upstream_walk",
103             "comments_for_ecr",
104             "evidence_for_ecr",
105         ],
106     ),
107     AgentDescriptor(
108         key="correlation_agent",
109         name="Correlation Agent",
110         purpose=(
111             "Connect every retrieved artefact to the ECR with typed, explained links, find the "
112             "impacted components, decide which regression tests actually matter, check "
113             "whether each related defect could happen again, and use the LLM to find the "
114             "contradictions and gaps across the correlated sources."
115         ),
116         tools=[
117             "component_impact",
118             "tests_by_requirement",
119             "tests_by_component",
120             "semantic_test_search",
121             "selection_engine.select",
122             "selection_engine.prioritise",

```

```

123         "build_correlation_graph",
124         "analyze_defects",
125         "llm.review_correlation",
126     ],
127     depends_on=["retrieval_agent"],
128 ),
129 AgentDescriptor(
130     key="summarization_agent",
131     name="Summarization Agent",
132     purpose=(
133         "Answer the user's question from the correlated bundle with citations, explain the "
134         "related defects in plain language, and publish the full ECR intelligence report."
135     ),
136     tools=["build_answer_context", "llm.summarize", "report_service.build"],
137     depends_on=["correlation_agent"],
138 ),
139 ]
140
141 # Steps each agent runs, in order. Keys match the step keys on the event stream.
142 AGENT_STEPS: dict[str, list[dict[str, str]]] = {
143     "retrieval_agent": [
144         {"key": "understand_ecr", "name": "Understand the ECR"},
145         {"key": "gather_evidence", "name": "Gather evidence"},
146     ],
147     "correlation_agent": [
148         {"key": "assess_impact", "name": "Assess impact"},
149         {"key": "select_tests", "name": "Select regression tests"},
150     ],
151     "summarization_agent": [
152         {"key": "summarize", "name": "Summarize and report"},
153     ],
154 }
155
156 AGENT_BY_KEY = {descriptor.key: descriptor for descriptor in AGENT_CATALOG}
157 STEP_NAMES = {step["key"]: step["name"] for steps in AGENT_STEPS.values() for step in steps}
158
159
160 def initial_state(
161     workflow_id: str, ecr_input: dict[str, Any], options: dict[str, Any] | None = None
162 ) -> ECRWorkflowState:
163     return ECRWorkflowState(
164         workflow_id=workflow_id,
165         ecr_id=ecr_input.get("ecr_id", ""),
166         ecr_input=ecr_input,
167         options=options or {},
168         skipped_steps=[],
169         execution_trace=[],
170         errors=[],
171         step_confidence={},
172         tools_used={},
173     )

```

backend/app/agents/orchestrator.py

```

1  """LangGraph orchestrator.
2
3  START -> retrieval_agent -> correlation_agent
4  -> (approval gate) -> [human_approval] -> summarization_agent -> END
5
6  The gate is a real conditional edge: the approval checkpoint is only entered
7  when the user ticks "Require approval" for the run. The checkpoint blocks on the run registry, so the API c
8  an
9  resume it with an approve/reject decision.
10 """
11 from __future__ import annotations
12
13 import time
14 from functools import lru_cache
15 from typing import Any
16
17 from langgraph.graph import END, START, StateGraph
18
19 from app.agents.nodes.agents import correlation_agent, retrieval_agent, summarization_agent
20 from app.agents.runtime import emit_event, registry
21 from app.agents.state import ECRWorkflowState, initial_state
22 from app.config import settings
23 from app.database import session_scope
24 from app.logging import get_logger
25 from app.models import ImpactReport
26 from app.schemas.common import AgentStatus, WorkflowStatus
27
28 logger = get_logger("ecr.orchestrator")
29
30 APPROVAL_POLL_SECONDS = 0.25

```

```

30
31
32 def approval_gate(state: dict[str, Any]) -> dict[str, Any]:
33     """Human-in-the-loop checkpoint.
34
35     Blocks the graph until a decision arrives (or the workflow times out), then
36     records the decision. A rejection removes the recommendation from the report
37     without discarding the analysis.
38     """
39     workflow_id = state.get("workflow_id", "")
40     run = registry.get(workflow_id)
41     impact = state.get("impact_analysis") or {}
42     selection = state.get("test_selection") or {}
43
44     emit_event(
45         workflow_id,
46         agent="Workflow",
47         agent_key="human_approval",
48         status=WorkflowStatus.AWAITING_APPROVAL,
49         action="Waiting for a human decision",
50         detail=(
51             f"{len(impact.get('directly_impacted_components') or [])} directly impacted component(s) "
52             f"and {len(selection.get('selected_tests') or [])} recommended tests."
53         ),
54         ecr_id=state.get("ecr_id", ""),
55         payload={
56             "direct_components": len(impact.get("directly_impacted_components") or []),
57             "selected_tests": len(selection.get("selected_tests") or []),
58         },
59     )
60     if run is None: # pragma: no cover - defensive
61         return {"approval": {"required": True, "status": "AUTO_APPROVED"}}
62
63     run.request_approval()
64     granted = run.wait_for_approval(settings.workflow_timeout_seconds)
65     decision = run.approval_decision or {}
66     status = "APPROVED" if decision.get("approved") else ("REJECTED" if granted else "TIMED_OUT")
67     run.status = WorkflowStatus.RUNNING if status != "REJECTED" else WorkflowStatus.RUNNING
68
69     excluded = set(decision.get("excluded_test_ids") or [])
70     updates: dict[str, Any] = {
71         "approval": {
72             "required": True,
73             "status": status,
74             "decided_by": decision.get("decided_by", ""),
75             "comment": decision.get("comment", ""),
76             "excluded_test_ids": sorted(excluded),
77         }
78     }
79     if excluded:
80         kept = [t for t in (state.get("selected_tests") or []) if t["test_case_id"] not in excluded]
81         updates["selected_tests"] = kept
82         updates["prioritized_tests"] = [
83             t for t in (state.get("prioritized_tests") or []) if t["test_case_id"] not in excluded
84         ]
85         updates["test_selection"] = {
86             **selection,
87             "selected_tests": kept,
88             "excluded_count": selection.get("excluded_count", 0) + len(excluded),
89         }
90
91     emit_event(
92         workflow_id,
93         agent="Workflow",
94         agent_key="human_approval",
95         status=AgentStatus.COMPLETED,
96         action="Human decision recorded",
97         detail=f"{status} by {decision.get('decided_by', 'timeout')}",
98         ecr_id=state.get("ecr_id", ""),
99         payload=updates["approval"],
100     )
101     return updates
102
103
104 def needs_approval(state: dict[str, Any]) -> str:
105     """Conditional edge: enter the checkpoint only when the user asked for approval."""
106     options = state.get("options") or {}
107     return "human_approval" if options.get("human_in_the_loop") else "summarization_agent"
108
109
110 def build_graph() -> StateGraph:
111     graph = StateGraph(ECRWorkflowState)
112     graph.add_node("retrieval_agent", retrieval_agent)
113     graph.add_node("correlation_agent", correlation_agent)

```

```

114     graph.add_node("human_approval", approval_gate)
115     graph.add_node("summarization_agent", summarization_agent)
116
117     graph.add_edge(START, "retrieval_agent")
118     graph.add_edge("retrieval_agent", "correlation_agent")
119     graph.add_conditional_edges(
120         "correlation_agent",
121         needs_approval,
122         {"human_approval": "human_approval", "summarization_agent": "summarization_agent"},
123     )
124     graph.add_edge("human_approval", "summarization_agent")
125     graph.add_edge("summarization_agent", END)
126     return graph
127
128
129 @lru_cache
130 def get_compiled_graph():
131     compiled = build_graph().compile()
132     logger.info("orchestrator.compiled", nodes=4)
133     return compiled
134
135
136 def graph_topology() -> dict[str, Any]:
137     """Static topology for the workflow visualisation."""
138     from app.agents.state import AGENT_CATALOG, AGENT_STEPS
139
140     return {
141         "nodes": [
142             {
143                 "key": descriptor.key,
144                 "name": descriptor.name,
145                 "purpose": descriptor.purpose,
146                 "optional": descriptor.optional,
147                 "tools": descriptor.tools,
148                 "steps": AGENT_STEPS.get(descriptor.key, []),
149             }
150             for descriptor in AGENT_CATALOG
151         ],
152         "checkpoints": [
153             {
154                 "key": "human_approval",
155                 "name": "Human approval checkpoint",
156                 "purpose": "Pauses the analysis until a person approves or rejects it. "
157                 "It is a gate in the graph, not an agent - it makes no decision itself.",
158             }
159         ],
160         "edges": [
161             {"source": "START", "target": "retrieval_agent"},
162             {"source": "retrieval_agent", "target": "correlation_agent"},
163             {"source": "correlation_agent", "target": "human_approval", "condition": "approval requested"},
164             {"source": "correlation_agent", "target": "summarization_agent", "condition": "otherwise"},
165             {"source": "human_approval", "target": "summarization_agent"},
166             {"source": "summarization_agent", "target": "END"},
167         ],
168     }
169
170
171 def run_workflow(
172     ecr_input: dict[str, Any],
173     options: dict[str, Any] | None = None,
174     *,
175     existing_run=None,
176 ):
177     """Execute the graph synchronously and persist the result.
178
179     ``existing_run`` lets the caller register the run (and hand its id to a
180     client that wants to subscribe to events) before execution starts.
181     """
182     options = options or {}
183     run = existing_run or registry.create(ecr_input.get("ecr_id", ""), options)
184     started = time.perf_counter()
185     run.status = WorkflowStatus.RUNNING
186     emit_event(
187         run.workflow_id,
188         agent="Workflow",
189         agent_key="workflow",
190         status=WorkflowStatus.RUNNING,
191         action="Workflow started",
192         ecr_id=run.ecr_id,
193         detail=f"Analysing {run.ecr_id}",
194     )
195     state = initial_state(run.workflow_id, ecr_input, options)
196     try:
197         final_state = get_compiled_graph().invoke(

```

```

198         state, config={"recursion_limit": 25, "configurable": {"thread_id": run.workflow_id}}
199     )
200 except Exception as exc:
201     run.status = WorkflowStatus.FAILED
202     run.finished_at = _now()
203     run.errors.append({"agent": "workflow", "error": str(exc), "critical": True})
204     emit_event(
205         run.workflow_id,
206         agent="Workflow",
207         agent_key="workflow",
208         status=WorkflowStatus.FAILED,
209         action="Workflow failed",
210         detail=str(exc),
211         ecr_id=run.ecr_id,
212     )
213     logger.exception("workflow.failed", workflow_id=run.workflow_id)
214     raise
215
216 duration = round(time.perf_counter() - started, 2)
217 run.state = dict(final_state)
218 run.report = final_state.get("final_report")
219 run.plan = [key for key in (final_state.get("plan") or []) if key != "planning"]
220 run.skipped = list(final_state.get("skipped_steps") or [])
221 run.errors = list(final_state.get("errors") or [])
222 approval = final_state.get("approval") or {}
223 run.approval_required = bool(approval.get("required"))
224 run.approval_status = approval.get("status", run.approval_status)
225 run.confidence = float(
226     ((final_state.get("final_report") or {}).get("confidence") or {}).get("overall", 0.0)
227 )
228 run.status = (
229     WorkflowStatus.REJECTED if approval.get("status") == "REJECTED" else WorkflowStatus.COMPLETED
230 )
231 run.finished_at = _now()
232
233 _persist_report(run.workflow_id, final_state, duration)
234 emit_event(
235     run.workflow_id,
236     agent="Workflow",
237     agent_key="workflow",
238     status=WorkflowStatus.COMPLETED,
239     action="Workflow completed",
240     duration=duration,
241     confidence=run.confidence,
242     ecr_id=run.ecr_id,
243     detail=(final_state.get("final_answer") or "")[:400],
244 )
245 logger.info(
246     "workflow.completed",
247     workflow_id=run.workflow_id,
248     ecr_id=run.ecr_id,
249     duration=duration,
250 )
251 return run, final_state
252
253
254 def _persist_report(workflow_id: str, state: dict[str, Any], duration: float) -> None:
255     report = state.get("final_report") or {}
256     selection = state.get("test_selection") or {}
257     try:
258         with session_scope() as db:
259             db.add(
260                 ImpactReport(
261                     workflow_id=workflow_id,
262                     ecr_id=state.get("ecr_id", ""),
263                     confidence=float((report.get("confidence") or {}).get("overall", 0.0)),
264                     total_tests=int(selection.get("total_available", 0) or 0),
265                     selected_tests=len(selection.get("selected_tests") or []),
266                     reduction_percentage=float(selection.get("reduction_percentage", 0.0) or 0.0),
267                     execution_time=duration,
268                     approval_status=(state.get("approval") or {}).get("status", "NOT_REQUIRED"),
269                     report_json=report,
270                 )
271             )
272     except Exception as exc: # pragma: no cover
273         logger.warning("report.persist_failed", workflow_id=workflow_id, error=str(exc))
274
275
276 def _now():
277     from datetime import datetime, timezone
278
279     return datetime.now(timezone.utc)

```

backend/app/agents/runtime.py

```
1  """Workflow runtime: live run registry, event stream and the agent-node decorator.
2
3  Nodes execute inside LangGraph's worker threads, so the registry is guarded by a
4  lock and the SSE endpoint simply tails an append-only event list. That keeps the
5  streaming path free of cross-thread asyncio hand-offs.
6  """
7  from __future__ import annotations
8
9  import functools
10 import threading
11 import time
12 import uuid
13 from datetime import datetime, timezone
14 from typing import Any, Callable
15
16 from app.config import settings
17 from app.database import session_scope
18 from app.logging import get_logger
19 from app.models import AgentRun
20 from app.schemas.agent import AgentEvent
21 from app.schemas.common import AgentStatus, WorkflowStatus
22
23 logger = get_logger("ecr.runtime")
24
25
26 class WorkflowRun:
27     """Live state of one analysis run."""
28
29     def __init__(self, workflow_id: str, ecr_id: str, options: dict[str, Any]) -> None:
30         self.workflow_id = workflow_id
31         self.ecr_id = ecr_id
32         self.options = options
33         self.status: WorkflowStatus = WorkflowStatus.QUEUED
34         self.started_at = datetime.now(timezone.utc)
35         self.finished_at: datetime | None = None
36         self.events: list[AgentEvent] = []
37         self.plan: list[str] = []
38         self.skipped: list[str] = []
39         self.current_agent: str | None = None
40         self.errors: list[dict[str, Any]] = []
41         self.state: dict[str, Any] = {}
42         self.report: dict[str, Any] | None = None
43         self.confidence: float = 0.0
44         self.approval_required = False
45         self.approval_status = "NOT_REQUIRED"
46         self.approval_decision: dict[str, Any] | None = None
47         self._lock = threading.Lock()
48         self._approval_event = threading.Event()
49
50     # -- events -----
51     def emit(self, event: AgentEvent) -> None:
52         with self._lock:
53             self.events.append(event)
54
55     def events_after(self, index: int) -> tuple[int, list[AgentEvent]]:
56         with self._lock:
57             return len(self.events), self.events[index:]
58
59     @property
60     def duration(self) -> float:
61         end = self.finished_at or datetime.now(timezone.utc)
62         return round((end - self.started_at).total_seconds(), 2)
63
64     @property
65     def progress(self) -> float:
66         if not self.plan:
67             return 0.05
68         done = {
69             event.agent_key
70             for event in self.events
71             if event.status in (AgentStatus.COMPLETED, AgentStatus.FAILED, AgentStatus.SKIPPED)
72         }
73         return round(min(1.0, len(done & set(self.plan)) / max(len(self.plan), 1)), 3)
74
75     # -- human in the loop -----
76     def request_approval(self) -> None:
77         self.approval_required = True
78         self.approval_status = "PENDING"
79         self.status = WorkflowStatus.AWAITING_APPROVAL
80         self._approval_event.clear()
81
82     def wait_for_approval(self, timeout: float) -> bool:
```

```

83         return self._approval_event.wait(timeout)
84
85     def resolve_approval(self, decision: dict[str, Any]) -> None:
86         self.approval_decision = decision
87         self.approval_status = "APPROVED" if decision.get("approved") else "REJECTED"
88         self._approval_event.set()
89
90     def summary(self) -> dict[str, Any]:
91         return {
92             "workflow_id": self.workflow_id,
93             "ecr_id": self.ecr_id,
94             "status": self.status,
95             "started_at": self.started_at,
96             "finished_at": self.finished_at,
97             "duration": self.duration,
98             "plan": self.plan,
99             "skipped_steps": self.skipped,
100             "current_agent": self.current_agent,
101             "progress": self.progress,
102             "confidence": self.confidence,
103             "errors": self.errors,
104             "approval_required": self.approval_required,
105             "approval_status": self.approval_status,
106             "events": self.events,
107         }
108
109
110 class WorkflowRegistry:
111     """Process-wide registry of runs (bounded, newest wins)."""
112
113     MAX_RUNS = 100
114
115     def __init__(self) -> None:
116         self._runs: dict[str, WorkflowRun] = {}
117         self._by_ecr: dict[str, str] = {}
118         self._lock = threading.Lock()
119
120     def create(self, ecr_id: str, options: dict[str, Any]) -> WorkflowRun:
121         workflow_id = f"WF-{uuid.uuid4().hex[:12]}"
122         run = WorkflowRun(workflow_id, ecr_id, options)
123         with self._lock:
124             self._runs[workflow_id] = run
125             self._by_ecr[ecr_id] = workflow_id
126             if len(self._runs) > self.MAX_RUNS:
127                 oldest = sorted(self._runs.values(), key=lambda r: r.started_at)[0]
128                 self._runs.pop(oldest.workflow_id, None)
129         return run
130
131     def get(self, workflow_id: str) -> WorkflowRun | None:
132         return self._runs.get(workflow_id)
133
134     def latest_for_ecr(self, ecr_id: str) -> WorkflowRun | None:
135         workflow_id = self._by_ecr.get(ecr_id)
136         return self._runs.get(workflow_id) if workflow_id else None
137
138     def all(self) -> list[WorkflowRun]:
139         return sorted(self._runs.values(), key=lambda r: r.started_at, reverse=True)
140
141
142 registry = WorkflowRegistry()
143
144
145 def emit_event(
146     workflow_id: str,
147     *,
148     agent: str,
149     agent_key: str,
150     status: Any,
151     action: str = "",
152     detail: str = "",
153     duration: float | None = None,
154     confidence: float | None = None,
155     payload: dict[str, Any] | None = None,
156     ecr_id: str = "",
157 ) -> AgentEvent:
158     event = AgentEvent(
159         timestamp=datetime.now(timezone.utc),
160         workflow_id=workflow_id,
161         ecr_id=ecr_id,
162         agent=agent,
163         agent_key=agent_key,
164         status=status,
165         action=action,
166         detail=detail,

```

```

167         duration=duration,
168         confidence=confidence,
169         payload=payload or {},
170     )
171     run = registry.get(workflow_id)
172     if run:
173         run.emit(event)
174     return event
175
176
177 def persist_agent_run(
178     *,
179     workflow_id: str,
180     ecr_id: str,
181     agent_name: str,
182     agent_key: str,
183     status: str,
184     action: str,
185     output: dict[str, Any],
186     reasoning: str,
187     confidence: float,
188     execution_time: float,
189     error: str = "",
190 ) -> None:
191     try:
192         with session_scope() as db:
193             db.add(
194                 AgentRun(
195                     workflow_id=workflow_id,
196                     ecr_id=ecr_id,
197                     agent_name=agent_name,
198                     agent_key=agent_key,
199                     status=status,
200                     action=action,
201                     input={},
202                     output=output,
203                     reasoning=reasoning,
204                     confidence=confidence,
205                     execution_time=execution_time,
206                     error=error,
207                 )
208             )
209     except Exception as exc: # pragma: no cover - telemetry must never break a run
210         logger.warning("runtime.persist_failed", agent=agent_name, error=str(exc))
211
212
213 def agent_node(
214     key: str,
215     name: str,
216     action: str,
217     *,
218     critical: bool = False,
219     output_keys: tuple[str, ...] = (),
220 ) -> Callable:
221     """Decorate a node function with tracing, timing and graceful degradation.
222
223     A failing non-critical agent records the error, emits a ``failed`` event and
224     returns an empty partial state so the graph keeps going with reduced
225     confidence. A failing critical agent re-raises.
226     """
227
228     def decorator(func: Callable[..., dict[str, Any]]) -> Callable[..., dict[str, Any]]:
229         @functools.wraps(func)
230         def wrapper(state: dict[str, Any], *args: Any, **kwargs: Any) -> dict[str, Any]:
231             workflow_id = state.get("workflow_id", "")
232             ecr_id = state.get("ecr_id", "")
233             run = registry.get(workflow_id)
234             if run:
235                 run.current_agent = name
236                 if run.status == WorkflowStatus.QUEUED:
237                     run.status = WorkflowStatus.RUNNING
238             emit_event(
239                 workflow_id,
240                 agent=name,
241                 agent_key=key,
242                 status=AgentStatus.RUNNING,
243                 action=action,
244                 ecr_id=ecr_id,
245             )
246             started = time.perf_counter()
247             if settings.agent_step_delay_ms:
248                 time.sleep(settings.agent_step_delay_ms / 1000.0)
249             try:
250                 result = func(state, *args, **kwargs) or {}

```

```

251 except Exception as exc:
252     duration = round(time.perf_counter() - started, 3)
253     logger.exception("agent.failed", agent=name, workflow_id=workflow_id)
254     error = {"agent": name, "agent_key": key, "error": str(exc), "critical": critical}
255     if run:
256         run.errors.append(error)
257     emit_event(
258         workflow_id,
259         agent=name,
260         agent_key=key,
261         status=AgentStatus.FAILED,
262         action=action,
263         detail=str(exc),
264         duration=duration,
265         ecr_id=ecr_id,
266     )
267     persist_agent_run(
268         workflow_id=workflow_id,
269         ecr_id=ecr_id,
270         agent_name=name,
271         agent_key=key,
272         status="FAILED",
273         action=action,
274         output={},
275         reasoning="",
276         confidence=0.0,
277         execution_time=duration,
278         error=str(exc),
279     )
280     if critical:
281         raise
282     return {
283         "errors": [error],
284         "execution_trace": [
285             _trace(name, key, action, "failed", duration, detail=str(exc))
286         ],
287         "skipped_steps": [key],
288     }
289
290 duration = round(time.perf_counter() - started, 3)
291 confidence = float(result.get("_confidence", 0.0) or 0.0)
292 reasoning = str(result.get("_reasoning", ""))
293 summary = result.get("_summary", {})
294 result.pop("_confidence", None)
295 result.pop("_reasoning", None)
296 result.pop("_summary", None)
297
298 emit_event(
299     workflow_id,
300     agent=name,
301     agent_key=key,
302     status=AgentStatus.COMPLETED,
303     action=action,
304     detail=reasoning[:400],
305     duration=duration,
306     confidence=confidence,
307     payload=summary,
308     ecr_id=ecr_id,
309 )
310 persist_agent_run(
311     workflow_id=workflow_id,
312     ecr_id=ecr_id,
313     agent_name=name,
314     agent_key=key,
315     status="COMPLETED",
316     action=action,
317     output=summary,
318     reasoning=reasoning,
319     confidence=confidence,
320     execution_time=duration,
321 )
322 trace = _trace(name, key, action, "completed", duration, detail=reasoning[:400])
323 trace["confidence"] = confidence
324 trace["summary"] = summary
325 merged: dict[str, Any] = dict(result)
326 merged["execution_trace"] = [*(merged.get("execution_trace") or []), trace]
327 if confidence:
328     merged["step_confidence"] = {**merged.get("step_confidence", {}), key: confidence}
329 return merged
330
331 wrapper.agent_key = key # type: ignore[attr-defined]
332 wrapper.agent_name = name # type: ignore[attr-defined]
333 return wrapper
334

```

```

335     return decorator
336
337
338 def agent_step(key: str, name: str, action: str, *, critical: bool = False) -> Callable:
339     """Decorate an internal step of an agent.
340
341     Steps are the units of work inside the three top-level agents. They emit
342     their own start/finish events (so the UI can show progress *inside* an
343     agent) and degrade gracefully: a failing non-critical step records the
344     error and returns an empty partial state instead of aborting the agent.
345     """
346
347     def decorator(func: Callable[..., dict[str, Any]]) -> Callable[..., dict[str, Any]]:
348         @functools.wraps(func)
349         def wrapper(state: dict[str, Any], *args: Any, **kwargs: Any) -> dict[str, Any]:
350             workflow_id = state.get("workflow_id", "")
351             ecr_id = state.get("ecr_id", "")
352             emit_event(
353                 workflow_id,
354                 agent=name,
355                 agent_key=key,
356                 status=AgentStatus.RUNNING,
357                 action=action,
358                 ecr_id=ecr_id,
359                 payload={"step": True},
360             )
361             started = time.perf_counter()
362             if settings.agent_step_delay_ms:
363                 time.sleep(settings.agent_step_delay_ms / 1000.0)
364             try:
365                 result = func(state, *args, **kwargs) or {}
366             except Exception as exc:
367                 duration = round(time.perf_counter() - started, 3)
368                 logger.exception("step.failed", step=name, workflow_id=workflow_id)
369                 error = {"agent": name, "agent_key": key, "error": str(exc), "critical": critical}
370                 run = registry.get(workflow_id)
371                 if run:
372                     run.errors.append(error)
373                 emit_event(
374                     workflow_id,
375                     agent=name,
376                     agent_key=key,
377                     status=AgentStatus.FAILED,
378                     action=action,
379                     detail=str(exc),
380                     duration=duration,
381                     ecr_id=ecr_id,
382                     payload={"step": True},
383                 )
384                 persist_agent_run(
385                     workflow_id=workflow_id,
386                     ecr_id=ecr_id,
387                     agent_name=name,
388                     agent_key=key,
389                     status="FAILED",
390                     action=action,
391                     output={},
392                     reasoning="",
393                     confidence=0.0,
394                     execution_time=duration,
395                     error=str(exc),
396                 )
397                 if critical:
398                     raise
399                 return {
400                     "errors": [error],
401                     "execution_trace": [
402                         _trace(name, key, action, "failed", duration, detail=str(exc))
403                     ],
404                     "skipped_steps": [key],
405                 }
406
407             duration = round(time.perf_counter() - started, 3)
408             confidence = float(result.pop("_confidence", 0.0) or 0.0)
409             reasoning = str(result.pop("_reasoning", ""))
410             summary = result.pop("_summary", {})
411             emit_event(
412                 workflow_id,
413                 agent=name,
414                 agent_key=key,
415                 status=AgentStatus.COMPLETED,
416                 action=action,
417                 detail=reasoning[:400],
418                 duration=duration,

```

```

419         confidence=confidence,
420         payload={**summary, "step": True},
421         ecr_id=ecr_id,
422     )
423     persist_agent_run(
424         workflow_id=workflow_id,
425         ecr_id=ecr_id,
426         agent_name=name,
427         agent_key=key,
428         status="COMPLETED",
429         action=action,
430         output=summary,
431         reasoning=reasoning,
432         confidence=confidence,
433         execution_time=duration,
434     )
435     trace = _trace(name, key, action, "completed", duration, detail=reasoning[:400])
436     trace["confidence"] = confidence
437     trace["summary"] = summary
438     merged = dict(result)
439     merged["execution_trace"] = [(merged.get("execution_trace") or []), trace]
440     if confidence:
441         merged["step_confidence"] = {**merged.get("step_confidence", {}), key: confidence}
442     return merged
443
444     wrapper.step_key = key # type: ignore[attr-defined]
445     wrapper.step_name = name # type: ignore[attr-defined]
446     return wrapper
447
448     return decorator
449
450
451 def build_view(state: dict[str, Any], accumulated: dict[str, Any]) -> dict[str, Any]:
452     """State as a step should see it: incoming state plus what was produced so far.
453
454     Applies the same reducers LangGraph applies between nodes. A plain
455     ``{**state, **accumulated}`` would let partial trace and confidence *replace*
456     what earlier agents contributed, so a step reading the whole run (the report)
457     would only ever see its own agent.
458     """
459     view = dict(state)
460     for key, value in accumulated.items():
461         if key.startswith("_"):
462             continue
463         if key in ("execution_trace", "errors"):
464             view[key] = [(state.get(key) or []), *(value or [])]
465         elif key == "skipped_steps":
466             merged = list(state.get(key) or [])
467             merged += [item for item in (value or []) if item not in merged]
468             view[key] = merged
469         elif key in ("step_confidence", "tools_used"):
470             view[key] = {**state.get(key) or {}, **value or {}}
471         else:
472             view[key] = value
473     return view
474
475
476 def merge_partial(target: dict[str, Any], partial: dict[str, Any]) -> dict[str, Any]:
477     """Merge a step result into an accumulating agent result.
478
479     Mirrors the LangGraph reducers: additive keys concatenate, dict keys merge,
480     everything else is last-write-wins.
481     """
482     for key, value in (partial or {}).items():
483         if key in ("execution_trace", "errors", "skipped_steps"):
484             existing = list(target.get(key) or [])
485             for item in value or []:
486                 if key == "skipped_steps" and item in existing:
487                     continue
488                 existing.append(item)
489             target[key] = existing
490         elif key in ("step_confidence", "tools_used"):
491             target[key] = {**target.get(key) or {}, **value or {}}
492         else:
493             target[key] = value
494     return target
495
496
497 def _trace(
498     name: str, key: str, action: str, status: str, duration: float, detail: str = ""
499 ) -> dict[str, Any]:
500     return {
501         "timestamp": datetime.now(timezone.utc).isoformat(),
502         "agent": name,

```

```

503         "agent_key": key,
504         "action": action,
505         "status": status,
506         "duration": duration,
507         "detail": detail,
508     }

```

backend/app/agents/nodes/agents.py

```

1  """The three agents. There are no others.
2
3  Retrieval Agent    -> LLM plans the search, gather the evidence
4  Correlation Agent  -> assess the impact, select the regression tests,
5                      LLM reviews the correlated sources for contradictions and gaps
6  Summarization Agent -> LLM answers the user, publish the report
7
8  Each agent runs one or two of the five steps in :mod:`app.agents.nodes.steps`,
9  and each one reasons with the LLM (with a rule-based fallback when none is set).
10 A step is not an agent: it has no graph node of its own, it cannot route the
11 workflow, and it is labelled on the event stream with the agent that owns it.
12 """
13 from __future__ import annotations
14
15 from typing import Any
16
17 from app.agents.nodes.steps import (
18     assess_impact,
19     gather_evidence,
20     select_tests,
21     summarize,
22     understand_ecr,
23 )
24 from app.agents.runtime import agent_node, build_view, merge_partial
25
26
27 def _run(state: dict[str, Any], accumulated: dict[str, Any], step) -> dict[str, Any]:
28     """Run one step against state + everything this agent produced so far."""
29     return merge_partial(accumulated, step(build_view(state, accumulated)))
30
31
32 def _average(confidences: dict[str, float]) -> float:
33     return round(sum(confidences.values()) / len(confidences), 3) if confidences else 0.6
34
35
36 @agent_node(
37     "retrieval_agent",
38     "Retrieval Agent",
39     "Understanding the ECR and gathering its evidence",
40     critical=True,
41 )
42 def retrieval_agent(state: dict[str, Any]) -> dict[str, Any]:
43     result: dict[str, Any] = {}
44     result = _run(state, result, understand_ecr)
45     result = _run(state, result, gather_evidence)
46
47     sources = {
48         "requirements": len(result.get("requirements") or []),
49         "defects": len(result.get("defects") or []),
50         "comments": len(result.get("comments") or []),
51         "evidence": len(result.get("evidence") or []),
52     }
53     plan = result.get("retrieval_plan") or {}
54     result["_confidence"] = _average(result.get("step_confidence") or {})
55     result["_reasoning"] = (
56         "Retrieved "
57         + ", ".join(f"{value} {name}" for name, value in sources.items() if value)
58         + f" for {state.get('ecr_id')}"
59         + (
60             f", searching with {len(plan.get('search_terms') or [])} LLM-planned term(s)."
61             if plan.get("source") == "llm"
62             else " with rule-based keywords (LLM unavailable)."
63         )
64     )
65     result["_summary"] = {"sources": sources, "reasoned_by": plan.get("source", "rules")}
66     return result
67
68
69 @agent_node(
70     "correlation_agent",
71     "Correlation Agent",
72     "Assessing impact and selecting regression tests",
73     critical=True,
74 )
75 def correlation_agent(state: dict[str, Any]) -> dict[str, Any]:

```

```

76     result: dict[str, Any] = {}
77     result = _run(state, result, assess_impact)
78     result = _run(state, result, select_tests)
79
80     impact = result.get("impact_analysis") or {}
81     selection = result.get("test_selection") or {}
82     review = result.get("correlation_review") or {}
83     result["_confidence"] = _average(result.get("step_confidence") or {})
84     result["_reasoning"] = (
85         f"{len(impact.get('directly_impacted_components') or [])} component(s) change directly; selected "
86         f"{len(selection.get('selected_tests') or [])} of {selection.get('total_available')} tests. "
87         f"'LLM' if review.get('source') == 'llm' else 'Rule-based' review found "
88         f"{len(review.get('conflicts') or [])} contradiction(s) and {len(review.get('gaps') or [])} gap(s).
+     "
89     )
90     result["_summary"] = {
91         "direct_components": len(impact.get("directly_impacted_components") or []),
92         "selected_tests": len(selection.get("selected_tests") or []),
93         "contradictions": len(review.get("conflicts") or []),
94         "gaps": len(review.get("gaps") or []),
95         "reasoned_by": review.get("source", "rules"),
96     }
97     return result
98
99
100 @agent_node(
101     "summarization_agent",
102     "Summarization Agent",
103     "Answering the question and publishing the report",
104     critical=True,
105 )
106 def summarization_agent(state: dict[str, Any]) -> dict[str, Any]:
107     result: dict[str, Any] = {}
108     result = _run(state, result, summarize)
109
110     report = result.get("final_report") or {}
111     result["_confidence"] = float((report.get("confidence") or {}).get("overall", 0.7))
112     result["_reasoning"] = result.get("final_answer", "")[:400]
113     result["_summary"] = {"citations": result.get("answer_citations", [])[:12]}
114     return result
115
116
117 AGENTS = (retrieval_agent, correlation_agent, summarization_agent)

```

backend/app/agents/nodes/steps.py

```

1  """The five steps. The three agents run them; nothing else is traced.
2
3      Retrieval Agent      1. understand_ecr      classify the change, LLM plans the search,
4                          2. gather_evidence     plan the investigation
5                          3. assess_impact        requirements, code impact, related defects,
6                          4. select_tests         [dependencies], review comments and evidence
7      Correlation Agent    impacted components and recommended actions
8                          discover, select and order regression tests,
9                          link every artefact to the ECR, analyse defects,
10                         LLM reviews the correlated bundle
11      Summarization Agent  5. summarize          plain-language answer, defect summary, report
12
13  Every agent reasons with the LLM once it has something to reason about: the
14  Retrieval Agent decides what to search for, the Correlation Agent finds the
15  contradictions and gaps across sources, the Summarization Agent writes the
16  answer and the defect summary. That is exactly one LLM request per agent - three
17  per analysis. Each LLM part has a rule-based fallback, so no key still means a result.
18
19  Each step calls smaller *parts* (the functions in the sibling modules). A part
20  that fails is recorded and skipped, so one missing source lowers confidence
21  instead of stopping the analysis.
22  """
23  from __future__ import annotations
24
25  from typing import Any, Callable
26
27  from app.agents.nodes.code_impact import code_impact_step
28  from app.agents.nodes.collaboration import collaboration_step
29  from app.agents.nodes.correlation import correlation_step
30  from app.agents.nodes.correlation_review import correlation_review_step
31  from app.agents.nodes.defect_insights import defect_insights_step
32  from app.agents.nodes.dependencies import dependency_step
33  from app.agents.nodes.ecr_understanding import ecr_understanding_step
34  from app.agents.nodes.historical_defects import defect_step
35  from app.agents.nodes.impact_assessment import impact_assessment_step
36  from app.agents.nodes.planning import planning_step
37  from app.agents.nodes.regression_tests import (
38      test_discovery_step,

```

```

39     test_prioritization_step,
40     test_selection_step,
41 )
42 from app.agents.nodes.requirements import requirement_step
43 from app.agents.nodes.retrieval_planning import retrieval_planning_step
44 from app.agents.nodes.summarization import report_step, summarization_step
45 from app.agents.runtime import agent_step, build_view, merge_partial, registry
46 from app.logging import get_logger
47
48 logger = get_logger("ecr.steps")
49
50 Part = Callable[[dict[str, Any]], dict[str, Any]]
51
52
53 def _part(
54     state: dict[str, Any],
55     acc: dict[str, Any],
56     parts: list[dict[str, Any]],
57     key: str,
58     fn: Part,
59     *,
60     owner: str,
61     required: bool = False,
62 ) -> None:
63     """Run one part against state + what this step produced so far."""
64     try:
65         out = dict(fn(build_view(state, acc)) or {})
66     except Exception as exc:
67         if required:
68             raise
69         logger.warning("step.part_failed", part=key, error=str(exc))
70         error = {"agent": owner, "agent_key": key, "error": str(exc), "critical": False}
71         run = registry.get(state.get("workflow_id", ""))
72         if run:
73             run.errors.append(error)
74         merge_partial(acc, {"errors": [error], "skipped_steps": [key]})
75         parts.append({"confidence": None, "reasoning": f"{key.replace('_', ' ').capitalize()} unavailable."
76 + })
77     return
78     parts.append({"confidence": out.pop("_confidence", None), "reasoning": str(out.pop("_reasoning", ""))})
79     out.pop("_summary", None)
80     merge_partial(acc, out)
81
82 def _finish(acc: dict[str, Any], parts: list[dict[str, Any]], summary: dict[str, Any]) -> dict[str, Any]:
83     confidences = [part["confidence"] for part in parts if part["confidence"]]
84     acc["_confidence"] = round(sum(confidences) / len(confidences), 3) if confidences else 0.5
85     acc["_reasoning"] = " ".join(part["reasoning"] for part in parts if part["reasoning"])
86     acc["_summary"] = summary
87     return acc
88
89
90 RETRIEVAL = "Retrieval Agent"
91 CORRELATION = "Correlation Agent"
92 SUMMARIZATION = "Summarization Agent"
93
94
95 @agent_step("understand_ecr", RETRIEVAL, "Understanding the ECR and planning the investigation", critical=True)
96 +
97 def understand_ecr(state: dict[str, Any]) -> dict[str, Any]:
98     acc: dict[str, Any] = {}
99     parts: list[dict[str, Any]] = []
100     _part(state, acc, parts, "ecr_understanding", ecr_understanding_step, owner=RETRIEVAL, required=True)
101     # The LLM plans the search between the rule classification and the plan it may extend.
102     _part(state, acc, parts, "retrieval_planning", retrieval_planning_step, owner=RETRIEVAL)
103     _part(state, acc, parts, "planning", planning_step, owner=RETRIEVAL, required=True)
104     detail = acc.get("plan_detail") or {}
105     retrieval = acc.get("retrieval_plan") or {}
106     return _finish(
107         acc,
108         parts,
109         {
110             "change_type": (acc.get("ecr_analysis") or {}).get("change_type"),
111             "investigations": detail.get("investigations", []),
112             "skipped": detail.get("skipped", []),
113             "search_terms": retrieval.get("search_terms", []),
114             "reasoned_by": retrieval.get("source", "rules"),
115         },
116     )
117
118 @agent_step("gather_evidence", RETRIEVAL, "Gathering requirements, code impact, history, comments and evidence")
119 +
120 def gather_evidence(state: dict[str, Any]) -> dict[str, Any]:

```

```

120     planned = set((state.get("plan_detail") or {}).get("investigations") or [])
121     acc: dict[str, Any] = {}
122     parts: list[dict[str, Any]] = []
123     _part(state, acc, parts, "requirements", requirement_step, owner=RETRIEVAL)
124     _part(state, acc, parts, "code_impact", code_impact_step, owner=RETRIEVAL)
125     _part(state, acc, parts, "historical_defects", defect_step, owner=RETRIEVAL)
126     if "dependencies" in planned:
127         _part(state, acc, parts, "dependencies", dependency_step, owner=RETRIEVAL)
128     _part(state, acc, parts, "collaboration_retrieval", collaboration_step, owner=RETRIEVAL)
129     return _finish(
130         acc,
131         parts,
132         {
133             "requirements": len(acc.get("requirements") or []),
134             "defects": len(acc.get("defects") or []),
135             "comments": len(acc.get("comments") or []),
136             "evidence": len(acc.get("evidence") or []),
137         },
138     )
139
140
141 @agent_step("assess_impact", CORRELATION, "Identifying impacted components")
142 def assess_impact(state: dict[str, Any]) -> dict[str, Any]:
143     acc: dict[str, Any] = {}
144     parts: list[dict[str, Any]] = []
145     _part(state, acc, parts, "impact_assessment", impact_assessment_step, owner=CORRELATION, required=True)
146     impact = acc.get("impact_analysis") or {}
147     return _finish(
148         acc,
149         parts,
150         {
151             "direct": len(impact.get("directly_impacted_components") or []),
152             "indirect": len(impact.get("indirectly_impacted_components") or []),
153         },
154     )
155
156
157 @agent_step("select_tests", CORRELATION, "Selecting regression tests, linking evidence and analysing related defects")
158 def select_tests(state: dict[str, Any]) -> dict[str, Any]:
159     acc: dict[str, Any] = {}
160     parts: list[dict[str, Any]] = []
161     _part(state, acc, parts, "test_discovery", test_discovery_step, owner=CORRELATION)
162     _part(state, acc, parts, "test_selection", test_selection_step, owner=CORRELATION)
163     _part(state, acc, parts, "test_prioritization", test_prioritization_step, owner=CORRELATION)
164     _part(state, acc, parts, "correlation", correlation_step, owner=CORRELATION)
165     # Runs last: it checks each related defect against the tests selected above.
166     _part(state, acc, parts, "defect_insights", defect_insights_step, owner=CORRELATION)
167     # Last: the LLM reviews the whole correlated bundle, defect insights included.
168     _part(state, acc, parts, "correlation_review", correlation_review_step, owner=CORRELATION)
169     selection = acc.get("test_selection") or {}
170     insights = acc.get("defect_insights") or {}
171     review = acc.get("correlation_review") or {}
172     return _finish(
173         acc,
174         parts,
175         {
176             "selected_tests": len(selection.get("selected_tests") or []),
177             "total_available": selection.get("total_available"),
178             "reduction_percentage": selection.get("reduction_percentage"),
179             "defects_analysed": insights.get("total", 0),
180             "defects_likely_to_return": insights.get("high", 0),
181             "contradictions": len(review.get("conflicts") or []),
182             "gaps": len(review.get("gaps") or []),
183             "reasoned_by": review.get("source", "rules"),
184         },
185     )
186
187
188 @agent_step("summarize", SUMMARIZATION, "Writing the answer, the defect summary and the report", critical=True)
189 def summarize(state: dict[str, Any]) -> dict[str, Any]:
190     acc: dict[str, Any] = {}
191     parts: list[dict[str, Any]] = []
192     # One LLM request writes both the answer and the defect summary.
193     _part(state, acc, parts, "summarization", summarization_step, owner=SUMMARIZATION)
194     _part(state, acc, parts, "report", report_step, owner=SUMMARIZATION, required=True)
195     return _finish(acc, parts, {"citations": (acc.get("answer_citations") or [])[:12]})

```

Backend - agent steps and parts

[backend/app/agents/nodes/__init__.py](#)

backend/app/agents/nodes/code_impact.py

```
1  """Retrieval Agent step: code impact analysis.
2
3  Direct impact = the files declared on the change. Indirect impact = modules that
4  import them, walked over the AST-derived import graph. When an ECR declares no
5  files (a description-only request), the agent falls back to keyword discovery
6  over the parsed symbol table.
7  """
8  from __future__ import annotations
9
10 from typing import Any
11
12 from app.agents.tools.code_tools import analyze_changed_files, repository_stats
13 from app.llm.provider import get_llm
14 from app.schemas.impact import CodeImpactAnalysis, CodeSymbol
15
16
17 def code_impact_step(state: dict[str, Any]) -> dict[str, Any]:
18     ecr = state["ecr_input"]
19     analysis = state.get("ecr_analysis") or {}
20     changed_files = list(ecr.get("changed_files") or [])
21     keywords = analysis.get("technical_keywords") or []
22
23     facts = analyze_changed_files(changed_files, keywords=keywords)
24     stats = repository_stats()
25
26     symbols = [
27         CodeSymbol(
28             file=item["file"],
29             symbol=item["symbol"],
30             kind=item["kind"],
31             component=item["component"],
32             calls=item["calls"][:6],
33         )
34         for item in facts["symbols"][:40]
35     ]
36
37     declared_only = not facts["resolved_modules"] and bool(changed_files)
38     confidence = round(
39         min(
40             0.96,
41             0.35
42             + 0.35 * bool(facts["resolved_modules"])
43             + 0.15 * bool(facts["dependent_modules"])
44             + 0.1 * bool(changed_files),
45         ),
46         3,
47     )
48     if declared_only:
49         confidence = min(confidence, 0.55)
50
51     baseline = (
52         f"Parsed {stats['modules']} modules ({stats['import_edges']} import edges) and resolved "
53         f"{len(facts['resolved_modules'])} changed module(s). "
54         f"Direct impact: {' '.join(facts['direct_components']) or 'none resolved'}. "
55         f"{len(facts['dependent_modules'])} module(s) import the changed code, giving indirect "
56         f"impact on {' '.join(facts['indirect_components']) or 'no further components'}."
57     )
58     if declared_only:
59         baseline += (
60             "The declared files are not present in the analysed repository, so components were "
61             "resolved by path convention only."
62         )
63
64     reasoning = get_llm().narrate(
65         task="Explain the code level impact of this change",
66         context={
67             "changed_files": changed_files,
68             "resolved_modules": facts["resolved_modules"],
69             "dependent_modules": list(facts["dependent_modules"]),
70             "direct_components": facts["direct_components"],
71             "indirect_components": facts["indirect_components"],
72             "apis": facts["apis"],
73         },
74         fallback=baseline,
75     )
76
77     result = CodeImpactAnalysis(
78         directly_impacted_files=facts["direct_files"],
79         impacted_symbols=symbols,
80         impacted_services=facts["direct_components"],
81         potentially_impacted_services=facts["indirect_components"],
```

```

82         impacted_apis=facts["apis"],
83         database_entities=facts["database_entities"],
84         change_surface=facts["change_surface"],
85         language_analyzers=facts["analyzers"],
86         confidence=confidence,
87         reasoning=reasoning,
88     )
89     return {
90         "code_impact": {
91             **result.model_dump(mode="json"),
92             "dependent_files": facts["dependent_files"],
93             "unresolved_files": facts["unresolved_files"],
94             "repository": facts["repository"],
95             "modules_indexed": facts["modules_indexed"],
96         },
97         "tools_used": {
98             "code_impact": ["analyze_changed_files", "walk_import_graph", "repository_stats"]
99         },
100         "_confidence": confidence,
101         "_reasoning": reasoning,
102         "_summary": {
103             "direct_files": facts["direct_files"][:8],
104             "direct_components": facts["direct_components"],
105             "indirect_components": facts["indirect_components"],
106             "apis": facts["apis"][:6],
107         },
108     }

```

backend/app/agents/nodes/collaboration.py

```

1  """Retrieval step: review comments and evidence artefacts.
2
3  This is the step that replaces "scroll through every comment thread and
4  evidence folder to work out what was decided".
5  """
6  from __future__ import annotations
7
8  from collections import Counter
9  from typing import Any
10
11  from app.agents.tools.collaboration_tools import (
12      comments_for_ecr,
13      evidence_for_ecr,
14      evidence_gaps,
15      extract_comment_signals,
16  )
17  from app.agents.nodes.scope import is_scoped
18  from app.database import session_scope
19  from app.llm.provider import get_llm
20
21
22  def collaboration_step(state: dict[str, Any]) -> dict[str, Any]:
23      ecr_id = state.get("ecr_id", "")
24      requirements = state.get("requirements") or []
25      defects = state.get("defects") or []
26      related_ids = [
27          *(r.get("requirement_id") for r in requirements),
28          *(d.get("defect_id") for d in defects),
29          ecr_id,
30      ]
31      if is_scoped(state.get("ecr_input")):
32          related_ids = [ecr_id] # a record with its own scope: only what is attached to it
33
34      with session_scope() as db:
35          comments = comments_for_ecr(db, ecr_id, related_ids=related_ids)
36          evidence = evidence_for_ecr(db, ecr_id, related_ids=related_ids)
37
38          signals = extract_comment_signals(comments)
39          gaps = evidence_gaps(evidence, [r.get("requirement_id", "") for r in requirements])
40          outcomes = Counter(item.get("outcome", "") for item in evidence)
41          concerns = [c for c in comments if c.get("sentiment") == "CONCERN"]
42
43          confidence = round(
44              min(0.95, 0.45 + 0.25 * bool(comments) + 0.2 * bool(evidence) + 0.05 * bool(signals["decisions"])),
45              3,
46          )
47          baseline = (
48              f"Collected {len(comments)} comment(s) and {len(evidence)} evidence artefact(s) for {ecr_id}. "
49              f"{len(signals['decisions'])} explicit decision(s), {len(concerns)} raised concern(s). "
50              + (
51                  f"Evidence outcomes: {'', '.join(f'{k} x{v}' for k, v in outcomes.items() if k)}. "
52                  if outcomes
53                  else "No evidence outcomes recorded. "
54              )
55          )

```

```

55         + (
56             f"{len(gaps)} impacted requirement(s) have no evidence attached."
57             if gaps
58             else "Every impacted requirement has at least one evidence artefact."
59         )
60     )
61     reasoning = get_llm().narrate(
62         task="Summarise what the team has already decided and flagged on this change",
63         context={
64             "decisions": signals["decisions"][:4],
65             "risks": signals["risks"][:4],
66             "scope": signals["scope"][:3],
67             "evidence": [
68                 {"id": e["evidence_id"], "title": e["title"], "outcome": e["outcome"]}
69                 for e in evidence[:5]
70             ],
71         },
72         fallback=baseline,
73     )
74
75     return {
76         "comments": comments,
77         "evidence": evidence,
78         "collaboration_analysis": {
79             "comment_count": len(comments),
80             "evidence_count": len(evidence),
81             "signals": signals,
82             "evidence_outcomes": {k: v for k, v in outcomes.items() if k},
83             "evidence_gaps": gaps,
84             "open_concerns": [
85                 {"comment_id": c["comment_id"], "author": c["author"], "text": c["body"]}
86                 for c in concerns
87             ],
88             "confidence": confidence,
89             "reasoning": reasoning,
90         },
91         "tools_used": {
92             "collaboration_retrieval": [
93                 "comments_for_ecr",
94                 "evidence_for_ecr",
95                 "extract_comment_signals",
96                 "evidence_gaps",
97             ]
98         },
99         "_confidence": confidence,
100         "_reasoning": reasoning,
101         "_summary": {
102             "comments": len(comments),
103             "evidence": len(evidence),
104             "decisions": len(signals["decisions"]),
105             "concerns": len(concerns),
106         },
107     }

```

backend/app/agents/nodes/correlation.py

```

1  """Correlation layer.
2
3  Builds one explicit traceability bundle for the ECR:
4
5      ECR -> requirements -> test cases
6          -> historical defects -> components
7          -> review comments -> evidence
8
9  Every link records *why* it exists, which is what makes the final answer
10 citable rather than a guess.
11 """
12 from __future__ import annotations
13
14 from collections import defaultdict
15 from typing import Any
16
17 from app.agents.tools.registry import tool
18 from app.database import session_scope
19 from app.models import Component
20
21
22 @tool("build_correlation_graph", "Link every retrieved artefact to the ECR with a typed edge.")
23 def build_correlation_graph(state: dict[str, Any]) -> dict[str, Any]:
24     ecr_id = state.get("ecr_id", "")
25     requirements = state.get("requirements") or []
26     defects = state.get("defects") or []
27     comments = state.get("comments") or []
28     evidence = state.get("evidence") or []

```

```

29     selected = state.get("selected_tests") or []
30     impact = state.get("impact_analysis") or {}
31
32     nodes: list[dict[str, Any]] = [
33         {"id": ecr_id, "type": "ECR", "label": state.get("ecr_input", {}).get("title", ecr_id)}
34     ]
35     edges: list[dict[str, Any]] = []
36
37     def add_node(node_id: str, node_type: str, label: str, **extra: Any) -> None:
38         if not node_id or any(n["id"] == node_id for n in nodes):
39             return
40         nodes.append({"id": node_id, "type": node_type, "label": label, **extra})
41
42     def add_edge(source: str, target: str, relation: str, reason: str, weight: float = 1.0) -> None:
43         if not source or not target:
44             return
45         edges.append(
46             {"source": source, "target": target, "relation": relation, "reason": reason, "weight": weight}
47         )
48
49     for requirement in requirements:
50         rid = requirement["requirement_id"]
51         add_node(rid, "REQUIREMENT", requirement.get("title", rid), priority=requirement.get("priority"))
52         add_edge(
53             ecr_id,
54             rid,
55             "IMPACTS_REQUIREMENT",
56             requirement.get("reason", ""),
57             round(float(requirement.get("relevance", 0)) / 100.0, 3),
58         )
59         for component_id in [requirement.get("component"), *(requirement.get("linked_components") or [])]:
60             if component_id:
61                 add_edge(rid, component_id, "IMPLEMENTED_BY", "Requirement is realised by this component",
+ 0.8)
62
63     for defect in defects:
64         did = defect["defect_id"]
65         add_node(did, "DEFECT", defect.get("title", did), severity=defect.get("severity"))
66         add_edge(
67             ecr_id,
68             did,
69             "SIMILAR_HISTORY",
70             defect.get("reason", ""),
71             round(float(defect.get("similarity", 0)) / 100.0, 3),
72         )
73         add_edge(did, defect.get("affected_component", ""), "AFFECTED", "Defect was raised on this componen
+ t", 0.7)
74
75     for test in selected:
76         tid = test["test_case_id"]
77         add_node(tid, "TEST_CASE", test.get("title", tid), priority=test.get("priority"))
78         add_edge(
79             ecr_id,
80             tid,
81             "RECOMMENDED_TEST",
82             test.get("reason", ""),
83             round(float(test.get("relevance_score", 0)) / 100.0, 3),
84         )
85         if test.get("requirement_id"):
86             add_edge(test["requirement_id"], tid, "VERIFIED_BY", "Test verifies this requirement", 0.9)
87
88     for comment in comments:
89         cid = comment["comment_id"]
90         add_node(cid, "COMMENT", f"{comment.get('author', '')}: {comment.get('category', '')}")
91         add_edge(
92             comment.get("target_id") or ecr_id,
93             cid,
94             "DISCUSSED_IN",
95             f"{comment.get('category', 'Discussion')} raised by {comment.get('author', 'a reviewer')}",
96             0.6,
97         )
98         for reference in comment.get("references") or []:
99             add_edge(cid, reference, "REFERENCES", "Comment explicitly references this artefact", 0.5)
100
101     for item in evidence:
102         eid = item["evidence_id"]
103         add_node(eid, "EVIDENCE", item.get("title", eid), outcome=item.get("outcome"))
104         add_edge(
105             item.get("target_id") or ecr_id,
106             eid,
107             "EVIDENCED_BY",
108             f"{item.get('evidence_type', 'Artefact')} with outcome {item.get('outcome', 'UNKNOWN')}",
109             0.7,
110         )

```

```

111
112 for component in [
113     *(impact.get("directly_impacted_components") or []),
114     *(impact.get("indirectly_impacted_components") or []),
115 ]:
116     add_node(
117         component["component_id"],
118         "COMPONENT",
119         component.get("name", component["component_id"]),
120         criticality=component.get("business_criticality"),
121     )
122     add_edge(
123         ecr_id,
124         component["component_id"],
125         "IMPACTS_COMPONENT" if component.get("impact_type") == "DIRECT" else "TOUCHES_COMPONENT",
126         "; ".join(component.get("reasons") or []),
127         round(float(component.get("impact_score", 0)) / 100.0, 3),
128     )
129
130 known = {node["id"] for node in nodes}
131 edges = [edge for edge in edges if edge["source"] in known and edge["target"] in known]
132 return {"nodes": nodes, "edges": edges}
133
134
135 def correlation_step(state: dict[str, Any]) -> dict[str, Any]:
136     graph = build_correlation_graph(state)
137     ecr_id = state.get("ecr_id", "")
138
139     by_type: dict[str, int] = defaultdict(int)
140     for node in graph["nodes"]:
141         by_type[node["type"]] += 1
142     relations: dict[str, int] = defaultdict(int)
143     for edge in graph["edges"]:
144         relations[edge["relation"]] += 1
145
146     # Cross-source corroboration: artefacts named by more than one source are the
147     # ones a human would otherwise have to join up manually.
148     mention_count: dict[str, set[str]] = defaultdict(set)
149     for comment in state.get("comments") or []:
150         for reference in comment.get("references") or []:
151             mention_count[reference].add("comment")
152     for item in state.get("evidence") or []:
153         if item.get("target_id"):
154             mention_count[item["target_id"]].add("evidence")
155     for requirement in state.get("requirements") or []:
156         mention_count[requirement["requirement_id"]].add("requirement_search")
157     for defect in state.get("defects") or []:
158         mention_count[defect["defect_id"]].add("defect_search")
159     for test in state.get("selected_tests") or []:
160         mention_count[test["test_case_id"]].add("test_selection")
161
162     corroborated = sorted(
163         (
164             {"artefact": artefact, "sources": sorted(sources), "source_count": len(sources)}
165             for artefact, sources in mention_count.items()
166             if len(sources) > 1
167         ),
168         key=lambda item: item["source_count"],
169         reverse=True,
170     )
171
172     with session_scope() as db:
173         component_names = {c.component_id: c.name for c in db.query(Component).all()}
174
175     coverage = {
176         "requirements": len(state.get("requirements") or []),
177         "tests_selected": len(state.get("selected_tests") or []),
178         "defects": len(state.get("defects") or []),
179         "comments": len(state.get("comments") or []),
180         "evidence": len(state.get("evidence") or []),
181         "components": by_type.get("COMPONENT", 0),
182     }
183     confidence = round(
184         min(0.96, 0.5 + 0.08 * sum(1 for value in coverage.values() if value)), 3
185     )
186     reasoning = (
187         f"Correlated {len(graph['nodes'])} artefact(s) and {len(graph['edges'])} typed link(s) "
188         f"for {ecr_id} across {sum(1 for v in coverage.values() if v)} source(s). "
189         f"{len(corroborated)} artefact(s) are corroborated by more than one source."
190     )
191
192     return {
193         "correlation": {
194             "graph": graph,

```

```

195         "node_types": dict(by_type),
196         "relations": dict(relations),
197         "coverage": coverage,
198         "corroborated": corroborated[:12],
199         "component_names": component_names,
200         "confidence": confidence,
201         "reasoning": reasoning,
202     },
203     "tools_used": {"correlation": ["build_correlation_graph"]},
204     "_confidence": confidence,
205     "_reasoning": reasoning,
206     "_summary": {
207         "nodes": len(graph["nodes"]),
208         "edges": len(graph["edges"]),
209         "coverage": coverage,
210         "corroborated": len(corroborated),
211     },
212 }

```

backend/app/agents/nodes/correlation_review.py

```

1  """Part of the *select_tests* step: the Correlation Agent's LLM reviews the bundle.
2
3  The graph in :mod:`correlation` records *which* artefacts are linked. It cannot
4  see that a reviewer's concern contradicts a passing test run, or that a defect
5  likely to return has no evidence. The model reads the whole correlated bundle
6  and reports the links that matter, the contradictions and the gaps - the
7  joining-up a person would otherwise do by hand across five sources.
8
9  Guardrails: every identifier the model cites must exist in the bundle (unknown
10 ones are dropped, and a finding left with none is discarded), and nothing it
11 says changes a score, a test selection or a recurrence rating. The
12 Summarization Agent receives the findings to explain.
13 """
14 from __future__ import annotations
15
16 import json
17 from collections import defaultdict
18 from typing import Any
19
20 from pydantic import BaseModel, Field
21
22 from app.llm.provider import get_llm
23 from app.prompts import CORRELATION_SYSTEM_PROMPT, CORRELATION_TASK
24
25 MAX_FINDINGS = 6
26 LLM_FINDINGS = 4 # per list, so the model's reply fits the token limit
27 PASSING = {"PASS", "PASSED", "APPROVED", "SUCCESS"}
28
29
30 class Finding(BaseModel):
31     ids: list[str] = Field(default_factory=list)
32     text: str = ""
33
34
35 class CorrelationReview(BaseModel):
36     summary: str = ""
37     key_links: list[Finding] = Field(default_factory=list)
38     conflicts: list[Finding] = Field(default_factory=list)
39     gaps: list[Finding] = Field(default_factory=list)
40     source: str = "llm"
41
42
43 def _selected_tests(state: dict[str, Any]) -> list[dict[str, Any]]:
44     return state.get("selected_tests") or (state.get("test_selection") or {}).get("selected_tests") or []
45
46
47 def _known_ids(state: dict[str, Any]) -> set[str]:
48     impact = state.get("impact_analysis") or {}
49     ids = {
50         state.get("ecr_id", ""),
51         *(r["requirement_id"] for r in state.get("requirements") or []),
52         *(d["defect_id"] for d in state.get("defects") or []),
53         *(t["test_case_id"] for t in _selected_tests(state)),
54         *(c["comment_id"] for c in state.get("comments") or []),
55         *(e["evidence_id"] for e in state.get("evidence") or []),
56         *(
57             c["component_id"]
58             for c in [
59                 *(impact.get("directly_impacted_components") or []),
60                 *(impact.get("indirectly_impacted_components") or []),
61             ]
62         ),
63     }

```

```

64     ids.discard("")
65     return ids
66
67
68 def _rule_review(state: dict[str, Any]) -> CorrelationReview:
69     """What the rules alone can say. Used when the LLM is unavailable, and as its starting point."""
70     correlation = state.get("correlation") or {}
71     collaboration = state.get("collaboration_analysis") or {}
72     insights = state.get("defect_insights") or {}
73
74     key_links = [
75         Finding(
76             ids=[item["artefact"]],
77             text=f"{item['artefact']} is backed by {item['source_count']} separate sources "
78             f"({', '.join(item['sources'])}).",
79         )
80         for item in (correlation.get("corroborated") or [])[:4]
81     ]
82
83     passing: dict[str, list[str]] = defaultdict(list)
84     for item in state.get("evidence") or []:
85         if item.get("target_id") and str(item.get("outcome", "")).upper() in PASSING:
86             passing[item["target_id"]].append(item["evidence_id"])
87     conflicts: list[Finding] = []
88     for comment in state.get("comments") or []:
89         if comment.get("sentiment") != "CONCERN":
90             continue
91         for target in [comment.get("target_id"), *(comment.get("references") or [])]:
92             runs = passing.get(target or "")
93             if runs:
94                 conflicts.append(
95                     Finding(
96                         ids=[comment["comment_id"], target, *runs[:2]],
97                         text=f"{comment.get('author') or 'A reviewer'} raised a concern about {target} "
98                         f"({comment['comment_id']}), but ({', '.join(runs[:2])}) recorded a pass. "
99                         "Check that the evidence really covers the concern.",
100                     )
101                 )
102             break
103
104     gaps = [
105         Finding(ids=[rid], text=f"{rid} is impacted by this change but has no evidence attached.")
106         for rid in (collaboration.get("evidence_gaps") or [])[:4]
107     ]
108     gaps += [
109         Finding(
110             ids=[row["defect_id"]],
111             text=f"{row['defect_id']} could happen again and no recommended test covers it.",
112         )
113         for row in insights.get("defects") or []
114         if row.get("chance_of_recurrence") == "High" and not row.get("covered")
115     ][:4]
116
117     summary = (
118         f"{len(key_links)} artefact(s) are confirmed by more than one source, "
119         f"{len(conflicts)} contradiction(s) and {len(gaps)} gap(s) were found."
120     )
121     return CorrelationReview(
122         summary=summary,
123         key_links=key_links[:MAX_FINDINGS],
124         conflicts=conflicts[:MAX_FINDINGS],
125         gaps=gaps[:MAX_FINDINGS],
126         source="rules",
127     )
128
129
130 def _bundle(state: dict[str, Any]) -> dict[str, Any]:
131     impact = state.get("impact_analysis") or {}
132     insights = {row["defect_id"]: row for row in (state.get("defect_insights") or {}).get("defects") or []}
133     return {
134         "ecr": {
135             "id": state.get("ecr_id"),
136             "title": (state.get("ecr_input") or {}).get("title"),
137             "description": (state.get("ecr_input") or {}).get("description"),
138         },
139         "requirements": [
140             {"id": r["requirement_id"], "title": r.get("title"), "relevance": r.get("relevance")}
141             for r in (state.get("requirements") or [])[:10]
142         ],
143         "recommended_tests": [
144             {
145                 "id": t["test_case_id"],
146                 "title": t.get("title"),
147                 "priority": t.get("priority"),

```

```

148         "requirement": t.get("requirement_id"),
149     }
150     for t in _selected_tests(state)[:15]
151 ],
152 "past_defects": [
153     {
154         "id": d["defect_id"],
155         "title": d.get("title"),
156         "component": d.get("affected_component"),
157         "chance_it_happens_again": (insights.get(d["defect_id"]) or {}).get("chance_of_recurrence")
158     },
159     {
160         "covered_by_tests": (insights.get(d["defect_id"]) or {}).get("covering_tests", []),
161     }
162     for d in (state.get("defects") or [])[:8]
163 ],
164 "comments": [
165     {
166         "id": c["comment_id"],
167         "author": c.get("author"),
168         "type": c.get("category"),
169         "sentiment": c.get("sentiment"),
170         "on": c.get("target_id"),
171         "references": c.get("references") or [],
172         "text": str(c.get("body", ""))[:280],
173     }
174     for c in (state.get("comments") or [])[:12]
175 ],
176 "evidence": [
177     {
178         "id": e["evidence_id"],
179         "title": e.get("title"),
180         "type": e.get("evidence_type"),
181         "outcome": e.get("outcome"),
182         "on": e.get("target_id"),
183     }
184     for e in (state.get("evidence") or [])[:10]
185 ],
186 "impacted_components": [
187     {"id": c["component_id"], "name": c.get("name"), "impact": c.get("impact_type")}
188     for c in [
189         *(impact.get("directly_impacted_components") or []),
190         *(impact.get("indirectly_impacted_components") or []),
191     ][:10]
192 ],
193 }
194
195 def _grounded(findings: list[Finding], known: set[str]) -> list[dict[str, Any]]:
196     kept = []
197     for finding in findings:
198         ids = [i for i in dict.fromkeys(finding.ids) if i in known]
199         text = " ".join(finding.text.split())
200         if ids and text:
201             kept.append({"ids": ids, "text": text[:400]})
202     return kept[:MAX_FINDINGS]
203
204 def correlation_review_step(state: dict[str, Any]) -> dict[str, Any]:
205     rules = _rule_review(state)
206     review = rules
207
208     llm = get_llm()
209     if not llm.is_mock:
210         review = llm.generate_structured(
211             CORRELATION_TASK.format(
212                 bundle=json.dumps(_bundle(state), default=str)[:12000],
213                 rule_findings=json.dumps(rules.model_dump(exclude={"source"})),
214                 max_findings=LLM_FINDINGS,
215             ),
216             CorrelationReview,
217             system=CORRELATION_SYSTEM_PROMPT,
218             fallback=rules.model_dump(),
219         )
220
221     used_llm = review.source != "rules"
222     known = _known_ids(state)
223     key_links = _grounded(review.key_links, known)
224     conflicts = _grounded(review.conflicts, known)
225     gaps = _grounded(review.gaps, known)
226     if used_llm and not (key_links or conflicts or gaps):
227         # Everything the model cited was ungrounded: keep the rule findings instead.
228         used_llm = False
229         key_links, conflicts, gaps = (
230             _grounded(rules.key_links, known),

```

```

231         _grounded(rules.conflicts, known),
232         _grounded(rules.gaps, known),
233     )
234     review = rules
235
236     reasoning = (
237         f"{'The LLM' if used_llm else 'Rule-based'} review of the correlated bundle found "
238         f"{len(key_links)} key link(s), {len(conflicts)} contradiction(s) and {len(gaps)} gap(s)."

```

backend/app/agents/nodes/defect_insights.py

```

1  """Correlation Agent part: analyse the past defects related to this ECR.
2
3  For every related defect it answers the questions an engineer otherwise works
4  out by hand: what went wrong, why it matters for this change, can it happen
5  again, and which recommended test would catch it. It is deterministic, so the
6  same ECR always gets the same defect analysis.
7  """
8  from __future__ import annotations
9
10 from collections import Counter
11 from datetime import timezone
12 from typing import Any
13
14 from app.agents.nodes.scope import is_scoped
15 from app.database import session_scope
16 from app.models import Component, Defect
17 from app.models.base import utcnow
18
19 CLOSED_STATUSES = {"CLOSED", "RESOLVED", "DONE", "VERIFIED"}
20
21
22 def _days_ago(stamp) -> int | None:
23     if stamp is None:
24         return None
25     if stamp.tzinfo is None:
26         stamp = stamp.replace(tzinfo=timezone.utc)
27     return max(0, (utcnow() - stamp).days)
28
29
30 def component_history(
31     defects: list[Defect], ordered_components: list[tuple[str, str]], names: dict[str, str]
32 ) -> tuple[list[dict[str, Any]], list[dict[str, Any]]:
33     """Defect history of the components the change touches, plus what is open now.
34
35     This is shown even when no past defect matches the change itself, so the
36     section always tells the reader how defect-prone the affected area is.
37     """
38     by_component: dict[str, list[Defect]] = {}
39     for defect in defects:
40         by_component.setdefault(defect.affected_component, []).append(defect)
41
42     history = []
43     for component_id, impact in ordered_components:
44         rows = by_component.get(component_id) or []
45         if not rows:
46             continue
47         causes = Counter(d.root_cause_category for d in rows if d.root_cause_category)
48         ages = [age for age in (_days_ago(d.detected_at) for d in rows) if age is not None]
49         history.append(
50             {
51                 "component": component_id,
52                 "component_name": names.get(component_id, component_id),
53                 "impact": impact,
54                 "total": len(rows),
55                 "severe": sum(1 for d in rows if d.severity in ("CRITICAL", "HIGH")),
56                 "open": sum(1 for d in rows if (d.status or "").upper() not in CLOSED_STATUSES),
57                 "reached_production": sum(1 for d in rows if d.escaped_to_production),
58                 "last_seen_days": min(ages) if ages else None,
59                 "top_cause": causes.most_common(1)[0][0] if causes else "",
60             }
61         )

```

```

62
63 touched = {component_id for component_id, _ in ordered_components}
64 open_defects = [
65     {
66         "defect_id": d.defect_id,
67         "title": d.title,
68         "severity": d.severity,
69         "status": d.status,
70         "component": d.affected_component,
71         "component_name": names.get(d.affected_component, d.affected_component),
72         "open_days": _days_ago(d.detected_at),
73     }
74     for d in defects
75     if (d.status or "").upper() not in CLOSED_STATUSES
76     and (d.affected_component in touched or touched & set(d.related_components or []))
77 ]
78 open_defects.sort(key=lambda row: SEVERITY_WEIGHT.get(row["severity"], 0.45), reverse=True)
79 return history, open_defects
80
81 SEVERITY_WEIGHT = {"CRITICAL": 1.0, "HIGH": 0.75, "MEDIUM": 0.45, "LOW": 0.2}
82 HIGH_CHANCE = 0.7
83 MEDIUM_CHANCE = 0.45
84 MAX_TESTS = 3
85 CHANCE_ORDER = {"High": 0, "Medium": 1, "Low": 2}
86
87
88 def analyze_defects(
89     defects: list[dict[str, Any]],
90     *,
91     requirement_ids: set[str],
92     direct: set[str],
93     indirect: set[str],
94     selected_tests: list[dict[str, Any]],
95     linked_requirements: dict[str, list[str]],
96     reopen_counts: dict[str, int],
97     component_names: dict[str, str],
98 ) -> list[dict[str, Any]]:
99     rows = []
100     for defect in defects:
101         defect_id = defect["defect_id"]
102         component = defect.get("affected_component", "")
103         name = component_names.get(component, component)
104         requirements = linked_requirements.get(defect_id, [])
105         shared_requirements = [r for r in requirements if r in requirement_ids]
106
107         tests = [
108             test
109             for test in selected_tests
110             if (component and test.get("component") == component)
111             or (test.get("requirement_id") and test["requirement_id"] in requirements)
112         ]
113         tests.sort(key=lambda test: test.get("relevance_score", 0), reverse=True)
114         test_ids = [test["test_case_id"] for test in tests[:MAX_TESTS]]
115
116         touches = component in direct or component in indirect
117         score = (
118             0.3 * SEVERITY_WEIGHT.get(defect.get("severity", "MEDIUM"), 0.45)
119             + 0.2 * min(1.0, float(defect.get("similarity", 0)) / 100.0)
120             + (0.3 if component in direct else 0.15 if component in indirect else 0.0)
121             + (0.1 if shared_requirements else 0.0)
122             + (0.05 if defect.get("escaped_to_production") else 0.0)
123             + min(0.05, 0.025 * reopen_counts.get(defect_id, 0))
124         )
125         chance = "High" if score >= HIGH_CHANCE else "Medium" if score >= MEDIUM_CHANCE else "Low"
126         # A defect can only come back through code this change actually reaches.
127         if not touches:
128             chance = "Low"
129
130         why = []
131         if component in direct:
132             why.append(f"This change touches {name}, where the defect happened.")
133         elif component in indirect:
134             why.append(f"This change indirectly affects {name}, where the defect happened.")
135         if shared_requirements:
136             why.append(
137                 f"It broke {' '.join(shared_requirements)}, which this ECR also affects"
138                 + (". " if touches else ", but the change does not touch the code where it happened.")
139             )
140         if not why:
141             why.append("Its description is similar to this change, but the change does not touch its code.")
142     + )
143     if defect.get("escaped_to_production"):
144         why.append("It reached production last time.")

```

```

145     cause = (defect.get("root_cause") or defect.get("title") or "").rstrip(".")
146     if test_ids:
147         check = f"Run {'', '.join(test_ids)} to make sure it does not come back."
148     elif chance == "Low":
149         check = "Low chance with this change. No extra check needed."
150     else:
151         check = f"No recommended test covers this. Check that the fix still holds: {cause}."
152
153     rows.append(
154         {
155             "defect_id": defect_id,
156             "title": defect.get("title", ""),
157             "severity": defect.get("severity", "MEDIUM"),
158             "status": defect.get("status", ""),
159             "component": component,
160             "component_name": name,
161             "root_cause": defect.get("root_cause", ""),
162             "root_cause_category": defect.get("root_cause_category", ""),
163             "similarity": defect.get("similarity", 0),
164             "why_it_matters": " ".join(why),
165             "chance_of_recurrence": chance,
166             "covering_tests": test_ids,
167             "covered": bool(test_ids),
168             "what_to_check": check,
169         }
170     )
171     rows.sort(
172         key=lambda row: (
173             CHANCE_ORDER[row["chance_of_recurrence"]],
174             -SEVERITY_WEIGHT.get(row["severity"], 0.45),
175             -float(row["similarity"] or 0),
176         )
177     )
178     return rows
179
180
181 def defect_insights_step(state: dict[str, Any]) -> dict[str, Any]:
182     defects = state.get("defects") or []
183     impact = state.get("impact_analysis") or {}
184     direct = {c["component_id"]: c["name"] for c in impact.get("directly_impacted_components") or []}
185     indirect = {c["component_id"]: c["name"] for c in impact.get("indirectly_impacted_components") or []}
186     selected = (state.get("test_selection") or {}).get("selected_tests") or []
187
188     defect_ids = {d["defect_id"] for d in defects}
189     linked: dict[str, list[str]] = {}
190     reopened: dict[str, int] = {}
191     names: dict[str, str] = {}
192     ordered = [(c, "Direct") for c in direct] + [(c, "Indirect") for c in indirect if c not in direct]
193     with session_scope() as db:
194         names = {row.component_id: row.name for row in db.query(Component)}
195         all_defects = db.query(Defect).all()
196         for row in all_defects:
197             if row.defect_id in defect_ids:
198                 linked[row.defect_id] = list(row.linked_requirements or [])
199                 reopened[row.defect_id] = int(row.reopen_count or 0)
200         history, open_defects = component_history(all_defects, ordered, names)
201     scoped = is_scoped(state.get("ecr_input"))
202     if scoped: # only this record's own defects, not the components' whole history
203         history = []
204         open_defects = [row for row in open_defects if row["defect_id"] in defect_ids]
205     names.update(indirect)
206     names.update(direct)
207
208     rows = analyze_defects(
209         defects,
210         requirement_ids={r["requirement_id"] for r in state.get("requirements") or []},
211         direct=set(direct),
212         indirect=set(indirect),
213         selected_tests=selected,
214         linked_requirements=linked,
215         reopen_counts=reopened,
216         component_names=names,
217     )
218     chances = Counter(row["chance_of_recurrence"] for row in rows)
219     causes = Counter(row["root_cause_category"] for row in rows if row["root_cause_category"])
220     # Only defects that could realistically come back need a manual check.
221     not_covered = [
222         row["defect_id"] for row in rows if not row["covered"] and row["chance_of_recurrence"] != "Low"
223     ]
224     insights = {
225         "defects": rows,
226         "total": len(rows),
227         "high": chances["High"],
228         "medium": chances["Medium"],

```

```

229         "low": chances["Low"],
230         "not_covered": not_covered,
231         "common_causes": [cause for cause, _ in causes.most_common(3)],
232         "component_history": history,
233         "scoped": scoped,
234         "open_defects": open_defects,
235     }
236     reasoning = (
237         f"Analysed {len(rows)} related defect(s): {chances['High']} could come back with this change, "
238         f"{len(not_covered)} not covered by the recommended tests."
239     )
240     return {
241         "defect_insights": insights,
242         "tools_used": {"defect_insights": ["analyze_defects"]},
243         "_confidence": 0.9 if rows else 0.75,
244         "_reasoning": reasoning,
245     }

```

backend/app/agents/nodes/dependencies.py

```

1  """Retrieval Agent step: dependency analysis.
2
3  Walks the component graph in both directions from the changed components:
4  downstream (who depends on us = regression blast radius) and upstream (what we
5  rely on), with a decayed propagation score per hop and the critical paths that
6  reach the change from an entry point.
7  """
8  from __future__ import annotations
9
10 from typing import Any
11
12 from app.agents.tools.dependency_tools import (
13     build_graph,
14     critical_paths,
15     dependency_subgraph,
16     downstream_walk,
17     upstream_walk,
18 )
19 from app.agents.tools.test_tools import resolve_domain
20 from app.database import session_scope
21 from app.llm.provider import get_llm
22 from app.models import Component
23 from app.schemas.impact import DependencyAnalysis, DependencyEdge, DependencyNode
24
25
26 def dependency_step(state: dict[str, Any]) -> dict[str, Any]:
27     analysis = state.get("ecr_analysis") or {}
28     code_impact = state.get("code_impact") or {}
29     requirement_analysis = state.get("requirement_analysis") or {}
30
31     ecr = state.get("ecr_input") or {}
32
33     with session_scope() as db:
34         graph = build_graph(db)
35         # Stay inside the ECR's product line, decided by its linked requirements:
36         # searched requirements can belong to another domain (a rail change must
37         # not walk the payments graph).
38         domain = resolve_domain(db, [], [], list(ecr.get("linked_requirements") or []))
39         in_domain = (
40             {c.component_id for c in db.query(Component).filter(Component.domain == domain)}
41             if domain
42             else None
43         )
44
45     def keep(ids: list[str]) -> list[str]:
46         return sorted({c for c in ids if in_domain is None or c in in_domain})
47
48     # Seed from the strongest evidence: resolved code beats keyword candidates,
49     # which beat requirement links. Mixing them would walk the graph from
50     # components the change never actually touches.
51     code_components = list(code_impact.get("impacted_services") or [])
52     seeds = (
53         keep(code_components)
54         or keep(analysis.get("candidate_components") or [])
55         or keep(requirement_analysis.get("impacted_components") or [])
56     )
57
58     downstream = downstream_walk(graph, seeds, depth=3)
59     upstream = upstream_walk(graph, seeds, depth=2)
60     propagation = graph.propagation_scores(seeds, max_depth=3)
61     paths = critical_paths(graph, seeds, max_paths=5)
62     subgraph = dependency_subgraph(graph, seeds, depth=2)
63
64     direct = sorted([c for c, distance in downstream.items() if distance == 1])

```

```

65     indirect = sorted([c for c, distance in downstream.items() if distance > 1])
66
67     nodes = [
68         DependencyNode(
69             component_id=component_id,
70             name=graph.node(component_id).get("name", component_id),
71             type=graph.node(component_id).get("type", "SERVICE"),
72             business_criticality=graph.node(component_id).get("business_criticality", "MEDIUM"),
73             distance=0 if component_id in seeds else downstream.get(component_id, upstream.get(component_id
74 + , 0)),
75             relation=(
76                 "SEED"
77                 if component_id in seeds
78                 else "DOWNSTREAM"
79                 if component_id in downstream
80                 else "UPSTREAM"
81             ),
82             for component_id in [*seeds, *downstream, *upstream]
83         ]
84     edges = [
85         DependencyEdge(
86             source=edge["source"],
87             target=edge["target"],
88             dependency_type=edge.get("dependency_type", "SYNC_API"),
89             criticality=edge.get("criticality", "MEDIUM"),
90         )
91         for edge in subgraph["edges"]
92     ]
93
94     confidence = round(
95         min(0.96, 0.45 + 0.3 * bool(seeds) + 0.15 * min(1.0, len(downstream) / 4.0) + 0.06),
96         3,
97     )
98     named = lambda ids: ", ".join(graph.node(c).get("name", c) for c in ids) or "none"
99     baseline = (
100         f"Seeded the walk from {named(seeds)}. "
101         f"{len(direct)} component(s) depend on the change directly ({named(direct)}) and "
102         f"{len(indirect)} indirectly ({named(indirect)}). "
103         f"The change reaches {len(downstream)} component(s) in total."
104     )
105     reasoning = get_llm().narrate(
106         task="Explain how this change propagates through the service dependency graph",
107         context={
108             "seeds": seeds,
109             "direct_dependents": direct,
110             "indirect_dependents": indirect,
111             "critical_paths": paths[:3],
112         },
113         fallback=baseline,
114     )
115
116     result = DependencyAnalysis(
117         seed_components=seeds,
118         direct_dependencies=direct,
119         indirect_dependencies=indirect,
120         upstream_components=sorted(upstream),
121         downstream_components=sorted(downstream),
122         critical_dependency_paths=paths,
123         blast_radius=len(downstream),
124         nodes=nodes,
125         edges=edges,
126         confidence=confidence,
127         reasoning=reasoning,
128     )
129     return {
130         "dependency_analysis": {
131             **result.model_dump(mode="json"),
132             "propagation": propagation,
133             "distances": downstream,
134             "graph_stats": graph.stats(),
135         },
136         "tools_used": {
137             "dependencies": [
138                 "build_graph",
139                 "downstream_walk",
140                 "upstream_walk",
141                 "critical_paths",
142             ]
143         },
144         "_confidence": confidence,
145         "_reasoning": reasoning,
146         "_summary": {
147             "seeds": seeds,

```

```

148         "blast_radius": len(downstream),
149         "direct": direct,
150         "indirect": indirect,
151     },
152 }

```

backend/app/agents/nodes/ecr_understanding.py

```

1  """Retrieval Agent step: understand and classify the ECR.
2
3  Deterministic classification, so the change type and complexity are
4  reproducible and testable. The Retrieval Agent's LLM reasoning runs right after
5  this, in :mod:`retrieval_planning`; numeric fields are never taken from the model.
6  """
7  from __future__ import annotations
8
9  import re
10 from collections import Counter
11 from typing import Any
12
13 from app.agents.tools.registry import tool
14 from app.database import session_scope
15 from app.llm.base import tokenize
16 from app.models import Component, Requirement
17 from app.schemas.common import ChangeType
18 from app.schemas.ecr import ECRUnderstanding
19 from app.services.code_analysis_service import KEYWORD_COMPONENT_MAP
20
21 CHANGE_TYPE_SIGNALS: dict[str, list[str]] = {
22     ChangeType.UI.value: [
23         "button", "label", "page", "screen", "css", "layout", "wording", "text",
24         "portal", "frontend", "ux", "tooltip", "banner", "template",
25     ],
26     ChangeType.API.value: [
27         "api", "endpoint", "contract", "rest", "payload", "route", "version",
28         "request", "response", "schema v2", "openapi",
29     ],
30     ChangeType.BACKEND.value: [
31         "logic", "service", "calculation", "processing", "algorithm", "handler",
32         "validation", "rounding", "orchestration", "workflow",
33     ],
34     ChangeType.DATABASE.value: [
35         "database", "schema", "column", "table", "migration", "index", "backfill",
36         "sql", "query", "partition", "constraint",
37     ],
38     ChangeType.SECURITY.value: [
39         "authentication", "authorisation", "authorization", "mfa", "token",
40         "credential", "encryption", "security", "session", "step-up", "oauth",
41     ],
42     ChangeType.INFRASTRUCTURE.value: [
43         "infrastructure", "redis", "cache", "kubernetes", "deployment", "scaling",
44         "cluster", "pipeline", "terraform", "network",
45     ],
46     ChangeType.CONFIGURATION.value: [
47         "configuration", "config", "feature flag", "toggle", "threshold",
48         "parameter", "setting", "property",
49     ],
50     ChangeType.INTEGRATION.value: [
51         "integration", "third-party", "third party", "vendor", "webhook",
52         "external", "partner", "connector", "anti-corruption",
53     ],
54 }
55
56 RISK_INDICATOR_RULES: list[tuple[str, tuple[str, ...]]] = [
57     ("financial_transaction", ("payment", "transaction", "refund", "settlement", "ledger", "invoice", "currency")),
58     ("validation_logic_change", ("validation", "validator", "validate", "reject", "rule")),
59     ("schema_migration", ("schema", "migration", "column", "backfill", "index")),
60     ("authentication_change", ("authentication", "mfa", "login", "sign-in", "step-up")),
61     ("authorization_change", ("authorisation", "authorization", "permission", "role", "token")),
62     ("shared_library_change", ("shared", "library", "common", "validator", "helper")),
63     ("third_party_integration", ("third-party", "third party", "vendor", "external", "partner")),
64     ("concurrency_change", ("concurrent", "lock", "race", "idempotency", "parallel")),
65     ("data_migration", ("backfill", "migrate", "historical rows", "data migration")),
66     ("customer_facing", ("customer", "portal", "page", "email", "notification")),
67     ("performance_change", ("performance", "latency", "cache", "optimise", "optimize", "index")),
68 ]
69
70 FILE_EXTENSION_SIGNALS = {
71     ".sql": ChangeType.DATABASE.value,
72     ".tf": ChangeType.INFRASTRUCTURE.value,
73     ".yaml": ChangeType.CONFIGURATION.value,
74     ".yml": ChangeType.CONFIGURATION.value,
75     ".json": ChangeType.CONFIGURATION.value,

```

```

76     ".tsx": ChangeType.UI.value,
77     ".jsx": ChangeType.UI.value,
78     ".css": ChangeType.UI.value,
79 }
80
81
82 @tool("classify_change", "Score the change against every change-type signal vocabulary.")
83 def classify_change(text: str, changed_files: list[str]) -> dict[str, float]:
84     lowered = text.lower()
85     scores: dict[str, float] = {}
86     for change_type, signals in CHANGE_TYPE_SIGNALS.items():
87         hits = sum(1 for signal in signals if signal in lowered)
88         scores[change_type] = float(hits)
89     for path in changed_files:
90         lower_path = path.lower()
91         for extension, change_type in FILE_EXTENSION_SIGNALS.items():
92             if lower_path.endswith(extension):
93                 scores[change_type] = scores.get(change_type, 0.0) + 1.5
94         if "schema" in lower_path or "migration" in lower_path:
95             scores[ChangeType.DATABASE.value] += 2.0
96         if "portal" in lower_path or "page" in lower_path:
97             scores[ChangeType.UI.value] += 2.0
98         if "router" in lower_path or "gateway" in lower_path or "api" in lower_path:
99             scores[ChangeType.API.value] += 1.5
100         if "service.py" in lower_path or "validator" in lower_path:
101             scores[ChangeType.BACKEND.value] += 1.5
102         if "auth" in lower_path:
103             scores[ChangeType.SECURITY.value] += 1.5
104         if "infra" in lower_path:
105             scores[ChangeType.INFRASTRUCTURE.value] += 1.5
106     total = sum(scores.values()) or 1.0
107     return {key: round(value / total, 3) for key, value in scores.items() if value}
108
109
110 @tool("extract_entities", "Extract features, keywords, domains and candidate components.")
111 def extract_entities(
112     text: str, changed_files: list[str], linked_requirements: list[str] | None = None
113 ) -> dict[str, Any]:
114     lowered = text.lower()
115     with session_scope() as db:
116         known_features: set[str] = set()
117         domain_by_feature: dict[str, str] = {}
118         requirement_domain: dict[str, str] = {}
119         for requirement in db.query(Requirement).all():
120             requirement_domain[requirement.requirement_id] = requirement.domain
121             for feature in requirement.features or []:
122                 known_features.add(feature)
123                 domain_by_feature[feature.lower()] = requirement.business_domain
124         components = db.query(Component).all()
125         component_names = {c.name.lower(): c.component_id for c in components}
126         component_domain = {c.component_id: c.domain for c in components}
127
128     features = sorted({f for f in known_features if f.lower() in lowered})
129     if not features:
130         # fall back to partial token overlap with the feature catalogue
131         text_tokens = set(tokenize(text))
132         for feature in known_features:
133             feature_tokens = set(tokenize(feature))
134             if feature_tokens and len(feature_tokens & text_tokens) >= max(1, len(feature_tokens) - 1):
135                 features.append(feature)
136     features = sorted(set(features))
137
138     counts = Counter(tokenize(text))
139     keywords = [word for word, _ in counts.most_common(24) if len(word) > 3][12:]
140     for path in changed_files:
141         stem = re.split(r"[/\.\.]", path)
142         keywords.extend(part for part in stem if len(part) > 3 and part not in keywords)
143
144     candidates: set[str] = set()
145     for keyword, component_id in KEYWORD_COMPONENT_MAP.items():
146         # Match at the start of a word only: "orders" and "reporting" count, but
147         # "recorder", "reorder" and "require" must not hit "order", "order", "ui".
148         if re.search(rf"\b{re.escape(keyword)}", lowered):
149             candidates.add(component_id)
150     named: set[str] = set()
151     for name, component_id in component_names.items():
152         if name in lowered:
153             named.add(component_id)
154     candidates |= named
155
156     # Keep candidates inside the ECR's own product domain. The requirements the ECR
157     # is linked to are the strongest evidence of that domain, then the components it
158     # names. Without this, commerce keywords ("validation", "report") in a rail
159     # change pull payment services into its impact and its test suite.

```

```

160     domain_votes = Counter(
161         requirement_domain[r] for r in (linked_requirements or []) if r in requirement_domain
162     ) or Counter(component_domain[c] for c in named if c in component_domain)
163     product_domain = domain_votes.most_common(1)[0][0] if domain_votes else None
164     if product_domain:
165         candidates = {c for c in candidates if component_domain.get(c) == product_domain}
166
167     domains = [domain_by_feature.get(f.lower()) for f in features]
168     domains = [d for d in domains if d]
169     business_domain = Counter(domains).most_common(1)[0][0] if domains else "General"
170
171     indicators = [
172         indicator
173         for indicator, signals in RISK_INDICATOR_RULES
174         if any(signal in lowered for signal in signals)
175     ]
176     return {
177         "affected_features": features[:8],
178         "technical_keywords": keywords[:12],
179         "candidate_components": sorted(candidates),
180         "business_domain": business_domain,
181         "product_domain": product_domain,
182         "risk_indicators": indicators,
183     }
184
185
186 def _complexity(scores: dict[str, float], entities: dict[str, Any], changed_files: list[str], text: str) ->
187     + float:
188     distinct_types = len([v for v in scores.values() if v >= 0.15])
189     value = (
190         0.3 * min(1.0, distinct_types / 3.0)
191         + 0.25 * min(1.0, len(entities["risk_indicators"]) / 4.0)
192         + 0.2 * min(1.0, len(changed_files) / 5.0)
193         + 0.15 * min(1.0, len(entities["candidate_components"]) / 4.0)
194         + 0.1 * min(1.0, len(text.split()) / 120.0)
195     )
196     return round(min(1.0, value), 3)
197
198 def ecr_understanding_step(state: dict[str, Any]) -> dict[str, Any]:
199     ecr = state["ecr_input"]
200     changed_files = list(ecr.get("changed_files") or [])
201     text = " ".join(
202         part
203         for part in (
204             f"{ecr.get('title', '')}. {ecr.get('description', '')}",
205             ecr.get("steps_to_reproduce") or "",
206             ecr.get("observed_behavior") or "",
207             ecr.get("expected_behavior") or "",
208         )
209         if part
210     )
211
212     scores = classify_change(text, changed_files)
213     entities = extract_entities(text, changed_files, list(ecr.get("linked_requirements") or []))
214     ranked = sorted(scores.items(), key=lambda item: item[1], reverse=True)
215     if not ranked:
216         change_type = ChangeType.MIXED.value
217     else:
218         top, top_score = ranked[0]
219         runner_up, runner_score = ranked[1] if len(ranked) > 1 else (None, 0.0)
220         if runner_up and abs(top_score - runner_score) < 0.08:
221             pair = {top, runner_up}
222             if pair == {ChangeType.API.value, ChangeType.BACKEND.value}:
223                 change_type = ChangeType.API_AND_BACKEND.value
224             else:
225                 change_type = ChangeType.MIXED.value
226         else:
227             change_type = top
228
229     complexity = _complexity(scores, entities, changed_files, text)
230     confidence = round(
231         min(0.97, 0.6 + 0.12 * bool(entities["affected_features"]) + 0.12 * bool(changed_files)
232             + 0.08 * bool(entities["candidate_components"]) + 0.05 * bool(scores)),
233         3,
234     )
235     baseline_summary = (
236         f"{ecr.get('ecr_id')} is a {change_type.replace('_', ' ').lower()} in the "
237         f"{entities['business_domain']} domain affecting "
238         f"{', '.join(entities['affected_features']) or 'unmapped functionality'}. "
239         f"{len(changed_files)} file(s) are declared on the change and "
240         f"{len(entities['candidate_components'])} candidate component(s) were identified."
241     )
242

```

```

243     fallback = {
244         "ecr_id": ecr.get("ecr_id", ""),
245         "change_type": change_type,
246         "change_type_scores": scores,
247         "business_domain": entities["business_domain"],
248         "affected_features": entities["affected_features"],
249         "technical_keywords": entities["technical_keywords"],
250         "candidate_components": entities["candidate_components"],
251         "risk_indicators": entities["risk_indicators"],
252         "change_complexity": complexity,
253         "summary": baseline_summary,
254         "confidence": confidence,
255     }
256
257     understanding = ECRUnderstanding.model_validate(fallback)
258
259     reasoning = (
260         f"Classified as {understanding.change_type.value} "
261         f"(confidence {understanding.confidence:.0%}) from {len(scores)} signal groups. "
262         f"Extracted {len(understanding.technical_keywords)} technical keywords, "
263         f"{len(understanding.affected_features)} affected feature(s) and "
264         f"{len(understanding.risk_indicators)} risk indicator(s). "
265     )
266
267     return {
268         "ecr_analysis": understanding.model_dump(mode="json"),
269         "tools_used": {"ecr_understanding": ["classify_change", "extract_entities"]},
270         "_confidence": understanding.confidence,
271         "_reasoning": reasoning,
272         "_summary": {
273             "change_type": understanding.change_type.value,
274             "business_domain": understanding.business_domain,
275             "affected_features": understanding.affected_features,
276             "risk_indicators": understanding.risk_indicators,
277         },
278     }

```

backend/app/agents/nodes/historical_defects.py

```

1  """Retrieval Agent step: similar historical defects.
2
3  Answers "has a change like this hurt us before?" by fusing semantic similarity
4  over incident write-ups with component-scoped defect history, then clustering
5  root causes into recurring patterns.
6  """
7  from __future__ import annotations
8
9  from collections import Counter
10 from typing import Any
11
12 from app.agents.nodes.retrieval_planning import search_query
13 from app.agents.tools.defect_tools import (
14     cluster_root_causes,
15     defects_by_component,
16     semantic_defect_search,
17 )
18 from app.agents.nodes.scope import is_scoped, linked_requirement_ids
19 from app.database import session_scope
20 from app.llm.provider import get_llm
21 from app.models import Defect
22 from app.schemas.impact import DefectAnalysis, DefectMatch
23
24 MIN_SIMILARITY = 40.0
25 MAX_DEFECTS = 10
26
27
28 def defect_step(state: dict[str, Any]) -> dict[str, Any]:
29     ecr = state["ecr_input"]
30     analysis = state.get("ecr_analysis") or {}
31     code_impact = state.get("code_impact") or {}
32     requirement_analysis = state.get("requirement_analysis") or {}
33
34     components = sorted(
35         {
36             *(code_impact.get("impacted_services") or []),
37             *(code_impact.get("potentially_impacted_services") or []),
38             *(requirement_analysis.get("impacted_components") or []),
39             *(analysis.get("candidate_components") or []),
40         }
41     )
42     query = search_query(state) # widened by the Retrieval Agent's LLM search plan
43     keywords = analysis.get("technical_keywords") or []
44
45     scoped = is_scoped(ecr)
46     semantic_hits = [] if scoped else semantic_defect_search(query, keywords=keywords, k=14)

```

```

47 with session_scope() as db:
48     if scoped: # only defects raised against the requirements the record links
49         wanted = linked_requirement_ids(ecr)
50         component_hits = [
51             **row.as_dict(), "similarity": 90.0}
52             for row in db.query(Defect).all()
53             if wanted & set(row.linked_requirements or [])
54         ]
55     else:
56         component_hits = defects_by_component(db, components)
57         searched_total = db.query(Defect).count()
58
59 merged: dict[str, dict[str, Any]] = {}
60 for payload in [*semantic_hits, *component_hits]:
61     defect_id = payload.get("defect_id")
62     if not defect_id:
63         continue
64     existing = merged.get(defect_id)
65     if existing is None:
66         merged[defect_id] = dict(payload)
67         continue
68     # found through both channels -> stronger evidence
69     best = dict(existing if existing["similarity"] >= payload["similarity"] else payload)
70     best["similarity"] = round(
71         min(100.0, max(existing["similarity"], payload["similarity"]) + 12.0), 1
72     )
73     merged[defect_id] = best
74
75 # Traceability beats text similarity: a defect raised against a requirement this
76 # ECR changes is relevant even when older, wordier defects look more similar.
77 ecr_requirements = set(ecr.get("linked_requirements") or [])
78 matched_requirements = {r["requirement_id"] for r in state.get("requirements") or []}
79 for payload in merged.values():
80     linked = set(payload.get("linked_requirements") or [])
81     if linked & ecr_requirements:
82         payload["similarity"] = round(min(100.0, payload["similarity"] + 15.0), 1)
83     elif linked & matched_requirements:
84         payload["similarity"] = round(min(100.0, payload["similarity"] + 6.0), 1)
85
86 candidates = [d for d in merged.values() if d["similarity"] >= MIN_SIMILARITY]
87 candidates.sort(key=lambda d: d["similarity"], reverse=True)
88 candidates = candidates[:MAX_DEFECTS]
89
90 matches = [
91     DefectMatch(
92         defect_id=d["defect_id"],
93         title=d.get("title", ""),
94         severity=d.get("severity", "MEDIUM"),
95         status=d.get("status", "CLOSED"),
96         root_cause=d.get("root_cause", ""),
97         root_cause_category=d.get("root_cause_category", ""),
98         affected_component=d.get("affected_component", ""),
99         similarity=d["similarity"],
100         escaped_to_production=bool(d.get("escaped_to_production")),
101         reason=_reason(d, components, ecr_requirements),
102     )
103     for d in candidates
104 ]
105
106 patterns = cluster_root_causes(candidates)
107 severity_distribution = dict(Counter(m.severity for m in matches))
108 component_counter = Counter(m.affected_component for m in matches if m.affected_component)
109 defect_prone_components = [comp for comp, count in component_counter.most_common(5) if count >= 1]
110
111 confidence = round(
112     min(0.95, 0.4 + 0.3 * min(1.0, len(matches) / 5.0) + 0.2 * bool(components) + 0.05),
113     3,
114 )
115 baseline = (
116     f"Searched {searched_total} historical defects and found {len(matches)} related to this "
117     f"change. "
118     + (
119         f"Recurring patterns: {'; '.join(patterns)}. "
120         if patterns
121         else "No recurring root-cause pattern emerged. "
122     )
123     + (
124         f"Components with the most related defects: {'; '.join(defect_prone_components[:3])}."
125         if defect_prone_components
126         else "No component concentration detected."
127     )
128 )
129 reasoning = get_llm().narrate(
130     task="Explain what past defects tell us about this change",

```

```

131         context={
132             "defects": [
133                 {"id": m.defect_id, "title": m.title, "severity": m.severity, "similarity": m.similarity}
134                 for m in matches[:6]
135             ],
136             "patterns": patterns,
137         },
138         fallback=baseline,
139     )
140
141     result = DefectAnalysis(
142         similar_defects=matches,
143         defect_prone_components=defect_prone_components,
144         recurring_failure_patterns=patterns,
145         severity_distribution=severity_distribution,
146         searched_count=searched_total,
147         confidence=confidence,
148         reasoning=reasoning,
149     )
150     return {
151         "defects": [m.model_dump(mode="json") for m in matches],
152         "defect_analysis": result.model_dump(mode="json"),
153         "tools_used": {
154             "historical_defects": [
155                 "semantic_defect_search",
156                 "defects_by_component",
157                 "cluster_root_causes",
158             ]
159         },
160         "_confidence": confidence,
161         "_reasoning": reasoning,
162         "_summary": {
163             "matched": len(matches),
164             "searched": searched_total,
165             "patterns": patterns,
166         },
167     }
168
169
170 def _reason(defect: dict[str, Any], components: list[str], ecr_requirements: set[str]) -> str:
171     parts = []
172     shared = sorted(set(defect.get("linked_requirements") or []) & ecr_requirements)
173     if shared:
174         parts.append(f"linked to {' '.join(shared)}, which this ECR changes")
175     if defect.get("affected_component") in components:
176         parts.append(f"same component ({defect['affected_component']}) as the current change")
177     if defect.get("root_cause_category"):
178         parts.append(f"root cause '{defect['root_cause_category']}'")
179     if defect.get("escaped_to_production"):
180         parts.append("escaped to production")
181     if defect.get("reopen_count"):
182         parts.append(f"reopened {defect['reopen_count']} time(s)")
183     if not parts:
184         return "Textually similar to the change description."
185     return "Matched on " + " "; ".join(parts) + "."

```

backend/app/agents/nodes/impact_assessment.py

```

1  """Correlation step: which components the change touches, and what to do about it.
2
3  Direct impact comes from the code the change resolves to; indirect impact is
4  what the dependency walk shows the change actually propagates to. Each
5  component is listed with the reasons it is in scope, together with concrete
6  recommended actions. No score is computed here.
7  """
8  from __future__ import annotations
9
10 from typing import Any
11
12 from app.agents.nodes.scope import is_scoped, linked_requirement_ids
13 from app.database import session_scope
14 from app.llm.provider import get_llm
15 from app.models import Component, Requirement
16 from app.schemas.impact import ComponentImpact, ImpactAssessment
17
18 # Downstream components below this propagation strength are reachable but not
19 # meaningfully impacted; they stay out of the blast radius and the test scope.
20 INDIRECT_PROPAGATION_FLOOR = 0.25
21
22 CRITICALITY_ORDER = ["LOW", "MEDIUM", "HIGH", "CRITICAL"]
23
24 MITIGATION_RULES: list[tuple[str, str]] = [
25     (
26         "shared_library_change",

```

```

27         "Version the shared library contract and run the consumer regression packs "
28         "(refund and payment) before release.",
29     ),
30     (
31         "schema_migration",
32         "Use expand-migrate-contract sequencing: deploy a nullable column, backfill in "
33         "batches outside the settlement window, then enforce the constraint.",
34     ),
35     (
36         "financial_transaction",
37         "Run the money-movement regression pack and reconcile ledger balances in a "
38         "staging dry run before promoting.",
39     ),
40     (
41         "authentication_change",
42         "Verify the claim survives token refresh and add an explicit step-up bypass test.",
43     ),
44     (
45         "concurrency_change",
46         "Add a concurrent-request test that proves the aggregate lock holds under load.",
47     ),
48     (
49         "third_party_integration",
50         "Configure an explicit timeout and a fail-open policy for the vendor call.",
51     ),
52     (
53         "performance_change",
54         "Measure the query plan before and after, and assert the job budget in CI.",
55     ),
56 ]
57
58
59 def impact_assessment_step(state: dict[str, Any]) -> dict[str, Any]:
60     analysis = state.get("ecr_analysis") or {}
61     requirement_analysis = state.get("requirement_analysis") or {}
62     defect_analysis = state.get("defect_analysis") or {}
63     code_impact = state.get("code_impact") or {}
64     dependency_analysis = state.get("dependency_analysis") or {}
65     skipped = set(state.get("skipped_steps") or [])
66
67     degraded = []
68     if not defect_analysis or "historical_defects" in skipped:
69         degraded.append("defect_analysis")
70     if not dependency_analysis or "dependencies" in skipped:
71         degraded.append("dependency_analysis")
72     if not code_impact or "code_impact" in skipped:
73         degraded.append("code_impact")
74
75     # Direct impact must rest on the strongest evidence available. When the code
76     # analyzer actually resolved the changed files, keyword-derived candidates are
77     # only guesses about topic - including them turns a copy tweak on a page into
78     # a "change to the user service".
79     code_resolved = bool(code_impact.get("impacted_services"))
80     direct_ids = sorted(
81         {
82             *(code_impact.get("impacted_services") or []),
83             *(dependency_analysis.get("seed_components") or []),
84         }
85         if code_resolved
86         else {
87             *(dependency_analysis.get("seed_components") or []),
88             *(analysis.get("candidate_components") or []),
89         }
90     )
91     if not direct_ids:
92         # Nothing resolved from code or named in the text: the components of the
93         # requirements the ECR is explicitly linked to are the next best evidence.
94         linked = list((state.get("ecr_input") or {}).get("linked_requirements") or [])
95         if linked:
96             with session_scope() as db:
97                 direct_ids = sorted(
98                     {
99                         row.component
100                     for row in db.query(Requirement).filter(Requirement.requirement_id.in_(linked))
101                     if row.component
102                 }
103             )
104     propagation = dependency_analysis.get("propagation") or {}
105     # Everything downstream is "reachable"; only what the change actually
106     # propagates to is *impacted*. Without this floor a two-hop async edge drags
107     # an unrelated critical service into the blast radius of a copy tweak.
108     candidate_indirect = {
109         *(code_impact.get("potentially_impacted_services") or []),
110         *(dependency_analysis.get("downstream_components") or []),

```

```

111     } - set(direct_ids)
112     indirect_ids = sorted(
113         component_id
114         for component_id in candidate_indirect
115         if propagation.get(component_id, 1.0) >= INDIRECT_PROPAGATION_FLOOR
116     )
117     weak_links = sorted(candidate_indirect - set(indirect_ids))
118
119     named_by: dict[str, list[str]] = {}
120     ecr = state.get("ecr_input") or {}
121     if is_scoped(ecr):
122         # A record with its own scope: the component of each linked requirement changes
123         # directly, and the other components those requirements name are reached.
124         with session_scope() as db:
125             rows = db.query(Requirement).filter(
126                 Requirement.requirement_id.in_(sorted(linked_requirement_ids(ecr)))
127             ).all()
128             direct_ids = sorted({row.component for row in rows if row.component})
129             for row in rows:
130                 for component_id in row.linked_components or []:
131                     if component_id not in direct_ids:
132                         named_by.setdefault(component_id, []).append(row.requirement_id)
133             indirect_ids = sorted(named_by)
134             weak_links = []
135
136     with session_scope() as db:
137         components = {c.component_id: c.as_dict() for c in db.query(Component).all()}
138
139     distances = dependency_analysis.get("distances") or {}
140     defect_components = set(defect_analysis.get("defect-prone-components") or [])
141
142     def build(component_id: str, impact_type: str) -> ComponentImpact:
143         meta = components.get(component_id, {})
144         reasons: list[str] = []
145         if impact_type == "DIRECT":
146             reasons.append("Directly modified by this change")
147         else:
148             hops = distances.get(component_id)
149             if hops:
150                 reasons.append(f"{hops} dependency hop(s) downstream of the change")
151             if component_id in named_by:
152                 reasons.append(f"Named by {' '.join(named_by[component_id])}")
153             if component_id in defect_components:
154                 reasons.append("Has related past defects")
155             if meta.get("business_criticality") in ("CRITICAL", "HIGH"):
156                 reasons.append(f"Business criticality {meta.get('business_criticality')}")
157             # How strongly the change reaches this component (100 = changed directly).
158             strength = 100.0 if impact_type == "DIRECT" else round(100 * propagation.get(component_id, 0.4), 1)
159             return ComponentImpact(
160                 component_id=component_id,
161                 name=meta.get("name", component_id),
162                 type=meta.get("type", "SERVICE"),
163                 business_criticality=meta.get("business_criticality", "MEDIUM"),
164                 impact_type=impact_type,
165                 impact_score=strength,
166                 distance=0 if impact_type == "DIRECT" else int(distances.get(component_id, 1)),
167                 reasons=reasons,
168             )
169
170     direct = [build(cid, "DIRECT") for cid in direct_ids]
171     indirect = sorted(
172         (build(cid, "INDIRECT") for cid in indirect_ids),
173         key=lambda item: item.impact_score,
174         reverse=True,
175     )
176
177     matched_requirements = requirement_analysis.get("matched_requirements") or []
178     indicators = set(analysis.get("risk_indicators") or [])
179     mitigations = [text for indicator, text in MITIGATION_RULES if indicator in indicators]
180     for concern in (state.get("collaboration_analysis") or {}).get("open_concerns", []):
181         mitigations.append(f"Address the open review concern from {concern['author']}.")
182     if not mitigations:
183         mitigations.append("Run the recommended regression suite and review the results before release.")
184
185     functional = (
186         f"{len(matched_requirements)} requirement(s) are in scope, led by "
187         f"{matched_requirements[0]['requirement_id']} ({matched_requirements[0]['title']})."
188         if matched_requirements
189         else "No requirement could be traced to this change; treat scope as unverified."
190     )
191     technical = (
192         f"{len(direct)} component(s) change directly and {len(indirect)} more are reached "
193         f"through dependencies."
194     )

```

```

195     peak_criticality = max(
196         (c.business_criticality for c in [*direct, *indirect]),
197         key=lambda value: CRITICALITY_ORDER.index(value) if value in CRITICALITY_ORDER else 0,
198         default="MEDIUM",
199     )
200     business = f"The most business-critical affected component is rated {peak_criticality}."
201     baseline = f"{functional} {technical} {business}"
202     summary = get_llm().narrate(
203         task="Summarise which parts of the system this change affects",
204         context={
205             "ecr": state.get("ecr_id"),
206             "direct_components": [c.name for c in direct],
207             "indirect_components": [c.name for c in indirect],
208             "requirements": [r["requirement_id"] for r in matched_requirements[:5]],
209             "highest_business_criticality": peak_criticality,
210         },
211         fallback=baseline,
212     )
213
214     confidence = round(min(0.96, 0.9 - 0.12 * len(degraded)), 3)
215     assessment = ImpactAssessment(
216         directly_impacted_components=direct,
217         indirectly_impacted_components=indirect,
218         functional_impact=functional,
219         technical_impact=technical,
220         business_impact=business,
221         mitigations=mitigations,
222         executive_summary=summary,
223         confidence=confidence,
224         degraded_inputs=degraded,
225         metadata={
226             "propagation_floor": INDIRECT_PROPAGATION_FLOOR,
227             "weakly_coupled_components": weak_links,
228         },
229     )
230     return {
231         "impact_analysis": assessment.model_dump(mode="json"),
232         "tools_used": {"impact_assessment": ["component_impact", "llm.narrate"]},
233         "_confidence": confidence,
234         "_reasoning": baseline,
235         "_summary": {
236             "direct": [c.component_id for c in direct],
237             "indirect": [c.component_id for c in indirect],
238         },
239     }

```

backend/app/agents/nodes/planning.py

```

1  """Part of the *understand_ecr* step: plan the investigation.
2
3  The plan has five fixed steps. Related defects are always gathered and
4  analysed. What varies is the dependency investigation: a copy tweak on a
5  profile page does not pay for it, while a schema change does. Every step and
6  every skipped investigation gets a stated reason.
7  """
8  from __future__ import annotations
9
10 from typing import Any
11
12 from app.agents.tools.registry import tool
13 from app.schemas.common import ChangeType
14
15 STEPS = ["understand_ecr", "gather_evidence", "assess_impact", "select_tests", "summarize"]
16
17 # Change types where the extra investigation is worth its cost
18 DEPENDENCY_HEAVY = {
19     ChangeType.DATABASE.value,
20     ChangeType.API.value,
21     ChangeType.API_AND_BACKEND.value,
22     ChangeType.BACKEND.value,
23     ChangeType.INTEGRATION.value,
24     ChangeType.SECURITY.value,
25     ChangeType.INFRASTRUCTURE.value,
26     ChangeType.MIXED.value,
27 }
28
29 HIGH_SIGNAL_INDICATORS = {
30     "financial_transaction",
31     "schema_migration",
32     "authentication_change",
33     "shared_library_change",
34     "concurrency_change",
35 }
36

```

```

37 INVESTIGATION_LABELS = {
38     "requirements": "impacted requirements",
39     "code_impact": "code impact",
40     "historical_defects": "similar historical defects",
41     "dependencies": "component dependencies",
42     "collaboration_retrieval": "review comments and evidence",
43 }
44
45
46 @tool("build_plan", "Decide which evidence to gather for this change.")
47 def build_plan(
48     analysis: dict[str, Any],
49     options: dict[str, Any],
50     retrieval_plan: dict[str, Any] | None = None,
51 ) -> dict[str, Any]:
52     change_type = analysis.get("change_type", ChangeType.MIXED.value)
53     acronyms = {"ui": "UI", "api": "API"}
54     change_label = " ".join(
55         acronyms.get(word, word) for word in change_type.replace("_", " ").lower().split()
56     )
57     indicators = set(analysis.get("risk_indicators") or [])
58     force_full = bool(options.get("force_full_analysis"))
59
60     by_rule = force_full or change_type in DEPENDENCY_HEAVY or bool(indicators & HIGH_SIGNAL_INDICATORS)
61     # The Retrieval Agent's LLM may add the dependency walk the rules would skip, never remove it.
62     llm_asked = bool((retrieval_plan or {}).get("needs_dependency_check"))
63     run_dependency = by_rule or llm_asked
64
65     # Defects are always analysed: even a small change can bring back an old bug.
66     investigations = ["requirements", "code_impact", "historical_defects"]
67     skipped: list[str] = []
68     reasons: dict[str, str] = {}
69
70     if run_dependency:
71         investigations.append("dependencies")
72         if not by_rule:
73             reasons["dependencies"] = "Requested by the Retrieval Agent's LLM review: " + (
74                 (retrieval_plan or {}).get("dependency_reason") or "the change may reach other components."
75             )
76     else:
77         skipped.append("dependencies")
78         reasons["dependencies"] = "The change stays inside a single presentation component."
79     investigations.append("collaboration_retrieval")
80
81     reasons.update(
82         {
83             "understand_ecr": "Classifies the change and decides which evidence is worth gathering.",
84             "gather_evidence": "Collects "
85             + " ".join(INVESTIGATION_LABELS[key] for key in investigations)
86             + ". ",
87             "assess_impact": "Works out which components the change touches directly and which it reaches t
+ hrough dependencies.",
88             "select_tests": "Keeps only the regression tests that matter, orders them by priority and "
89             "checks whether each related defect could happen again.",
90             "summarize": "Answers in plain language, summarises the defects and publishes the report.",
91         }
92     )
93
94     base = change_label[:1].upper() + change_label[1:]
95     if force_full:
96         routing = "Full analysis was requested, so all evidence is gathered."
97     elif not skipped:
98         routing = f"{base} with {len(indicators)} risk indicator(s) detected. Gathering all evidence."
99     else:
100         routing = (
101             f"{base} with {len(indicators)} risk indicator(s) detected. Not needed: "
102             + " ".join(INVESTIGATION_LABELS[key] for key in skipped)
103             + ". "
104         )
105
106     return {
107         "plan": list(STEPS),
108         "steps": list(STEPS),
109         "investigations": investigations,
110         "skipped": skipped,
111         "reasons": reasons,
112         "routing_reason": routing,
113         "rationale": [reasons[key] for key in [*STEPS, *skipped]],
114         "checkpoint": bool(options.get("human_in_the_loop")),
115     }
116
117
118 def planning_step(state: dict[str, Any]) -> dict[str, Any]:
119     analysis = state.get("ecr_analysis") or {}

```

```

120     options = state.get("options") or {}
121     decision = build_plan(analysis, options, state.get("retrieval_plan"))
122
123     from app.agents.runtime import registry
124
125     run = registry.get(state.get("workflow_id", ""))
126     if run:
127         run.plan = list(decision["steps"])
128         run.skipped = decision["skipped"]
129
130     return {
131         "plan": decision["steps"],
132         "plan_detail": {
133             "steps": decision["steps"],
134             "investigations": decision["investigations"],
135             "skipped": decision["skipped"],
136             "reasons": decision["reasons"],
137             "routing_reason": decision["routing_reason"],
138             "approval_checkpoint": decision["checkpoint"],
139             "source": "policy",
140         },
141         "skipped_steps": decision["skipped"],
142         "tools_used": {"planning": ["build_plan"]},
143         "_confidence": 0.95,
144         "_reasoning": decision["routing_reason"],
145     }

```

backend/app/agents/nodes/regression_tests.py

```

1  """Test discovery, selection and prioritisation steps.
2
3  Discovery casts a wide net through three channels; selection then scores every
4  candidate with the explainable algorithm and drops what does not earn its place;
5  prioritisation orders what remains for the fastest feedback.
6  """
7  from __future__ import annotations
8
9  from typing import Any
10
11  from app.agents.tools.test_tools import (
12      baseline_duration,
13      resolve_domain,
14      semantic_test_search,
15      tests_by_component,
16      tests_by_requirement,
17      total_test_inventory,
18  )
19  from app.agents.nodes.scope import is_scoped, linked_requirement_ids, listed_test_ids
20  from app.database import session_scope
21  from app.llm.provider import get_llm
22  from app.models import Component, TestCase
23  from app.schemas.test_case import (
24      SelectedTest,
25      TestCaseRead,
26      TestDiscovery,
27      TestPrioritization,
28      TestSelection,
29  )
30  from app.services.test_selection_service import (
31      SelectionContext,
32      get_selection_engine,
33      priority_distribution,
34  )
35
36
37  def test_discovery_step(state: dict[str, Any]) -> dict[str, Any]:
38      ecr = state["ecr_input"]
39      analysis = state.get("ecr_analysis") or {}
40      impact = state.get("impact_analysis") or {}
41      requirements = state.get("requirements") or []
42
43      requirement_ids = [r["requirement_id"] for r in requirements]
44      component_ids = sorted(
45          {
46              *(c["component_id"] for c in impact.get("directly_impacted_components") or []),
47              *(c["component_id"] for c in impact.get("indirectly_impacted_components") or []),
48          }
49      )
50      query = f"{ecr.get('title', '')}. {ecr.get('description', '')}"
51      keywords = analysis.get("technical_keywords") or []
52
53      listed = listed_test_ids(ecr)
54      scoped = is_scoped(ecr) # the ECR's own requirements and listed tests only
55      with session_scope() as db:

```

```

56     by_listing = (
57         [row.as_dict() for row in db.query(TestCase).filter(TestCase.test_case_id.in_(listed))]
58         if listed
59         else []
60     )
61     by_requirement = tests_by_requirement(
62         db, sorted(linked_requirement_ids(ecr)) if scoped else requirement_ids
63     )
64     by_component = [] if scoped else tests_by_component(db, component_ids)
65     domain = resolve_domain(
66         db, component_ids, requirement_ids, list(ecr.get("linked_requirements") or [])
67     )
68     total_available = total_test_inventory(db, domain)
69     baseline_minutes = baseline_duration(db, domain)
70     semantic_hits = semantic_test_search(query, keywords=keywords, k=80)
71     by_semantic = [] if scoped else semantic_hits # in scope, similarity only scores
72
73     candidates: dict[str, dict[str, Any]] = {}
74     raw_semantic = {
75         payload["test_case_id"]: float(payload.get("semantic_score", 0.0)) for payload in semantic_hits
76     }
77     # Hybrid retrieval scores saturate well below 100 on short enterprise
78     # artefacts, so normalise against the best hit in this candidate set. The
79     # factor then measures "how close to the best semantic match is this test",
80     # which is what the P0..P3 bands are calibrated against.
81     best_semantic = max(raw_semantic.values(), default=0.0)
82     semantic_scores = {
83         test_id: round(100.0 * score / best_semantic, 1) if best_semantic else 0.0
84         for test_id, score in raw_semantic.items()
85     }
86     for payload in by_listing: # named on the ECR: always a candidate
87         candidates.setdefault(payload["test_case_id"], dict(payload))
88     for payload in [*by_requirement, *by_component, *by_semantic]:
89         test_id = payload.get("test_case_id")
90         if not test_id or payload.get("retired"):
91             continue
92         if domain and payload.get("domain") and payload["domain"] != domain:
93             continue # never pull another product line into this suite
94         candidates.setdefault(test_id, dict(payload))
95
96     channels = {
97         "listed_on_ecr": len(by_listing),
98         "requirement_trace": len(by_requirement),
99         "component_coverage": len(by_component),
100         "semantic_search": len(by_semantic),
101         "unique_candidates": len(candidates),
102     }
103     confidence = round(min(0.95, 0.5 + 0.3 * bool(by_requirement) + 0.15 * bool(candidates)), 3)
104     reasoning = (
105         f"Discovered {len(candidates)} unique candidate test(s) out of {total_available} "
106         f"{domain or 'catalogue'} test(s): {channels['requirement_trace']} via requirement traceability, "
107         f"{channels['component_coverage']} via component coverage and "
108         f"{channels['semantic_search']} via semantic search."
109     )
110
111     discovery = TestDiscovery(
112         candidate_tests=[
113             TestCaseRead(**{k: v for k, v in payload.items() if k in TestCaseRead.model_fields})
114             for payload in candidates.values()
115         ],
116         total_available=total_available,
117         discovery_channels=channels,
118         confidence=confidence,
119         reasoning=reasoning,
120     )
121     return {
122         "discovered_tests": list(candidates.values()),
123         "test_discovery": {
124             **discovery.model_dump(mode="json"),
125             "semantic_scores": semantic_scores,
126             "baseline_duration_minutes": baseline_minutes,
127             "domain": domain,
128         },
129         "tools_used": {
130             "test_discovery": [
131                 "tests_by_requirement",
132                 "tests_by_component",
133                 "semantic_test_search",
134                 "total_test_inventory",
135             ]
136         },
137         "_confidence": confidence,
138         "_reasoning": reasoning,
139         "_summary": {

```

```

140         "candidates": len(candidates),
141         "total_available": total_available,
142         "domain": domain or "all",
143         **channels,
144     },
145 }
146
147
148 def _selection_context(state: dict[str, Any]) -> SelectionContext:
149     impact = state.get("impact_analysis") or {}
150     dependency = state.get("dependency_analysis") or {}
151     defect = state.get("defect_analysis") or {}
152     discovery = state.get("test_discovery") or {}
153     analysis = state.get("ecr_analysis") or {}
154
155     with session_scope() as db:
156         criticality = {c.component_id: c.business_criticality for c in db.query(Component).all()}
157
158     return SelectionContext(
159         matched_requirements={
160             r["requirement_id"]: float(r.get("relevance", 0.0))
161             for r in state.get("requirements") or []
162         },
163         direct_components={
164             c["component_id"] for c in impact.get("directly_impacted_components") or []
165         },
166         indirect_components={
167             c["component_id"] for c in impact.get("indirectly_impacted_components") or []
168         },
169         dependency_distance={k: int(v) for k, v in (dependency.get("distances") or {}).items()},
170         propagation={k: float(v) for k, v in (dependency.get("propagation") or {}).items()},
171         component_criticality=criticality,
172         defect_components=set(defect.get("defect_prone_components") or []),
173         semantic_scores={k: float(v) for k, v in (discovery.get("semantic_scores") or {}).items()},
174         affected_features=set(analysis.get("affected_features") or []),
175         listed_tests=set(listed_test_ids(state.get("ecr_input") or {})),
176     )
177
178
179 def test_selection_step(state: dict[str, Any]) -> dict[str, Any]:
180     discovered = state.get("discovered_tests") or []
181     discovery = state.get("test_discovery") or {}
182     context = _selection_context(state)
183     engine = get_selection_engine()
184     selected, rejected = engine.select(discovered, context)
185
186     total_available = int(discovery.get("total_available", 0) or 0)
187     baseline_minutes = float(discovery.get("baseline_duration_minutes", 0.0) or 0.0)
188     estimated = round(sum(t.average_execution_time for t in selected) / 60.0, 1)
189     reduction = round(100 * (1 - len(selected) / total_available), 1) if total_available else 0.0
190
191     confidence = round(min(0.95, 0.55 + 0.25 * bool(selected) + 0.15 * bool(context.matched_requirements)),
192 + 3)
193     reasoning = (
194         f"Scored {len(discovered)} candidate(s) against the impact model and selected "
195         f"{len(selected)} test(s) out of a {total_available}-test regression suite "
196         f"({reduction:.1f}% reduction). Estimated execution {estimated:.0f} minutes versus "
197         f"{baseline_minutes:.0f} minutes for the full suite."
198     )
199
200     selection = TestSelection(
201         selected_tests=selected,
202         excluded_count=len(rejected),
203         total_available=total_available,
204         total_candidates=len(discovered),
205         reduction_percentage=reduction,
206         estimated_duration_minutes=estimated,
207         baseline_duration_minutes=baseline_minutes,
208         priority_distribution=priority_distribution(selected),
209         confidence=confidence,
210         reasoning=reasoning,
211     )
212     return {
213         "selected_tests": [t.model_dump(mode="json") for t in selected],
214         "test_selection": selection.model_dump(mode="json"),
215         "tools_used": {"test_selection": ["selection_engine.select"]},
216         "_confidence": confidence,
217         "_reasoning": reasoning,
218         "_summary": {
219             "selected": len(selected),
220             "total_available": total_available,
221             "reduction_percentage": reduction,
222             "priority_distribution": priority_distribution(selected),
223         },
224     },

```

```

223     }
224
225
226 def test_prioritization_step(state: dict[str, Any]) -> dict[str, Any]:
227     selected = [SelectedTest.model_validate(t) for t in state.get("selected_tests") or []]
228     context = _selection_context(state)
229     ordered = get_selection_engine().prioritise(selected, context)
230
231     waves: dict[str, list[str]] = {"P0": [], "P1": [], "P2": [], "P3": []}
232     for test in ordered:
233         waves[test.priority.value].append(test.test_case_id)
234     total_minutes = round(sum(t.average_execution_time for t in ordered) / 60.0, 1)
235     critical_minutes = round(
236         sum(t.average_execution_time for t in ordered if t.priority.value in ("P0", "P1")) / 60.0, 1
237     )
238
239     strategy = (
240         "Priority first: highest priority band, then the most business critical component, then "
241         "relevance, then shortest runtime so failures surface early."
242     )
243     confidence = round(min(0.95, 0.6 + 0.3 * bool(ordered)), 3)
244     baseline = (
245         f"Ordered {len(ordered)} test(s): {len(waves['P0'])} P0, {len(waves['P1'])} P1, "
246         f"{len(waves['P2'])} P2, {len(waves['P3'])} P3. P0 and P1 complete in about "
247         f"{critical_minutes:.0f} minutes, the whole selection in {total_minutes:.0f} minutes."
248     )
249     reasoning = get_llm().narrate(
250         task="Explain the recommended regression execution order",
251         context={
252             "waves": {k: v[:6] for k, v in waves.items()},
253             "critical_minutes": critical_minutes,
254             "total_minutes": total_minutes,
255             "strategy": strategy,
256         },
257         fallback=baseline,
258     )
259
260     prioritization = TestPrioritization(
261         prioritized_tests=ordered,
262         waves=waves,
263         estimated_duration_minutes=total_minutes,
264         critical_path_minutes=critical_minutes,
265         strategy=strategy,
266         confidence=confidence,
267         reasoning=reasoning,
268     )
269     return {
270         "prioritized_tests": [t.model_dump(mode="json") for t in ordered],
271         "test_prioritization": prioritization.model_dump(mode="json"),
272         "tools_used": {"test_prioritization": ["selection_engine.prioritise"]},
273         "_confidence": confidence,
274         "_reasoning": reasoning,
275         "_summary": {
276             "waves": {k: len(v) for k, v in waves.items()},
277             "critical_path_minutes": critical_minutes,
278             "estimated_duration_minutes": total_minutes,
279         },
280     }

```

backend/app/agents/nodes/requirements.py

```

1  """Retrieval Agent step: impacted requirements.
2
3  Three retrieval channels are fused so an identifier-heavy enterprise corpus is
4  not left to dense similarity alone:
5      1. explicit traceability (ECR -> requirement links, component -> requirement),
6      2. hybrid semantic search over the requirement corpus,
7      3. literal keyword search.
8  """
9  from __future__ import annotations
10
11  from typing import Any
12
13  from app.agents.nodes.retrieval_planning import search_query
14  from app.agents.tools.requirement_tools import (
15      keyword_requirement_search,
16      semantic_requirement_search,
17      trace_requirements,
18  )
19  from app.agents.nodes.scope import is_scoped, linked_requirement_ids
20  from app.database import session_scope
21  from app.llm.provider import get_llm
22  from app.models import Requirement
23  from app.schemas.impact import RequirementAnalysis, RequirementMatch

```

```

24
25 MATCH_WEIGHT = {"TRACEABILITY": 1.0, "COMPONENT_LINK": 0.85, "SEMANTIC": 0.9, "KEYWORD": 0.8}
26 MIN_RELEVANCE = 45.0
27 MAX_MATCHES = 10
28
29
30 def requirement_step(state: dict[str, Any]) -> dict[str, Any]:
31     ecr = state["ecr_input"]
32     analysis = state.get("ecr_analysis") or {}
33     keywords = analysis.get("technical_keywords") or []
34     query = search_query(state) # widened by the Retrieval Agent's LLM search plan
35
36     scoped = is_scoped(ecr)
37     with session_scope() as db:
38         traced = trace_requirements(
39             db,
40             ecr.get("linked_requirements") or [],
41             analysis.get("candidate_components") or [],
42         )
43         keyword_hits = [] if scoped else keyword_requirement_search(db, keywords)
44         searched_total = db.query(Requirement).count()
45         semantic_hits = [] if scoped else semantic_requirement_search(query, keywords=keywords, k=12)
46         if scoped: # only the requirements the change record itself links
47             wanted = linked_requirement_ids(ecr)
48             traced = [payload for payload in traced if payload.get("requirement_id") in wanted]
49
50         merged: dict[str, dict[str, Any]] = {}
51         for payload in [*traced, *semantic_hits, *keyword_hits]:
52             requirement_id = payload.get("requirement_id")
53             if not requirement_id:
54                 continue
55             candidate = dict(payload)
56             candidate["relevance"] = round(
57                 float(payload.get("relevance", 0.0))
58                 * MATCH_WEIGHT.get(payload.get("match_type", "SEMANTIC"), 0.8),
59                 1,
60             )
61             existing = merged.get(requirement_id)
62             if existing is None:
63                 merged[requirement_id] = candidate
64                 continue
65             # a requirement found through several channels is more certain
66             best, other = (
67                 (existing, candidate)
68                 if existing["relevance"] >= candidate["relevance"]
69                 else (candidate, existing)
70             )
71             best = dict(best)
72             best["relevance"] = round(min(100.0, best["relevance"] + 0.25 * other["relevance"]), 1)
73             best["match_type"] = f"{best.get('match_type')}+{other.get('match_type')}"
74             merged[requirement_id] = best
75
76         matches: list[RequirementMatch] = []
77         for payload in merged.values():
78             if payload["relevance"] < MIN_RELEVANCE:
79                 continue
80             matches.append(
81                 RequirementMatch(
82                     requirement_id=payload["requirement_id"],
83                     title=payload.get("title", ""),
84                     description=payload.get("description", ""),
85                     business_domain=payload.get("business_domain", ""),
86                     priority=payload.get("priority", "MEDIUM"),
87                     component=payload.get("component", ""),
88                     linked_components=list(payload.get("linked_components") or []),
89                     features=list(payload.get("features") or []),
90                     relevance=payload["relevance"],
91                     match_type=payload.get("match_type", "SEMANTIC"),
92                     reason=_reason(payload),
93                 )
94             )
95         matches.sort(key=lambda m: m.relevance, reverse=True)
96         matches = matches[:MAX_MATCHES]
97
98         impacted_components = sorted(
99             {c for match in matches for c in [match.component, *match.linked_components] if c}
100         )
101         confidence = _confidence(matches)
102         baseline = (
103             f"Searched {searched_total} requirements across traceability, semantic and keyword "
104             f"channels and matched {len(matches)}. Strongest match is "
105             f"{matches[0].requirement_id} ({matches[0].title}) at {matches[0].relevance:.0f}% relevance."
106             if matches
107             else f"Searched {searched_total} requirements and found no match above the {MIN_RELEVANCE:.0f}% thr

```

```

+ eshold."
108     )
109     reasoning = get_llm().narrate(
110         task="Explain which requirements are impacted by this change and why",
111         context={
112             "ecr": ecr.get("ecr_id"),
113             "matches": [
114                 {"id": m.requirement_id, "title": m.title, "relevance": m.relevance, "via": m.match_type}
115                 for m in matches[:5]
116             ],
117         },
118         fallback=baseline,
119     )
120
121     result = RequirementAnalysis(
122         matched_requirements=matches,
123         impacted_components=impacted_components,
124         searched_count=searched_total,
125         confidence=confidence,
126         reasoning=reasoning,
127     )
128     return {
129         "requirements": [m.model_dump(mode="json") for m in matches],
130         "requirement_analysis": result.model_dump(mode="json"),
131         "tools_used": {
132             "requirements": [
133                 "trace_requirements",
134                 "semantic_requirement_search",
135                 "keyword_requirement_search",
136             ]
137         },
138         "_confidence": confidence,
139         "_reasoning": reasoning,
140         "_summary": {
141             "matched": len(matches),
142             "searched": searched_total,
143             "top": [m.requirement_id for m in matches[:5]],
144         },
145     }
146
147
148 def _reason(payload: dict[str, Any]) -> str:
149     match_type = payload.get("match_type", "SEMANTIC")
150     if "TRACEABILITY" in match_type:
151         return "Explicitly linked to the ECR in the change record."
152     if "COMPONENT_LINK" in match_type:
153         return f"Traced through component {payload.get('component')} identified in the change."
154     if "KEYWORD" in match_type:
155         terms = payload.get("matched_terms") or []
156         return f"Literal match on change keywords: {' '.join(terms[:4])}."
157     return "Strong semantic similarity between the change description and the requirement text."
158
159
160 def _confidence(matches: list[RequirementMatch]) -> float:
161     if not matches:
162         return 0.35
163     top = matches[0].relevance / 100.0
164     breadth = min(1.0, len(matches) / 5.0)
165     traced = any("TRACEABILITY" in m.match_type for m in matches)
166     return round(min(0.97, 0.45 + 0.35 * top + 0.12 * breadth + (0.08 if traced else 0.0)), 3)

```

backend/app/agents/nodes/retrieval_planning.py

```

1  """Part of the *understand_ecr* step: the Retrieval Agent's LLM plans the search.
2
3  Rules only find what the ECR literally says. Before anything is fetched, the
4  model reads the change and names what else to look for - synonyms, the
5  behaviour that could break, related functions - and decides whether the dependency walk
6  is needed. Its search terms drive the requirement and defect searches in
7  *gather_evidence*, so the LLM changes what is retrieved, not just how it reads.
8
9  Guardrails: the model never supplies identifiers or scores. Its terms only
10 widen the semantic query (the relevance threshold still decides what is kept),
11 and it can only ask for *more* investigation, never skip one.
12 """
13 from __future__ import annotations
14
15 import json
16 import re
17 from typing import Any
18
19 from pydantic import BaseModel, Field
20
21 from app.llm.provider import get_llm

```

```

22 from app.prompts import RETRIEVAL_SYSTEM_PROMPT, RETRIEVAL_TASK
23
24 MAX_TERMS = 10
25 MAX_BEHAVIOURS = 5
26 _IDENTIFIER = re.compile(r"[a-z]{1,6}-?\d[\w-]*")
27
28
29 class RetrievalPlan(BaseModel):
30     summary: str = ""
31     search_terms: list[str] = Field(default_factory=list)
32     could_break: list[str] = Field(default_factory=list)
33     needs_dependency_check: bool = False
34     dependency_reason: str = ""
35     source: str = "llm"
36
37
38 def _clean(values: list[str], limit: int) -> list[str]:
39     """Short, lower-case, de-duplicated phrases; identifiers are not search terms."""
40     kept: list[str] = []
41     for value in values or []:
42         term = " ".join(str(value).split()).strip(".,;:").lower()[:60]
43         if len(term) < 3 or _IDENTIFIER.fullmatch(term) or term in kept:
44             continue
45         kept.append(term)
46     return kept[:limit]
47
48
49 def search_query(state: dict[str, Any]) -> str:
50     """The query the evidence searches run: the ECR text plus the LLM's search terms."""
51     ecr = state.get("ecr_input") or {}
52     base = f"{ecr.get('title', '')}. {ecr.get('description', '')}"
53     plan = state.get("retrieval_plan") or {}
54     terms = plan.get("search_terms") or []
55     if plan.get("source") != "llm" or not terms:
56         return base
57     return f"{base} Related: {'; '.join(terms)}."
58
59
60 def retrieval_planning_step(state: dict[str, Any]) -> dict[str, Any]:
61     ecr = state["ecr_input"]
62     analysis = dict(state.get("ecr_analysis") or {})
63     fallback = {
64         "summary": analysis.get("summary", ""),
65         "search_terms": list(analysis.get("technical_keywords") or [])[:MAX_TERMS],
66         "could_break": [],
67         "needs_dependency_check": False,
68         "dependency_reason": "",
69         "source": "rules",
70     }
71
72     llm = get_llm()
73     if llm.is_mock:
74         plan = RetrievalPlan.model_validate(fallback)
75     else:
76         plan = llm.generate_structured(
77             RETRIEVAL_TASK.format(
78                 ecr_id=ecr.get("ecr_id"),
79                 title=ecr.get("title"),
80                 description=ecr.get("description"),
81                 changed_files=json.dumps(ecr.get("changed_files") or []),
82                 linked_requirements=json.dumps(ecr.get("linked_requirements") or []),
83                 classification=json.dumps(
84                     {
85                         "change_type": analysis.get("change_type"),
86                         "business_domain": analysis.get("business_domain"),
87                         "keywords": analysis.get("technical_keywords"),
88                         "candidate_components": analysis.get("candidate_components"),
89                         "risk_indicators": analysis.get("risk_indicators"),
90                     }
91                 ),
92                 max_terms=MAX_TERMS,
93                 max_behaviours=MAX_BEHAVIOURS,
94             ),
95             RetrievalPlan,
96             system=RETRIEVAL_SYSTEM_PROMPT,
97             fallback=fallback,
98         )
99
100     used_llm = plan.source != "rules"
101     terms = _clean(plan.search_terms, MAX_TERMS)
102     behaviours = _clean(plan.could_break, MAX_BEHAVIOURS)
103     summary = " ".join(plan.summary.split()) or analysis.get("summary", "")
104     wants_dependencies = used_llm and plan.needs_dependency_check
105

```

```

106     if used_llm:
107         reasoning = (
108             f"The LLM planned the search with {len(terms)} term(s)"
109             + (f" ({', '.join(terms[:4])})" if terms else "")
110             + (f" and flagged {len(baviours)} behaviour(s) that could break." if behaviours else ".")
111             + (" It asked for the dependency walk." if wants_dependencies else "")
112         )
113     else:
114         reasoning = "LLM unavailable, so the search uses the rule-based keywords only."
115
116     return {
117         # Only the language field changes; classification and numbers stay rule-based.
118         "ecr_analysis": {**analysis, "summary": summary} if used_llm else analysis,
119         "retrieval_plan": {
120             "source": "llm" if used_llm else "rules",
121             "summary": summary,
122             "search_terms": terms,
123             "could_break": behaviours,
124             "needs_dependency_check": wants_dependencies,
125             "dependency_reason": " ".join(plan.dependency_reason.split()) if wants_dependencies else "",
126         },
127         "tools_used": {"retrieval_planning": [llm.plan_retrieval] if used_llm else ["rule_keywords"]},
128         "_reasoning": reasoning,
129     }

```

backend/app/agents/nodes/scope.py

```

1  """What belongs to an ECR.
2
3  Every change record names its own scope: the requirements it links and the
4  test cases it lists as affected. The analysis uses only that scope - the
5  linked requirements, the listed tests (and the tests traced to those
6  requirements), the defects raised against those requirements, the components
7  those requirements name, and the comments and evidence attached to the ECR
8  itself. There is no keyword, semantic or component-wide widening, so the page
9  shows only data that belongs to that ECR.
10
11  Similarity is still used to score and order what is in scope; it never adds
12  anything. An ECR created without any linked requirement or test falls back to
13  the broader search, because it has no scope of its own yet.
14  """
15  from __future__ import annotations
16
17  from typing import Any
18
19
20  def is_scoped(ecr: dict[str, Any] | None) -> bool:
21      ecr = ecr or {}
22      return bool(ecr.get("affected_test_cases") or ecr.get("linked_requirements"))
23
24
25  def listed_test_ids(ecr: dict[str, Any] | None) -> list[str]:
26      """Affected test cases named on the ECR ("L2R26: TC2" -> "L2R26:TC2")."""
27      return [
28          "".join(str(value).split())
29          for value in (ecr or {}).get("affected_test_cases") or []
30          if str(value).strip()
31      ]
32
33
34  def linked_requirement_ids(ecr: dict[str, Any] | None) -> set[str]:
35      """Linked requirements, plus the requirement named in an "L2R26:TC1" style test id."""
36      linked = {str(value).strip() for value in (ecr or {}).get("linked_requirements") or [] if str(value).strip()}
37      linked |= {test_id.split(":", 1)[0] for test_id in listed_test_ids(ecr) if ":" in test_id}
38      return linked

```

backend/app/agents/nodes/summarization.py

```

1  """Summarization steps: the plain answer, then the full report.
2
3  The answer is what replaces "read every comment and system by hand": it is
4  generated *only* from the correlated bundle, and it always ships with the
5  citations it was built from.
6  """
7  from __future__ import annotations
8
9  import json
10  from typing import Any
11
12  from pydantic import BaseModel
13
14  from app.llm.provider import get_llm
15  from app.prompts import (

```

```

16     ANSWER_SYSTEM_PROMPT,
17     SUMMARY_DEFECT_INSTRUCTION,
18     SUMMARY_NO_DEFECT_INSTRUCTION,
19     SUMMARY_STYLE,
20     SUMMARY_TASK,
21 )
22 from app.services.report_service import build_report
23
24 DEFAULT_QUESTION = (
25     "What do I need to know about this change, and what should we test?"
26 )
27
28
29 def build_answer_context(state: dict[str, Any]) -> dict[str, Any]:
30     """The grounded, citable context handed to the model (and to the API)."""
31     impact = state.get("impact_analysis") or {}
32     selection = state.get("test_selection") or {}
33     collaboration = state.get("collaboration_analysis") or {}
34     signals = collaboration.get("signals") or {}
35     review = state.get("correlation_review") or {}
36     return {
37         "ecr": {
38             "id": state.get("ecr_id"),
39             "title": (state.get("ecr_input") or {}).get("title"),
40             "description": (state.get("ecr_input") or {}).get("description"),
41             "change_type": (state.get("ecr_analysis") or {}).get("change_type"),
42             "business_domain": (state.get("ecr_analysis") or {}).get("business_domain"),
43         },
44         "requirements": [
45             {
46                 "id": r["requirement_id"],
47                 "title": r["title"],
48                 "relevance": r["relevance"],
49                 "why": r["reason"],
50             }
51             for r in (state.get("requirements") or [])[:8]
52         ],
53         "historical_defects": _defect_context(state),
54         "decisions": [
55             {"by": item["author"], "text": item["text"], "ref": item["comment_id"]}
56             for item in signals.get("decisions", [])[:5]
57         ],
58         "raised_concerns": [
59             {"by": item["author"], "text": item["text"], "ref": item["comment_id"]}
60             for item in signals.get("risks", [])[:5]
61         ],
62         "evidence": [
63             {
64                 "id": e["evidence_id"],
65                 "title": e["title"],
66                 "type": e["evidence_type"],
67                 "outcome": e["outcome"],
68             }
69             for e in (state.get("evidence") or [])[:6]
70         ],
71         "evidence_gaps": collaboration.get("evidence_gaps", []),
72         # What the other two agents' LLM reasoning concluded.
73         "could_break": (state.get("retrieval_plan") or {}).get("could_break", []),
74         "correlation_findings": {
75             "summary": review.get("summary", ""),
76             "contradictions": review.get("conflicts", []),
77             "gaps": review.get("gaps", []),
78             "key_links": review.get("key_links", []),
79         },
80         "impact_summary": {
81             "direct_components": len(impact.get("directly_impacted_components") or []),
82             "indirect_components": len(impact.get("indirectly_impacted_components") or []),
83         },
84         "human_approval": _approval_context(state),
85         "impacted_components": [
86             {"id": c["component_id"], "name": c["name"], "impact": c["impact_type"]}
87             for c in [
88                 *(impact.get("directly_impacted_components") or []),
89                 *(impact.get("indirectly_impacted_components") or []),
90             ][:10]
91         ],
92         "tests": {
93             "total_available": selection.get("total_available"),
94             "recommended": len(selection.get("selected_tests") or []),
95             "reduction_percentage": selection.get("reduction_percentage"),
96             "priority_distribution": selection.get("priority_distribution"),
97             "top": [
98                 {
99                     "id": t["test_case_id"],

```

```

100         "title": t["title"],
101         "priority": t["priority"],
102         "relevance": t["relevance_score"],
103         "why": t["reason"],
104     }
105     for t in (selection.get("selected_tests") or []):8]
106 ],
107 },
108 }
109
110
111 def _defect_context(state: dict[str, Any]) -> list[dict[str, Any]]:
112     analysed = {row["defect_id"]: row for row in (state.get("defect_insights") or {}).get("defects") or []}
113     out = []
114     for d in (state.get("defects") or []):6]:
115         row = analysed.get(d["defect_id"], {})
116         out.append(
117             {
118                 "id": d["defect_id"],
119                 "title": d["title"],
120                 "severity": d["severity"],
121                 "root_cause": d["root_cause"],
122                 "why": row.get("why_it_matters") or d["reason"],
123                 "chance_it_happens_again": row.get("chance_of_recurrence"),
124                 "covered_by_tests": row.get("covering_tests", []),
125             }
126         )
127     return out
128
129
130 def _baseline_answer(context: dict[str, Any]) -> str:
131     ecr = context["ecr"]
132     impact = context["impact_summary"]
133     tests = context["tests"]
134     parts = [
135         f"{ecr['id']} ({ecr['title']}) is a "
136         f"{str(ecr.get('change_type') or 'change').replace('_', ' ').lower()} in the "
137         f"{ecr.get('business_domain')} domain. It changes {impact['direct_components']} "
138         f"component(s) directly and reaches {impact['indirect_components']} more through "
139         f"dependencies."
140     ]
141     if context["requirements"]:
142         parts.append(
143             "It touches "
144             + ", ".join(f"{r['id']} ({r['title']})" for r in context["requirements"][:3])
145             + "."
146         )
147     if context["historical_defects"]:
148         parts.append(
149             "History is relevant here: "
150             + ", ".join(
151                 f"{d['id']} ({d['title']})" for d in context["historical_defects"][:3]
152             )
153             + "."
154         )
155     if context["decisions"]:
156         parts.append("Decisions already recorded: " + context["decisions"][0]["text"])
157     if context["raised_concerns"]:
158         parts.append("Open concern: " + context["raised_concerns"][0]["text"])
159     contradictions = context["correlation_findings"]["contradictions"]
160     if contradictions:
161         parts.append("To resolve: " + contradictions[0]["text"])
162     if tests.get("recommended"):
163         distribution = tests.get("priority_distribution") or {}
164         parts.append(
165             f"Recommended regression: {tests['recommended']} of {tests['total_available']} tests "
166             f"f({tests['reduction_percentage']}% reduction), "
167             f"f({distribution.get('P0', 0)} P0 and {distribution.get('P1', 0)} P1."
168         )
169     if context["evidence_gaps"]:
170         parts.append(
171             "Evidence is missing for: " + ", ".join(context["evidence_gaps"][:4]) + "."
172         )
173     approval_sentence = _approval_sentence(context.get("human_approval") or {})
174     if approval_sentence:
175         parts.append(approval_sentence)
176     return " ".join(parts)
177
178
179 def _approval_context(state: dict[str, Any]) -> dict[str, Any]:
180     """Whether a person was asked to approve this analysis, and what they decided."""
181     approval = state.get("approval") or {}
182     if not approval.get("required"):
183         return {"requested": False}

```

```

184     return {
185         "requested": True,
186         "status": approval.get("status", ""),
187         "decided_by": approval.get("decided_by", ""),
188         "comment": approval.get("comment", ""),
189         "excluded_tests": approval.get("excluded_test_ids", []),
190     }
191
192
193 def _approval_sentence(approval: dict[str, Any]) -> str:
194     if not approval.get("requested"):
195         return ""
196     who = approval.get("decided_by") or "The reviewer"
197     status = approval.get("status")
198     if status == "APPROVED":
199         excluded = approval.get("excluded_tests") or []
200         return f"{who} approved this analysis" + (
201             f" and removed {' '.join(excluded)} from the recommendation." if excluded else "."
202         )
203     if status == "REJECTED":
204         return f"{who} rejected this analysis, so the test recommendation is on hold."
205     if status == "TIMED_OUT":
206         return "Approval was requested but no decision arrived in time."
207     return ""
208
209
210 class SummaryReply(BaseModel):
211     """The Summarization Agent's one LLM reply: the answer and the defect summary together."""
212
213     answer: str = ""
214     defect_summary: str = ""
215
216
217 def _baseline_defect_summary(ecr_id: str, insights: dict[str, Any]) -> str:
218     rows = insights.get("defects") or []
219     history = insights.get("component_history") or []
220     open_now = insights.get("open_defects") or []
221     open_sentence = (
222         "Still open in the affected area: "
223         + "; ".join(f"{d['defect_id']} ({d['title']}, {d['severity'].lower()})" for d in open_now[:3])
224         + "."
225     )
226     if open_now:
227         else ""
228     if not rows:
229         if not history:
230             if insights.get("scoped"):
231                 return f"No past defects are linked to the requirements or test cases of {ecr_id}." + (
232                     f" {open_sentence}" if open_sentence else ""
233                 )
234             return f"No defects are recorded for {ecr_id} or the components it touches." + (
235                 f" {open_sentence}" if open_sentence else ""
236             )
237         total = sum(h["total"] for h in history)
238         latest = min(
239             (h for h in history if h["last_seen_days"] is not None),
240             key=lambda h: h["last_seen_days"],
241             default=history[0],
242         )
243         busiest = max(history, key=lambda h: h["total"])
244         parts = [
245             f"No past defect matches {ecr_id} directly, but the components it touches have "
246             f"{total} recorded defect(s)."
247         ]
248         if latest["last_seen_days"] is not None:
249             parts.append(f"The most recent was {latest['last_seen_days']} days ago on {latest['component_name']}")
250         if busiest["top_cause"]:
251             parts.append(
252                 f"{busiest['component_name']} has the most ({busiest['total']}), usually caused by "
253                 f"{busiest['top_cause'][:1].lower() + busiest['top_cause'][1:]}"
254             )
255         if open_sentence:
256             parts.append(open_sentence)
257         return " ".join(parts)
258     parts = [f"{len(rows)} past defect(s) are related to {ecr_id}."]
259     high = [row for row in rows if row["chance_of_recurrence"] == "High"]
260     medium = [row for row in rows if row["chance_of_recurrence"] == "Medium"]
261     if high:
262         parts.append(
263             "These could happen again with this change: "
264             + "; ".join(f"{row['defect_id']} ({row['title']})" for row in high[:3])
265             + "."
266         )

```

```

267     elif medium:
268         parts.append(
269             "None is likely to come back, but keep an eye on "
270             + ", ".join(row["defect_id"] for row in medium[:3])
271             + "."
272         )
273     else:
274         parts.append("None of them is likely to happen again, because this change does not touch their code
+ ."")
275     # Lower-case the first letter only, and never an acronym ("UI error handling").
276     causes = [
277         cause if cause[:2].isupper() else cause[:1].lower() + cause[1:]
278         for cause in (insights.get("common_causes") or [])[:2]
279     ]
280     if causes and (high or medium):
281         parts.append(
282             ("The most common causes were " if len(causes) > 1 else "The most common cause was ")
283             + " and ".join(causes)
284             + "."
285         )
286     uncovered = insights.get("not_covered") or []
287     if uncovered:
288         verb = "is" if len(uncovered) == 1 else "are"
289         parts.append(
290             f"{'', '.join(uncovered[:4])} {verb} not covered by the recommended tests, so check "
291             f"{'it' if len(uncovered) == 1 else 'them'} manually."
292         )
293     elif high or medium:
294         parts.append("The recommended tests cover every defect that could come back.")
295     if open_sentence:
296         parts.append(open_sentence)
297     return " ".join(parts)
298
299
300 def summarization_step(state: dict[str, Any]) -> dict[str, Any]:
301     """The Summarization Agent's single LLM request: the answer and the defect summary in one reply."""
302     options = state.get("options") or {}
303     question = (options.get("question") or "").strip() or DEFAULT_QUESTION
304     ecr_id = state.get("ecr_id", "this ECR")
305     context = build_answer_context(state)
306     insights = state.get("defect_insights") or {}
307     baseline = _baseline_answer(context)
308     defect_baseline = _baseline_defect_summary(ecr_id, insights)
309     # Matched defects are rewritten by the model; a history-only summary stays factual.
310     rewrite_defects = bool(insights.get("defects"))
311
312     answer, defect_summary, used_llm = baseline, defect_baseline, False
313     llm = get_llm()
314     if not llm.is_mock:
315         reply = llm.generate_structured(
316             SUMMARY_TASK.format(
317                 question=question,
318                 bundle=json.dumps(context, default=str)[:14000],
319                 defect_instruction=(
320                     SUMMARY_DEFECT_INSTRUCTION.format(ecr_id=ecr_id)
321                     if rewrite_defects
322                     else SUMMARY_NO_DEFECT_INSTRUCTION
323                 ),
324             ),
325             SummaryReply,
326             system=ANSWER_SYSTEM_PROMPT + SUMMARY_STYLE,
327             fallback={"answer": baseline, "defect_summary": defect_baseline},
328             temperature=0.2,
329             # The fast model, like the other two agents: the main model took longer
330             # than the request timeout on this bundle, and a timed-out request is
331             # billed but its answer is lost.
332             fast=True,
333         )
334     answer = reply.answer.strip() or baseline
335     if rewrite_defects:
336         defect_summary = reply.defect_summary.strip() or defect_baseline
337     used_llm = answer != baseline
338
339     citations = sorted(
340         {
341             *(r["id"] for r in context["requirements"]),
342             *(d["id"] for d in context["historical_defects"]),
343             *(t["id"] for t in context["tests"]["top"]),
344             *(item["ref"] for item in context["decisions"]),
345             *(item["ref"] for item in context["raised_concerns"]),
346             *(e["id"] for e in context["evidence"]),
347         }
348     )
349     confidence = round(min(0.95, 0.55 + 0.05 * min(8, len(citations))), 3)

```

```

350     return {
351         "final_answer": answer,
352         "answer_context": context,
353         "answer_citations": citations,
354         "question": question,
355         "defect_summary": defect_summary,
356         "tools_used": {
357             "summarization": ["build_answer_context", "llm.summarize" if used_llm else "rule_summary"]
358         },
359         "_confidence": confidence,
360         "_reasoning": baseline,
361     }
362
363
364 def report_step(state: dict[str, Any]) -> dict[str, Any]:
365     report = build_report(state)
366     return {
367         "final_report": report.model_dump(mode="json"),
368         "tools_used": {"report": ["report_service.build"]},
369         "_confidence": report.confidence.overall,
370         "_reasoning": (
371             f"Assembled {len(report.sections)} report sections with overall confidence "
372             f"{report.confidence.overall:.0%} ({report.confidence.band.value})."
373         ),
374         "_summary": {
375             "sections": len(report.sections),
376             "confidence": report.confidence.overall,
377             "metrics": report.metrics,
378         },
379     }

```

Backend - agent tools

backend/app/agents/tools/_init_.py

backend/app/agents/tools/code_tools.py

```

1  """Tools of the Code Impact Analysis Agent."""
2  from __future__ import annotations
3
4  from typing import Any, Iterable
5
6  from app.agents.tools.registry import tool
7  from app.services.code_analysis_service import get_code_analysis_service
8
9
10 @tool("analyze_changed_files", "Parse the repository with the Python AST analyzer and map impact.")
11 def analyze_changed_files(
12     changed_files: Iterable[str], *, keywords: Iterable[str] = ()
13 ) -> dict[str, Any]:
14     return get_code_analysis_service().analyze_change(changed_files, keywords=keywords)
15
16
17 @tool("resolve_component_owner", "Map a repository path onto the owning component.")
18 def resolve_component_owner(file_path: str) -> str | None:
19     return get_code_analysis_service().component_for_path(file_path)
20
21
22 @tool("walk_import_graph", "Walk the module import graph to find transitive dependents.")
23 def walk_import_graph(modules: Iterable[str], depth: int = 2) -> dict[str, int]:
24     return get_code_analysis_service().dependents(modules, max_depth=depth)
25
26
27 @tool("repository_stats", "Summarise the analysed repository (modules, edges, languages).")
28 def repository_stats() -> dict[str, Any]:
29     return get_code_analysis_service().stats()

```

backend/app/agents/tools/collaboration_tools.py

```

1  """Tools for retrieving human artefacts: review comments and evidence.
2
3  These are the sources a user would otherwise open one by one (work item
4  discussions, review threads, test evidence folders).
5  """
6  from __future__ import annotations
7
8  from typing import Any, Iterable
9
10 from sqlalchemy import or_, select
11 from sqlalchemy.orm import Session
12
13 from app.agents.tools.registry import tool

```

```

14 from app.models import Comment, Evidence
15
16
17 @tool("comments_for_ecr", "Fetch every review comment attached to an ECR or its linked artefacts.")
18 def comments_for_ecr(
19     db: Session, ecr_id: str, *, related_ids: Iterable[str] = ()
20 ) -> list[dict[str, Any]]:
21     related = [rid for rid in related_ids if rid]
22     stmt = select(Comment).where(
23         or_(Comment.ecr_id == ecr_id, Comment.target_id.in_(related) if related else False)
24     )
25     rows = db.execute(stmt).scalars().all()
26     return sorted(
27         (row.as_dict() for row in rows),
28         key=lambda item: item.get("created_at") or "",
29         reverse=True,
30     )
31
32
33 @tool("evidence_for_ecr", "Fetch evidence artefacts (runs, reviews, documents) for an ECR.")
34 def evidence_for_ecr(
35     db: Session, ecr_id: str, *, related_ids: Iterable[str] = ()
36 ) -> list[dict[str, Any]]:
37     related = [rid for rid in related_ids if rid]
38     stmt = select(Evidence).where(
39         or_(Evidence.ecr_id == ecr_id, Evidence.target_id.in_(related) if related else False)
40     )
41     rows = db.execute(stmt).scalars().all()
42     return sorted(
43         (row.as_dict() for row in rows),
44         key=lambda item: item.get("captured_at") or "",
45         reverse=True,
46     )
47
48
49 @tool("extract_comment_signals", "Pull decisions, risks and scope calls out of a comment thread.")
50 def extract_comment_signals(comments: list[dict[str, Any]]) -> dict[str, list[dict[str, str]]]:
51     signals: dict[str, list[dict[str, str]]] = {
52         "decisions": [],
53         "risks": [],
54         "scope": [],
55         "test_scope": [],
56         "implementation": [],
57     }
58     bucket_by_category = {
59         "DECISION": "decisions",
60         "RISK": "risks",
61         "SCOPE": "scope",
62         "TEST_SCOPE": "test_scope",
63         "IMPLEMENTATION": "implementation",
64         "HISTORY": "risks",
65     }
66     for comment in comments:
67         bucket = bucket_by_category.get(comment.get("category", ""), None)
68         if bucket is None:
69             continue
70         signals[bucket].append(
71             {
72                 "comment_id": comment.get("comment_id", ""),
73                 "author": comment.get("author", ""),
74                 "role": comment.get("author_role", ""),
75                 "text": comment.get("body", ""),
76                 "references": comment.get("references", []),
77             }
78         )
79     return signals
80
81
82 @tool("evidence_gaps", "Identify artefacts that have no supporting evidence yet.")
83 def evidence_gaps(evidence: list[dict[str, Any]], expected_targets: Iterable[str]) -> list[str]:
84     covered = {item.get("target_id") for item in evidence if item.get("target_id")}
85     return sorted({target for target in expected_targets if target and target not in covered})

```

backend/app/agents/tools/defect_tools.py

```

1 """Tools of the Defect Intelligence Agent."""
2 from __future__ import annotations
3
4 from collections import Counter
5 from typing import Any, Iterable
6
7 from sqlalchemy.orm import Session
8
9 from app.agents.tools.registry import tool

```

```

10 from app.repositories import DefectRepository
11 from app.services.rag_service import get_rag_service
12 from app.vectorstore import COLLECTION_DEFECTS
13
14
15 @tool("semantic_defect_search", "Hybrid search over historical defects and incident write-ups.")
16 def semantic_defect_search(
17     query: str, *, keywords: Iterable[str] = (), k: int = 12
18 ) -> list[dict[str, Any]]:
19     documents = get_rag_service().retrieve(
20         COLLECTION_DEFECTS, query, k=k, keywords=keywords, min_relevance=20
21     )
22     return [{"doc.metadata", "similarity": doc.relevance} for doc in documents]
23
24
25 @tool("defects_by_component", "Retrieve every defect recorded against the impacted components.")
26 def defects_by_component(db: Session, component_ids: Iterable[str]) -> list[dict[str, Any]]:
27     components = list(component_ids)
28     results = []
29     for defect in DefectRepository(db).by_components(components):
30         payload = defect.as_dict()
31         payload["similarity"] = 70.0 if defect.affected_component in components else 55.0
32         results.append(payload)
33     return results
34
35
36 @tool("cluster_root_causes", "Group defects by root-cause category to expose recurring patterns.")
37 def cluster_root_causes(defects: list[dict[str, Any]]) -> list[str]:
38     counter = Counter(
39         defect.get("root_cause_category", "").strip()
40         for defect in defects
41         if defect.get("root_cause_category")
42     )
43     patterns = []
44     for category, count in counter.most_common(5):
45         if count >= 2:
46             patterns.append(f"{category} ({count} occurrences)")
47         elif counter.total() <= 3:
48             patterns.append(category)
49     return patterns
50
51

```

backend/app/agents/tools/dependency_tools.py

```

1  """Tools of the Dependency Analysis Agent."""
2  from __future__ import annotations
3
4  from typing import Any, Iterable
5
6  from sqlalchemy.orm import Session
7
8  from app.agents.tools.registry import tool
9  from app.graph.dependency_graph import (
10     DependencyGraphService,
11     build_dependency_graph,
12 )
13
14
15 @tool("build_graph", "Load the component dependency graph from the database (NetworkX).")
16 def build_graph(db: Session) -> DependencyGraphService:
17     return build_dependency_graph(db)
18
19
20 @tool("downstream_walk", "Find components that depend on the changed ones (blast radius).")
21 def downstream_walk(
22     graph: DependencyGraphService, components: Iterable[str], depth: int = 3
23 ) -> dict[str, int]:
24     return graph.downstream(components, max_depth=depth)
25
26
27 @tool("upstream_walk", "Find components the changed ones rely on.")
28 def upstream_walk(
29     graph: DependencyGraphService, components: Iterable[str], depth: int = 2
30 ) -> dict[str, int]:
31     return graph.upstream(components, max_depth=depth)
32
33
34 @tool("critical_paths", "Highest-propagation paths from an entry point into the changed component.")
35 def critical_paths(
36     graph: DependencyGraphService, components: Iterable[str], max_paths: int = 5
37 ) -> list[list[str]]:
38     return graph.critical_paths(components, max_paths=max_paths)
39

```

```

40
41 @tool("dependency_subgraph", "Nodes and edges around the change for the UI graph view.")
42 def dependency_subgraph(
43     graph: DependencyGraphService, components: Iterable[str], depth: int = 2
44 ) -> dict[str, Any]:
45     return graph.subgraph_payload(components, max_depth=depth)

```

backend/app/agents/tools/registry.py

```

1  """Tool registry.
2
3  Every capability an agent can invoke is registered here, which gives the UI an
4  honest list of the tools each agent actually called (agent transparency) and
5  keeps tool implementations independent of the graph.
6  """
7  from __future__ import annotations
8
9  from dataclasses import dataclass
10 from typing import Any, Callable
11
12 @dataclass(slots=True)
13 class ToolSpec:
14     name: str
15     description: str
16     func: Callable[..., Any]
17
18
19 TOOLS: dict[str, ToolSpec] = {}
20
21
22 def tool(name: str, description: str) -> Callable:
23     def decorator(func: Callable[..., Any]) -> Callable[..., Any]:
24         TOOLS[name] = ToolSpec(name=name, description=description, func=func)
25         func.tool_name = name # type: ignore[attr-defined]
26         return func
27     return decorator
28
29
30
31
32 def list_tools() -> list[dict[str, str]]:
33     return [{"name": spec.name, "description": spec.description} for spec in TOOLS.values()]
34
35
36 def call_tool(name: str, *args: Any, **kwargs: Any) -> Any:
37     spec = TOOLS.get(name)
38     if spec is None:
39         raise KeyError(f"unknown tool: {name}")
40     return spec.func(*args, **kwargs)

```

backend/app/agents/tools/requirement_tools.py

```

1  """Tools of the Requirement Intelligence Agent."""
2  from __future__ import annotations
3
4  from typing import Any, Iterable
5
6  from sqlalchemy.orm import Session
7
8  from app.agents.tools.registry import tool
9  from app.repositories import RequirementRepository
10 from app.services.rag_service import get_rag_service
11 from app.vectorstore import COLLECTION_REQUIREMENTS
12
13
14 @tool("semantic_requirement_search", "Vector + keyword hybrid search over the requirement corpus.")
15 def semantic_requirement_search(
16     query: str, *, keywords: Iterable[str] = (), k: int = 12
17 ) -> list[dict[str, Any]]:
18     documents = get_rag_service().retrieve(
19         COLLECTION_REQUIREMENTS, query, k=k, keywords=keywords, min_relevance=20
20     )
21     return [
22         {**doc.metadata, "relevance": doc.relevance, "match_type": "SEMANTIC",
23          "dense": doc.dense_score, "lexical": doc.lexical_score}
24         for doc in documents
25     ]
26
27
28 @tool("keyword_requirement_search", "Literal keyword match across requirement titles and text.")
29 def keyword_requirement_search(
30     db: Session, keywords: Iterable[str], *, limit: int = 12
31 ) -> list[dict[str, Any]]:
32     terms = [k.lower() for k in keywords if len(k) > 2]

```

```

33     if not terms:
34         return []
35     results: list[dict[str, Any]] = []
36     for requirement in RequirementRepository(db).list():
37         haystack = f"{requirement.title} {requirement.description} {' '.join(requirement.features or [])}".
+ lower()
38         hits = [term for term in terms if term in haystack]
39         if not hits:
40             continue
41         payload = requirement.as_dict()
42         payload["relevance"] = round(min(100.0, 45 + 18 * len(hits)), 1)
43         payload["match_type"] = "KEYWORD"
44         payload["matched_terms"] = hits
45         results.append(payload)
46     results.sort(key=lambda item: item["relevance"], reverse=True)
47     return results[:limit]
48
49
50 @tool("trace_requirements", "Follow explicit ECR -> requirement and component -> requirement links.")
51 def trace_requirements(
52     db: Session, requirement_ids: Iterable[str], component_ids: Iterable[str]
53 ) -> list[dict[str, Any]]:
54     repository = RequirementRepository(db)
55     found: dict[str, dict[str, Any]] = {}
56     for requirement in repository.get_many(list(requirement_ids)):
57         payload = requirement.as_dict()
58         payload["relevance"] = 100.0
59         payload["match_type"] = "TRACEABILITY"
60         found[requirement.requirement_id] = payload
61     for component_id in component_ids:
62         for requirement in repository.by_component(component_id):
63             if requirement.requirement_id in found:
64                 continue
65             payload = requirement.as_dict()
66             payload["relevance"] = 78.0
67             payload["match_type"] = "COMPONENT_LINK"
68             found[requirement.requirement_id] = payload
69     return list(found.values())

```

backend/app/agents/tools/test_tools.py

```

1  """Tools of the Test Discovery / Selection / Prioritisation agents."""
2  from __future__ import annotations
3
4  from typing import Any, Iterable
5
6  from sqlalchemy.orm import Session
7
8  from app.agents.tools.registry import tool
9  from app.repositories import TestRepository
10 from app.services.rag_service import get_rag_service
11 from app.vectorstore import COLLECTION_TESTS
12
13
14 @tool("tests_by_requirement", "Fetch every test traced to the impacted requirements.")
15 def tests_by_requirement(db: Session, requirement_ids: Iterable[str]) -> list[dict[str, Any]]:
16     return [t.as_dict() for t in TestRepository(db).by_requirements(list(requirement_ids))]
17
18
19 @tool("tests_by_component", "Fetch every test covering the impacted components.")
20 def tests_by_component(db: Session, component_ids: Iterable[str]) -> list[dict[str, Any]]:
21     return [t.as_dict() for t in TestRepository(db).by_components(list(component_ids))]
22
23
24 @tool("semantic_test_search", "Hybrid semantic search over the test catalogue.")
25 def semantic_test_search(
26     query: str, *, keywords: Iterable[str] = (), k: int = 60
27 ) -> list[dict[str, Any]]:
28     documents = get_rag_service().retrieve(
29         COLLECTION_TESTS, query, k=k, keywords=keywords, min_relevance=15
30     )
31     return [{"doc.metadata", "semantic_score": doc.relevance} for doc in documents]
32
33
34 @tool("total_test_inventory", "Size of the regression suite that is the optimisation baseline.")
35 def total_test_inventory(db: Session, domain: str | None = None) -> int:
36     """Baseline = the suite a team would otherwise run.
37
38     Scoped to the ECR's own domain: a payments change would never trigger the
39     rail suite, so counting it would inflate the reduction figure.
40     """
41     rows = TestRepository(db).list()
42     return sum(1 for t in rows if not t.retired and (domain is None or t.domain == domain))
43

```

```

44
45 @tool("baseline_duration", "Execution time of the baseline regression suite in minutes.")
46 def baseline_duration(db: Session, domain: str | None = None) -> float:
47     rows = TestRepository(db).list()
48     total = sum(
49         t.average_execution_time
50         for t in rows
51         if not t.retired and (domain is None or t.domain == domain)
52     )
53     return round(total / 60.0, 1)
54
55
56 @tool("resolve_domain", "Infer which product domain an ECR belongs to.")
57 def resolve_domain(
58     db: Session,
59     component_ids: list[str],
60     requirement_ids: list[str],
61     linked_requirement_ids: list[str] | None = None,
62 ) -> str | None:
63     from collections import Counter
64
65     from app.models import Component, Requirement
66
67     # The requirements the ECR is explicitly linked to decide its product line;
68     # searched requirements and impacted components only vote when there are none.
69     if linked_requirement_ids:
70         linked = Counter(
71             row.domain
72             for row in db.query(Requirement).filter(
73                 Requirement.requirement_id.in_(list(linked_requirement_ids))
74             )
75         )
76         if linked:
77             return linked.most_common(1)[0][0]
78
79     votes: Counter[str] = Counter()
80     if component_ids:
81         for row in db.query(Component).filter(Component.component_id.in_(component_ids)).all():
82             votes[row.domain] += 2
83     if requirement_ids:
84         for row in db.query(Requirement).filter(
85             Requirement.requirement_id.in_(requirement_ids)
86         ).all():
87             votes[row.domain] += 1
88     return votes.most_common(1)[0][0] if votes else None

```

Backend - services (engines, retrieval, reporting)

backend/app/services/__init__.py

backend/app/services/chat_service.py

```

1  """Natural-language interface over the correlated ECR data.
2
3  Intent detection is deliberately deterministic (a regex for the ECR number plus
4  verb matching): routing a user to the wrong ECR is worse than asking them to be
5  explicit. Once an ECR is resolved, the answer is generated strictly from the
6  correlated bundle, with citations.
7  """
8  from __future__ import annotations
9
10 import json
11 import re
12 from typing import Any
13
14 from sqlalchemy.orm import Session
15
16 from app.agents.nodes.summarization import build_answer_context
17 from app.prompts import ANSWER_SYSTEM_PROMPT
18 from app.agents.runtime import registry
19 from app.llm.provider import get_llm
20 from app.logging import get_logger
21 from app.services import ecr_service
22 from app.services.rag_service import get_rag_service
23 from app.vectorstore import COLLECTION_ECRS
24
25 logger = get_logger("ecr.chat")
26
27 # ECR-2026-001 as well as short ids such as ECR-1
28 ECR_ID_RE = re.compile(r"\b(ECR[- ]?\d{1,6})(?:[- ]\d{1,6})?\b", re.IGNORECASE)
29 # L2R26:TC1 (test ids from the export) before the plain identifier forms
30 ARTEFACT_RE = re.compile(

```

```

31     r"\b(L2R\d{1,6}:\s?TC\d{1,4}|(?:REQ|BUG|TC|CMT|EV|PTC|L2R)[- ]?\d{2,6})\b", re.IGNORECASE
32 )
33
34 ANALYZE_VERBS = ("analyze", "analys", "run ", "assess", "impact of", "evaluate")
35 TEST_VERBS = ("test", "regression", "suite", "execute", "p0", "p1")
36 WHY_VERBS = ("why", "reason", "justif", "explain")
37
38
39 def detect_intent(query: str) -> dict[str, Any]:
40     lowered = query.lower()
41     ecr_match = ECR_ID_RE.search(query)
42     ecr_id = None
43     if ecr_match:
44         digits = re.split(r"[- ]", re.sub(r"^ECR[- ]?", "", ecr_match.group(1).upper()))
45         ecr_id = "ECR-" + "-".join(part for part in digits if part)
46     artefacts = [
47         m.group(1).upper().replace(" ", "") if ":" in m.group(1) else m.group(1).upper().replace(" ", "-")
48         for m in ARTEFACT_RE.finditer(query)
49     ]
50
51     if any(verb in lowered for verb in WHY_VERBS) and artefacts:
52         intent = "EXPLAIN_ARTEFACT"
53     elif any(verb in lowered for verb in TEST_VERBS):
54         intent = "TEST_RECOMMENDATION"
55     elif any(verb in lowered for verb in ANALYZE_VERBS):
56         intent = "ANALYZE"
57     elif ecr_id:
58         intent = "QUESTION"
59     else:
60         intent = "SEARCH"
61     return {"intent": intent, "ecr_id": ecr_id, "artefacts": artefacts}
62
63
64 def resolve_ecr(db: Session, query: str, ecr_id: str | None) -> str | None:
65     """Explicit id wins; otherwise fall back to semantic search over ECRs."""
66     if ecr_id:
67         try:
68             return ecr_service.get_ecr(db, ecr_id).ecr_id
69         except ecr_service.ECRNotFound:
70             return None
71     hits = get_rag_service().retrieve(COLLECTION_ECRS, query, k=1, min_relevance=35)
72     return hits[0].id if hits else None
73
74
75 def _bundle_for(ecr_id: str) -> dict[str, Any] | None:
76     run = registry.latest_for_ecr(ecr_id)
77     if run and run.state:
78         return run.state
79     return None
80
81
82 def answer_question(
83     db: Session, question: str, *, ecr_id: str | None = None, run_if_missing: bool = True
84 ) -> dict[str, Any]:
85     """Answer a question about an ECR from its correlated bundle."""
86     detection = detect_intent(question)
87     resolved = resolve_ecr(db, question, ecr_id or detection["ecr_id"])
88     if not resolved:
89         return {
90             "answer": (
91                 "I could not work out which ECR you mean. Give me an ECR number "
92                 "(for example ECR-2026-001) and I will pull its requirements, defects, "
93                 "comments, evidence and recommended tests."
94             ),
95             "intent": detection["intent"],
96             "ecr_id": None,
97             "citations": [],
98             "analysis_available": False,
99         }
100
101     state = _bundle_for(resolved)
102     if state is None and run_if_missing:
103         logger.info("chat.analysis_missing", ecr_id=resolved)
104         run = ecr_service.start_analysis(
105             db, resolved, {"human_in_the_loop": False, "question": question}, background=False
106         )
107         state = run.state
108     if state is None:
109         return {
110             "answer": f"{resolved} has not been analysed yet. Run an analysis first.",
111             "intent": detection["intent"],
112             "ecr_id": resolved,
113             "citations": [],
114             "analysis_available": False,

```

```

115         }
116
117     context = build_answer_context(state)
118     focus = _focus_context(state, detection)
119     llm = get_llm()
120     if llm.is_mock:
121         answer = _deterministic_answer(state, detection, focus)
122     else:
123         baseline = _deterministic_answer(state, detection, focus)
124         answer = llm.generate(
125             f"QUESTION: {question}\n\n"
126             f"CORRELATED BUNDLE FOR {resolved}\n"
127             f"{json.dumps(context, indent=2, default=str)[:12000]}\n\n"
128             + (f"FOCUS ARTEFACTS:\n{json.dumps(focus, indent=2, default=str)[:4000]}\n\n" if focus else "")
129             + "Answer in at most 6 sentences, citing the identifiers you used.",
130             system=ANSWER_SYSTEM_PROMPT,
131             max_tokens=800,
132             temperature=0.2,
133             fallback=baseline,
134         ).strip() or baseline
135
136     citations = sorted(
137         {
138             *detection["artefacts"],
139             *(r["id"] for r in context["requirements"][:5]),
140             *(t["id"] for t in context["tests"]["top"][:5]),
141             *(d["id"] for d in context["historical_defects"][:3]),
142         }
143     )
144     return {
145         "answer": answer,
146         "intent": detection["intent"],
147         "ecr_id": resolved,
148         "citations": citations,
149         "focus": focus,
150         "analysis_available": True,
151         "workflow_id": state.get("workflow_id"),
152     }
153
154
155 def _focus_context(state: dict[str, Any], detection: dict[str, Any]) -> dict[str, Any]:
156     """Pull the exact artefacts the question names (e.g. 'why was TC-1042 selected?')."""
157     wanted = set(detection["artefacts"])
158     if not wanted:
159         return {}
160     focus: dict[str, Any] = {}
161     for test in state.get("selected_tests") or []:
162         if test["test_case_id"] in wanted:
163             focus[test["test_case_id"]] = {
164                 "title": test["title"],
165                 "priority": test["priority"],
166                 "relevance": test["relevance_score"],
167                 "why_selected": test["reason"],
168                 "score_breakdown": test["breakdown"],
169                 "requirement": test["requirement_id"],
170                 "component": test["component"],
171             }
172     for requirement in state.get("requirements") or []:
173         if requirement["requirement_id"] in wanted:
174             focus[requirement["requirement_id"]] = {
175                 "title": requirement["title"],
176                 "relevance": requirement["relevance"],
177                 "matched_via": requirement["match_type"],
178                 "why": requirement["reason"],
179             }
180     for defect in state.get("defects") or []:
181         if defect["defect_id"] in wanted:
182             focus[defect["defect_id"]] = {
183                 "title": defect["title"],
184                 "severity": defect["severity"],
185                 "root_cause": defect["root_cause"],
186                 "why": defect["reason"],
187             }
188     for comment in state.get("comments") or []:
189         if comment["comment_id"] in wanted:
190             focus[comment["comment_id"]] = {
191                 "author": comment["author"],
192                 "category": comment["category"],
193                 "body": comment["body"],
194             }
195     return focus
196
197
198 def _deterministic_answer(

```

```

199     state: dict[str, Any], detection: dict[str, Any], focus: dict[str, Any]
200 ) -> str:
201     """Rule-based answer used in demo (mock) mode."""
202     selection = state.get("test_selection") or {}
203     if focus:
204         parts = []
205         for artefact, detail in focus.items():
206             reason = detail.get("why_selected") or detail.get("why") or detail.get("body", "")
207             parts.append(f"{artefact}: {reason}")
208         return " ".join(parts)
209     if detection["intent"] == "TEST_RECOMMENDATION":
210         distribution = selection.get("priority_distribution", {})
211         top = " ".join(t["test_case_id"] for t in (selection.get("selected_tests") or [])[:8])
212         return (
213             f"{len(selection.get('selected_tests') or [])} of {selection.get('total_available')} "
214             f"regression tests are recommended ({selection.get('reduction_percentage')}% reduction): "
215             f"{distribution.get('P0', 0)} P0, {distribution.get('P1', 0)} P1, "
216             f"{distribution.get('P2', 0)} P2, {distribution.get('P3', 0)} P3. "
217             f"Start with {top}."
218         )
219     return state.get("final_answer") or (
220         f"The analysis of {state.get('ecr_id')} is ready - ask about its requirements, "
221         f"recommended tests or past defects."
222     )

```

backend/app/services/code_analysis_service.py

```

1  """Static code-impact analysis over a repository.
2
3  The Python analyzer is real: it parses the sample repository with :mod:`ast`,
4  builds a module import graph and a function call graph, and derives
5      direct impact = files listed on the change,
6      indirect impact = modules that import them (transitively, depth-limited).
7
8  ``CodeAnalyzer`` is the extension point: JavaAnalyzer / CSharpAnalyzer are
9  declared with the same contract so another language can be added without
10 touching the agent.
11 """
12 from __future__ import annotations
13
14 import ast
15 import re
16 from abc import ABC, abstractmethod
17 from dataclasses import dataclass, field
18 from functools import lru_cache
19 from pathlib import Path
20 from typing import Any, Iterable
21
22 from app.config import BACKEND_DIR
23 from app.logging import get_logger
24
25 logger = get_logger("ecr.code")
26
27 SAMPLE_REPO_PATH = BACKEND_DIR / "app" / "seed" / "sample_repo"
28
29 # Repository path prefix -> component id
30 PATH_COMPONENT_MAP: dict[str, str] = {
31     "portal/": "CMP-001",
32     "gateway/": "CMP-002",
33     "infra/": "CMP-002",
34     "payment/service.py": "CMP-003",
35     "payment/validator.py": "CMP-004",
36     "payment/currency.py": "CMP-005",
37     "payment/": "CMP-003",
38     "currency/": "CMP-005",
39     "order/": "CMP-006",
40     "refund/": "CMP-007",
41     "schema/": "CMP-008",
42     "user/": "CMP-009",
43     "auth/": "CMP-010",
44     "notification/": "CMP-011",
45     "invoice/": "CMP-012",
46     "reporting/": "CMP-013",
47     "fraud/": "CMP-014",
48     "ledger/": "CMP-015",
49     "inventory/": "CMP-016",
50     "shipping/": "CMP-017",
51     "catalog/": "CMP-018",
52     "promotion/": "CMP-019",
53     "loyalty/": "CMP-020",
54     "subscription/": "CMP-021",
55     "wallet/": "CMP-022",
56     "chargeback/": "CMP-023",
57     "payout/": "CMP-024",

```

```

58     "tax_engine/": "CMP-025",
59     "kyc/": "CMP-026",
60     "consent/": "CMP-027",
61     "audit/": "CMP-028",
62     "search/": "CMP-029",
63     "mobile_bff/": "CMP-030",
64     "broker/": "CMP-031",
65     "support/": "CMP-032",
66     "warehouse/": "CMP-033",
67     "giftcard/": "CMP-034",
68     "returns/": "CMP-035",
69 }
70
71 # Fallback for files that do not exist in the sample repository
72 KEYWORD_COMPONENT_MAP: dict[str, str] = {
73     "portal": "CMP-001",
74     "ui": "CMP-001",
75     "gateway": "CMP-002",
76     "validator": "CMP-004",
77     "validation": "CMP-004",
78     "currency": "CMP-005",
79     "fx": "CMP-005",
80     "payment": "CMP-003",
81     "order": "CMP-006",
82     "checkout": "CMP-006",
83     "refund": "CMP-007",
84     "schema": "CMP-008",
85     "migration": "CMP-008",
86     "database": "CMP-008",
87     "user": "CMP-009",
88     "profile": "CMP-009",
89     "auth": "CMP-010",
90     "notification": "CMP-011",
91     "email": "CMP-011",
92     "template": "CMP-011",
93     "invoice": "CMP-012",
94     "report": "CMP-013",
95     "fraud": "CMP-014",
96     "ledger": "CMP-015",
97 }
98
99 API_ROUTE_HINTS = ("ROUTES", "router", "endpoint")
100
101
102 @dataclass(slots=True)
103 class ModuleInfo:
104     module: str
105     path: str
106     functions: list[str] = field(default_factory=list)
107     classes: list[str] = field(default_factory=list)
108     imports: dict[str, list[str]] = field(default_factory=dict) # module -> symbols
109     calls: dict[str, list[str]] = field(default_factory=dict) # function -> called names
110     routes: list[str] = field(default_factory=list)
111     constants: list[str] = field(default_factory=list)
112
113
114 class CodeAnalyzer(ABC):
115     """Contract for a per-language static analyzer."""
116
117     language: str = "unknown"
118     extensions: tuple[str, ...] = ()
119
120     @abstractmethod
121     def parse(self, root: Path) -> dict[str, ModuleInfo]: ...
122
123
124 class PythonASTAnalyzer(CodeAnalyzer):
125     language = "python"
126     extensions = (".py",)
127
128     def parse(self, root: Path) -> dict[str, ModuleInfo]:
129         modules: dict[str, ModuleInfo] = {}
130         for path in sorted(root.rglob("*.py")):
131             rel = path.relative_to(root).as_posix()
132             module_name = rel[:-3].replace("/", ".")
133             try:
134                 tree = ast.parse(path.read_text(encoding="utf-8"), filename=rel)
135             except SyntaxError as exc: # pragma: no cover - sample repo is valid
136                 logger.warning("code.parse_failed", file=rel, error=str(exc))
137                 continue
138             info = ModuleInfo(module=module_name, path=rel)
139             for node in ast.walk(tree):
140                 if isinstance(node, ast.FunctionDef):
141                     info.functions.append(node.name)

```

```

142         info.calls[node.name] = sorted(
143             {
144                 _call_name(child)
145                 for child in ast.walk(node)
146                 if isinstance(child, ast.Call) and _call_name(child)
147             }
148         )
149     elif isinstance(node, ast.ClassDef):
150         info.classes.append(node.name)
151     elif isinstance(node, ast.ImportFrom) and node.module:
152         info.imports.setdefault(node.module, []).extend(
153             alias.name for alias in node.names
154         )
155     elif isinstance(node, ast.Import):
156         for alias in node.names:
157             info.imports.setdefault(alias.name, [])
158     elif isinstance(node, ast.Assign):
159         for target in node.targets:
160             if isinstance(target, ast.Name) and target.id.isupper():
161                 info.constants.append(target.id)
162                 if any(hint in target.id for hint in ("ROUTE", "ENDPOINT")) and isinstance(
163                     node.value, ast.Dict
164                 ):
165                     info.routes.extend(
166                         key.value
167                         for key in node.value.keys
168                         if isinstance(key, ast.Constant) and isinstance(key.value, str)
169                     )
170         modules[module_name] = info
171     return modules
172
173
174 class JavaAnalyzer(CodeAnalyzer): # pragma: no cover - extension point
175     """Placeholder for a JavaParser/tree-sitter based analyzer."""
176
177     language = "java"
178     extensions = (".java",)
179
180     def parse(self, root: Path) -> dict[str, ModuleInfo]:
181         return {}
182
183
184 class CSharpAnalyzer(CodeAnalyzer): # pragma: no cover - extension point
185     """Placeholder for a Roslyn based analyzer."""
186
187     language = "csharp"
188     extensions = (".cs",)
189
190     def parse(self, root: Path) -> dict[str, ModuleInfo]:
191         return {}
192
193
194 class CodeAnalysisService:
195     """Repository-level analysis with an import-graph based impact walk."""
196
197     def __init__(self, root: Path | None = None) -> None:
198         self.root = root or SAMPLE_REPO_PATH
199         self.analyzers: list[CodeAnalyzer] = [PythonASTAnalyzer(), JavaAnalyzer(), CSharpAnalyzer()]
200         self.modules: dict[str, ModuleInfo] = {}
201         self.importers: dict[str, set[str]] = {}
202         self._load()
203
204     # -- loading -----
205     def _load(self) -> None:
206         if not self.root.exists():
207             logger.warning("code.repo_missing", path=str(self.root))
208             return
209         for analyzer in self.analyzers:
210             self.modules.update(analyzer.parse(self.root))
211         for module, info in self.modules.items():
212             for imported in info.imports:
213                 if imported in self.modules:
214                     self.importers.setdefault(imported, set()).add(module)
215         logger.info("code.repo_loaded", modules=len(self.modules), edges=len(self.importers))
216
217     # -- helpers -----
218     @staticmethod
219     def component_for_path(file_path: str) -> str | None:
220         normalised = file_path.replace("\\", "/").rstrip("/").lower()
221         for prefix, component in PATH_COMPONENT_MAP.items():
222             if normalised.startswith(prefix) or f"/{prefix}" in normalised:
223                 return component
224         for keyword, component in KEYWORD_COMPONENT_MAP.items():
225             # Only at the start of a path segment or name part: "order/api.py" and

```

```

226         # "order_total.py" match, but "event_recorder.py" is not the order service.
227         if re.search(rf"(?:^|[/_\.\\-]){re.escape(keyword)}", normalised):
228             return component
229     return None
230
231 def module_for_path(self, file_path: str) -> str | None:
232     normalised = file_path.replace("\\", "/").rstrip("/")
233     if normalised.endswith(".py"):
234         candidate = normalised[:-3].replace("/", ".")
235         if candidate in self.modules:
236             return candidate
237     for module in self.modules:
238         if module.endswith(candidate.split(".")[1]):
239             if self.modules[module].path == normalised:
240                 return module
241     return None
242
243 def dependents(self, modules: Iterable[str], max_depth: int = 2) -> dict[str, int]:
244     """Modules that (transitively) import the given ones."""
245     distances: dict[str, int] = {}
246     frontier = [m for m in modules if m in self.modules]
247     seen = set(frontier)
248     depth = 0
249     while frontier and depth < max_depth:
250         depth += 1
251         nxt: list[str] = []
252         for module in frontier:
253             for importer in self.importers.get(module, set()):
254                 if importer in seen:
255                     continue
256                 seen.add(importer)
257                 distances[importer] = depth
258                 nxt.append(importer)
259         frontier = nxt
260     return distances
261
262 # -- public API -----
263 def analyze_change(
264     self, changed_files: Iterable[str], *, keywords: Iterable[str] = ()
265 ) -> dict[str, Any]:
266     """Return the raw code-impact facts for the Code Impact Agent."""
267     files = [f for f in changed_files if f]
268     resolved_modules: list[str] = []
269     unresolved: list[str] = []
270     for file_path in files:
271         module = self.module_for_path(file_path)
272         if module:
273             resolved_modules.append(module)
274         else:
275             unresolved.append(file_path)
276
277     # Keyword-driven discovery only when the ECR lists no files at all. Declared
278     # files that are not in this repository belong to another codebase (e.g. the
279     # rail onboard software); searching this repository for their keywords would
280     # report unrelated modules as directly changed.
281     if not resolved_modules and not files:
282         for module, info in self.modules.items():
283             haystack = f"{module} {' '.join(info.functions)} {' '.join(info.constants)}".lower()
284             if any(kw.lower() in haystack for kw in keywords):
285                 resolved_modules.append(module)
286
287     direct_components = {
288         comp
289         for comp in (self.component_for_path(f) for f in files)
290         if comp
291     }
292     direct_components |= {
293         comp
294         for comp in (self.component_for_path(self.modules[m].path) for m in resolved_modules)
295         if comp
296     }
297
298     dependents = self.dependents(resolved_modules, max_depth=2)
299     indirect_components = {
300         comp
301         for comp in (self.component_for_path(self.modules[m].path) for m in dependents)
302         if comp
303     } - direct_components
304
305     symbols: list[dict[str, Any]] = []
306     apis: list[str] = []
307     db_entities: list[str] = []
308     for module in resolved_modules:
309         info = self.modules[module]

```

```

310         component = self.component_for_path(info.path) or ""
311         for function in info.functions:
312             symbols.append(
313                 {
314                     "file": info.path,
315                     "symbol": function,
316                     "kind": "function",
317                     "component": component,
318                     "calls": info.calls.get(function, []),
319                 }
320             )
321         apis.extend(info.routes)
322         if "schema" in module or "TRANSACTION_COLUMNS" in info.constants:
323             db_entities.extend(info.constants)
324
325         # routes exposed by dependent modules are part of the API surface too
326         for module in dependents:
327             apis.extend(self.modules[module].routes)
328
329         analysed = len(resolved_modules)
330         total = max(len(self.modules), 1)
331         return {
332             "resolved_modules": resolved_modules,
333             "unresolved_files": unresolved,
334             "direct_files": [self.modules[m].path for m in resolved_modules] or files,
335             "dependent_modules": dependents,
336             "dependent_files": [self.modules[m].path for m in dependents],
337             "direct_components": sorted(direct_components),
338             "indirect_components": sorted(indirect_components),
339             "symbols": symbols,
340             "apis": sorted(set(apis)),
341             "database_entities": sorted(set(db_entities)),
342             "change_surface": round(min(1.0, (analysed + 0.5 * len(dependents)) / total), 3),
343             "repository": str(self.root),
344             "analyzers": [a.language for a in self.analyzers if a.language == "python"],
345             "modules_indexed": len(self.modules),
346         }
347
348     def stats(self) -> dict[str, Any]:
349         return {
350             "repository": str(self.root),
351             "modules": len(self.modules),
352             "import_edges": sum(len(v) for v in self.importers.values()),
353             "languages": [a.language for a in self.analyzers],
354         }
355
356
357     def _call_name(node: ast.Call) -> str | None:
358         func = node.func
359         if isinstance(func, ast.Name):
360             return func.id
361         if isinstance(func, ast.Attribute):
362             return func.attr
363         return None
364
365
366     @lru_cache
367     def get_code_analysis_service() -> CodeAnalysisService:
368         return CodeAnalysisService()

```

backend/app/services/confidence.py

```

1  """How sure an analysis run is, from its steps' own confidence values."""
2  from __future__ import annotations
3
4  from typing import Any
5
6
7  def compute_confidence(
8      per_step: dict[str, float], *, errors: list[dict[str, Any]] | None = None
9  ) -> tuple[float, list[str]]:
10     """Blend the step confidences and apply penalties for degraded execution."""
11     penalties: list[str] = []
12     values = [v for v in per_step.values() if v is not None]
13     base = sum(values) / len(values) if values else 0.5
14     for error in errors or []:
15         agent = error.get("agent", "an agent")
16         base -= 0.08
17         penalties.append(f"{agent} failed - dependent evidence is missing")
18     if len(values) < 5:
19         base -= 0.05
20     penalties.append("Fewer than five agents contributed evidence")
21     return round(max(0.05, min(1.0, base)), 3), penalties

```

backend/app/services/ecr_service.py

```
1  """ECR application service: creation, lookup and workflow launch."""
2  from __future__ import annotations
3
4  import re
5  import threading
6  from datetime import datetime, timezone
7  from typing import Any
8
9  from sqlalchemy.orm import Session
10
11  from app.agents.orchestrator import run_workflow
12  from app.agents.runtime import WorkflowRun, registry
13  from app.logging import get_logger
14  from app.models import ECR
15  from app.repositories import ECRRepository, ImpactReportRepository
16  from app.schemas.ecr import ECRCreate
17  from app.services.rag_service import get_rag_service
18
19  logger = get_logger("ecr.service")
20
21  ECR_ID_PATTERN = re.compile(r"^ECR-\d{4}-\d{1,6}$", re.IGNORECASE)
22
23
24  class ECRNotFound(LookupError):
25      pass
26
27
28  def normalise_ecr_id(value: str) -> str:
29      return (value or "").strip().upper()
30
31
32  def next_ecr_id(db: Session) -> str:
33      year = datetime.now(timezone.utc).year
34      prefix = f"ECR-{year}-"
35      existing = [
36          int(row.ecr_id.rsplit("-", 1)[-1])
37          for row in ECRRepository(db).list()
38          if row.ecr_id.startswith(prefix) and row.ecr_id.rsplit("-", 1)[-1].isdigit()
39      ]
40      return f"{prefix}{max(existing, default=0) + 1:03d}"
41
42
43  def create_ecr(db: Session, payload: ECRCreate) -> ECR:
44      repository = ECRRepository(db)
45      ecr_id = normalise_ecr_id(payload.ecr_id or "") or next_ecr_id(db)
46      if repository.get(ecr_id):
47          raise ValueError(f"{ecr_id} already exists")
48      ecr = ECR(ecr_id=ecr_id, status="NEW", **payload.model_dump(exclude={"ecr_id"}))
49      repository.add(ecr)
50      try:
51          get_rag_service().index_ecr(ecr)
52          db.commit()
53      except Exception as exc: # pragma: no cover - indexing must not block creation
54          logger.warning("ecr.index_failed", ecr_id=ecr_id, error=str(exc))
55      logger.info("ecr.created", ecr_id=ecr_id)
56      return ecr
57
58
59  def get_ecr(db: Session, ecr_id: str) -> ECR:
60      ecr = ECRRepository(db).get(normalise_ecr_id(ecr_id))
61      if ecr is None:
62          raise ECRNotFound(ecr_id)
63      return ecr
64
65
66  def start_analysis(
67      db: Session, ecr_id: str, options: dict[str, Any], *, background: bool = True
68  ) -> WorkflowRun:
69      """Launch the agent workflow. Returns immediately when ``background``."""
70      ecr = get_ecr(db, ecr_id)
71      payload = ecr.as_dict()
72      if options.get("description_override"):
73          payload["description"] = options["description_override"]
74
75      ecr.status = "ANALYSING"
76      db.commit()
77
78      if not background:
79          run, _ = run_workflow(payload, options)
80          _mark_analysed(ecr_id)
81          return run
82
```

```

83     # Create the run up front so the caller can subscribe to its event stream
84     # before the first agent starts.
85     run = registry.create(payload["ecr_id"], options)
86
87     def _target() -> None:
88         try:
89             run_workflow(payload, options, existing_run=run)
90             _mark_analysed(ecr_id)
91         except Exception: # pragma: no cover - already logged and recorded
92             logger.warning("ecr.analysis_failed", ecr_id=ecr_id)
93
94     threading.Thread(target=_target, name=f"workflow-{run.workflow_id}", daemon=True).start()
95     return run
96
97
98 def _mark_analysed(ecr_id: str) -> None:
99     from app.database import session_scope
100
101     with session_scope() as db:
102         ecr = ECRRepository(db).get(ecr_id)
103         if ecr:
104             ecr.status = "ANALYSED"
105
106
107 def latest_report(db: Session, ecr_id: str) -> dict[str, Any] | None:
108     """Report from the live run if present, else the last persisted one."""
109     run = registry.latest_for_ecr(normalise_ecr_id(ecr_id))
110     if run and run.report:
111         return run.report
112     record = ImpactReportRepository(db).latest_for_ecr(normalise_ecr_id(ecr_id))
113     return record.report_json if record else None
114
115
116 def latest_state(ecr_id: str) -> dict[str, Any] | None:
117     run = registry.latest_for_ecr(normalise_ecr_id(ecr_id))
118     return run.state if run else None

```

backend/app/services/rag_service.py

```

1  """Retrieval Augmented Generation layer.
2
3  Indexes requirements, defects, test cases and ECR descriptions, then serves
4  hybrid retrieval (dense vector similarity + lexical keyword overlap) to the
5  agents. Hybrid scoring matters here: enterprise artefacts are short and full of
6  identifiers, where pure dense retrieval alone is brittle.
7
8  Embeddings are cached in a file next to the data (not in the CSVs), keyed by a
9  hash of each document's text. A restart reuses them, and editing a CSV row only
10 re-embeds the rows whose text changed.
11 """
12 from __future__ import annotations
13
14 import hashlib
15 import json
16 import os
17 import tempfile
18 import threading
19 from collections import OrderedDict
20 from dataclasses import dataclass
21 from pathlib import Path
22 from typing import Any, Iterable, Sequence
23
24 from sqlalchemy.orm import Session
25
26 from app.config import settings
27 from app.llm.base import hashed_embedding, tokenize
28 from app.llm.provider import get_llm
29 from app.logging import get_logger
30 from app.models import Defect, ECR, Requirement, TestCase
31 from app.vectorstore import (
32     COLLECTION_DEFECTS,
33     COLLECTION_ECRS,
34     COLLECTION_REQUIREMENTS,
35     COLLECTION_TESTS,
36     VectorRecord,
37     get_vector_store,
38 )
39
40 logger = get_logger("ecr.rag")
41
42
43 @dataclass(slots=True)
44 class RetrievedDocument:
45     id: str

```

```

46     text: str
47     metadata: dict[str, Any]
48     dense_score: float
49     lexical_score: float
50     hybrid_score: float
51
52     @property
53     def relevance(self) -> float:
54         """Hybrid score mapped onto a 0-100 relevance percentage."""
55         return round(max(0.0, min(1.0, self.hybrid_score)) * 100, 1)
56
57
58 def _digest(text: str) -> str:
59     return hashlib.sha1(text.encode("utf-8")).hexdigest()[:16]
60
61
62 class RAGService:
63     """Owns the embedding index lifecycle and hybrid retrieval."""
64
65     DENSE_WEIGHT = 0.65
66     LEXICAL_WEIGHT = 0.35
67
68     def __init__(self) -> None:
69         self._store = get_vector_store()
70         self._llm = get_llm()
71         self._lock = threading.RLock()
72         self._indexed = False
73         # Vectors from different embedders are not comparable. The index records
74         # which embedder built it so a mid-flight fallback triggers a rebuild.
75         self._index_mode: str | None = None
76         self._embed_mode: str = "unknown"
77         self._pinned_mode: str | None = None
78         self._embed_mode_changed = False
79         self._vectors: dict[str, dict[str, dict[str, Any]]] = {}
80         # query text -> (embedder mode, vector); repeat analyses skip the remote call
81         self._query_cache: OrderedDict[str, tuple[str, list[float]]] = OrderedDict()
82
83     # -- document text builders -----
84     @staticmethod
85     def _requirement_text(r: Requirement) -> str:
86         return (
87             f"{{r.requirement_id}} {{r.title}}. {{r.description}} "
88             f"domain={{r.business_domain}} component={{r.component}} "
89             f"features={{' '.join(r.features or [])}}"
90         )
91
92     @staticmethod
93     def _defect_text(d: Defect) -> str:
94         return (
95             f"{{d.defect_id}} {{d.title}}. {{d.description}} root_cause={{d.root_cause}} "
96             f"category={{d.root_cause_category}} component={{d.affected_component}} "
97             f"severity={{d.severity}}"
98         )
99
100     @staticmethod
101     def _test_text(t: TestCase) -> str:
102         return (
103             f"{{t.test_case_id}} {{t.title}}. {{t.description}} feature={{t.feature}} "
104             f"folder={{t.folder}} component={{t.component}} requirement={{t.requirement_id}} "
105             f"type={{t.test_polarity}} technique={{t.test_technique}} "
106             f"tags={{' '.join(t.tags or [])}}"
107         )
108
109     @staticmethod
110     def _ecr_text(e: ECR) -> str:
111         return f"{{e.ecr_id}} {{e.title}}. {{e.description}}"
112
113     def _specs(self):
114         return [
115             (COLLECTION_REQUIREMENTS, Requirement, "requirement_id", self._requirement_text),
116             (COLLECTION_DEFECTS, Defect, "defect_id", self._defect_text),
117             (COLLECTION_TESTS, TestCase, "test_case_id", self._test_text),
118             (COLLECTION_ECRS, ECR, "ecr_id", self._ecr_text),
119         ]
120
121     # -- indexing -----
122     def index_all(self, db: Session, *, force: bool = False) -> dict[str, int]:
123         """Build every collection, reusing cached embeddings whose text is unchanged."""
124         with self._lock:
125             if self._indexed and not force:
126                 return {name: self._store.count(name) for name, *_ in self._specs()}
127             cache_mode, cache = (None, {}) if force else self._read_cache()
128             self._pinned_mode = "local" if cache_mode == "local" else None
129             self._embed_mode = cache_mode or "unknown"

```

```

130         self._embed_mode_changed = False
131
132         counts = self._build(db, cache)
133         # Cached vectors and freshly embedded ones must come from the same
134         # embedder; if they do not (or the embedder fell back mid-way), rebuild.
135         mismatch = cache_mode is not None and self._embed_mode not in (cache_mode, "unknown")
136         if mismatch or (self._pinned_mode == "local" and self._embed_mode_changed):
137             logger.warning("rag.reindexing", cache_mode=cache_mode, current=self._embed_mode)
138             self._embed_mode_changed = False
139             counts = self._build(db, {})
140
141         self._indexed = True
142         self._index_mode = self._embed_mode
143         self._write_cache()
144         logger.info("rag.indexed", mode=self._index_mode, **counts)
145         return counts
146
147     def _build(self, db: Session, cache: dict[str, dict[str, dict[str, Any]]]) -> dict[str, int]:
148         counts: dict[str, int] = {}
149         vectors_out: dict[str, dict[str, dict[str, Any]]] = {}
150         reused = embedded = 0
151         for collection, model, key, text_fn in self._specs():
152             rows = db.query(model).all()
153             ids = [getattr(row, key) for row in rows]
154             texts = [text_fn(row) for row in rows]
155             hashes = [_digest(text) for text in texts]
156             cached = cache.get(collection, {})
157             vectors: list[list[float] | None] = [None] * len(rows)
158             missing: list[int] = []
159             for index, (doc_id, digest) in enumerate(zip(ids, hashes)):
160                 entry = cached.get(doc_id)
161                 if entry and entry.get("h") == digest and len(entry.get("v") or []) == settings.embedding_d
+ im:
162                     vectors[index] = entry["v"]
163                 else:
164                     missing.append(index)
165             if missing:
166                 fresh = self._embed([texts[index] for index in missing])
167                 for index, vector in zip(missing, fresh):
168                     vectors[index] = vector
169             reused += len(rows) - len(missing)
170             embedded += len(missing)
171             try:
172                 self._store.reset(collection) # rows removed from a CSV must disappear
173             except NotImplementedError: # pragma: no cover
174                 pass
175             records = [
176                 VectorRecord(id=ids[i], text=texts[i], vector=vectors[i], metadata=rows[i].as_dict())
177                 for i in range(len(rows))
178             ]
179             counts[collection] = self._store.upsert(collection, records)
180             vectors_out[collection] = {ids[i]: {"h": hashes[i], "v": vectors[i]} for i in range(len(rows))}
181         self._vectors = vectors_out
182         logger.info("rag.embeddings", reused=reused, embedded=embedded)
183         return counts
184
185     def index_ecr(self, ecr: ECR) -> None:
186         """Index a single (newly created) ECR without a full rebuild."""
187         text = self._ecr_text(ecr)
188         vector = self._embed([text])[0]
189         self._store.upsert(
190             COLLECTION_ECRS, [VectorRecord(id=ecr.ecr_id, text=text, vector=vector, metadata=ecr.as_dict())
+ ]
191         )
192         self._vectors.setdefault(COLLECTION_ECRS, {})[ecr.ecr_id] = {"h": _digest(text), "v": vector}
193         self._write_cache()
194
195     def _embed(self, texts: Sequence[str]) -> list[list[float]]:
196         """Embed, pinning the mode for the whole process once it degrades."""
197         if not texts:
198             return []
199         if self._pinned_mode == "local":
200             self._embed_mode = "local"
201             return [hashed_embedding(text) for text in texts]
202         vectors, mode = self._llm.embed_with_mode(list(texts))
203         if mode == "local" and self._embed_mode == "remote":
204             self._embed_mode_changed = True
205         if mode == "local":
206             self._pinned_mode = "local"
207         self._embed_mode = mode
208         return vectors
209
210     # -- embedding cache file -----
211     @staticmethod

```

```

212 def _cache_path() -> Path:
213     return Path(settings.embedding_cache_path)
214
215 def _remote_embeddings_available(self) -> bool:
216     if not self._llm.supports_remote_embeddings:
217         return False
218     return not getattr(self._llm, "is_open", False)
219
220 def _read_cache(self) -> tuple[str | None, dict[str, dict[str, dict[str, Any]]]]:
221     path = self._cache_path()
222     if not path.exists():
223         return None, {}
224     try:
225         payload = json.loads(path.read_text(encoding="utf-8"))
226     except (OSError, ValueError):
227         return None, {}
228     mode = payload.get("mode")
229     if payload.get("dim") != settings.embedding_dim or mode not in ("remote", "local"):
230         return None, {}
231     if mode == "remote" and (
232         not self._remote_embeddings_available()
233         or payload.get("model") != settings.openai_embedding_model
234     ):
235         return None, {}
236     if mode == "local" and self._remote_embeddings_available():
237         logger.info("rag.cache_stale_local_index") # better vectors are available now
238         return None, {}
239     return mode, payload.get("collections") or {}
240
241 def _write_cache(self) -> None:
242     # Only remote vectors are worth keeping: local ones take well under a second
243     # to recompute, and writing them would throw away a remote cache that cost
244     # minutes of gateway quota whenever the gateway is briefly unreachable.
245     if self._embed_mode != "remote":
246         return
247     payload = {
248         "mode": self._embed_mode,
249         "model": settings.openai_embedding_model if self._embed_mode == "remote" else "hashed",
250         "dim": settings.embedding_dim,
251         "collections": self._vectors,
252     }
253     path = self._cache_path()
254     try:
255         path.parent.mkdir(parents=True, exist_ok=True)
256         fd, tmp = tempfile.mkstemp(dir=path.parent, suffix=".tmp")
257         with os.fdopen(fd, "w", encoding="utf-8") as handle:
258             json.dump(payload, handle)
259         os.replace(tmp, path)
260     except OSError as exc: # pragma: no cover - cache is best effort
261         logger.warning("rag.cache_write_failed", error=str(exc))
262
263 def _ensure_index_consistency(self) -> None:
264     """Rebuild the index if the embedder changed under us (remote -> local)."""
265     if not self._indexed or self._index_mode is None or self._embed_mode == "unknown":
266         return
267     if self._embed_mode == self._index_mode:
268         return
269     logger.warning("rag.embedder_changed", index_mode=self._index_mode, current=self._embed_mode)
270     from app.database import session_scope
271
272     with session_scope() as db:
273         self._indexed = False
274         self.index_all(db, force=True)
275
276 # -- retrieval -----
277 def retrieve(
278     self,
279     collection: str,
280     query: str,
281     *,
282     k: int = 10,
283     keywords: Iterable[str] | None = None,
284     where: dict[str, Any] | None = None,
285     min_relevance: float = 0.0,
286 ) -> list[RetrievedDocument]:
287     """Hybrid search: dense cosine similarity blended with keyword overlap."""
288     if not query.strip():
289         return []
290     query_vector = self._query_vector(query)
291     self._ensure_index_consistency()
292     # over-fetch, then re-rank with the lexical signal
293     hits = self._store.search(collection, query_vector, k=max(k * 4, 20), where=where)
294     query_tokens = set(tokenize(query)) | {kw.lower() for kw in (keywords or [])}
295     documents: list[RetrievedDocument] = []

```

```

296         for hit in hits:
297             doc_tokens = set(tokenize(hit.text))
298             overlap = len(query_tokens & doc_tokens)
299             lexical = overlap / max(len(query_tokens), 1)
300             dense = max(0.0, hit.score)
301             hybrid = self.DENSE_WEIGHT * dense + self.LEXICAL_WEIGHT * min(1.0, lexical * 1.6)
302             if hybrid * 100 < min_relevance:
303                 continue
304             documents.append(
305                 RetrievedDocument(
306                     id=hit.id,
307                     text=hit.text,
308                     metadata=hit.metadata,
309                     dense_score=round(dense, 4),
310                     lexical_score=round(lexical, 4),
311                     hybrid_score=round(hybrid, 4),
312                 )
313             )
314         documents.sort(key=lambda doc: doc.hybrid_score, reverse=True)
315         return documents[:k]
316
317     QUERY_CACHE_SIZE = 512
318
319     def _query_vector(self, query: str) -> list[float]:
320         """Embed a query once; reuse it while the index is built by the same embedder."""
321         with self._lock:
322             cached = self._query_cache.get(query)
323             if cached and cached[0] == (self._index_mode or self._embed_mode):
324                 self._query_cache.move_to_end(query)
325                 return cached[1]
326             vector = self._embed([query])[0]
327             with self._lock:
328                 self._query_cache[query] = (self._embed_mode, vector)
329                 while len(self._query_cache) > self.QUERY_CACHE_SIZE:
330                     self._query_cache.popitem(last=False)
331             return vector
332
333     def health(self) -> dict[str, Any]:
334         return {
335             "indexed": self._indexed,
336             "store": self._store.health(),
337             "embedding_mode": self._embed_mode,
338             "index_mode": self._index_mode,
339             "embedding_cache": str(self._cache_path()),
340             "embedding_provider": "remote" if self._llm.supports_remote_embeddings else "local-hashed",
341         }
342
343     _rag_service: RAGService | None = None
344     _rag_lock = threading.Lock()
345
346     def get_rag_service() -> RAGService:
347         global _rag_service
348         with _rag_lock:
349             if _rag_service is None:
350                 _rag_service = RAGService()
351             return _rag_service

```

backend/app/services/report_service.py

```

1  """Report assembly and rendering (JSON / Markdown / HTML)."""
2  from __future__ import annotations
3
4  import html
5  from datetime import datetime, timezone
6  from typing import Any
7
8  from app.schemas.common import ConfidenceBand
9  from app.schemas.report import (
10     ConfidenceReport,
11     ECRIntelligenceReport,
12     ReportSection,
13 )
14 from app.services.confidence import compute_confidence
15
16 SECTION_ORDER = [
17     ("executive_summary", "Executive Summary"),
18     ("ecr_overview", "ECR Overview"),
19     ("change_classification", "Change Classification"),
20     ("affected_requirements", "Affected Requirements"),
21     ("historical_defects", "Defect Analysis"),
22     ("team_signals", "Review Comments and Evidence"),
23     ("correlation_findings", "Correlation Findings"),

```

```

24     ("code_impact", "Code Impact"),
25     ("dependency_impact", "Dependency Impact"),
26     ("impacted_components", "Impacted Components"),
27     ("recommended_tests", "Recommended Regression Tests"),
28     ("test_prioritization", "Test Prioritization"),
29     ("mitigations", "Recommended Actions"),
30     ("confidence", "Confidence Score"),
31     ("execution_trace", "Agent Execution Trace"),
32 ]
33
34
35 def build_report(state: dict[str, Any]) -> ECRIntelligenceReport:
36     """Assemble the final report object from the workflow state."""
37     ecr = state.get("ecr_input") or {}
38     impact = state.get("impact_analysis") or {}
39     selection = state.get("test_selection") or {}
40     prioritization = state.get("test_prioritization") or {}
41     collaboration = state.get("collaboration_analysis") or {}
42     correlation = state.get("correlation") or {}
43     errors = state.get("errors") or []
44
45     overall, penalties = compute_confidence(state.get("step_confidence") or {}, errors=errors)
46     confidence = ConfidenceReport(
47         overall=overall,
48         band=ConfidenceBand.from_score(overall),
49         per_step=state.get("step_confidence") or {},
50         penalties=penalties,
51     )
52
53     report = ECRIntelligenceReport(
54         workflow_id=state.get("workflow_id", ""),
55         ecr_id=state.get("ecr_id", ""),
56         generated_at=datetime.now(timezone.utc),
57         title=ecr.get("title", ""),
58         executive_summary=state.get("final_answer") or impact.get("executive_summary", ""),
59         ecr_overview={
60             "ecr_id": state.get("ecr_id", ""),
61             "title": ecr.get("title", ""),
62             "description": ecr.get("description", ""),
63             "business_domain": ecr.get("business_domain", ""),
64             "priority": ecr.get("priority", ""),
65             "requested_by": ecr.get("requested_by", ""),
66             "target_release": ecr.get("target_release", ""),
67             "changed_files": ecr.get("changed_files", []),
68         },
69         change_classification=state.get("ecr_analysis"),
70         retrieval_plan=state.get("retrieval_plan") or {},
71         correlation_review=state.get("correlation_review") or {},
72         requirement_analysis=state.get("requirement_analysis"),
73         defect_analysis=state.get("defect_analysis"),
74         defect_insights=state.get("defect_insights") or {},
75         defect_summary=state.get("defect_summary") or "",
76         code_impact=_strip_extras(state.get("code_impact")),
77         dependency_analysis=_strip_extras(state.get("dependency_analysis")),
78         impact_assessment=impact or None,
79         test_selection=selection or None,
80         test_prioritization=prioritization or None,
81         mitigations=impact.get("mitigations", []),
82         confidence=confidence,
83         execution_trace=state.get("execution_trace") or [],
84         errors=errors,
85         approval=state.get("approval") or {},
86         metrics={
87             "total_tests": selection.get("total_available", 0),
88             "candidate_tests": selection.get("total_candidates", 0),
89             "selected_tests": len(selection.get("selected_tests") or []),
90             "reduction_percentage": selection.get("reduction_percentage", 0.0),
91             "estimated_duration_minutes": selection.get("estimated_duration_minutes", 0.0),
92             "baseline_duration_minutes": selection.get("baseline_duration_minutes", 0.0),
93             "priority_distribution": selection.get("priority_distribution", {}),
94             "correlated_artefacts": len((correlation.get("graph") or {}).get("nodes") or []),
95             "correlated_links": len((correlation.get("graph") or {}).get("edges") or []),
96             "comments": collaboration.get("comment_count", 0),
97             "evidence": collaboration.get("evidence_count", 0),
98         },
99     )
100     report.sections = _build_sections(report, state)
101     return report
102
103
104 def _strip_extras(payload: dict[str, Any] | None) -> dict[str, Any] | None:
105     """Drop the internal-only keys that are not part of the published schema."""
106     if not payload:
107         return None

```

```

108     drop = {"propagation", "distances", "graph_stats", "dependent_files", "unresolved_files",
109            "repository", "modules_indexed"}
110     return {k: v for k, v in payload.items() if k not in drop}
111
112
113 def _build_sections(report: ECRIntelligenceReport, state: dict[str, Any]) -> list[ReportSection]:
114     impact = state.get("impact_analysis") or {}
115     selection = state.get("test_selection") or {}
116     prioritization = state.get("test_prioritization") or {}
117     collaboration = state.get("collaboration_analysis") or {}
118     requirement_analysis = state.get("requirement_analysis") or {}
119     defect_analysis = state.get("defect_analysis") or {}
120     dependency = state.get("dependency_analysis") or {}
121     code_impact = state.get("code_impact") or {}
122     classification = state.get("ecr_analysis") or {}
123
124     sections: list[ReportSection] = []
125     for key, title in SECTION_ORDER:
126         body = ""
127         data: dict[str, Any] = {}
128         if key == "executive_summary":
129             body = report.executive_summary
130         elif key == "ecr_overview":
131             body = report.ecr_overview.get("description", "")
132             data = report.ecr_overview
133         elif key == "change_classification":
134             body = classification.get("summary", "")
135             data = {"classification": classification, "retrieval_plan": state.get("retrieval_plan") or {}}
136         elif key == "affected_requirements":
137             body = requirement_analysis.get("reasoning", "")
138             data = {"requirements": requirement_analysis.get("matched_requirements", [])}
139         elif key == "historical_defects":
140             insights = state.get("defect_insights") or {}
141             body = state.get("defect_summary") or defect_analysis.get("reasoning", "")
142             data = {
143                 "defects": insights.get("defects") or defect_analysis.get("similar_defects", []),
144                 "not_covered": insights.get("not_covered", []),
145                 "common_causes": insights.get("common_causes", []),
146             }
147         elif key == "team_signals":
148             body = collaboration.get("reasoning", "")
149             data = {
150                 "decisions": (collaboration.get("signals") or {}).get("decisions", []),
151                 "risks": (collaboration.get("signals") or {}).get("risks", []),
152                 "open_concerns": collaboration.get("open_concerns", []),
153                 "evidence_gaps": collaboration.get("evidence_gaps", []),
154             }
155         elif key == "correlation_findings":
156             review = state.get("correlation_review") or {}
157             lines = [
158                 f"- {label}: {item['text']}"
159                 for label, bucket in (("Contradiction", "conflicts"), ("Gap", "gaps"), ("Key link", "key_li
160 + nks"))
161                 for item in review.get(bucket) or []
162             ]
163             body = "\n".join([review.get("summary", ""), *lines]).strip()
164             data = review
165         elif key == "code_impact":
166             body = code_impact.get("reasoning", "")
167             data = {
168                 "files": code_impact.get("directly_impacted_files", []),
169                 "apis": code_impact.get("impacted_apis", []),
170                 "symbols": code_impact.get("impacted_symbols", [])[:15],
171             }
172         elif key == "dependency_impact":
173             body = dependency.get("reasoning", "Dependency analysis was not required for this change.")
174             data = {
175                 "direct": dependency.get("direct_dependencies", []),
176                 "indirect": dependency.get("indirect_dependencies", []),
177                 "critical_paths": dependency.get("critical_dependency_paths", []),
178             }
179         elif key == "impacted_components":
180             body = impact.get("technical_impact", "")
181             data = {
182                 "direct": impact.get("directly_impacted_components", []),
183                 "indirect": impact.get("indirectly_impacted_components", []),
184             }
185         elif key == "recommended_tests":
186             body = selection.get("reasoning", "")
187             data = {
188                 "selected": selection.get("selected_tests", []),
189                 "metrics": report.metrics,
190             }
191         elif key == "test_prioritization":

```

```

191         body = prioritization.get("reasoning", "")
192         data = {
193             "waves": prioritization.get("waves", {}),
194             "strategy": prioritization.get("strategy", ""),
195         }
196     elif key == "mitigations":
197         body = "\n".join(f"- {item}" for item in report.mitigations)
198         data = {"mitigations": report.mitigations}
199     elif key == "confidence":
200         body = (
201             f"Overall confidence {report.confidence.overall:.0%} "
202             f"({report.confidence.band.value})."
203         )
204         data = report.confidence.model_dump(mode="json")
205     elif key == "execution_trace":
206         body = f"{len(report.execution_trace)} step(s) executed."
207         data = {"trace": report.execution_trace}
208     sections.append(ReportSection(key=key, title=title, body=body, data=data))
209 return sections
210
211
212 # -----
213 # Renderers
214 # -----
215 def render_markdown(report: ECRIntelligenceReport) -> str:
216     lines: list[str] = [
217         f"# ECR Intelligence Report - {report.ecr_id}",
218         "",
219         f"***{report.title}***",
220         "",
221         f"- Generated: {report.generated_at.isoformat()}",
222         f"- Workflow: `{report.workflow_id}`",
223         f"- Confidence: **{report.confidence.overall:.0%} ({report.confidence.band.value})***",
224         "",
225     ]
226     metrics = report.metrics
227     if metrics.get("total_tests"):
228         lines += [
229             "## Regression Optimisation",
230             "",
231             f"| Metric | Value |",
232             f"| --- | --- |",
233             f"| Full regression suite | {metrics.get('total_tests', 0)} tests |",
234             f"| Candidates discovered | {metrics.get('candidate_tests', 0)} tests |",
235             f"| AI recommended | {metrics.get('selected_tests', 0)} tests |",
236             f"| Reduction | {metrics.get('reduction_percentage', 0)}% |",
237             f"| Estimated runtime | {metrics.get('estimated_duration_minutes', 0)} min |",
238             f"| (baseline {metrics.get('baseline_duration_minutes', 0)} min) |",
239             "",
240         ]
241     for section in report.sections:
242         if section.key in ("executive_summary",):
243             lines += [f"## {section.title}", "", section.body or "_Not available._", ""]
244             continue
245         lines += [f"## {section.title}", "", section.body or "_Not available._", ""]
246         if section.key == "affected_requirements":
247             for requirement in section.data.get("requirements", []):
248                 lines.append(
249                     f"- **{requirement['requirement_id']}** {requirement['title']} - "
250                     f"{requirement['relevance']:.0f}% ({requirement['match_type']}: {requirement['reason']"
251                 + }"
252             )
253             lines.append("")
254         if section.key == "historical_defects":
255             for defect in section.data.get("defects", []):
256                 if defect.get("chance_of_recurrence"):
257                     lines.append(
258                         f"- **{defect['defect_id']}** [{defect['severity']}] {defect['title']}. "
259                         f"Can it happen again: {defect['chance_of_recurrence']}. "
260                         f"{defect['why_it_matters']} {defect['what_to_check']}"
261                     )
262                 else: # reports saved before defect analysis existed
263                     lines.append(
264                         f"- **{defect['defect_id']}** [{defect['severity']}] {defect['title']} - "
265                         f"{defect.get('similarity', 0):.0f}% similar. {defect.get('reason', '')}"
266                     )
267             lines.append("")
268         if section.key == "recommended_tests":
269             lines += ["| Test | Priority | Relevance | Component | Reason |", "| --- | --- | --- | --- | --"
270                 + - "|"]
271             for test in section.data.get("selected", [])[:60]:
272                 lines.append(
273                     f"| {test['test_case_id']} | {test['priority']} | {test['relevance_score']:.0f}% | "
274                     f"{test['component']} | {test['reason']} |"

```

```

273         )
274         lines.append("")
275     return "\n".join(lines)
276
277
278 def render_html(report: ECRIntelligenceReport) -> str:
279     def esc(value: Any) -> str:
280         return html.escape(str(value))
281
282     rows = "".join(
283         f"<tr><td>{esc(t['test_case_id'])}</td><td><span class='pill {esc(t['priority'])}'>"
284         f"{esc(t['priority'])}</span></td><td>{esc(round(t['relevance_score'])}%</td>"
285         f"<td>{esc(t['component'])}</td><td>{esc(t['reason'])}</td></tr>"
286         for section in report.sections
287         if section.key == "recommended_tests"
288         for t in section.data.get("selected", [])[:80]
289     )
290     sections_html = "".join(
291         f"<section><h2>{esc(s.title)}</h2><p>{esc(s.body).replace(chr(10), '<br/>')}</p></section>"
292         for s in report.sections
293         if s.key not in ("recommended_tests", "execution_trace")
294     )
295     return f"""<!doctype html>
296 <html lang="en"><head><meta charset="utf-8"/>
297 <title>ECR Intelligence Report - {esc(report.ecr_id)}</title>
298 <style>
299 body {{ font-family: 'Segoe UI', system-ui, sans-serif; background:#0b1020; color:#e6ebff; margin:0; paddi
+ ng:40px; }}
300 h1 {{ margin:0 0 4px; font-size:28px; }}
301 h2 {{ font-size:16px; text-transform:uppercase; letter-spacing:.08em; color:#8ea2ff; margin:32px 0 8px; }}
302 .meta {{ color:#9aa7c7; margin-bottom:24px; }}
303 .score {{ display:inline-block; padding:12px 20px; border-radius:12px; background:#161d38; margin-right:12
+ px; }}
304 .score b {{ font-size:28px; }}
305 table {{ width:100%; border-collapse:collapse; margin-top:12px; font-size:13px; }}
306 th, td {{ text-align:left; padding:8px 10px; border-bottom:1px solid #223; vertical-align:top; }}
307 th {{ color:#8ea2ff; text-transform:uppercase; font-size:11px; letter-spacing:.06em; }}
308 .pill {{ padding:2px 8px; border-radius:999px; font-size:11px; font-weight:600; }}
309 .P0 {{ background:#5b1a2a; color:#fffb3c2; }} .P1 {{ background:#5a3c12; color:#fffd79a; }}
310 .P2 {{ background:#14385a; color:#a8d5ff; }} .P3 {{ background:#233; color:#9aa7c7; }}
311 section p {{ line-height:1.6; color:#c9d3ee; }}
312 </style></head><body>
313 <h1>ECR Intelligence Report</h1>
314 <div class="meta">{esc(report.ecr_id)} &middot; {esc(report.title)} &middot; generated {esc(report.generate
+ d_at.isoformat())}</div>
315 <div class="score">Confidence<br/><b>{esc(round(report.confidence.overall * 100))}%</b> {esc(report.confide
+ nce.band.value)}</div>
316 <div class="score">Regression<br/><b>{esc(report.metrics.get('selected_tests', 0))}</b> of {esc(report.metr
+ ics.get('total_tests', 0))} tests
317 ({esc(report.metrics.get('reduction_percentage', 0))}% reduction)</div>
318 {sections_html}
319 <section><h2>Recommended Regression Tests</h2>
320 <table><thead><tr><th>Test</th><th>Priority</th><th>Relevance</th><th>Component</th><th>Why selected</th></
+ tr></thead>
321 <tbody>{rows}</tbody></table></section>
322 </body></html>"""

```

backend/app/services/test_selection_service.py

```

1  """Explainable regression test selection and prioritisation.
2
3  Relevance (0-100) per test:
4
5      0.30 requirement match
6      + 0.25 component match
7      + 0.15 dependency relevance
8      + 0.15 historical failure probability
9      + 0.15 semantic similarity
10
11  Weights come from ``TEST_WEIGHT_*``. Tests scoring below
12  ``TEST_SELECTION_THRESHOLD`` are excluded, the rest are banded
13  P0 (>=90) / P1 (>=75) / P2 (>=50) / P3.
14  """
15  from __future__ import annotations
16
17  from dataclasses import dataclass, field
18  from typing import Any, Iterable
19
20  from app.config import settings
21  from app.schemas.common import Priority
22  from app.schemas.test_case import (
23      SelectedTest,
24      TestCaseRead,
25      TestScoreBreakdown,

```

```

26 )
27
28 CRITICALITY_INDEX = {"CRITICAL": 1.0, "HIGH": 0.75, "MEDIUM": 0.45, "LOW": 0.2}
29
30
31 @dataclass(slots=True)
32 class SelectionContext:
33     """Impact facts the selection algorithm scores tests against."""
34
35     matched_requirements: dict[str, float] = field(default_factory=dict) # req -> relevance 0..100
36     direct_components: set[str] = field(default_factory=set)
37     indirect_components: set[str] = field(default_factory=set)
38     dependency_distance: dict[str, int] = field(default_factory=dict) # component -> hops
39     propagation: dict[str, float] = field(default_factory=dict) # component -> 0..1
40     component_criticality: dict[str, str] = field(default_factory=dict)
41     defect_components: set[str] = field(default_factory=set)
42     semantic_scores: dict[str, float] = field(default_factory=dict) # test id -> 0..100
43     affected_features: set[str] = field(default_factory=set)
44     listed_tests: set[str] = field(default_factory=set) # affected test cases named on the ECR
45
46
47 class TestSelectionEngine:
48     def __init__(self, weights=None, threshold: float | None = None) -> None:
49         self.weights = weights or settings.test_weights
50         self.threshold = threshold if threshold is not None else settings.test_selection_threshold
51
52     # -- individual factors (0..1) -----
53     def requirement_factor(self, test: dict[str, Any], ctx: SelectionContext) -> tuple[float, str]:
54         relevance = ctx.matched_requirements.get(test.get("requirement_id", ""), 0.0)
55         if relevance:
56             return min(1.0, relevance / 100.0), (
57                 f"covers impacted requirement {test['requirement_id']} "
58                 f"({relevance:.0f}% relevance)"
59             )
60         if test.get("feature") in ctx.affected_features:
61             return 0.45, f"exercises affected feature '{test.get('feature')}'"
62         return 0.0, ""
63
64     def component_factor(self, test: dict[str, Any], ctx: SelectionContext) -> tuple[float, str]:
65         component = test.get("component", "")
66         linked = set(test.get("linked_components") or [])
67         if component in ctx.direct_components:
68             return 1.0, f"targets directly modified component {component}"
69         if linked & ctx.direct_components:
70             hit = sorted(linked & ctx.direct_components)[0]
71             return 0.7, f"touches directly modified component {hit}"
72         if component in ctx.indirect_components:
73             return 0.55, f"targets indirectly impacted component {component}"
74         if linked & ctx.indirect_components:
75             return 0.35, f"touches an indirectly impacted component"
76         return 0.0, ""
77
78     def dependency_factor(self, test: dict[str, Any], ctx: SelectionContext) -> tuple[float, str]:
79         component = test.get("component", "")
80         candidates = [component, *(test.get("linked_components") or [])]
81         best = 0.0
82         reason = ""
83         for candidate in candidates:
84             propagation = ctx.propagation.get(candidate, 0.0)
85             if propagation > best:
86                 best = propagation
87                 distance = ctx.dependency_distance.get(candidate, 0)
88                 reason = (
89                     f"{candidate} is {distance} dependency hop(s) from the change "
90                     f"({propagation:.2f})"
91                 )
92             if candidate in ctx.direct_components and best < 1.0:
93                 best = 1.0
94                 reason = f"{candidate} is at the centre of the change"
95         return best, reason
96
97     def historical_factor(self, test: dict[str, Any], ctx: SelectionContext) -> tuple[float, str]:
98         rate = float(test.get("historical_failure_rate", 0.0) or 0.0)
99         value = min(1.0, rate / 0.30)
100         reason = ""
101         if rate >= 0.15:
102             reason = f"historically fails in {rate * 100:.0f}% of runs"
103         component = test.get("component", "")
104         if component in ctx.defect_components:
105             value = min(1.0, value + 0.35)
106             reason = (reason + "; " if reason else "") + (
107                 f"{component} has related historical defects"
108             )
109         return value, reason

```

```

110
111 def semantic_factor(self, test: dict[str, Any], ctx: SelectionContext) -> tuple[float, str]:
112     score = ctx.semantic_scores.get(test.get("test_case_id", ""), 0.0)
113     value = min(1.0, score / 100.0)
114     reason = f"semantic similarity to the change description {score:.0f}%" if score >= 45 else ""
115     return value, reason
116
117 # -- scoring -----
118 def score_test(self, test: dict[str, Any], ctx: SelectionContext) -> SelectedTest:
119     req_v, req_r = self.requirement_factor(test, ctx)
120     cmp_v, cmp_r = self.component_factor(test, ctx)
121     dep_v, dep_r = self.dependency_factor(test, ctx)
122     his_v, his_r = self.historical_factor(test, ctx)
123     sem_v, sem_r = self.semantic_factor(test, ctx)
124
125     breakdown = TestScoreBreakdown(
126         requirement_match=round(100 * self.weights.requirement_match * req_v, 1),
127         component_match=round(100 * self.weights.component_match * cmp_v, 1),
128         dependency_relevance=round(100 * self.weights.dependency_relevance * dep_v, 1),
129         historical_failure=round(100 * self.weights.historical_failure * his_v, 1),
130         semantic_similarity=round(100 * self.weights.semantic_similarity * sem_v, 1),
131     )
132     score = round(
133         breakdown.requirement_match
134         + breakdown.component_match
135         + breakdown.dependency_relevance
136         + breakdown.historical_failure
137         + breakdown.semantic_similarity,
138         1,
139     )
140     reasons = [r for r in (req_r, cmp_r, dep_r, his_r, sem_r) if r]
141     # A test the ECR names as affected is always kept, whatever it scores.
142     listed = test.get("test_case_id") in ctx.listed_tests
143     if listed:
144         reasons.insert(0, "listed as an affected test case on the ECR")
145     payload = {k: v for k, v in test.items() if k in TestCaseRead.model_fields}
146     return SelectedTest(
147         **payload,
148         relevance_score=score,
149         priority=Priority.from_score(score),
150         breakdown=breakdown,
151         reasons=reasons,
152         reason=_compose_reason(reasons, score),
153         selected=listed or score >= self.threshold,
154     )
155
156 def select(
157     self,
158     tests: Iterable[dict[str, Any]],
159     ctx: SelectionContext,
160     *,
161     max_tests: int | None = None,
162 ) -> tuple[list[SelectedTest], list[SelectedTest]]:
163     """Return (selected, rejected) scored tests, highest relevance first."""
164     scored = [self.score_test(test, ctx) for test in tests]
165     scored.sort(key=lambda t: t.relevance_score, reverse=True)
166     selected = [t for t in scored if t.selected]
167     limit = max_tests if max_tests is not None else settings.max_selected_tests
168     if limit and len(selected) > limit:
169         for test in selected[limit:]:
170             test.selected = False
171         selected = selected[:limit]
172     rejected = [t for t in scored if not t.selected]
173     return selected, rejected
174
175 # -- prioritisation -----
176 def prioritise(self, tests: list[SelectedTest], ctx: SelectionContext) -> list[SelectedTest]:
177     """Order execution by priority first, then fastest feedback.
178
179     Sort key: priority band, then tests the ECR names as affected, then
180     criticality of the target component, then relevance, then shortest
181     execution time (quick wins first inside a band so failures surface early).
182     """
183     band_order = {Priority.P0: 0, Priority.P1: 1, Priority.P2: 2, Priority.P3: 3}
184
185     def sort_key(test: SelectedTest):
186         criticality = CRITICALITY_INDEX.get(
187             ctx.component_criticality.get(test.component, "MEDIUM"), 0.45
188         )
189         return (
190             band_order[test.priority],
191             test.test_case_id not in ctx.listed_tests,
192             -criticality,
193             -test.relevance_score,
194         )

```

```

194         test.average_execution_time,
195     )
196
197     ordered = sorted(tests, key=sort_key)
198     for index, test in enumerate(ordered, start=1):
199         test.execution_order = index
200     return ordered
201
202
203 def _compose_reason(reasons: list[str], score: float) -> str:
204     if not reasons:
205         return "Retained for baseline coverage of the impacted area."
206     head = reasons[0][0].upper() + reasons[0][1:]
207     tail = "; ".join(reasons[1:3])
208     sentence = head + (";" + tail if tail else "")
209     return f"{sentence}. Relevance {score:.0f}%."
210
211
212 def priority_distribution(tests: list[SelectedTest]) -> dict[str, int]:
213     distribution = {"P0": 0, "P1": 0, "P2": 0, "P3": 0}
214     for test in tests:
215         distribution[test.priority.value] += 1
216     return distribution
217
218
219 def get_selection_engine() -> TestSelectionEngine:
220     return TestSelectionEngine()

```

Backend - LLM providers

backend/app/llm/__init__.py

backend/app/llm/anthropic_provider.py

```

1  """Anthropic provider (Messages API over plain HTTP)."""
2  from __future__ import annotations
3
4  import json
5  from typing import Any, TypeVar
6
7  import httpx
8  from pydantic import BaseModel
9
10 from app.config import settings
11 from app.llm.base import BaseLLMProvider, LLMError, hashed_embedding
12 from app.llm.openai_provider import _extract_json
13 from app.logging import get_logger
14
15 logger = get_logger("ecr.llm.anthropic")
16
17 T = TypeVar("T", bound=BaseModel)
18
19
20 class AnthropicProvider(BaseLLMProvider):
21     name = "anthropic"
22     supports_remote_embeddings = False # embeddings stay local (hashed vectors)
23
24     def __init__(self) -> None:
25         self.api_key = settings.anthropic_api_key or ""
26         self.endpoint = settings.anthropic_base_url.rstrip("/")
27         self.model = settings.anthropic_model
28         if not self.api_key:
29             raise LLMError("Anthropic provider selected but no API key is configured")
30
31     def _post(self, payload: dict[str, Any]) -> dict[str, Any]:
32         try:
33             response = httpx.post(
34                 f"{self.endpoint}/messages",
35                 headers={
36                     "x-api-key": self.api_key,
37                     "anthropic-version": "2023-06-01",
38                     "content-type": "application/json",
39                 },
40                 json=payload,
41                 timeout=settings.llm_timeout_seconds,
42             )
43             response.raise_for_status()
44             return response.json()
45         except httpx.HTTPError as exc:
46             raise LLMError(f"Anthropic request failed: {exc}") from exc
47
48     def generate(

```

```

49         self,
50         prompt: str,
51         *,
52         system: str | None = None,
53         max_tokens: int | None = None,
54         temperature: float = 0.2,
55         fast: bool = False,
56     ) -> str:
57         payload: dict[str, Any] = {
58             "model": self.model,
59             "max_tokens": max_tokens or settings.llm_max_tokens,
60             "temperature": temperature,
61             "messages": [{"role": "user", "content": prompt}],
62         }
63         if system:
64             payload["system"] = system
65         data = self._post(payload)
66         blocks = [b.get("text", "") for b in data.get("content", []) if b.get("type") == "text"]
67         return "".join(blocks)
68
69     def generate_structured(
70         self,
71         prompt: str,
72         schema: type[T],
73         *,
74         system: str | None = None,
75         fallback: dict[str, Any] | None = None,
76         temperature: float = 0.0,
77         fast: bool = True,
78     ) -> T:
79         instruction = (
80             f"{prompt}\n\nRespond with a single JSON object (no prose, no code fence) "
81             f"matching this schema:\n{json.dumps(schema.model_json_schema())[:4000]}"
82         )
83         try:
84             raw = self.generate(instruction, system=system, temperature=temperature, fast=fast)
85             return schema.model_validate(_extract_json(raw))
86         except Exception as exc:
87             logger.warning("llm.structured_failed", provider=self.name, error=str(exc))
88             return schema.model_validate(fallback or {})
89
90     def embed(self, texts: list[str]) -> list[list[float]]:
91         return [hashed_embedding(text) for text in texts]

```

backend/app/llm/base.py

```

1  """LLM provider abstraction.
2
3  Every agent talks to the model through this interface only, which is what makes
4  the platform runnable with *no* API key at all (see :mod:`app.llm.mock_provider`).
5
6  Design rule enforced across the codebase:
7      deterministic engines compute the numbers, the LLM only explains them.
8  Hence :meth:`narrate` always receives a ``fallback`` narrative produced by the
9  deterministic layer, and any provider failure degrades to that fallback.
10 """
11 from __future__ import annotations
12
13 import hashlib
14 import math
15 import re
16 from abc import ABC, abstractmethod
17 from typing import Any, TypeVar
18
19 from pydantic import BaseModel
20
21 from app.config import settings
22 from app.logging import get_logger
23 from app.prompts import NARRATION_SYSTEM_PROMPT, NARRATION_TASK
24
25 logger = get_logger("ecr.llm")
26
27 T = TypeVar("T", bound=BaseModel)
28
29 _TOKEN_RE = re.compile(r"[a-z0-9_]+")
30 _STOPWORDS = {
31     "the", "a", "an", "and", "or", "to", "of", "for", "in", "on", "with", "is",
32     "are", "be", "should", "must", "that", "this", "it", "as", "by", "from",
33     "when", "will", "shall", "at", "we", "if", "not", "no", "than", "then",
34 }
35
36
37 def tokenize(text: str) -> list[str]:
38     """Lower-cased, stop-word filtered tokens (also used by the vector store)."""

```

```

39     return [t for t in _TOKEN_RE.findall((text or "").lower()) if t not in _STOPWORDS and len(t) > 2]
40
41
42 def hashed_embedding(text: str, dim: int | None = None) -> list[float]:
43     """Deterministic hashing-trick embedding.
44
45     Behaves like an L2-normalised sparse lexical vector: cosine similarity
46     approximates weighted token overlap, which gives the PoC meaningful
47     "semantic" search with zero external calls and zero model download.
48     """
49     dim = dim or settings.embedding_dim
50     vector = [0.0] * dim
51     tokens = tokenize(text)
52     if not tokens:
53         return vector
54     counts: dict[str, int] = {}
55     for token in tokens:
56         counts[token] = counts.get(token, 0) + 1
57     for token, count in counts.items():
58         digest = hashlib.md5(token.encode("utf-8")).digest()
59         idx = int.from_bytes(digest[:4], "big") % dim
60         sign = 1.0 if digest[4] % 2 == 0 else -1.0
61         # sub-linear term weighting keeps long documents from dominating
62         vector[idx] += sign * (1.0 + math.log(count))
63         # a second bucket reduces hash collisions
64         idx2 = int.from_bytes(digest[5:9], "big") % dim
65         vector[idx2] += sign * 0.5
66     norm = math.sqrt(sum(v * v for v in vector)) or 1.0
67     return [v / norm for v in vector]
68
69
70 class LLMError(RuntimeError):
71     """Raised when a remote provider cannot fulfil a request."""
72
73     def __init__(self, message: str, *, status_code: int | None = None) -> None:
74         super().__init__(message)
75         self.status_code = status_code # HTTP status from the endpoint, if any
76
77
78 class BaseLLMProvider(ABC):
79     """Contract implemented by every provider."""
80
81     name: str = "base"
82     is_mock: bool = False
83     supports_remote_embeddings: bool = False
84
85     @abstractmethod
86     def generate(
87         self,
88         prompt: str,
89         *,
90         system: str | None = None,
91         max_tokens: int | None = None,
92         temperature: float = 0.2,
93         fast: bool = False,
94     ) -> str:
95         """Free-form completion. ``fast`` selects the cheap narration model."""
96
97     @abstractmethod
98     def generate_structured(
99         self,
100         prompt: str,
101         schema: type[T],
102         *,
103         system: str | None = None,
104         fallback: dict[str, Any] | None = None,
105         temperature: float = 0.0,
106         fast: bool = True,
107     ) -> T:
108         """Validated structured completion (always returns a valid ``schema``).
109
110         ``fast`` selects the cheap model; pass ``False`` for text a user reads first.
111         """
112
113     def embed(self, texts: list[str]) -> list[list[float]]:
114         """Embed documents. Providers without an embedding API use the local one."""
115         return [hashed_embedding(text) for text in texts]
116
117     def narrate(
118         self,
119         *,
120         task: str,
121         context: dict[str, Any],
122         fallback: str,

```

```

123         system: str | None = None,
124         max_tokens: int | None = None,
125     ) -> str:
126         """Explain a deterministic result in prose.
127
128         ``fallback`` is the deterministic narrative; it is returned verbatim by
129         the mock provider, whenever a remote call fails, and unless
130         ``LLM_NARRATION`` is on (each narration is a full LLM round trip, and a
131         user is waiting on the analysis).
132         """
133         if self.is_mock or not settings.llm_narration:
134             return fallback
135         try:
136             prompt = _build_narration_prompt(task, context, fallback)
137             text = self.generate(
138                 prompt,
139                 system=system or NARRATION_SYSTEM_PROMPT,
140                 max_tokens=max_tokens or settings.narration_max_tokens,
141                 temperature=0.2,
142                 fast=True,
143             )
144             return text.strip() or fallback
145         except Exception as exc: # graceful degradation - never break a workflow
146             logger.warning("llm.narrate_failed", provider=self.name, error=str(exc))
147             return fallback
148
149     def health(self) -> dict[str, Any]:
150         return {"provider": self.name, "mock": self.is_mock, "available": True}
151
152
153     def _build_narration_prompt(task: str, context: dict[str, Any], fallback: str) -> str:
154         import json
155
156         return NARRATION_TASK.format(
157             task=task,
158             context=json.dumps(context, indent=2, default=str)[:6000],
159             fallback=fallback,
160         )

```

backend/app/llm/mock_provider.py

```

1  """Zero-dependency provider used when no LLM credentials are configured.
2
3  It is not a stub: agent intelligence in mock mode comes from the deterministic
4  rule engines (selection algorithm, dependency traversal, hashed
5  embeddings). This provider supplies the *language* layer around them.
6  """
7  from __future__ import annotations
8
9  from typing import Any, TypeVar
10
11  from pydantic import BaseModel
12
13  from app.llm.base import BaseLLMProvider, hashed_embedding
14
15  T = TypeVar("T", bound=BaseModel)
16
17
18  class MockProvider(BaseLLMProvider):
19     name = "mock"
20     is_mock = True
21
22     def generate(
23         self,
24         prompt: str,
25         *,
26         system: str | None = None,
27         max_tokens: int | None = None,
28         temperature: float = 0.2,
29         fast: bool = False,
30     ) -> str:
31         """Echo the deterministic baseline embedded in the prompt, if present."""
32         marker = "DETERMINISTIC BASELINE NARRATIVE:"
33         if marker in prompt:
34             return prompt.split(marker, 1)[1].split("\n\nRewrite")[0].strip()
35         return prompt.strip()[: max_tokens or 800]
36
37     def generate_structured(
38         self,
39         prompt: str,
40         schema: type[T],
41         *,
42         system: str | None = None,
43         fallback: dict[str, Any] | None = None,

```

```

44         temperature: float = 0.0,
45         fast: bool = True,
46     ) -> T:
47         return schema.model_validate(fallback or {})
48
49     def embed(self, texts: list[str]) -> list[list[float]]:
50         return [hashed_embedding(text) for text in texts]
51
52     def health(self) -> dict[str, Any]:
53         return {
54             "provider": self.name,
55             "mock": True,
56             "available": True,
57             "note": "Rule-based reasoning with hashed local embeddings. "
58             "Set OPENAI_API_KEY or ANTHROPIC_API_KEY to enable an LLM.",
59         }

```

backend/app/llm/openai_provider.py

```

1  """OpenAI / Azure OpenAI provider (plain HTTP, no vendor SDK required)."""
2  from __future__ import annotations
3
4  import json
5  import time
6  from typing import Any, TypeVar
7
8  import httpx
9  from pydantic import BaseModel
10
11 from app.config import settings
12 from app.llm.base import BaseLLMProvider, LLMError, hashed_embedding
13 from app.logging import get_logger
14
15 logger = get_logger("ecr.llm.openai")
16
17 T = TypeVar("T", bound=BaseModel)
18
19
20 class OpenAIProvider(BaseLLMProvider):
21     name = "openai"
22     supports_remote_embeddings = True
23
24     def __init__(self, *, azure: bool = False) -> None:
25         self.azure = azure
26         if azure:
27             self.name = "azure_openai"
28             self.api_key = settings.azure_openai_api_key or ""
29             self.endpoint = (settings.azure_openai_endpoint or "").rstrip("/")
30             self.model = settings.azure_openai_deployment
31             self.supports_remote_embeddings = False
32         else:
33             self.api_key = settings.openai_api_key or ""
34             self.endpoint = settings.openai_base_url.rstrip("/")
35             self.model = settings.openai_model
36             # Off by default: a remote embedding is a gateway request of its own.
37             self.supports_remote_embeddings = settings.remote_embeddings
38             self.fast_model = settings.openai_fast_model or ""
39         if not self.api_key:
40             raise LLMError("OpenAI provider selected but no API key is configured")
41
42     # -- transport -----
43     def _chat_url(self, model: str | None = None) -> str:
44         if self.azure:
45             return (
46                 f"{self.endpoint}/openai/deployments/{model or self.model}/chat/completions"
47                 f"?api-version={settings.azure_openai_api_version}"
48             )
49         return f"{self.endpoint}/chat/completions"
50
51     def _headers(self) -> dict[str, str]:
52         if self.azure:
53             return {"api-key": self.api_key, "content-type": "application/json"}
54         return {
55             "authorization": f"Bearer {self.api_key}",
56             "content-type": "application/json",
57         }
58
59     def _post(self, url: str, payload: dict[str, Any]) -> dict[str, Any]:
60         """POST, retrying only requests the gateway did not process.
61
62         A throttled (429) or never-connected request is retried with a short
63         backoff, so a burst limit does not degrade the answer. Anything else -
64         a timeout or a 5xx included - may already have been billed, so it fails
65         once and the caller falls back to its rules. One agent call therefore

```

```

66         never costs more than one processed request.
67         """
68         last_error: Exception | None = None
69         status: int | None = None
70         for attempt in range(settings.llm_max_retries + 1):
71             try:
72                 response = httpx.post(
73                     url,
74                     headers=self._headers(),
75                     json=payload,
76                     timeout=settings.llm_timeout_seconds,
77                 )
78                 response.raise_for_status()
79                 return response.json()
80             except httpx.HTTPError as exc:
81                 last_error = exc
82                 status = exc.response.status_code if isinstance(exc, httpx.HTTPStatusError) else None
83                 if not (status == 429 or isinstance(exc, httpx.ConnectError)):
84                     break # the gateway may have processed it: a retry would be a second request
85                 if attempt < settings.llm_max_retries:
86                     time.sleep(self._retry_delay(exc, attempt))
87             raise LLMError(f"OpenAI request failed: {last_error}", status_code=status) from last_error
88
89     @staticmethod
90     def _retry_delay(exc: httpx.HTTPError, attempt: int) -> float:
91         """Honour the gateway's Retry-After on 429; otherwise back off exponentially."""
92         if isinstance(exc, httpx.HTTPStatusError) and exc.response.status_code == 429:
93             header = exc.response.headers.get("retry-after", "")
94             try:
95                 return min(max(float(header), 1.0), settings.llm_rate_limit_max_wait_seconds)
96             except ValueError:
97                 return min(3.0 * (2**attempt), settings.llm_rate_limit_max_wait_seconds)
98         return 0.8 * (2**attempt)
99
100     # -- interface -----
101     def generate(
102         self,
103         prompt: str,
104         *,
105         system: str | None = None,
106         max_tokens: int | None = None,
107         temperature: float = 0.2,
108         fast: bool = False,
109     ) -> str:
110         messages = ([{"role": "system", "content": system}] if system else []) + [
111             {"role": "user", "content": prompt}
112         ]
113         model = self.fast_model if (fast and self.fast_model) else self.model
114         data = self._post(
115             self._chat_url(model),
116             {
117                 "model": model,
118                 "messages": messages,
119                 "max_tokens": max_tokens or settings.llm_max_tokens,
120                 "temperature": temperature,
121             },
122         )
123         return data["choices"][0]["message"]["content"] or ""
124
125     def generate_structured(
126         self,
127         prompt: str,
128         schema: type[T],
129         *,
130         system: str | None = None,
131         fallback: dict[str, Any] | None = None,
132         temperature: float = 0.0,
133         fast: bool = True,
134     ) -> T:
135         instruction = (
136             f"{prompt}\n\nRespond with a single JSON object matching this schema:\n"
137             f"{json.dumps(schema.model_json_schema())[1:4000]}"
138         )
139         try:
140             raw = self.generate(
141                 instruction,
142                 system=system,
143                 temperature=temperature,
144                 max_tokens=settings.llm_max_tokens,
145                 fast=fast,
146             )
147             return schema.model_validate(_extract_json(raw))
148         except Exception as exc:
149             logger.warning("llm.structured_failed", provider=self.name, error=str(exc))

```

```

150         return schema.model_validate(fallback or {})
151
152     def embed(self, texts: list[str]) -> list[list[float]]:
153         """Batched embeddings. Raises on failure so callers can switch modes."""
154         if not self.supports_remote_embeddings:
155             return [hashed_embedding(text) for text in texts]
156         batch = max(1, settings.embedding_batch_size)
157         vectors: list[list[float]] = []
158         for start in range(0, len(texts), batch):
159             chunk = texts[start : start + batch]
160             data = self._post(
161                 f"{self.endpoint}/embeddings",
162                 {"model": settings.openai_embedding_model, "input": chunk},
163             )
164             items = sorted(data["data"], key=lambda item: item.get("index", 0))
165             vectors.extend(item["embedding"] for item in items)
166         return vectors
167
168
169     def _extract_json(raw: str) -> dict[str, Any]:
170         text = raw.strip()
171         if text.startswith("`"):
172             text = text.split("`", 2)[1]
173             text = text.split("\n", 1)[1] if "\n" in text else text
174         start, end = text.find("{"), text.rfind("}")
175         if start == -1 or end == -1:
176             raise ValueError("no JSON object found in model response")
177         return json.loads(text[start : end + 1])

```

backend/app/llm/provider.py

```

1  """Provider factory, startup probe and circuit breaker.
2
3  Two failure modes are handled explicitly because they are common in enterprise
4  networks: (a) no credentials at all, (b) credentials present but the provider
5  endpoint is unreachable (proxy / TLS interception / quota). In both cases the
6  platform must keep working, so the runtime degrades to :class:`MockProvider`.
7  """
8  from __future__ import annotations
9
10 import threading
11 import time
12 from functools import lru_cache
13 from typing import Any, TypeVar
14
15 from pydantic import BaseModel
16
17 from app.config import settings
18 from app.llm.base import BaseLLMProvider
19 from app.llm.mock_provider import MockProvider
20 from app.logging import get_logger
21
22 logger = get_logger("ecr.llm")
23
24 T = TypeVar("T", bound=BaseModel)
25
26
27 def build_provider(name: str | None = None) -> BaseLLMProvider:
28     """Instantiate a raw provider, degrading to :class:`MockProvider`."""
29     resolved = name or settings.resolved_llm_provider()
30     try:
31         if resolved == "openai":
32             from app.llm.openai_provider import OpenAIProvider
33
34             return OpenAIProvider()
35         if resolved == "azure_openai":
36             from app.llm.openai_provider import OpenAIProvider
37
38             return OpenAIProvider(azure=True)
39         if resolved == "anthropic":
40             from app.llm.anthropic_provider import AnthropicProvider
41
42             return AnthropicProvider()
43     except Exception as exc:
44         logger.warning("llm.provider_fallback", requested=resolved, error=str(exc))
45     return MockProvider()
46
47
48 class ResilientProvider(BaseLLMProvider):
49     """Wraps a remote provider with a probe and a circuit breaker.
50
51     After ``max_failures`` consecutive remote errors the breaker opens and every
52     later call is served by the mock provider, so a dead endpoint can never
53     stall an agent workflow.

```

```

54     """
55
56     def __init__(
57         self, primary: BaseLLMProvider, max_failures: int = 3, cooldown_seconds: float = 120.0
58     ) -> None:
59         self._primary = primary
60         self._mock = MockProvider()
61         self._max_failures = max_failures
62         self._cooldown = cooldown_seconds
63         self._failures = 0
64         self._opened_at = 0.0
65         self._open = False
66         self._lock = threading.Lock()
67         self._probe_result: dict[str, Any] = {"probed": False}
68         self.name = primary.name
69         self.is_mock = primary.is_mock
70         self.supports_remote_embeddings = primary.supports_remote_embeddings
71
72     # -- breaker -----
73     @property
74     def is_open(self) -> bool:
75         """Open, unless the cooldown has elapsed - then try the primary again.
76
77         Rate limits and gateway blips are transient, so a permanently open
78         breaker would strand the platform in demo mode for the rest of the
79         process lifetime.
80         """
81         if not self._open:
82             return False
83         if self._cooldown and (time.monotonic() - self._opened_at) >= self._cooldown:
84             logger.info("llm.circuit_half_open", provider=self._primary.name)
85             self._open = False
86             self._failures = 0
87             self.is_mock = self._primary.is_mock
88             return False
89         return True
90
91     @property
92     def active(self) -> BaseLLMProvider:
93         return self._mock if self.is_open else self._primary
94
95     def _record_failure(self, exc: Exception) -> None:
96         with self._lock:
97             self._failures += 1
98             if self._failures >= self._max_failures and not self._open:
99                 self._open = True
100                 self._opened_at = time.monotonic()
101                 self.is_mock = True
102                 logger.warning(
103                     "llm.circuit_open",
104                     provider=self._primary.name,
105                     failures=self._failures,
106                     error=str(exc),
107                 )
108
109     def _record_success(self) -> None:
110         with self._lock:
111             self._failures = 0
112
113     def probe(self) -> dict[str, Any]:
114         """One cheap call verifying the endpoint really answers."""
115         if self._primary.is_mock:
116             self._probe_result = {"probed": True, "reachable": True, "mock": True}
117             return self._probe_result
118         try:
119             text = self._primary.generate(
120                 "Reply with the single word: READY", max_tokens=8, temperature=0.0
121             )
122             self._probe_result = {"probed": True, "reachable": True, "sample": text[:40]}
123             logger.info("llm.probe_ok", provider=self._primary.name)
124         except Exception as exc:
125             if getattr(exc, "status_code", None) == 429:
126                 # The endpoint answered - it is only throttling this key. Stay on the
127                 # real model; the per-call retries absorb short rate-limit windows.
128                 self._probe_result = {"probed": True, "reachable": True, "rate_limited": True}
129                 logger.warning("llm.probe_rate_limited", provider=self._primary.name)
130                 return self._probe_result
131             self._open = True
132             self._opened_at = time.monotonic()
133             self.is_mock = True
134             self._probe_result = {"probed": True, "reachable": False, "error": str(exc)[:300]}
135             logger.warning("llm.probe_failed", provider=self._primary.name, error=str(exc)[:300])
136         return self._probe_result
137

```

```

138 # -- interface -----
139 def generate(self, prompt: str, *, fallback: str | None = None, **kwargs: Any) -> str:
140     """Generate text, degrading to ``fallback`` when the endpoint is unavailable.
141
142     Callers that show the text to a user must pass their grounded baseline as
143     ``fallback``: the mock provider echoes the prompt, which must never reach
144     the page.
145     """
146     if self.is_open:
147         return fallback if fallback is not None else self._mock.generate(prompt, **kwargs)
148     try:
149         result = self._primary.generate(prompt, **kwargs)
150         self._record_success()
151         return result
152     except Exception as exc:
153         self._record_failure(exc)
154         return fallback if fallback is not None else self._mock.generate(prompt, **kwargs)
155
156 def generate_structured(self, prompt: str, schema: type[T], **kwargs: Any) -> T:
157     if self.is_open:
158         return self._mock.generate_structured(prompt, schema, **kwargs)
159     try:
160         result = self._primary.generate_structured(prompt, schema, **kwargs)
161         self._record_success()
162         return result
163     except Exception as exc:
164         self._record_failure(exc)
165         return self._mock.generate_structured(prompt, schema, **kwargs)
166
167 def embed(self, texts: list[str]) -> list[list[float]]:
168     return self.embed_with_mode(texts)[0]
169
170 def embed_with_mode(self, texts: list[str]) -> tuple[list[list[float]], str]:
171     """Embed and report which embedder produced the vectors.
172
173     Vectors from two different embedders are not comparable, so the RAG
174     layer needs to know when a fallback happened in order to re-index.
175     """
176     if self.is_open or not self._primary.supports_remote_embeddings:
177         return self._mock.embed(texts), "local"
178     try:
179         result = self._primary.embed(texts)
180         self._record_success()
181         return result, "remote"
182     except Exception as exc:
183         self._record_failure(exc)
184         logger.warning("llm.embed_fallback", provider=self._primary.name, error=str(exc)[:200])
185         return self._mock.embed(texts), "local"
186
187 def narrate(self, **kwargs: Any) -> str:
188     if self.is_open:
189         return self._mock.narrate(**kwargs)
190     try:
191         return self._primary.narrate(**kwargs)
192     except Exception as exc: # pragma: no cover - narrate already guards
193         self._record_failure(exc)
194         return self._mock.narrate(**kwargs)
195
196 def health(self) -> dict[str, Any]:
197     return {
198         "provider": self._primary.name,
199         "configured_provider": settings.llm_provider,
200         "effective_provider": "mock" if self.is_open else self._primary.name,
201         "mock": self.is_mock,
202         "circuit_open": self.is_open,
203         "cooldown_seconds": self._cooldown,
204         "consecutive_failures": self._failures,
205         "probe": self._probe_result,
206         "available": True,
207     }
208
209
210 @lru_cache
211 def get_llm() -> ResilientProvider:
212     provider = ResilientProvider(build_provider())
213     logger.info(
214         "llm.provider_selected", provider=provider.name, mock=provider.is_mock
215     )
216     return provider

```

Backend - vector store

backend/app/vectorstore/__init__.py

```
1  """Vector store factory.
2
3  Collection names are shared constants so the RAG service, the seeder and the
4  agents all address the same indexes.
5  """
6  from __future__ import annotations
7
8  from functools import lru_cache
9
10 from app.config import settings
11 from app.logging import get_logger
12 from app.vectorstore.base import SearchHit, VectorRecord, VectorStore
13 from app.vectorstore.memory_store import InMemoryVectorStore
14
15 logger = get_logger("ecr.vectorstore")
16
17 COLLECTION_REQUIREMENTS = "requirements"
18 COLLECTION_DEFECTS = "defects"
19 COLLECTION_TESTS = "test_cases"
20 COLLECTION_ECRS = "ecrs"
21 ALL_COLLECTIONS = (
22     COLLECTION_REQUIREMENTS,
23     COLLECTION_DEFECTS,
24     COLLECTION_TESTS,
25     COLLECTION_ECRS,
26 )
27
28
29 def build_vector_store(kind: str | None = None) -> VectorStore:
30     resolved = kind or settings.vector_store
31     try:
32         if resolved == "pgvector":
33             from app.vectorstore.pgvector_store import PgVectorStore
34
35             return PgVectorStore()
36         if resolved == "chroma":
37             from app.vectorstore.chroma_store import ChromaVectorStore
38
39             return ChromaVectorStore()
40     except Exception as exc:
41         logger.warning("vectorstore.fallback", requested=resolved, error=str(exc))
42     return InMemoryVectorStore()
43
44
45 @lru_cache
46 def get_vector_store() -> VectorStore:
47     store = build_vector_store()
48     logger.info("vectorstore.selected", store=store.name)
49     return store
50
51
52 __all__ = [
53     "ALL_COLLECTIONS",
54     "COLLECTION_DEFECTS",
55     "COLLECTION_ECRS",
56     "COLLECTION_REQUIREMENTS",
57     "COLLECTION_TESTS",
58     "SearchHit",
59     "VectorRecord",
60     "VectorStore",
61     "build_vector_store",
62     "get_vector_store",
63 ]
```

backend/app/vectorstore/base.py

```
1  """Vector store abstraction used by the RAG layer."""
2  from __future__ import annotations
3
4  from abc import ABC, abstractmethod
5  from dataclasses import dataclass, field
6  from typing import Any, Iterable
7
8
9  @dataclass(slots=True)
10 class VectorRecord:
11     id: str
12     text: str
13     vector: list[float]
```

```

14     metadata: dict[str, Any] = field(default_factory=dict)
15
16
17 @dataclass(slots=True)
18 class SearchHit:
19     id: str
20     score: float
21     text: str
22     metadata: dict[str, Any] = field(default_factory=dict)
23
24
25 class VectorStore(ABC):
26     """Minimal contract: upsert documents, search by vector, inspect size."""
27
28     name: str = "base"
29
30     @abstractmethod
31     def upsert(self, collection: str, records: Iterable[VectorRecord]) -> int: ...
32
33     @abstractmethod
34     def search(
35         self,
36         collection: str,
37         query_vector: list[float],
38         *,
39         k: int = 10,
40         min_score: float = 0.0,
41         where: dict[str, Any] | None = None,
42     ) -> list[SearchHit]: ...
43
44     @abstractmethod
45     def count(self, collection: str) -> int: ...
46
47     def reset(self, collection: str | None = None) -> None: # pragma: no cover
48         raise NotImplementedError
49
50     def health(self) -> dict[str, Any]:
51         return {"vector_store": self.name}

```

backend/app/vectorstore/chroma_store.py

```

1  """ChromaDB backed store (optional local alternative to pgvector).
2
3  ``chromadb`` is intentionally *not* a hard dependency: import errors surface as
4  a clean RuntimeError so the factory can fall back to the in-memory store.
5  """
6  from __future__ import annotations
7
8  from typing import Any, Iterable
9
10 from app.config import settings
11 from app.logging import get_logger
12 from app.vectorstore.base import SearchHit, VectorRecord, VectorStore
13
14 logger = get_logger("ecr.vectorstore.chroma")
15
16
17 class ChromaVectorStore(VectorStore):
18     name = "chroma"
19
20     def __init__(self) -> None:
21         try:
22             import chromadb
23         except ImportError as exc: # pragma: no cover - optional dependency
24             raise RuntimeError("chromadb is not installed (pip install chromadb)") from exc
25         self._client = chromadb.PersistentClient(path=settings.chroma_path)
26         self._collections: dict[str, Any] = {}
27         logger.info("chroma.ready", path=settings.chroma_path)
28
29     def _collection(self, name: str):
30         if name not in self._collections:
31             self._collections[name] = self._client.get_or_create_collection(
32                 name=name, metadata={"hnsw:space": "cosine"}
33             )
34         return self._collections[name]
35
36     def upsert(self, collection: str, records: Iterable[VectorRecord]) -> int:
37         items = list(records)
38         if not items:
39             return 0
40         self._collection(collection).upsert(
41             ids=[r.id for r in items],
42             embeddings=[r.vector for r in items],
43             documents=[r.text for r in items],

```

```

44         metadatas=[_flatten(r.metadata) for r in items],
45     )
46     return len(items)
47
48     def search(
49         self,
50         collection: str,
51         query_vector: list[float],
52         *,
53         k: int = 10,
54         min_score: float = 0.0,
55         where: dict[str, Any] | None = None,
56     ) -> list[SearchHit]:
57         result = self._collection(collection).query(
58             query_embeddings=[query_vector],
59             n_results=max(k * 3, k),
60             where=_flatten(where) if where else None,
61         )
62         hits: list[SearchHit] = []
63         ids = (result.get("ids") or [[]])[0]
64         docs = (result.get("documents") or [[]])[0]
65         metas = (result.get("metadatas") or [[]])[0]
66         distances = (result.get("distances") or [[]])[0]
67         for idx, doc_id in enumerate(ids):
68             score = 1.0 - float(distances[idx])
69             if score < min_score:
70                 continue
71             hits.append(
72                 SearchHit(
73                     id=doc_id,
74                     score=score,
75                     text=docs[idx] if idx < len(docs) else "",
76                     metadata=dict(metas[idx] or {}) if idx < len(metas) else {},
77                 )
78             )
79             if len(hits) >= k:
80                 break
81         return hits
82
83     def count(self, collection: str) -> int:
84         return int(self._collection(collection).count())
85
86     def reset(self, collection: str | None = None) -> None:
87         names = [collection] if collection else list(self._collections)
88         for name in names:
89             try:
90                 self._client.delete_collection(name)
91             except Exception: # pragma: no cover
92                 pass
93         self._collections.pop(name, None)
94
95     def _flatten(metadata: dict[str, Any]) -> dict[str, Any]:
96         """Chroma only accepts scalar metadata values."""
97         clean: dict[str, Any] = {}
98         for key, value in (metadata or {}).items():
99             if isinstance(value, (str, int, float, bool)):
100                 clean[key] = value
101             elif isinstance(value, (list, tuple)):
102                 clean[key] = ",".join(str(item) for item in value)
103             elif value is not None:
104                 clean[key] = str(value)
105         return clean
106

```

backend/app/vectorstore/memory_store.py

```

1  """In-process numpy vector store -- the default for local/PoC runs."""
2  from __future__ import annotations
3
4  import threading
5  from typing import Any, Iterable
6
7  import numpy as np
8
9  from app.vectorstore.base import SearchHit, VectorRecord, VectorStore
10
11
12  class InMemoryVectorStore(VectorStore):
13     name = "memory"
14
15     def __init__(self) -> None:
16         self._lock = threading.RLock()
17         self._matrix: dict[str, np.ndarray] = {}
18         self._records: dict[str, list[VectorRecord]] = {}

```

```

19
20 def upsert(self, collection: str, records: Iterable[VectorRecord]) -> int:
21     items = list(records)
22     if not items:
23         return 0
24     with self._lock:
25         existing = {rec.id: idx for idx, rec in enumerate(self._records.get(collection, []))}
26         bucket = self._records.setdefault(collection, [])
27         for record in items:
28             if record.id in existing:
29                 bucket[existing[record.id]] = record
30             else:
31                 existing[record.id] = len(bucket)
32                 bucket.append(record)
33         self._matrix[collection] = _normalise(
34             np.asarray([r.vector for r in bucket], dtype=np.float32)
35         )
36     return len(items)
37
38 def search(
39     self,
40     collection: str,
41     query_vector: list[float],
42     *,
43     k: int = 10,
44     min_score: float = 0.0,
45     where: dict[str, Any] | None = None,
46 ) -> list[SearchHit]:
47     with self._lock:
48         matrix = self._matrix.get(collection)
49         records = list(self._records.get(collection, []))
50         if matrix is None or not records:
51             return []
52         query = np.asarray(query_vector, dtype=np.float32)
53         norm = float(np.linalg.norm(query)) or 1.0
54         scores = matrix @ (query / norm)
55         order = np.argsort(-scores)
56         hits: list[SearchHit] = []
57         for idx in order:
58             record = records[int(idx)]
59             score = float(scores[int(idx)])
60             if score < min_score:
61                 break
62             if where and any(record.metadata.get(key) != value for key, value in where.items()):
63                 continue
64             hits.append(
65                 SearchHit(id=record.id, score=score, text=record.text, metadata=record.metadata)
66             )
67             if len(hits) >= k:
68                 break
69     return hits
70
71 def count(self, collection: str) -> int:
72     with self._lock:
73         return len(self._records.get(collection, []))
74
75 def reset(self, collection: str | None = None) -> None:
76     with self._lock:
77         if collection is None:
78             self._matrix.clear()
79             self._records.clear()
80         else:
81             self._matrix.pop(collection, None)
82             self._records.pop(collection, None)
83
84 def health(self) -> dict[str, Any]:
85     with self._lock:
86         return {
87             "vector_store": self.name,
88             "collections": {name: len(recs) for name, recs in self._records.items()},
89         }
90
91
92 def _normalise(matrix: np.ndarray) -> np.ndarray:
93     if matrix.size == 0:
94         return matrix
95     norms = np.linalg.norm(matrix, axis=1, keepdims=True)
96     norms[norms == 0] = 1.0
97     return matrix / norms

```

backend/app/vectorstore/pgvector_store.py

```

1 """PostgreSQL + pgvector backed store.
2

```

```

3 Used automatically when ``VECTOR_STORE=pgvector`` and the database is
4 PostgreSQL with the ``vector`` extension available. Falls back to the in-memory
5 store at construction time when either precondition is missing, so the same
6 code path works on a laptop and in docker-compose.
7 """
8 from __future__ import annotations
9
10 from typing import Any, Iterable
11
12 from sqlalchemy import text
13
14 from app.config import settings
15 from app.database import engine
16 from app.logging import get_logger
17 from app.vectorstore.base import SearchHit, VectorRecord, VectorStore
18
19 logger = get_logger("ecr.vectorstore.pgvector")
20
21 _TABLE = "vector_documents"
22
23
24 class PgVectorStore(VectorStore):
25     name = "pgvector"
26
27     def __init__(self) -> None:
28         if settings.is_sqlite:
29             raise RuntimeError("pgvector requires a PostgreSQL DATABASE_URL")
30         dim = settings.embedding_dim
31         with engine.begin() as conn:
32             conn.execute(text("CREATE EXTENSION IF NOT EXISTS vector"))
33             conn.execute(
34                 text(
35                     f"""
36                     CREATE TABLE IF NOT EXISTS {_TABLE} (
37                         collection TEXT NOT NULL,
38                         doc_id TEXT NOT NULL,
39                         content TEXT NOT NULL,
40                         metadata JSONB NOT NULL DEFAULT '{{}}'::jsonb,
41                         embedding vector({dim}),
42                         PRIMARY KEY (collection, doc_id)
43                     )
44                     """
45                 )
46             )
47             conn.execute(
48                 text(
49                     f"CREATE INDEX IF NOT EXISTS idx_{_TABLE}_collection "
50                     f"ON {_TABLE} (collection)"
51                 )
52             )
53         logger.info("pgvector.ready", dim=dim)
54
55     def upsert(self, collection: str, records: Iterable[VectorRecord]) -> int:
56         import json
57
58         items = list(records)
59         if not items:
60             return 0
61         with engine.begin() as conn:
62             for record in items:
63                 conn.execute(
64                     text(
65                         f"""
66                         INSERT INTO {_TABLE} (collection, doc_id, content, metadata, embedding)
67                         VALUES (:collection, :doc_id, :content, CAST(:metadata AS jsonb),
68                             CAST(:embedding AS vector))
69                         ON CONFLICT (collection, doc_id) DO UPDATE
70                         SET content = EXCLUDED.content,
71                             metadata = EXCLUDED.metadata,
72                             embedding = EXCLUDED.embedding
73                         """
74                     ),
75                     {
76                         "collection": collection,
77                         "doc_id": record.id,
78                         "content": record.text,
79                         "metadata": json.dumps(record.metadata),
80                         "embedding": _to_vector_literal(record.vector),
81                     },
82                 )
83         return len(items)
84
85     def search(
86         self,

```

```

87         collection: str,
88         query_vector: list[float],
89         *,
90         k: int = 10,
91         min_score: float = 0.0,
92         where: dict[str, Any] | None = None,
93     ) -> list[SearchHit]:
94         with engine.connect() as conn:
95             rows = conn.execute(
96                 text(
97                     f"""
98                     SELECT doc_id, content, metadata,
99                         1 - (embedding <=> CAST(:embedding AS vector)) AS score
100                     FROM {_TABLE}
101                     WHERE collection = :collection
102                     ORDER BY embedding <=> CAST(:embedding AS vector)
103                     LIMIT :limit
104                     """
105                 ),
106                 {
107                     "collection": collection,
108                     "embedding": _to_vector_literal(query_vector),
109                     "limit": max(k * 3, k),
110                 },
111             ).mappings()
112         hits: list[SearchHit] = []
113         for row in rows:
114             metadata = dict(row["metadata"] or {})
115             score = float(row["score"])
116             if score < min_score:
117                 continue
118             if where and any(metadata.get(key) != value for key, value in where.items()):
119                 continue
120             hits.append(
121                 SearchHit(
122                     id=row["doc_id"], score=score, text=row["content"], metadata=metadata
123                 )
124             )
125             if len(hits) >= k:
126                 break
127         return hits
128
129     def count(self, collection: str) -> int:
130         with engine.connect() as conn:
131             return int(
132                 conn.execute(
133                     text(f"SELECT COUNT(*) FROM {_TABLE} WHERE collection = :c"),
134                     {"c": collection},
135                 ).scalar()
136                 or 0
137             )
138
139     def reset(self, collection: str | None = None) -> None:
140         with engine.begin() as conn:
141             if collection is None:
142                 conn.execute(text(f"DELETE FROM {_TABLE}"))
143             else:
144                 conn.execute(
145                     text(f"DELETE FROM {_TABLE} WHERE collection = :c"), {"c": collection}
146                 )
147
148
149     def _to_vector_literal(vector: list[float]) -> str:
150         return "[" + ",".join(f"{value:.6f}" for value in vector) + "]"

```

Backend - dependency graph

backend/app/graph/_init_.py

backend/app/graph/dependency_graph.py

```

1  """Component dependency graph (NetworkX).
2
3  The graph is directed: an edge ``A -> B`` means "A depends on B".
4  Therefore, when B changes:
5  * successors of B (things B depends on) are *upstream* of the change,
6  * predecessors of B (things that depend on B) are *downstream* and carry the
7    regression risk.
8
9  ``DependencyGraphService`` is the only place that knows about NetworkX, which
10 keeps a future Neo4j implementation a drop-in replacement
11 (see :class:`BaseDependencyGraph`).

```

```

12 """
13 from __future__ import annotations
14
15 from abc import ABC, abstractmethod
16 from typing import Any, Iterable
17
18 import networkx as nx
19 from sqlalchemy.orm import Session
20
21 from app.logging import get_logger
22 from app.models import Component, Dependency
23
24 logger = get_logger("ecr.graph")
25
26 CRITICALITY_WEIGHT = {"CRITICAL": 1.0, "HIGH": 0.8, "MEDIUM": 0.5, "LOW": 0.25}
27 DEPENDENCY_TYPE_WEIGHT = {
28     "LIBRARY": 1.0, # compiled in - change propagates immediately
29     "DB_ACCESS": 0.9,
30     "SYNC_API": 0.85,
31     "ASYNC_EVENT": 0.55,
32     "CONFIG": 0.4,
33 }
34
35
36 class BaseDependencyGraph(ABC):
37     """Contract a Neo4j-backed implementation would also satisfy."""
38
39     @abstractmethod
40     def load(self, db: Session) -> None: ...
41
42     @abstractmethod
43     def downstream(self, components: Iterable[str], max_depth: int = 3) -> dict[str, int]: ...
44
45     @abstractmethod
46     def upstream(self, components: Iterable[str], max_depth: int = 3) -> dict[str, int]: ...
47
48
49 class DependencyGraphService(BaseDependencyGraph):
50     """In-process NetworkX implementation."""
51
52     def __init__(self) -> None:
53         self.graph = nx.DiGraph()
54
55     # -- lifecycle -----
56     def load(self, db: Session) -> "DependencyGraphService":
57         graph = nx.DiGraph()
58         for component in db.query(Component).all():
59             graph.add_node(component.component_id, **component.as_dict())
60         for dependency in db.query(Dependency).all():
61             if not dependency.source or not dependency.target:
62                 continue
63             graph.add_edge(
64                 dependency.source.component_id,
65                 dependency.target.component_id,
66                 dependency_type=dependency.dependency_type,
67                 criticality=dependency.criticality,
68                 weight=dependency.weight,
69                 propagation=self.edge_propagation(dependency.dependency_type, dependency.criticality),
70                 description=dependency.description,
71             )
72         self.graph = graph
73         logger.info("graph.loaded", nodes=graph.number_of_nodes(), edges=graph.number_of_edges())
74         return self
75
76     @staticmethod
77     def edge_propagation(dependency_type: str, criticality: str) -> float:
78         """How strongly a change propagates along this edge (0..1)."""
79         return round(
80             DEPENDENCY_TYPE_WEIGHT.get(dependency_type, 0.6)
81             * (0.6 + 0.4 * CRITICALITY_WEIGHT.get(criticality, 0.5)),
82             3,
83         )
84
85     # -- traversal -----
86     def _bfs(self, components: Iterable[str], max_depth: int, reverse: bool) -> dict[str, int]:
87         graph = self.graph.reverse(copy=False) if reverse else self.graph
88         distances: dict[str, int] = {}
89         frontier = [c for c in components if c in graph]
90         depth = 0
91         seen = set(frontier)
92         while frontier and depth < max_depth:
93             depth += 1
94             next_frontier: list[str] = []
95             for node in frontier:

```

```

96         for neighbour in graph.successors(node):
97             if neighbour in seen:
98                 continue
99             seen.add(neighbour)
100             distances[neighbour] = depth
101             next_frontier.append(neighbour)
102         frontier = next_frontier
103     return distances
104
105 def downstream(self, components: Iterable[str], max_depth: int = 3) -> dict[str, int]:
106     """Components that depend on the changed ones (regression blast radius)."""
107     return self._bfs(components, max_depth, reverse=True)
108
109 def upstream(self, components: Iterable[str], max_depth: int = 3) -> dict[str, int]:
110     """Components the changed ones rely on."""
111     return self._bfs(components, max_depth, reverse=False)
112
113 def critical_paths(
114     self, components: Iterable[str], max_paths: int = 6, cutoff: int = 4
115 ) -> list[list[str]]:
116     """Highest-propagation paths from an entry point into a changed component."""
117     seeds = [c for c in components if c in self.graph]
118     if not seeds:
119         return []
120     entry_points = [
121         node for node in self.graph.nodes if self.graph.in_degree(node) == 0
122     ] or [node for node in self.graph.nodes if self.graph.in_degree(node) <= 1]
123     scored: list[tuple[float, list[str]]] = []
124     for entry in entry_points:
125         for seed in seeds:
126             if entry == seed:
127                 continue
128             try:
129                 paths = nx.all_simple_paths(self.graph, entry, seed, cutoff=cutoff)
130                 for path in paths:
131                     score = 1.0
132                     for left, right in zip(path, path[1:]):
133                         score *= self.graph.edges[left, right].get("propagation", 0.5)
134                     crit = CRITICALITY_WEIGHT.get(
135                         self.graph.nodes[seed].get("business_criticality", "MEDIUM"), 0.5
136                     )
137                     scored.append((score * crit, path))
138             except (nx.NetworkXNoPath, nx.NodeNotFound): # pragma: no cover
139                 continue
140     scored.sort(key=lambda item: item[0], reverse=True)
141     unique: list[list[str]] = []
142     for _, path in scored:
143         if path not in unique:
144             unique.append(path)
145         if len(unique) >= max_paths:
146             break
147     return unique
148
149 def propagation_scores(
150     self, components: Iterable[str], max_depth: int = 3
151 ) -> dict[str, float]:
152     """Decayed change-propagation strength per downstream component (0.1)."""
153     reverse = self.graph.reverse(copy=False)
154     scores: dict[str, float] = {}
155     frontier: list[tuple[str, float]] = [(c, 1.0) for c in components if c in self.graph]
156     seen = {c for c, _ in frontier}
157     depth = 0
158     while frontier and depth < max_depth:
159         depth += 1
160         next_frontier: list[tuple[str, float]] = []
161         for node, strength in frontier:
162             for neighbour in reverse.successors(node):
163                 propagation = reverse.edges[node, neighbour].get("propagation", 0.5)
164                 value = strength * propagation
165                 if value <= scores.get(neighbour, 0.0):
166                     continue
167                 scores[neighbour] = round(value, 4)
168                 if neighbour not in seen:
169                     seen.add(neighbour)
170                 next_frontier.append((neighbour, value))
171         frontier = next_frontier
172     return scores
173
174 def node(self, component_id: str) -> dict[str, Any]:
175     return dict(self.graph.nodes.get(component_id, {}))
176
177 def subgraph_payload(self, components: Iterable[str], max_depth: int = 2) -> dict[str, Any]:
178     """Nodes+edges for the UI dependency visualisation."""
179     seeds = {c for c in components if c in self.graph}

```

```

180         include = set(seeds)
181         include |= set(self.downstream(seeds, max_depth))
182         include |= set(self.upstream(seeds, max_depth))
183         nodes = [
184             **self.node(node_id), "component_id": node_id, "seed": node_id in seeds}
185             for node_id in include
186         ]
187         edges = [
188             {
189                 "source": source,
190                 "target": target,
191                 **{k: v for k, v in data.items() if k != "description"},
192             }
193             for source, target, data in self.graph.edges(data=True)
194             if source in include and target in include
195         ]
196         return {"nodes": nodes, "edges": edges}
197
198     def stats(self) -> dict[str, Any]:
199         return {
200             "nodes": self.graph.number_of_nodes(),
201             "edges": self.graph.number_of_edges(),
202             "density": round(nx.density(self.graph), 4) if self.graph.number_of_nodes() else 0.0,
203             "is_dag": nx.is_directed_acyclic_graph(self.graph),
204         }
205
206
207     def build_dependency_graph(db: Session) -> DependencyGraphService:
208         """Build a fresh graph. Cheap enough (tens of nodes) to do per workflow."""
209         return DependencyGraphService().load(db)

```

Backend - repositories

backend/app/repositories/__init__.py

```

1  """Repositories -- the only place that issues queries for a given aggregate."""
2  from __future__ import annotations
3
4  from sqlalchemy import desc, or_, select
5
6  from app.models import (
7      AgentRun,
8      Component,
9      Defect,
10     Dependency,
11     ECR,
12     Feedback,
13     ImpactReport,
14     Requirement,
15     TestCase,
16 )
17 from app.repositories.base import BaseRepository
18
19
20 class ECRRepository(BaseRepository[ECR]):
21     model = ECR
22     key_field = "ecr_id"
23
24     def recent(self, limit: int = 10) -> list[ECR]:
25         return list(
26             self.db.execute(select(ECR).order_by(desc(ECR.created_at)).limit(limit))
27             .scalars()
28             .all()
29         )
30
31     def search(self, term: str, limit: int = 20) -> list[ECR]:
32         like = f"%{term.lower()}%"
33         stmt = (
34             select(ECR)
35             .where(or_(ECR.ecr_id.ilike(like), ECR.title.ilike(like), ECR.description.ilike(like)))
36             .limit(limit)
37         )
38         return list(self.db.execute(stmt).scalars().all())
39
40
41 class RequirementRepository(BaseRepository[Requirement]):
42     model = Requirement
43     key_field = "requirement_id"
44
45     def by_component(self, component_id: str) -> list[Requirement]:
46         return [
47             row
48             for row in self.list()

```

```

49         if row.component == component_id or component_id in (row.linked_components or [])
50     ]
51
52
53 class DefectRepository(BaseRepository[Defect]):
54     model = Defect
55     key_field = "defect_id"
56
57     def by_components(self, component_ids: list[str]) -> list[Defect]:
58         wanted = set(component_ids)
59         return [
60             row
61             for row in self.list()
62             if row.affected_component in wanted or wanted & set(row.related_components or [])
63         ]
64
65
66 class TestRepository(BaseRepository[TestCase]):
67     model = TestCase
68     key_field = "test_case_id"
69
70     def by_requirements(self, requirement_ids: list[str]) -> list[TestCase]:
71         wanted = set(requirement_ids)
72         return [row for row in self.list() if row.requirement_id in wanted]
73
74     def by_components(self, component_ids: list[str]) -> list[TestCase]:
75         wanted = set(component_ids)
76         return [
77             row
78             for row in self.list()
79             if row.component in wanted or wanted & set(row.linked_components or [])
80         ]
81
82
83 class ComponentRepository(BaseRepository[Component]):
84     model = Component
85     key_field = "component_id"
86
87
88 class DependencyRepository(BaseRepository[Dependency]):
89     model = Dependency
90     key_field = "id"
91
92
93 class AgentRunRepository(BaseRepository[AgentRun]):
94     model = AgentRun
95     key_field = "workflow_id"
96
97     def for_workflow(self, workflow_id: str) -> list[AgentRun]:
98         return list(
99             self.db.execute(
100                 select(AgentRun).where(AgentRun.workflow_id == workflow_id).order_by(AgentRun.id)
101             )
102             .scalars()
103             .all()
104         )
105
106     def recent(self, limit: int = 50) -> list[AgentRun]:
107         return list(
108             self.db.execute(select(AgentRun).order_by(desc(AgentRun.id)).limit(limit))
109             .scalars()
110             .all()
111         )
112
113
114 class ImpactReportRepository(BaseRepository[ImpactReport]):
115     model = ImpactReport
116     key_field = "ecr_id"
117
118     def latest_for_ecr(self, ecr_id: str) -> ImpactReport | None:
119         return (
120             self.db.execute(
121                 select(ImpactReport)
122                 .where(ImpactReport.ecr_id == ecr_id)
123                 .order_by(desc(ImpactReport.id))
124                 .limit(1)
125             )
126             .scalars()
127             .first()
128         )
129
130     def all_reports(self, limit: int = 200) -> list[ImpactReport]:
131         return list(
132             self.db.execute(select(ImpactReport).order_by(desc(ImpactReport.id)).limit(limit))

```

```

133         .scalars()
134         .all()
135     )
136
137
138 class FeedbackRepository(BaseRepository[Feedback]):
139     model = Feedback
140     key_field = "ecr_id"
141
142
143 __all__ = [
144     "AgentRunRepository",
145     "ComponentRepository",
146     "DefectRepository",
147     "DependencyRepository",
148     "ECRRepository",
149     "FeedbackRepository",
150     "ImpactReportRepository",
151     "RequirementRepository",
152     "TestRepository",
153 ]

```

backend/app/repositories/base.py

```

1  """Generic repository helpers (repository pattern over SQLAlchemy)."""
2  from __future__ import annotations
3
4  from typing import Generic, Iterable, Sequence, TypeVar
5
6  from sqlalchemy import select
7  from sqlalchemy.orm import Session
8
9  from app.database import Base
10
11 ModelT = TypeVar("ModelT", bound=Base)
12
13
14 class BaseRepository(Generic[ModelT]):
15     model: type[ModelT]
16     key_field: str = "id"
17
18     def __init__(self, db: Session) -> None:
19         self.db = db
20
21     def list(self, limit: int | None = None) -> list[ModelT]:
22         stmt = select(self.model)
23         if limit:
24             stmt = stmt.limit(limit)
25         return list(self.db.execute(stmt).scalars().all())
26
27     def get(self, key: str) -> ModelT | None:
28         column = getattr(self.model, self.key_field)
29         return self.db.execute(select(self.model).where(column == key)).scalars().first()
30
31     def get_many(self, keys: Iterable[str]) -> list[ModelT]:
32         keys = list(keys)
33         if not keys:
34             return []
35         column = getattr(self.model, self.key_field)
36         return list(self.db.execute(select(self.model).where(column.in_(keys))).scalars().all())
37
38     def count(self) -> int:
39         from sqlalchemy import func
40
41         return int(self.db.execute(select(func.count()).select_from(self.model)).scalar() or 0)
42
43     def add(self, entity: ModelT) -> ModelT:
44         self.db.add(entity)
45         self.db.commit()
46         self.db.refresh(entity)
47         return entity
48
49     def bulk_add(self, entities: Sequence[ModelT]) -> int:
50         self.db.add_all(entities)
51         self.db.commit()
52         return len(entities)

```

Backend - schemas

backend/app/schemas/__init__.py

```

1  """Pydantic schemas (API contracts and agent I/O contracts)."""

```

backend/app/schemas/agent.py

```
1  """Agent execution / workflow telemetry schemas."""
2  from __future__ import annotations
3
4  from datetime import datetime
5  from typing import Any
6
7  from pydantic import BaseModel, Field
8
9  from app.schemas.common import AgentStatus, WorkflowStatus
10
11
12  class AgentDescriptor(BaseModel):
13      """Static catalogue entry describing an agent in the graph."""
14
15      key: str
16      name: str
17      purpose: str
18      tools: list[str] = Field(default_factory=list)
19      optional: bool = False
20      depends_on: list[str] = Field(default_factory=list)
21
22
23  class AgentEvent(BaseModel):
24      """A single event on the workflow event stream (also the execution trace)."""
25
26      timestamp: datetime
27      workflow_id: str
28      ecr_id: str = ""
29      agent: str
30      agent_key: str = ""
31      status: AgentStatus | WorkflowStatus | str
32      action: str = ""
33      detail: str = ""
34      duration: float | None = None
35      confidence: float | None = None
36      payload: dict[str, Any] = Field(default_factory=dict)
37
38
39  class AgentRunRead(BaseModel):
40      workflow_id: str
41      ecr_id: str
42      agent: str
43      agent_key: str
44      status: str
45      action: str
46      reasoning: str
47      confidence: float
48      execution_time: float
49      error: str = ""
50      timestamp: str | None = None
51      output: dict[str, Any] = Field(default_factory=dict)
52
53
54  class WorkflowSummary(BaseModel):
55      workflow_id: str
56      ecr_id: str
57      status: WorkflowStatus
58      started_at: datetime
59      finished_at: datetime | None = None
60      duration: float = 0.0
61      plan: list[str] = Field(default_factory=list)
62      skipped_steps: list[str] = Field(default_factory=list)
63      current_agent: str | None = None
64      progress: float = 0.0
65      confidence: float = 0.0
66      errors: list[dict[str, Any]] = Field(default_factory=list)
67      approval_required: bool = False
68      approval_status: str = "NOT_REQUIRED"
69      events: list[AgentEvent] = Field(default_factory=list)
70
71
72  class ApprovalDecision(BaseModel):
73      approved: bool
74      decided_by: str = "qa.lead@enterprise.com"
75      comment: str = ""
76      excluded_test_ids: list[str] = Field(default_factory=list)
```

backend/app/schemas/common.py

```
1  """Enumerations and small shared schemas."""
2  from __future__ import annotations
3
```

```

4 from enum import Enum
5
6 from pydantic import BaseModel, Field
7
8
9 class ChangeType(str, Enum):
10     UI = "UI_CHANGE"
11     BACKEND = "BACKEND_CHANGE"
12     API = "API_CHANGE"
13     DATABASE = "DATABASE_CHANGE"
14     INFRASTRUCTURE = "INFRASTRUCTURE_CHANGE"
15     SECURITY = "SECURITY_CHANGE"
16     CONFIGURATION = "CONFIGURATION_CHANGE"
17     INTEGRATION = "INTEGRATION_CHANGE"
18     MIXED = "MIXED_CHANGE"
19     API_AND_BACKEND = "API_AND_BACKEND"
20
21
22 class Priority(str, Enum):
23     P0 = "P0"
24     P1 = "P1"
25     P2 = "P2"
26     P3 = "P3"
27
28     @classmethod
29     def from_score(cls, score: float) -> "Priority":
30         if score >= 90:
31             return cls.P0
32         if score >= 75:
33             return cls.P1
34         if score >= 50:
35             return cls.P2
36         return cls.P3
37
38
39 class AgentStatus(str, Enum):
40     PENDING = "pending"
41     RUNNING = "running"
42     COMPLETED = "completed"
43     FAILED = "failed"
44     SKIPPED = "skipped"
45
46
47 class WorkflowStatus(str, Enum):
48     QUEUED = "QUEUED"
49     RUNNING = "RUNNING"
50     AWAITING_APPROVAL = "AWAITING_APPROVAL"
51     COMPLETED = "COMPLETED"
52     FAILED = "FAILED"
53     REJECTED = "REJECTED"
54
55
56 class ConfidenceBand(str, Enum):
57     LOW = "LOW CONFIDENCE"
58     MEDIUM = "MEDIUM CONFIDENCE"
59     HIGH = "HIGH CONFIDENCE"
60
61     @classmethod
62     def from_score(cls, score: float) -> "ConfidenceBand":
63         if score >= 0.85:
64             return cls.HIGH
65         if score >= 0.65:
66             return cls.MEDIUM
67         return cls.LOW
68
69
70 class ScoredItem(BaseModel):
71     """Generic id/score/reason triple used across agent outputs."""
72
73     id: str
74     title: str = ""
75     score: float = Field(0.0, ge=0.0, le=100.0)
76     reason: str = ""

```

backend/app/schemas/ecr.py

```

1 """ECR request/response schemas plus the ECR Understanding Agent contract."""
2 from __future__ import annotations
3
4 from datetime import datetime
5 from typing import Any
6
7 from pydantic import BaseModel, ConfigDict, Field, field_validator
8

```

```

9 from app.schemas.common import ChangeType
10
11
12 class ECRCreate(BaseModel):
13     ecr_id: str | None = Field(
14         None, description="Optional. Auto-generated (ECR-YYYY-NNN) when omitted."
15     )
16     title: str = Field(..., min_length=3, max_length=240)
17     description: str = Field("", max_length=8000)
18     business_domain: str = "General"
19     requested_by: str = "Change Board"
20     priority: str = "MEDIUM"
21     target_release: str = ""
22     linked_requirements: list[str] = Field(default_factory=list)
23     changed_files: list[str] = Field(default_factory=list)
24     change_metadata: dict[str, Any] = Field(default_factory=dict)
25
26     # Change-tracking export columns (all optional).
27     record_type: str = "ECR"
28     lifecycle_status: str = ""
29     severity: str = ""
30     assigned_testers: list[str] = Field(default_factory=list)
31     actual_modified_objects: list[str] = Field(default_factory=list)
32     planned_modified_objects: list[str] = Field(default_factory=list)
33     build_resolved_in: str = ""
34     test_estimate_archived: str = ""
35     estimate: str = ""
36     test_estimate: str = ""
37     creation_date: str = ""
38     modified_date: str = ""
39     sit_assigned_testers: list[str] = Field(default_factory=list)
40     test_actual_time: str = ""
41     affected_test_cases: list[str] = Field(default_factory=list)
42     steps_to_reproduce: str = ""
43     observed_behavior: str = ""
44     expected_behavior: str = ""
45     attachments: list[str] = Field(default_factory=list)
46
47     @field_validator("ecr_id")
48     @classmethod
49     def _strip(cls, value: str | None) -> str | None:
50         return value.strip().upper() if value else None
51
52
53 class ECRRead(BaseModel):
54     model_config = ConfigDict(from_attributes=True)
55
56     ecr_id: str
57     title: str
58     description: str
59     status: str
60     change_type: str
61     business_domain: str
62     requested_by: str
63     priority: str
64     target_release: str
65     linked_requirements: list[str] = Field(default_factory=list)
66     changed_files: list[str] = Field(default_factory=list)
67     change_metadata: dict[str, Any] = Field(default_factory=dict)
68
69     # Change-tracking export columns (all optional).
70     record_type: str = "ECR"
71     lifecycle_status: str = ""
72     severity: str = ""
73     assigned_testers: list[str] = Field(default_factory=list)
74     actual_modified_objects: list[str] = Field(default_factory=list)
75     planned_modified_objects: list[str] = Field(default_factory=list)
76     build_resolved_in: str = ""
77     test_estimate_archived: str = ""
78     estimate: str = ""
79     test_estimate: str = ""
80     creation_date: str = ""
81     modified_date: str = ""
82     sit_assigned_testers: list[str] = Field(default_factory=list)
83     test_actual_time: str = ""
84     affected_test_cases: list[str] = Field(default_factory=list)
85     steps_to_reproduce: str = ""
86     observed_behavior: str = ""
87     expected_behavior: str = ""
88     attachments: list[str] = Field(default_factory=list)
89     created_at: datetime | None = None
90     updated_at: datetime | None = None
91
92

```

```

93 class AnalyzeRequest(BaseModel):
94     """Options for a workflow run."""
95
96     human_in_the_loop: bool | None = None
97     force_full_analysis: bool = False
98     description_override: str | None = None
99     demo_mode: bool = False
100
101
102 class ECRUnderstanding(BaseModel):
103     """Structured output of Agent 1 (ECR Understanding)."""
104
105     ecr_id: str
106     change_type: ChangeType = ChangeType.MIXED
107     change_type_scores: dict[str, float] = Field(default_factory=dict)
108     business_domain: str = "General"
109     affected_features: list[str] = Field(default_factory=list)
110     technical_keywords: list[str] = Field(default_factory=list)
111     candidate_components: list[str] = Field(default_factory=list)
112     risk_indicators: list[str] = Field(default_factory=list)
113     change_complexity: float = Field(0.5, ge=0.0, le=1.0)
114     summary: str = ""
115     confidence: float = Field(0.8, ge=0.0, le=1.0)

```

backend/app/schemas/impact.py

```

1  """Impact analysis schemas - outputs of agents 2 to 6."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  from pydantic import BaseModel, Field
7
8
9  class RequirementMatch(BaseModel):
10     requirement_id: str
11     title: str
12     description: str = ""
13     business_domain: str = ""
14     priority: str = "MEDIUM"
15     component: str = ""
16     linked_components: list[str] = Field(default_factory=list)
17     features: list[str] = Field(default_factory=list)
18     relevance: float = Field(0.0, ge=0.0, le=100.0)
19     match_type: str = "SEMANTIC"
20     reason: str = ""
21
22
23 class RequirementAnalysis(BaseModel):
24     matched_requirements: list[RequirementMatch] = Field(default_factory=list)
25     impacted_components: list[str] = Field(default_factory=list)
26     searched_count: int = 0
27     confidence: float = Field(0.0, ge=0.0, le=1.0)
28     reasoning: str = ""
29
30
31 class DefectMatch(BaseModel):
32     defect_id: str
33     title: str
34     severity: str = "MEDIUM"
35     status: str = "CLOSED"
36     root_cause: str = ""
37     root_cause_category: str = ""
38     affected_component: str = ""
39     similarity: float = Field(0.0, ge=0.0, le=100.0)
40     escaped_to_production: bool = False
41     reason: str = ""
42
43
44 class DefectAnalysis(BaseModel):
45     similar_defects: list[DefectMatch] = Field(default_factory=list)
46     defect_prone_components: list[str] = Field(default_factory=list)
47     recurring_failure_patterns: list[str] = Field(default_factory=list)
48     severity_distribution: dict[str, int] = Field(default_factory=dict)
49     searched_count: int = 0
50     confidence: float = Field(0.0, ge=0.0, le=1.0)
51     reasoning: str = ""
52
53
54 class CodeSymbol(BaseModel):
55     file: str
56     symbol: str
57     kind: str = "function"
58     component: str = ""

```

```

59     calls: list[str] = Field(default_factory=list)
60
61
62 class CodeImpactAnalysis(BaseModel):
63     directly_impacted_files: list[str] = Field(default_factory=list)
64     impacted_symbols: list[CodeSymbol] = Field(default_factory=list)
65     impacted_services: list[str] = Field(default_factory=list)
66     potentially_impacted_services: list[str] = Field(default_factory=list)
67     impacted_apis: list[str] = Field(default_factory=list)
68     database_entities: list[str] = Field(default_factory=list)
69     change_surface: float = Field(0.0, ge=0.0, le=1.0)
70     language_analyzers: list[str] = Field(default_factory=list)
71     confidence: float = Field(0.0, ge=0.0, le=1.0)
72     reasoning: str = ""
73
74
75 class DependencyEdge(BaseModel):
76     source: str
77     target: str
78     dependency_type: str = "SYNC_API"
79     criticality: str = "MEDIUM"
80
81
82 class DependencyNode(BaseModel):
83     component_id: str
84     name: str
85     type: str = "SERVICE"
86     business_criticality: str = "MEDIUM"
87     distance: int = 0
88     relation: str = "SEED"
89
90
91 class DependencyAnalysis(BaseModel):
92     seed_components: list[str] = Field(default_factory=list)
93     direct_dependencies: list[str] = Field(default_factory=list)
94     indirect_dependencies: list[str] = Field(default_factory=list)
95     upstream_components: list[str] = Field(default_factory=list)
96     downstream_components: list[str] = Field(default_factory=list)
97     critical_dependency_paths: list[list[str]] = Field(default_factory=list)
98     blast_radius: int = 0
99     nodes: list[DependencyNode] = Field(default_factory=list)
100     edges: list[DependencyEdge] = Field(default_factory=list)
101     confidence: float = Field(0.0, ge=0.0, le=1.0)
102     reasoning: str = ""
103
104
105 class ComponentImpact(BaseModel):
106     component_id: str
107     name: str
108     type: str = "SERVICE"
109     business_criticality: str = "MEDIUM"
110     impact_type: str = "DIRECT"
111     impact_score: float = 0.0
112     distance: int = 0
113     reasons: list[str] = Field(default_factory=list)
114
115
116 class ImpactAssessment(BaseModel):
117     directly_impacted_components: list[ComponentImpact] = Field(default_factory=list)
118     indirectly_impacted_components: list[ComponentImpact] = Field(default_factory=list)
119     functional_impact: str = ""
120     technical_impact: str = ""
121     business_impact: str = ""
122     mitigations: list[str] = Field(default_factory=list)
123     executive_summary: str = ""
124     confidence: float = Field(0.0, ge=0.0, le=1.0)
125     degraded_inputs: list[str] = Field(default_factory=list)
126     metadata: dict[str, Any] = Field(default_factory=dict)

```

backend/app/schemas/report.py

```

1  """Final report schema (Agent 10)."""
2  from __future__ import annotations
3
4  from datetime import datetime
5  from typing import Any
6
7  from pydantic import BaseModel, Field
8
9  from app.schemas.common import ConfidenceBand
10 from app.schemas.ecr import ECRUnderstanding
11 from app.schemas.impact import (
12     CodeImpactAnalysis,
13     DefectAnalysis,

```

```

14     DependencyAnalysis,
15     ImpactAssessment,
16     RequirementAnalysis,
17 )
18 from app.schemas.test_case import TestPrioritization, TestSelection
19
20
21 class ReportSection(BaseModel):
22     key: str
23     title: str
24     body: str = ""
25     data: dict[str, Any] = Field(default_factory=dict)
26
27
28 class ConfidenceReport(BaseModel):
29     overall: float = 0.0
30     band: ConfidenceBand = ConfidenceBand.MEDIUM
31     per_step: dict[str, float] = Field(default_factory=dict)
32     penalties: list[str] = Field(default_factory=list)
33
34
35 class ECRIntelligenceReport(BaseModel):
36     workflow_id: str
37     ecr_id: str
38     generated_at: datetime
39     title: str = ""
40     executive_summary: str = ""
41     ecr_overview: dict[str, Any] = Field(default_factory=dict)
42     change_classification: ECRUnderstanding | None = None
43     retrieval_plan: dict[str, Any] = Field(default_factory=dict)
44     correlation_review: dict[str, Any] = Field(default_factory=dict)
45     requirement_analysis: RequirementAnalysis | None = None
46     defect_analysis: DefectAnalysis | None = None
47     defect_insights: dict[str, Any] = Field(default_factory=dict)
48     defect_summary: str = ""
49     code_impact: CodeImpactAnalysis | None = None
50     dependency_analysis: DependencyAnalysis | None = None
51     impact_assessment: ImpactAssessment | None = None
52     test_selection: TestSelection | None = None
53     test_prioritization: TestPrioritization | None = None
54     mitigations: list[str] = Field(default_factory=list)
55     confidence: ConfidenceReport = Field(default_factory=ConfidenceReport)
56     execution_trace: list[dict[str, Any]] = Field(default_factory=list)
57     sections: list[ReportSection] = Field(default_factory=list)
58     metrics: dict[str, Any] = Field(default_factory=dict)
59     errors: list[dict[str, Any]] = Field(default_factory=list)
60     approval: dict[str, Any] = Field(default_factory=dict)

```

backend/app/schemas/test_case.py

```

1  """Test discovery / selection / prioritisation schemas."""
2  from __future__ import annotations
3
4  from pydantic import BaseModel, Field
5
6  from app.schemas.common import Priority
7
8
9  class TestCaseRead(BaseModel):
10     test_case_id: str
11     title: str
12     description: str = ""
13     feature: str = ""
14     suite: str = "Regression"
15     component: str = ""
16     linked_components: list[str] = Field(default_factory=list)
17     requirement_id: str = ""
18     test_type: str = "FUNCTIONAL"
19     test_level: str = "SYSTEM"
20     automation_status: str = "AUTOMATED"
21     folder: str = ""
22     optimization_technique: str = "Default"
23     test_polarity: str = "Positive"
24     test_technique: str = ""
25     retired: bool = False
26     scorable: bool = True
27     domain: str = "commerce"
28     average_execution_time: float = 60.0
29     historical_failure_rate: float = 0.0
30     tags: list[str] = Field(default_factory=list)
31
32
33 class TestScoreBreakdown(BaseModel):
34     requirement_match: float = 0.0

```

```

35     component_match: float = 0.0
36     dependency_relevance: float = 0.0
37     historical_failure: float = 0.0
38     semantic_similarity: float = 0.0
39
40
41 class SelectedTest(TestCaseRead):
42     relevance_score: float = Field(0.0, ge=0.0, le=100.0)
43     priority: Priority = Priority.P3
44     breakdown: TestScoreBreakdown = Field(default_factory=TestScoreBreakdown)
45     reasons: list[str] = Field(default_factory=list)
46     reason: str = ""
47     execution_order: int = 0
48     selected: bool = True
49
50
51 class TestDiscovery(BaseModel):
52     candidate_tests: list[TestCaseRead] = Field(default_factory=list)
53     total_available: int = 0
54     discovery_channels: dict[str, int] = Field(default_factory=dict)
55     confidence: float = Field(0.0, ge=0.0, le=1.0)
56     reasoning: str = ""
57
58
59 class TestSelection(BaseModel):
60     selected_tests: list[SelectedTest] = Field(default_factory=list)
61     excluded_count: int = 0
62     total_available: int = 0
63     total_candidates: int = 0
64     reduction_percentage: float = 0.0
65     estimated_duration_minutes: float = 0.0
66     baseline_duration_minutes: float = 0.0
67     priority_distribution: dict[str, int] = Field(default_factory=dict)
68     confidence: float = Field(0.0, ge=0.0, le=1.0)
69     reasoning: str = ""
70
71
72 class TestPrioritization(BaseModel):
73     prioritized_tests: list[SelectedTest] = Field(default_factory=list)
74     waves: dict[str, list[str]] = Field(default_factory=dict)
75     estimated_duration_minutes: float = 0.0
76     critical_path_minutes: float = 0.0
77     strategy: str = ""
78     confidence: float = Field(0.0, ge=0.0, le=1.0)
79     reasoning: str = ""

```

Backend - API routes

backend/app/api/__init__.py

backend/app/api/routes/__init__.py

backend/app/api/routes/agents.py

```

1  """Agent catalogue, tool registry and execution telemetry."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  from fastapi import APIRouter, Query
7
8  from app.agents.orchestrator import graph_topology
9  from app.agents.state import AGENT_CATALOG, AGENT_STEPS
10 from app.agents.tools import ( # noqa: F401 (imports register the tools)
11     code_tools,
12     collaboration_tools,
13     defect_tools,
14     dependency_tools,
15     requirement_tools,
16     test_tools,
17 )
18 from app.agents.tools.registry import list_tools
19 from app.dependencies import DbSession
20 from app.repositories import AgentRunRepository
21
22 router = APIRouter(prefix="/agents", tags=["agents"])
23
24
25 @router.get("")
26 def list_agents() -> dict[str, Any]:
27     """The three agents, their purpose, tools and internal steps."""

```

```

28     return {
29         "agents": [
30             {**descriptor.model_dump(), "steps": AGENT_STEPS.get(descriptor.key, [])}
31             for descriptor in AGENT_CATALOG
32         ],
33         "topology": graph_topology(),
34     }
35
36
37 @router.get("/tools")
38 def tools() -> dict[str, Any]:
39     # Importing the node modules registers the tools they define locally.
40     from app.agents.nodes import correlation, ecr_understanding, planning # noqa: F401
41
42     return {"tools": list_tools()}
43
44
45 @router.get("/runs")
46 def agent_runs(
47     db: DbSession,
48     workflow_id: str | None = None,
49     limit: int = Query(60, le=300),
50 ) -> list[dict[str, Any]]:
51     repository = AgentRunRepository(db)
52     rows = repository.for_workflow(workflow_id) if workflow_id else repository.recent(limit)
53     return [row.as_dict() for row in rows]

```

backend/app/api/routes/analysis.py

```

1  """Analysis endpoints: run the agents, stream progress, read the results."""
2  from __future__ import annotations
3
4  import asyncio
5  import json
6  from typing import Any
7
8  from fastapi import APIRouter, HTTPException, Query, status
9  from fastapi.responses import StreamingResponse
10
11 from app.agents.runtime import registry
12 from app.config import settings
13 from app.dependencies import DbSession
14 from app.schemas.agent import ApprovalDecision
15 from app.schemas.ecr import AnalyzeRequest
16 from app.services import ecr_service
17
18 router = APIRouter(prefix="/ecr", tags=["analysis"])
19
20
21 def _run_or_404(ecr_id: str):
22     run = registry.latest_for_ecr(ecr_service.normalise_ecr_id(ecr_id))
23     if run is None:
24         raise HTTPException(status_code=404, detail=f"no analysis run found for {ecr_id}")
25     return run
26
27
28 @router.post("/{ecr_id}/analyze", status_code=status.HTTP_202_ACCEPTED)
29 def analyze(
30     ecr_id: str,
31     db: DbSession,
32     payload: AnalyzeRequest | None = None,
33     wait: bool = Query(False, description="Block until the workflow finishes"),
34 ) -> dict[str, Any]:
35     """Trigger the agentic analysis for an ECR number."""
36     request = payload or AnalyzeRequest()
37     options: dict[str, Any] = {
38         "human_in_the_loop": (
39             settings.human_in_the_loop
40             if request.human_in_the_loop is None
41             else request.human_in_the_loop
42         ),
43         "force_full_analysis": request.force_full_analysis,
44         "demo_mode": request.demo_mode,
45     }
46     if request.description_override:
47         options["description_override"] = request.description_override
48     try:
49         run = ecr_service.start_analysis(db, ecr_id, options, background=not wait)
50     except ecr_service.ECRNotFound as exc:
51         raise HTTPException(status_code=404, detail=f"{ecr_id} not found") from exc
52     return {
53         "workflow_id": run.workflow_id,
54         "ecr_id": run.ecr_id,
55         "status": run.status,

```

```

56         "stream_url": f"{settings.api_prefix}/workflows/{run.workflow_id}/stream",
57     }
58
59
60 @router.get("/{ecr_id}/analysis")
61 def get_analysis(ecr_id: str, db: DbSession) -> dict[str, Any]:
62     """Full analysis payload (live run if available, else the last persisted)."""
63     run = registry.latest_for_ecr(ecr_service.normalise_ecr_id(ecr_id))
64     if run and run.state:
65         state = run.state
66         return {
67             "workflow_id": run.workflow_id,
68             "ecr_id": run.ecr_id,
69             "status": run.status,
70             "duration": run.duration,
71             "answer": state.get("final_answer"),
72             "citations": state.get("answer_citations", []),
73             "question": state.get("question"),
74             "ecr_analysis": state.get("ecr_analysis"),
75             "retrieval_plan": state.get("retrieval_plan"),
76             "correlation_review": state.get("correlation_review"),
77             "requirement_analysis": state.get("requirement_analysis"),
78             "defect_analysis": state.get("defect_analysis"),
79             "defect_insights": state.get("defect_insights"),
80             "defect_summary": state.get("defect_summary"),
81             "code_impact": state.get("code_impact"),
82             "dependency_analysis": state.get("dependency_analysis"),
83             "collaboration_analysis": state.get("collaboration_analysis"),
84             "correlation": state.get("correlation"),
85             "impact_analysis": state.get("impact_analysis"),
86             "test_discovery": {
87                 k: v for k, v in (state.get("test_discovery") or {}).items()
88                 if k != "candidate_tests"
89             },
90             "test_selection": state.get("test_selection"),
91             "test_prioritization": state.get("test_prioritization"),
92             "comments": state.get("comments", []),
93             "evidence": state.get("evidence", []),
94             "approval": state.get("approval", {}),
95             "plan": state.get("plan", []),
96             "plan_detail": state.get("plan_detail", {}),
97             "skipped_steps": state.get("skipped_steps", []),
98             "execution_trace": state.get("execution_trace", []),
99             "errors": state.get("errors", []),
100            "report": state.get("final_report"),
101        }
102    report = ecr_service.latest_report(db, ecr_id)
103    if not report:
104        raise HTTPException(status_code=404, detail=f"no analysis available for {ecr_id}")
105    from app.models import Comment, Evidence
106
107    resolved = ecr_service.normalise_ecr_id(ecr_id)
108    return {
109        **page_fields_from_report(report),
110        "ecr_id": resolved,
111        "status": "COMPLETED",
112        "comments": [c.as_dict() for c in db.query(Comment).filter(Comment.ecr_id == resolved)],
113        "evidence": [e.as_dict() for e in db.query(Evidence).filter(Evidence.ecr_id == resolved)],
114        "report": report,
115    }
116
117
118 def page_fields_from_report(report: dict[str, Any]) -> dict[str, Any]:
119     """Rebuild the analysis page fields from a saved report.
120
121     Live runs are held in memory, so after a backend restart only the saved report
122     is left. Without this the page would show an ECR as analysed but empty.
123     """
124     return {
125         "workflow_id": report.get("workflow_id"),
126         "duration": None,
127         "answer": report.get("executive_summary"),
128         "citations": [],
129         "ecr_analysis": report.get("change_classification"),
130         "retrieval_plan": report.get("retrieval_plan") or None,
131         "correlation_review": report.get("correlation_review") or None,
132         "requirement_analysis": report.get("requirement_analysis"),
133         "defect_analysis": report.get("defect_analysis"),
134         "defect_insights": report.get("defect_insights") or None,
135         "defect_summary": report.get("defect_summary") or "",
136         "code_impact": report.get("code_impact"),
137         "dependency_analysis": report.get("dependency_analysis"),
138         "impact_analysis": report.get("impact_assessment"),
139         "test_selection": report.get("test_selection"),

```

```

140         "test_prioritization": report.get("test_prioritization"),
141         "approval": report.get("approval") or {},
142         "plan_detail": {},
143         "skipped_steps": [],
144         "execution_trace": report.get("execution_trace") or [],
145         "errors": report.get("errors") or [],
146     }
147
148
149 @router.get("/{ecr_id}/impact")
150 def get_impact(ecr_id: str, db: DbSession) -> dict[str, Any]:
151     state = ecr_service.latest_state(ecr_id) or {}
152     if state.get("impact_analysis"):
153         return {
154             "impact": state["impact_analysis"],
155             "dependency": state.get("dependency_analysis"),
156             "code_impact": state.get("code_impact"),
157             "defects": state.get("defect_analysis"),
158         }
159     report = ecr_service.latest_report(db, ecr_id)
160     if not report:
161         raise HTTPException(status_code=404, detail=f"no impact analysis available for {ecr_id}")
162     return {
163         "impact": report.get("impact_assessment"),
164         "dependency": report.get("dependency_analysis"),
165         "code_impact": report.get("code_impact"),
166         "defects": report.get("defect_analysis"),
167     }
168
169
170 @router.get("/{ecr_id}/tests")
171 def get_tests(
172     ecr_id: str,
173     db: DbSession,
174     priority: str | None = Query(None, description="Filter by P0/P1/P2/P3"),
175 ) -> dict[str, Any]:
176     state = ecr_service.latest_state(ecr_id) or {}
177     selection = state.get("test_selection")
178     prioritization = state.get("test_prioritization")
179     if not selection:
180         report = ecr_service.latest_report(db, ecr_id)
181         if not report:
182             raise HTTPException(status_code=404, detail=f"no test selection available for {ecr_id}")
183         selection = report.get("test_selection") or {}
184         prioritization = report.get("test_prioritization") or {}
185     tests = (prioritization or {}).get("prioritized_tests") or selection.get("selected_tests") or []
186     if priority:
187         wanted = {p.strip().upper() for p in priority.split(",")}
188         tests = [test for test in tests if test.get("priority") in wanted]
189     return {
190         "ecr_id": ecr_service.normalise_ecr_id(ecr_id),
191         "total_available": selection.get("total_available", 0),
192         "total_candidates": selection.get("total_candidates", 0),
193         "selected": len(selection.get("selected_tests") or []),
194         "reduction_percentage": selection.get("reduction_percentage", 0.0),
195         "estimated_duration_minutes": selection.get("estimated_duration_minutes", 0.0),
196         "baseline_duration_minutes": selection.get("baseline_duration_minutes", 0.0),
197         "priority_distribution": selection.get("priority_distribution", {}),
198         "waves": (prioritization or {}).get("waves", {}),
199         "strategy": (prioritization or {}).get("strategy", ""),
200         "tests": tests,
201     }
202
203
204 @router.get("/{ecr_id}/workflow")
205 def get_workflow(ecr_id: str) -> dict[str, Any]:
206     from app.agents.orchestrator import graph_topology
207
208     run = _run_or_404(ecr_id)
209     summary = run.summary()
210     summary["events"] = [event.model_dump(mode="json") for event in summary["events"]]
211     summary["topology"] = graph_topology()
212     summary["tools_used"] = (run.state or {}).get("tools_used", {})
213     return summary
214
215
216 @router.post("/{ecr_id}/approve")
217 def approve(ecr_id: str, decision: ApprovalDecision) -> dict[str, Any]:
218     """Resume a workflow paused at the human approval checkpoint."""
219     run = _run_or_404(ecr_id)
220     if run.approval_status != "PENDING":
221         raise HTTPException(
222             status_code=409, detail=f"workflow is not awaiting approval ({run.approval_status})"
223         )

```

```

224     run.resolve_approval(decision.model_dump())
225     return {"workflow_id": run.workflow_id, "approval_status": run.approval_status}
226
227
228 # -----
229 # Workflow-scoped endpoints
230 # -----
231 workflow_router = APIRouter(prefix="/workflows", tags=["workflow"])
232
233
234 @workflow_router.get("")
235 def list_workflows() -> list[dict[str, Any]]:
236     return [
237         {
238             "workflow_id": run.workflow_id,
239             "ecr_id": run.ecr_id,
240             "status": run.status,
241             "started_at": run.started_at,
242             "duration": run.duration,
243             "progress": run.progress,
244             "confidence": run.confidence,
245         }
246         for run in registry.all()
247     ]
248
249
250 @workflow_router.get("/{workflow_id}")
251 def get_workflow_by_id(workflow_id: str) -> dict[str, Any]:
252     run = registry.get(workflow_id)
253     if run is None:
254         raise HTTPException(status_code=404, detail=f"workflow {workflow_id} not found")
255     summary = run.summary()
256     summary["events"] = [event.model_dump(mode="json") for event in summary["events"]]
257     return summary
258
259
260 @workflow_router.get("/{workflow_id}/stream")
261 async def stream_workflow(workflow_id: str) -> StreamingResponse:
262     """Server-Sent Events stream of live agent progress."""
263     run = registry.get(workflow_id)
264     if run is None:
265         raise HTTPException(status_code=404, detail=f"workflow {workflow_id} not found")
266
267     async def event_source():
268         cursor = 0
269         idle = 0.0
270         yield _sse("connected", {"workflow_id": workflow_id, "ecr_id": run.ecr_id})
271         while True:
272             cursor, events = run.events_after(cursor)
273             for event in events:
274                 yield _sse("agent", event.model_dump(mode="json"))
275                 idle = 0.0
276             if run.finished_at is not None:
277                 yield _sse(
278                     "done",
279                     {
280                         "workflow_id": workflow_id,
281                         "status": run.status,
282                         "duration": run.duration,
283                         "confidence": run.confidence,
284                         "approval_status": run.approval_status,
285                     },
286                 )
287             return
288         await asyncio.sleep(0.25)
289         idle += 0.25
290         if idle >= 10.0: # keep proxies from closing an idle connection
291             yield _sse("ping", {"progress": run.progress, "status": run.status})
292             idle = 0.0
293
294     return StreamingResponse(
295         event_source(),
296         media_type="text/event-stream",
297         headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no", "Connection": "keep-alive"},
298     )
299
300
301 def _sse(event: str, data: dict[str, Any]) -> str:
302     return f"event: {event}\ndata: {json.dumps(data, default=str)}\n\n"

```

backend/app/api/routes/chat.py

```

1 """Natural-language query endpoints."""
2 from __future__ import annotations

```

```

3
4 from typing import Any
5
6 from fastapi import APIRouter, HTTPException
7 from pydantic import BaseModel, Field
8
9 from app.dependencies import DbSession
10 from app.services import chat_service, ecr_service
11
12 router = APIRouter(tags=["chat"])
13
14
15 class ChatQuery(BaseModel):
16     query: str = Field(..., min_length=3, max_length=2000)
17     ecr_id: str | None = None
18     run_if_missing: bool = True
19
20
21 class AskRequest(BaseModel):
22     question: str = Field(..., min_length=3, max_length=2000)
23
24
25 @router.post("/chat/query")
26 def chat_query(payload: ChatQuery, db: DbSession) -> dict[str, Any]:
27     """Ask anything about an ECR.
28
29     Detects the intent and the ECR number, runs the analysis if it has not been
30     run yet, then answers from the correlated bundle with citations.
31     """
32     return chat_service.answer_question(
33         db, payload.query, ecr_id=payload.ecr_id, run_if_missing=payload.run_if_missing
34     )
35
36
37 @router.post("/ecr/{ecr_id}/ask")
38 def ask_about_ecr(ecr_id: str, payload: AskRequest, db: DbSession) -> dict[str, Any]:
39     """Follow-up question scoped to one ECR (e.g. 'why was TC-1042 selected?')."""
40     try:
41         ecr_service.get_ecr(db, ecr_id)
42     except ecr_service.ECRNotFound as exc:
43         raise HTTPException(status_code=404, detail=f"{ecr_id} not found") from exc
44     return chat_service.answer_question(db, payload.question, ecr_id=ecr_id)
45
46
47 @router.get("/chat/intent")
48 def detect_intent(query: str) -> dict[str, Any]:
49     """Expose intent detection on its own (useful for the UI and for debugging)."""
50     return chat_service.detect_intent(query)

```

backend/app/api/routes/ecr.py

```

1 """ECR CRUD and catalogue endpoints."""
2 from __future__ import annotations
3
4 from typing import Any
5
6 from fastapi import APIRouter, HTTPException, Query, status
7
8 from app.dependencies import DbSession
9 from app.models import Comment, Evidence
10 from app.repositories import ECRRepository, ImpactReportRepository
11 from app.schemas.ecr import ECRCreate, ECRRead
12 from app.services import ecr_service
13
14 router = APIRouter(prefix="/ecr", tags=["ecr"])
15
16
17 @router.get("", response_model=list[ECRRead])
18 def list_ecrs(db: DbSession, search: str | None = None, limit: int = Query(50, le=200)):
19     repository = ECRRepository(db)
20     rows = repository.search(search, limit) if search else repository.list(limit)
21     return [ECRRead.model_validate(row.as_dict()) for row in rows]
22
23
24 @router.post("", response_model=ECRRead, status_code=status.HTTP_201_CREATED)
25 def create_ecr(payload: ECRCreate, db: DbSession):
26     try:
27         ecr = ecr_service.create_ecr(db, payload)
28     except ValueError as exc:
29         raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(exc)) from exc
30     return ECRRead.model_validate(ecr.as_dict())
31
32
33 @router.get("/{ecr_id}", response_model=ECRRead)

```

```

34 def get_ecr(ecr_id: str, db: DbSession):
35     try:
36         ecr = ecr_service.get_ecr(db, ecr_id)
37     except ecr_service.ECRNotFound as exc:
38         raise HTTPException(status_code=404, detail=f"{ecr_id} not found") from exc
39     return ECRRead.model_validate(ecr.as_dict())
40
41
42 @router.get("/{ecr_id}/sources")
43 def ecr_sources(ecr_id: str, db: DbSession) -> dict[str, Any]:
44     """Everything attached to the ECR before any correlation is applied.
45
46     Useful on its own: it is the raw multi-source view a user would otherwise
47     assemble by opening several systems.
48     """
49     try:
50         ecr = ecr_service.get_ecr(db, ecr_id)
51     except ecr_service.ECRNotFound as exc:
52         raise HTTPException(status_code=404, detail=f"{ecr_id} not found") from exc
53     normalised = ecr.ecr_id
54     comments = db.query(Comment).filter(Comment.ecr_id == normalised).all()
55     evidence = db.query(Evidence).filter(Evidence.ecr_id == normalised).all()
56     reports = ImpactReportRepository(db)
57     latest = reports.latest_for_ecr(normalised)
58     return {
59         "ecr": ecr.as_dict(),
60         "comments": [row.as_dict() for row in comments],
61         "evidence": [row.as_dict() for row in evidence],
62         "last_analysis": latest.summary() if latest else None,
63     }
64
65
66 @router.get("/{ecr_id}/history")
67 def ecr_history(ecr_id: str, db: DbSession) -> list[dict[str, Any]]:
68     """Previous analyses of this ECR."""
69     normalised = ecr_service.normalise_ecr_id(ecr_id)
70     return [
71         report.summary()
72         for report in ImpactReportRepository(db).all_reports()
73         if report.ecr_id == normalised
74     ]

```

backend/app/api/routes/health.py

```

1  """Health and platform introspection endpoints."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  from fastapi import APIRouter
7
8  from app.config import settings
9  from app.database import engine
10 from app.dependencies import DbSession
11 from app.integrations.providers import source_systems
12 from app.data.csv_store import current_counts
13 from app.llm.provider import get_llm
14 from app.services.code_analysis_service import get_code_analysis_service
15 from app.services.rag_service import get_rag_service
16
17 router = APIRouter(tags=["health"])
18
19
20 @router.get("/health")
21 def health() -> dict[str, Any]:
22     """Liveness plus a truthful picture of which subsystems are degraded."""
23     from sqlalchemy import text
24
25     database_ok = True
26     try:
27         with engine.connect() as conn:
28             conn.execute(text("SELECT 1"))
29     except Exception:
30         database_ok = False
31
32     llm = get_llm().health()
33     return {
34         "status": "ok" if database_ok else "degraded",
35         "app": settings.app_name,
36         "version": settings.app_version,
37         "environment": settings.app_env,
38         "database": {
39             "ok": database_ok,
40             "source": "csv",

```

```

41         "data_dir": settings.data_dir,
42         "working_copy": engine.dialect.name,
43     },
44     "llm": llm,
45     "demo_mode": bool(llm.get("mock")),
46     "vector_store": get_rag_service().health(),
47     "code_analysis": get_code_analysis_service().stats(),
48     "source_systems": source_systems(),
49 }
50
51
52 @router.get("/health/data")
53 def data_health(db: DbSession) -> dict[str, Any]:
54     """Row counts of the data loaded from the CSV files."""
55     return current_counts(db)

```

backend/app/api/routes/reports.py

```

1  """Report retrieval and rendering, plus dashboard analytics and feedback."""
2  from __future__ import annotations
3
4  from typing import Any
5
6  from fastapi import APIRouter, HTTPException, Query
7  from fastapi.responses import HTMLResponse, PlainTextResponse
8
9  from app.dependencies import DbSession
10 from app.models import Component, Dependency, Feedback, TestCase
11 from app.repositories import (
12     ComponentRepository,
13     ECRRepository,
14     FeedbackRepository,
15     ImpactReportRepository,
16 )
17 from app.schemas.report import ECRIntelligenceReport
18 from app.services import ecr_service
19 from app.services.report_service import render_html, render_markdown
20
21 router = APIRouter(tags=["reports"])
22
23
24 @router.get("/ecr/{ecr_id}/report")
25 def get_report(
26     ecr_id: str,
27     db: DbSession,
28     format: str = Query("json", pattern="^(json|markdown|html)$"),
29 ):
30     report = ecr_service.latest_report(db, ecr_id)
31     if not report:
32         raise HTTPException(status_code=404, detail=f"no report available for {ecr_id}")
33     if format == "json":
34         return report
35     model = ECRIntelligenceReport.model_validate(report)
36     if format == "markdown":
37         return PlainTextResponse(render_markdown(model), media_type="text/markdown")
38     return HTMLResponse(render_html(model))
39
40
41 @router.get("/reports")
42 def list_reports(db: DbSession, limit: int = Query(50, le=200)) -> list[dict[str, Any]]:
43     return [report.summary() for report in ImpactReportRepository(db).all_reports(limit)]
44
45
46 @router.get("/dashboard/stats")
47 def dashboard_stats(db: DbSession) -> dict[str, Any]:
48     """Aggregates for the dashboard landing page."""
49     reports = ImpactReportRepository(db).all_reports(200)
50     ecrs = ECRRepository(db)
51     latest_by_ecr: dict[str, Any] = {}
52     for report in reports:
53         latest_by_ecr.setdefault(report.ecr_id, report)
54
55     analysed = list(latest_by_ecr.values())
56     average_reduction = (
57         round(sum(r.reduction_percentage for r in analysed) / len(analysed), 1) if analysed else 0.0
58     )
59     average_time = (
60         round(sum(r.execution_time for r in analysed) / len(analysed), 1) if analysed else 0.0
61     )
62     tests_saved = sum(max(0, r.total_tests - r.selected_tests) for r in analysed)
63
64     return {
65         "total_ecrs": ecrs.count(),
66         "analysed_ecrs": len(analysed),

```

```

67         "average_regression_reduction": average_reduction,
68         "average_analysis_seconds": average_time,
69         "tests_avoided": tests_saved,
70         "catalogue": {
71             "components": db.query(Component).count(),
72             "dependencies": db.query(Dependency).count(),
73             "test_cases": db.query(TestCase).count(),
74         },
75         "recent_ecrs": [row.as_dict() for row in ecrs.recent(8)],
76         "recent_analyses": [report.summary() for report in reports[:8]],
77     }
78
79
80 @router.get("/components")
81 def list_components(db: DbSession) -> list[dict[str, Any]]:
82     return [row.as_dict() for row in ComponentRepository(db).list()]
83
84
85 @router.get("/components/graph")
86 def component_graph(db: DbSession) -> dict[str, Any]:
87     """The full landscape graph (used by the dependency visualisation)."""
88     from app.graph.dependency_graph import build_dependency_graph
89
90     graph = build_dependency_graph(db)
91     return {
92         "nodes": [
93             {**graph.node(node_id), "component_id": node_id} for node_id in graph.graph.nodes
94         ],
95         "edges": [
96             {"source": source, "target": target, **data}
97             for source, target, data in graph.graph.edges(data=True)
98         ],
99         "stats": graph.stats(),
100     }
101
102
103 @router.post("/feedback")
104 def submit_feedback(payload: dict[str, Any], db: DbSession) -> dict[str, Any]:
105     """Record whether a recommendation was useful (feedback loop)."""
106     feedback = Feedback(
107         ecr_id=ecr_service.normalise_ecr_id(payload.get("ecr_id", "")),
108         workflow_id=payload.get("workflow_id", ""),
109         target_type=payload.get("target_type", "RECOMMENDATION"),
110         target_id=payload.get("target_id", ""),
111         useful=bool(payload.get("useful", True)),
112         comment=payload.get("comment", ""),
113     )
114     FeedbackRepository(db).add(feedback)
115     return feedback.as_dict()
116
117
118 @router.get("/feedback")
119 def list_feedback(db: DbSession) -> list[dict[str, Any]]:
120     return [row.as_dict() for row in FeedbackRepository(db).list()]

```

Backend - source system integrations

backend/app/integrations/__init__.py

backend/app/integrations/base.py

```

1  """Enterprise source-system integration interfaces.
2
3  The platform reads its data through these interfaces so a real connector can be
4  dropped in without touching an agent. Every interface ships with a local
5  implementation (the seeded database) so the PoC needs no credentials, and a
6  placeholder adapter that documents exactly what the real one must return.
7  """
8  from __future__ import annotations
9
10 from abc import ABC, abstractmethod
11 from dataclasses import dataclass
12 from typing import Any, Iterable
13
14
15 @dataclass(slots=True)
16 class SourceSystem:
17     """Describes a connected (or connectable) system for the UI and /health."""
18
19     key: str
20     name: str
21     kind: str # REQUIREMENTS | DEFECTS | TESTS | CODE | DOCS | WORKFLOW

```

```

22     connected: bool
23     detail: str = ""
24
25
26 class BaseRequirementProvider(ABC):
27     """Requirements, from wherever the organisation keeps them."""
28
29     key: str = "base"
30     name: str = "Base requirement provider"
31
32     @abstractmethod
33     def fetch(self, *, query: str = "", ids: Iterable[str] = ()) -> list[dict[str, Any]]: ...
34
35     def describe(self) -> SourceSystem:
36         return SourceSystem(self.key, self.name, "REQUIREMENTS", connected=False)
37
38
39 class BaseDefectProvider(ABC):
40     key: str = "base"
41     name: str = "Base defect provider"
42
43     @abstractmethod
44     def fetch(self, *, query: str = "", components: Iterable[str] = ()) -> list[dict[str, Any]]: ...
45
46     def describe(self) -> SourceSystem:
47         return SourceSystem(self.key, self.name, "DEFECTS", connected=False)
48
49
50 class BaseTestProvider(ABC):
51     key: str = "base"
52     name: str = "Base test provider"
53
54     @abstractmethod
55     def fetch(
56         self, *, requirement_ids: Iterable[str] = (), components: Iterable[str] = ()
57     ) -> list[dict[str, Any]]: ...
58
59     def describe(self) -> SourceSystem:
60         return SourceSystem(self.key, self.name, "TESTS", connected=False)
61
62
63 class BaseRepositoryProvider(ABC):
64     """Source control: which files a change actually touched."""
65
66     key: str = "base"
67     name: str = "Base repository provider"
68
69     @abstractmethod
70     def changed_files(self, reference: str) -> list[str]: ...
71
72     def describe(self) -> SourceSystem:
73         return SourceSystem(self.key, self.name, "CODE", connected=False)

```

backend/app/integrations/providers.py

```

1  """Local providers (used by the PoC) and enterprise adapters (placeholders).
2
3  The local providers are the ones actually wired in: they read the seeded
4  database, which is what makes the platform runnable with no credentials. The
5  Azure DevOps / Jira / TestRail / GitHub adapters implement the same interfaces
6  and raise a clear error until they are configured, so the wiring point is
7  obvious and the failure is never silent.
8  """
9  from __future__ import annotations
10
11  from dataclasses import asdict
12  from typing import Any, Iterable
13
14  from app.config import settings
15  from app.database import session_scope
16  from app.integrations.base import (
17      BaseDefectProvider,
18      BaseRepositoryProvider,
19      BaseRequirementProvider,
20      BaseTestProvider,
21      SourceSystem,
22  )
23  from app.logging import get_logger
24  from app.repositories import DefectRepository, RequirementRepository, TestRepository
25
26  logger = get_logger("ecr.integrations")
27
28
29 class NotConfigured(RuntimeError):

```

```

30     """Raised when an enterprise adapter is used without credentials."""
31
32
33 # -----
34 # Local (seeded database) - the default wiring
35 # -----
36 class LocalRequirementProvider(BaseRequirementProvider):
37     key = "local"
38     name = "Local requirement store"
39
40     def fetch(self, *, query: str = "", ids: Iterable[str] = ()) -> list[dict[str, Any]]:
41         with session_scope() as db:
42             repository = RequirementRepository(db)
43             rows = repository.get_many(list(ids)) if ids else repository.list()
44             return [row.as_dict() for row in rows]
45
46     def describe(self) -> SourceSystem:
47         return SourceSystem(self.key, self.name, "REQUIREMENTS", True, "CSV data files")
48
49
50 class LocalDefectProvider(BaseDefectProvider):
51     key = "local"
52     name = "Local defect store"
53
54     def fetch(self, *, query: str = "", components: Iterable[str] = ()) -> list[dict[str, Any]]:
55         with session_scope() as db:
56             repository = DefectRepository(db)
57             rows = repository.by_components(list(components)) if components else repository.list()
58             return [row.as_dict() for row in rows]
59
60     def describe(self) -> SourceSystem:
61         return SourceSystem(self.key, self.name, "DEFECTS", True, "CSV data files")
62
63
64 class LocalTestProvider(BaseTestProvider):
65     key = "local"
66     name = "Local test store"
67
68     def fetch(
69         self, *, requirement_ids: Iterable[str] = (), components: Iterable[str] = ()
70     ) -> list[dict[str, Any]]:
71         with session_scope() as db:
72             repository = TestRepository(db)
73             rows: list = []
74             if requirement_ids:
75                 rows += repository.by_requirements(list(requirement_ids))
76             if components:
77                 rows += repository.by_components(list(components))
78             if not rows:
79                 rows = repository.list()
80             unique = {row.test_case_id: row for row in rows}
81             return [row.as_dict() for row in unique.values()]
82
83     def describe(self) -> SourceSystem:
84         return SourceSystem(self.key, self.name, "TESTS", True, "CSV data files")
85
86
87 class LocalRepositoryProvider(BaseRepositoryProvider):
88     key = "local"
89     name = "Local sample repository"
90
91     def changed_files(self, reference: str) -> list[str]:
92         """The PoC takes changed files from the ECR record itself."""
93         with session_scope() as db:
94             from app.repositories import ECRRepository
95
96             ecr = ECRRepository(db).get(reference)
97             return list(ecr.changed_files or []) if ecr else []
98
99     def describe(self) -> SourceSystem:
100         return SourceSystem(self.key, self.name, "CODE", True, "Parsed with the Python AST analyzer")
101
102
103 # -----
104 # Enterprise adapters - interfaces implemented, credentials required
105 # -----
106 class AzureDevOpsRequirementProvider(BaseRequirementProvider):
107     """Work items (`/_apis/wit/wiql`) mapped onto the requirement contract."""
108
109     key = "azure_devops"
110     name = "Azure DevOps Work Items"
111
112     def __init__(self, organisation: str | None = None, project: str | None = None,
113                 token: str | None = None) -> None:

```

```

114         self.organisation = organisation
115         self.project = project
116         self.token = token
117
118     @property
119     def configured(self) -> bool:
120         return bool(self.organisation and self.project and self.token)
121
122     def fetch(self, *, query: str = "", ids: Iterable[str] = ()) -> list[dict[str, Any]]:
123         if not self.configured:
124             raise NotConfigured(
125                 "Azure DevOps is not configured. Provide organisation, project and a PAT, then map "
126                 "System.Id -> requirement_id, System.Title -> title, System.Description -> "
127                 "description, Microsoft.VSTS.Common.Priority -> priority."
128             )
129         raise NotConfigured("Azure DevOps client not implemented in this PoC")
130
131     def describe(self) -> SourceSystem:
132         return SourceSystem(
133             self.key, self.name, "REQUIREMENTS", self.configured,
134             "Interface ready; supply organisation/project/PAT to enable",
135         )
136
137
138 class JiraRequirementProvider(BaseRequirementProvider):
139     key = "jira"
140     name = "Jira Issues"
141
142     def __init__(self, base_url: str | None = None, token: str | None = None) -> None:
143         self.base_url = base_url
144         self.token = token
145
146     @property
147     def configured(self) -> bool:
148         return bool(self.base_url and self.token)
149
150     def fetch(self, *, query: str = "", ids: Iterable[str] = ()) -> list[dict[str, Any]]:
151         raise NotConfigured("Jira client not implemented in this PoC")
152
153     def describe(self) -> SourceSystem:
154         return SourceSystem(self.key, self.name, "REQUIREMENTS", self.configured,
155                             "Interface ready; supply base URL and API token")
156
157
158 class ServiceNowDefectProvider(BaseDefectProvider):
159     key = "servicenow"
160     name = "ServiceNow Incidents"
161
162     def __init__(self, instance: str | None = None, token: str | None = None) -> None:
163         self.instance = instance
164         self.token = token
165
166     @property
167     def configured(self) -> bool:
168         return bool(self.instance and self.token)
169
170     def fetch(self, *, query: str = "", components: Iterable[str] = ()) -> list[dict[str, Any]]:
171         raise NotConfigured("ServiceNow client not implemented in this PoC")
172
173     def describe(self) -> SourceSystem:
174         return SourceSystem(self.key, self.name, "DEFECTS", self.configured,
175                             "Interface ready; supply instance and token")
176
177
178 class TestRailTestProvider(BaseTestProvider):
179     key = "testrail"
180     name = "TestRail Test Cases"
181
182     def __init__(self, base_url: str | None = None, token: str | None = None) -> None:
183         self.base_url = base_url
184         self.token = token
185
186     @property
187     def configured(self) -> bool:
188         return bool(self.base_url and self.token)
189
190     def fetch(
191         self, *, requirement_ids: Iterable[str] = (), components: Iterable[str] = ()
192     ) -> list[dict[str, Any]]:
193         raise NotConfigured("TestRail client not implemented in this PoC")
194
195     def describe(self) -> SourceSystem:
196         return SourceSystem(self.key, self.name, "TESTS", self.configured,
197                             "Interface ready; supply base URL and API key")

```

```

198
199
200 class GitHubRepositoryProvider(BaseRepositoryProvider):
201     key = "github"
202     name = "GitHub Repositories"
203
204     def __init__(self, token: str | None = None, repo: str | None = None) -> None:
205         self.token = token
206         self.repo = repo
207
208     @property
209     def configured(self) -> bool:
210         return bool(self.token and self.repo)
211
212     def changed_files(self, reference: str) -> list[str]:
213         if not self.configured:
214             raise NotConfigured(
215                 "GitHub is not configured. Provide a token and owner/repo, then read the file list "
216                 "from GET /repos/{owner}/{repo}/pulls/{number}/files."
217             )
218         raise NotConfigured("GitHub client not implemented in this PoC")
219
220     def describe(self) -> SourceSystem:
221         return SourceSystem(self.key, self.name, "CODE", self.configured,
222                             "Interface ready; supply a token and owner/repo")
223
224
225 # -----
226 # Wiring
227 # -----
228 def get_requirement_provider() -> BaseRequirementProvider:
229     return LocalRequirementProvider()
230
231
232 def get_defect_provider() -> BaseDefectProvider:
233     return LocalDefectProvider()
234
235
236 def get_test_provider() -> BaseTestProvider:
237     return LocalTestProvider()
238
239
240 def get_repository_provider() -> BaseRepositoryProvider:
241     return LocalRepositoryProvider()
242
243
244 def source_systems() -> list[dict[str, Any]]:
245     """Everything the platform can read from, and whether it is connected."""
246     providers = [
247         LocalRequirementProvider(),
248         LocalDefectProvider(),
249         LocalTestProvider(),
250         LocalRepositoryProvider(),
251         AzureDevOpsRequirementProvider(),
252         JiraRequirementProvider(),
253         ServiceNowDefectProvider(),
254         TestRailTestProvider(),
255         GitHubRepositoryProvider(),
256     ]
257     return [asdict(provider.describe()) for provider in providers]

```

Backend - sample repository parsed by the code analyzer

backend/app/seed/__main__.py

```

1  """python -m app.seed -- check that the CSV data files load, and build embeddings.
2
3      python -m app.seed          load backend/data/*.csv and print row counts
4      python -m app.seed --index  also build the embedding cache (reuses unchanged rows)
5      python -m app.seed --index --force  re-embed every row
6
7  It loads into a private in-memory copy, so it is safe to run while the backend
8  is up, and it never changes the CSV files.
9  """
10 from __future__ import annotations
11
12 import os
13 import sys
14
15 # Before any app import: keep this check away from the running backend's working copy.
16 os.environ["DATABASE_URL"] = "sqlite://"
17
18 from app.config import settings # noqa: E402

```

```

19 from app.data.csv_store import CsvDataError # noqa: E402
20 from app.database import init_db, session_scope # noqa: E402
21 from app.logging import configure_logging # noqa: E402
22
23
24 def main() -> int:
25     configure_logging()
26     try:
27         counts = init_db()
28     except CsvDataError as exc:
29         print(f"CSV data error: {exc}")
30         return 1
31     print(f"Loaded from {settings.data_dir}:")
32     for table, rows in counts.items():
33         print(f" {table:<16} {rows:>5}")
34
35     if "--index" in sys.argv:
36         from app.services.rag_service import get_rag_service
37
38         with session_scope() as db:
39             print("Indexed:", get_rag_service().index_all(db, force="--force" in sys.argv))
40     return 0
41
42
43 if __name__ == "__main__":
44     raise SystemExit(main())

```

backend/app/seed/sample_repo/auth/service.py

```

1 """Auth Service (CMP-010): sessions, MFA and step-up assertions."""
2 from notification.service import send_refund_confirmation
3
4 ACCESS_TOKEN_TTL = 1800
5 HIGH_VALUE_THRESHOLD = 5000
6 _REVOKED = set()
7
8
9 def issue_token(user_id, device_trusted):
10     return {"sub": user_id, "device_trusted": device_trusted, "ttl": ACCESS_TOKEN_TTL}
11
12
13 def refresh_token(token):
14     """Device trust must survive a refresh (see BUG-228)."""
15     return {"sub": token["sub"], "device_trusted": token.get("device_trusted", False), "ttl": ACCESS_TOKEN_TTL}
16
17
18 def revoke_sessions(user_id):
19     _REVOKED.add(user_id)
20     return True
21
22
23 def requires_step_up(amount, token):
24     return amount >= HIGH_VALUE_THRESHOLD or not token.get("device_trusted")
25
26
27 def validate_step_up_assertion(assertion, token):
28     return bool(assertion) and assertion.get("sub") == token.get("sub")

```

backend/app/seed/sample_repo/currency/service.py

```

1 """Currency Service (CMP-005): FX rate sourcing and catalogue."""
2 from infra.redis_cache import cache_get, cache_set
3
4 RATE_TTL_SECONDS = 900
5 _CATALOGUE = ["EUR", "USD", "GBP", "JPY", "CHF", "SEK", "BHD"]
6
7
8 def supported_currencies():
9     return list(_CATALOGUE)
10
11
12 def refresh_rates():
13     """Pull the treasury feed and refresh the cache."""
14     rates = {"EUR:USD": 1.09, "USD:EUR": 0.92, "EUR:GBP": 0.85, "EUR:JPY": 158.4}
15     cache_set("fx_rates", rates, RATE_TTL_SECONDS)
16     return rates
17
18
19 def get_rate(from_currency, to_currency):
20     rates = cache_get("fx_rates") or refresh_rates()
21     key = from_currency + ":" + to_currency
22     if key not in rates:
23         raise KeyError("NO_RATE_AVAILABLE")

```

```
24         return rates[key]
```

backend/app/seed/sample_repo/fraud/service.py

```
1  """Fraud Screening Service (CMP-014)."""
2  from schema.transaction_schema import recent_transaction_count
3
4  THRESHOLD = 0.82
5
6
7  def score_transaction(request, base_amount):
8      velocity = recent_transaction_count(request.get("customer_id"))
9      return min(1.0, base_amount / 25000 + velocity * 0.05)
10
11
12  def screen_transaction(request, base_amount, base_currency):
13      score = score_transaction(request, base_amount)
14      return "HOLD" if score >= THRESHOLD else "ALLOW"
```

backend/app/seed/sample_repo/gateway/router.py

```
1  """API Gateway routing table (CMP-002)."""
2  from auth.service import refresh_token
3  from payment.service import authorise_payment
4  from order.service import checkout
5  from refund.service import process_refund
6
7  ROUTES = {
8      "POST /payments": "payment.service.authorise_payment",
9      "POST /orders/checkout": "order.service.checkout",
10     "POST /refunds": "refund.service.process_refund",
11     "POST /refunds/v2": "refund.service.process_refund",
12     "POST /auth/refresh": "auth.service.refresh_token",
13 }
14
15
16  def dispatch(route, payload, merchant_config):
17      if route == "POST /payments":
18          return authorise_payment(payload, merchant_config)
19      if route == "POST /orders/checkout":
20          return checkout(payload, merchant_config)
21      if route.startswith("POST /refunds"):
22          return process_refund(payload, merchant_config)
23      if route == "POST /auth/refresh":
24          return refresh_token(payload)
25      raise KeyError("NO_ROUTE")
```

backend/app/seed/sample_repo/infra/redis_cache.py

```
1  """Shared cache adapter (infrastructure)."""
2  _STORE = {}
3
4
5  def cache_get(key):
6      entry = _STORE.get(key)
7      return entry["value"] if entry else None
8
9
10  def cache_set(key, value, ttl_seconds):
11      _STORE[key] = {"value": value, "ttl": ttl_seconds}
12      return True
```

backend/app/seed/sample_repo/invoice/service.py

```
1  """Invoice Service (CMP-012)."""
2  from payment.currency import minor_units
3  from schema.transaction_schema import find_transaction
4
5  _SEQUENCE = {"EU": 0}
6
7
8  def next_invoice_number(entity="EU"):
9      _SEQUENCE[entity] += 1
10     return entity + "-INV-" + str(_SEQUENCE[entity]).zfill(6)
11
12
13  def create_invoice(transaction_id, amount, currency):
14      find_transaction(transaction_id)
15      return {
16          "invoice_number": next_invoice_number(),
17          "amount": round(amount, minor_units(currency)),
18          "currency": currency,
19      }
```

backend/app/seed/sample_repo/ledger/service.py

```
1 """Ledger Service (CMP-015): double-entry postings."""
2 from schema.transaction_schema import find_transaction
3
4 _POSTINGS = []
5
6
7 def post_capture(transaction_id, amount):
8     transaction = find_transaction(transaction_id)
9     _POSTINGS.append({"tx": transaction_id, "debit": amount, "credit": amount})
10    return transaction
11
12
13 def post_refund(transaction_id, amount, original_rate):
14     _POSTINGS.append({"tx": transaction_id, "debit": -amount, "credit": -amount, "rate": original_rate})
15     return True
16
17
18 def balance():
19     return sum(p["debit"] - p["credit"] for p in _POSTINGS)
```

backend/app/seed/sample_repo/notification/service.py

```
1 """Notification Service (CMP-011)."""
2 from notification.templates import ORDER_CONFIRMATION, REFUND_CONFIRMATION, render
3
4 _SENT = []
5
6
7 def send_order_confirmation(customer_id, invoice_number):
8     body = render(ORDER_CONFIRMATION, {"order_id": invoice_number})
9     _SENT.append({"to": customer_id, "body": body})
10    return True
11
12
13 def send_refund_confirmation(transaction_id):
14     body = render(REFUND_CONFIRMATION, {"transaction_id": transaction_id})
15     _SENT.append({"to": transaction_id, "body": body})
16    return True
```

backend/app/seed/sample_repo/notification/templates.py

```
1 """Notification templates (CMP-011). Content-only module."""
2 ORDER_CONFIRMATION = {
3     "subject": "Your order {{order_id}} is confirmed",
4     "header": "Thanks for your order",
5     "footer": "Enterprise Commerce, 1 Market Street",
6 }
7
8 REFUND_CONFIRMATION = {
9     "subject": "Your refund for {{transaction_id}} is on its way",
10    "header": "Refund processed",
11    "footer": "Enterprise Commerce, 1 Market Street",
12 }
13
14 SECURITY_ALERT = {
15     "subject": "Security check on your account",
16     "header": "We need to verify it is you",
17     "footer": "Enterprise Commerce Security",
18 }
19
20
21 def render(template, values):
22     body = template["header"]
23     for key, value in values.items():
24         body = body.replace("{{" + key + "}}", str(value))
25     return body
```

backend/app/seed/sample_repo/order/service.py

```
1 """Order Service (CMP-006): cart, checkout and totals."""
2 from payment.service import authorise_payment
3 from invoice.service import create_invoice
4 from notification.service import send_order_confirmation
5 from payment.currency import minor_units
6
7
8 def calculate_total(line_items, tax_rate, discount, currency):
9     subtotal = sum(item["price"] * item["quantity"] for item in line_items)
10    discounted = subtotal - discount
11    total = discounted * (1 + tax_rate)
12    return round(total, minor_units(currency))
13
```

```

14
15 def checkout(cart, merchant_config):
16     total = calculate_total(cart["items"], cart["tax_rate"], cart.get("discount", 0), cart["currency"])
17     result = authorise_payment(
18         {
19             "amount": total,
20             "currency": cart["currency"],
21             "instrument_token": cart["instrument_token"],
22             "idempotency_key": cart["idempotency_key"],
23         },
24         merchant_config,
25     )
26     if result["status"] != "AUTHORISED":
27         return {"status": "CHECKOUT_FAILED", "payment": result}
28     invoice = create_invoice(result["transaction_id"], total, cart["currency"])
29     send_order_confirmation(cart["customer_id"], invoice["invoice_number"])
30     return {"status": "CONFIRMED", "invoice": invoice}
31
32
33 def cancel_order(order_id, captured):
34     return {"status": "REFUND_REQUESTED" if captured else "VOIDED", "order_id": order_id}

```

backend/app/seed/sample_repo/payment/currency.py

```

1  """Currency helpers used inside the payment boundary (CMP-005)."""
2  from currency.service import get_rate, supported_currencies
3
4  ZERO_DECIMAL = {"JPY", "KRW", "VND"}
5  THREE_DECIMAL = {"BHD", "KWD", "OMR"}
6
7
8  def minor_units(currency):
9      if currency in ZERO_DECIMAL:
10         return 0
11     if currency in THREE_DECIMAL:
12         return 3
13     return 2
14
15
16 def is_supported_currency(currency):
17     return currency in supported_currencies()
18
19
20 def convert(amount, from_currency, to_currency):
21     """Convert using the rate effective now; raises for unknown currencies."""
22     if from_currency == to_currency:
23         return amount
24     rate = get_rate(from_currency, to_currency)
25     return round(amount * rate, minor_units(to_currency))

```

backend/app/seed/sample_repo/payment/service.py

```

1  """Payment Service (CMP-003): authorise, capture and settle payments."""
2  from payment.currency import convert
3  from payment.validator import ValidationError, validate_payment_request
4  from fraud.service import screen_transaction
5  from ledger.service import post_capture
6  from schema.transaction_schema import insert_transaction
7
8
9  def authorise_payment(request, merchant_config):
10     errors = validate_payment_request(request, merchant_config)
11     if errors:
12         raise ValidationError(errors)
13     settlement_currency = merchant_config["settlement_currency"]
14     settlement_amount = convert(request["amount"], request["currency"], settlement_currency)
15     decision = screen_transaction(request, settlement_amount, settlement_currency)
16     if decision == "HOLD":
17         return {"status": "PENDING_REVIEW"}
18     transaction = insert_transaction(
19         amount=request["amount"],
20         currency=request["currency"],
21         settlement_amount=settlement_amount,
22         settlement_currency=settlement_currency,
23         idempotency_key=request.get("idempotency_key"),
24     )
25     return {"status": "AUTHORISED", "transaction_id": transaction["id"]}
26
27
28 def capture_payment(transaction_id, amount):
29     post_capture(transaction_id, amount)
30     return {"status": "CAPTURED", "transaction_id": transaction_id}

```

backend/app/seed/sample_repo/payment/validator.py

```
1  """Shared payment/refund validation library (Payment Validator, CMP-004)."""
2  from payment.currency import is_supported_currency, minor_units
3
4  MAX_AMOUNT_DEFAULT = 10000
5
6
7  class ValidationError(Exception):
8      """Raised when a payment request cannot be processed."""
9
10
11  def validate_amount(amount, currency, merchant_config):
12      """Amount must be positive and within the merchant ceiling."""
13      errors = []
14      if amount is None or amount <= 0:
15          errors.append("INVALID_AMOUNT")
16      ceiling = merchant_config.get("max_amount", MAX_AMOUNT_DEFAULT)
17      if amount is not None and amount > ceiling:
18          errors.append("AMOUNT_ABOVE_CEILING")
19      if amount is not None and round(amount, minor_units(currency)) != amount:
20          errors.append("INVALID_MINOR_UNITS")
21      return errors
22
23
24  def validate_currency(currency, merchant_config):
25      """Reject any currency outside the merchant supported list."""
26      errors = []
27      if not currency or len(currency) != 3:
28          errors.append("MALFORMED_CURRENCY")
29          return errors
30      if not is_supported_currency(currency):
31          errors.append("UNSUPPORTED_CURRENCY")
32      if currency not in merchant_config.get("supported_currencies", []):
33          errors.append("UNSUPPORTED_CURRENCY")
34      return errors
35
36
37  def validate_payment_request(request, merchant_config):
38      """Entry point used by both the payment and the refund service."""
39      errors = []
40      errors += validate_amount(request.get("amount"), request.get("currency"), merchant_config)
41      errors += validate_currency(request.get("currency"), merchant_config)
42      if not request.get("instrument_token"):
43          errors.append("MISSING_INSTRUMENT")
44      return sorted(set(errors))
```

backend/app/seed/sample_repo/portal/profile_page.py

```
1  """Frontend Portal profile page controller (CMP-001)."""
2  from user.service import get_profile, update_profile
3
4  SAVE_BUTTON_LABEL = "Submit"
5  SAVE_CONFIRMATION = "Profile updated"
6
7
8  def render_profile_page(user_id):
9      profile = get_profile(user_id)
10     return {
11         "fields": profile,
12         "save_button": SAVE_BUTTON_LABEL,
13     }
14
15
16  def submit_profile_form(user_id, form_values):
17      result = update_profile(user_id, form_values)
18      return {"profile": result, "toast": SAVE_CONFIRMATION}
```

backend/app/seed/sample_repo/refund/service.py

```
1  """Refund Service (CMP-007): full and partial refunds."""
2  from payment.validator import validate_payment_request
3  from ledger.service import post_refund
4  from schema.transaction_schema import find_transaction
5  from notification.service import send_refund_confirmation
6
7
8  def refundable_amount(transaction_id, already_refunded):
9      transaction = find_transaction(transaction_id)
10     return (transaction["amount"] if transaction else 0) - already_refunded
11
12
13  def process_refund(request, merchant_config, already_refunded=0):
14     errors = validate_payment_request(request, merchant_config)
```

```

15     if errors:
16         return {"status": "REJECTED", "errors": errors}
17     remaining = refundable_amount(request["transaction_id"], already_refunded)
18     if request["amount"] > remaining:
19         return {"status": "REJECTED", "errors": ["AMOUNT_EXCEEDS_REFUNDABLE"]}
20     post_refund(request["transaction_id"], request["amount"], request.get("original_rate"))
21     send_refund_confirmation(request["transaction_id"])
22     return {"status": "REFUNDED"}

```

backend/app/seed/sample_repo/reporting/service.py

```

1  """Reporting Service (CMP-013): reconciliation extracts."""
2  from schema.transaction_schema import settled_transactions
3  from ledger.service import balance
4
5
6  def reconciliation_report(currency=None):
7      rows = settled_transactions(currency)
8      settled = sum(row["settlement_amount"] or 0 for row in rows)
9      return {
10         "currency": currency or "ALL",
11         "transaction_count": len(rows),
12         "settled_amount": settled,
13         "ledger_balance": balance(),
14     }

```

backend/app/seed/sample_repo/schema/transaction_schema.py

```

1  """Transaction Database access layer and schema (CMP-008)."""
2  TRANSACTION_COLUMNS = [
3      "id",
4      "amount",
5      "currency",
6      "settlement_amount",
7      "settlement_currency",
8      "status",
9      "idempotency_key",
10     "created_at",
11     "settled_at",
12 ]
13
14 _ROWS = []
15
16
17 def insert_transaction(amount, currency, settlement_amount, settlement_currency, idempotency_key):
18     row = {
19         "id": "TX-" + str(len(_ROWS) + 1),
20         "amount": amount,
21         "currency": currency,
22         "settlement_amount": settlement_amount,
23         "settlement_currency": settlement_currency,
24         "status": "AUTHORISED",
25         "idempotency_key": idempotency_key,
26     }
27     _ROWS.append(row)
28     return row
29
30
31 def find_transaction(transaction_id):
32     for row in _ROWS:
33         if row["id"] == transaction_id:
34             return row
35     return None
36
37
38 def recent_transaction_count(customer_id):
39     return len([row for row in _ROWS if row.get("customer_id") == customer_id])
40
41
42 def settled_transactions(currency=None):
43     return [r for r in _ROWS if currency is None or r["currency"] == currency]

```

backend/app/seed/sample_repo/user/service.py

```

1  """User Service (CMP-009): profile and address book."""
2  from auth.service import revoke_sessions
3
4  _PROFILES = {}
5
6
7  def get_profile(user_id):
8      return _PROFILES.setdefault(user_id, {"display_name": "", "addresses": []})
9
10

```

```

11 def update_profile(user_id, changes):
12     profile = get_profile(user_id)
13     profile.update(changes)
14     return profile
15
16
17 def upsert_address(user_id, address):
18     """Partial update must preserve the default flag (see BUG-244)."""
19     profile = get_profile(user_id)
20     for existing in profile["addresses"]:
21         if existing["id"] == address["id"]:
22             existing.update({k: v for k, v in address.items() if v is not None})
23             return existing
24     profile["addresses"].append(address)
25     return address
26
27
28 def change_password(user_id, new_password):
29     revoke_sessions(user_id)
30     return True

```

Backend - tests

backend/app/tests/__init__.py

backend/app/tests/conftest.py

```

1  """Test fixtures: a private copy of the CSV data files for every test worker."""
2  from __future__ import annotations
3
4  import os
5  import shutil
6  import tempfile
7  from pathlib import Path
8
9  import pytest
10
11 # Before app modules import: point the app at a throwaway copy of backend/data, so
12 # tests that create records never touch the real CSV files. One copy per xdist
13 # worker; values are assigned, not setdefault, because workers inherit the
14 # controller's environment.
15 _WORKER = os.environ.get("PYTEST_XDIST_WORKER", "main")
16 _ROOT = Path(tempfile.gettempdir()) / f"ecr_test_{_WORKER}"
17 shutil.rmtree(_ROOT, ignore_errors=True)
18 shutil.copytree(Path(__file__).resolve().parents[2] / "data", _ROOT / "data")
19 os.environ["DATA_DIR"] = str(_ROOT / "data")
20 os.environ["DATABASE_URL"] = f"sqlite:///({_ROOT / 'working_copy.db'}).as_posix()"
21 os.environ["EMBEDDING_CACHE_PATH"] = str(_ROOT / "embeddings.json")
22 os.environ["LLM_PROVIDER"] = "mock"
23 os.environ["AGENT_STEP_DELAY_MS"] = "0"
24 os.environ["APP_ENV"] = "test"
25
26
27 @pytest.fixture(scope="session", autouse=True)
28 def database():
29     from app.data.csv_store import writer
30     from app.database import init_db
31
32     init_db()
33     yield
34     writer.flush()
35
36
37 @pytest.fixture(scope="session")
38 def rag(database):
39     from app.database import session_scope
40     from app.services.rag_service import get_rag_service
41
42     service = get_rag_service()
43     with session_scope() as db:
44         service.index_all(db, force=True)
45     return service
46
47
48 @pytest.fixture(scope="session")
49 def client(database, rag):
50     from fastapi.testclient import TestClient
51
52     from app.main import app
53
54     with TestClient(app) as test_client:
55         yield test_client

```

```

56
57
58 @pytest.fixture
59 def db_session(database):
60     from app.database import SessionLocal
61
62     session = SessionLocal()
63     try:
64         yield session
65     finally:
66         session.close()

```

backend/app/tests/test_agents/__init__.py

backend/app/tests/test_agents/test_agent_llm_reasoning.py

```

1  """Each of the three agents reasons with the LLM, and what it says stays grounded."""
2  from __future__ import annotations
3
4  from app.agents.nodes import correlation_review, retrieval_planning
5  from app.agents.nodes.planning import build_plan
6  from app.agents.orchestrator import run_workflow
7  from app.database import session_scope
8  from app.llm import openai_provider
9  from app.llm import provider as provider_module
10 from app.llm.base import BaseLLMProvider
11 from app.llm.provider import get_llm
12 from app.repositories import ECRRepository
13 from app.schemas.common import ChangeType
14
15
16 class FakeLLM:
17     """A real (non-mock) provider as far as the agents can tell."""
18
19     is_mock = False
20
21     def __init__(self, replies: dict | None = None) -> None:
22         self.replies = replies or {}
23         self.calls: list[str] = []
24
25     def generate(self, prompt, *, fallback=None, **kwargs):
26         self.calls.append("generate")
27         return "Grounded answer from the model."
28
29     def generate_structured(self, prompt, schema, *, fallback=None, **kwargs):
30         self.calls.append(schema.__name__)
31         data = {*(fallback or {}), "source": "llm", **self.replies.get(schema.__name__, {})}
32         return schema.model_validate(data)
33
34
35 ECR_STATE = {
36     "ecr_input": {"ecr_id": "ECR-T", "title": "Tune braking", "description": "Adjust the curve."},
37     "ecr_analysis": {"summary": "Rule summary.", "technical_keywords": ["braking"]},
38 }
39
40
41 def _bundle_state() -> dict:
42     return {
43         "ecr_id": "ECR-T",
44         "requirements": [{"requirement_id": "REQ-1", "title": "Brake enforcement"}],
45         "comments": [
46             {"comment_id": "CMT-1", "author": "Ana", "sentiment": "CONCERN", "target_id": "REQ-1", "referen
47 + ces": []}],
48         "evidence": [{"evidence_id": "EV-1", "title": "HIL run", "outcome": "PASS", "target_id": "REQ-1"}],
49         "collaboration_analysis": {"evidence_gaps": []},
50     }
51
52
53 def test_retrieval_agent_llm_plans_the_search(monkeypatch):
54     fake = FakeLLM(
55         {
56             "RetrievalPlan": {
57                 "summary": "Changes the braking curve.",
58                 "search_terms": ["Braking curve", "braking curve", "REQ-101", "ab", "penalty brake"],
59                 "could_break": ["enforcement timing"],
60                 "needs_dependency_check": True,
61                 "dependency_reason": "The braking model is shared.",
62             }
63         }
64     )
65     monkeypatch.setattr(retrieval_planning, "get_llm", lambda: fake)
66

```

```

67 out = retrieval_planning.retrieval_planning_step(ECR_STATE)
68 plan = out["retrieval_plan"]
69
70 assert fake.calls == ["RetrievalPlan"]
71 assert plan["source"] == "llm"
72 assert plan["search_terms"] == ["braking curve", "penalty brake"] # no duplicates, ids or noise
73 assert out["ecr_analysis"]["summary"] == "Changes the braking curve."
74 assert "penalty brake" in retrieval_planning.search_query(**ECR_STATE, "retrieval_plan": plan})
75 # The LLM can add the dependency walk the rules would skip for a UI change.
76 ui = build_plan({"change_type": ChangeType.UI.value, "risk_indicators": []}, {}, plan)
77 assert "dependencies" in ui["investigations"]
78 assert "Retrieval Agent" in ui["reasons"]["dependencies"]
79
80
81 def test_without_an_llm_retrieval_uses_the_rule_keywords():
82 out = retrieval_planning.retrieval_planning_step(ECR_STATE)
83 assert out["retrieval_plan"]["source"] == "rules"
84 assert retrieval_planning.search_query(**ECR_STATE, **out) == "Tune braking. Adjust the curve."
85
86
87 def test_correlation_agent_llm_findings_must_cite_real_artefacts(monkeypatch):
88 fake = FakeLLM(
89     {
90         "CorrelationReview": {
91             "summary": "One contradiction.",
92             "conflicts": [
93                 {"ids": ["CMT-1", "EV-1", "BUG-999"], "text": "Concern against a pass."},
94                 {"ids": ["BUG-999"], "text": "Invented."},
95             ],
96             "key_links": [],
97             "gaps": [],
98         }
99     }
100 )
101 monkeypatch.setattr(correlation_review, "get_llm", lambda: fake)
102
103 review = correlation_review.correlation_review_step(_bundle_state())["correlation_review"]
104
105 assert fake.calls == ["CorrelationReview"]
106 assert review["source"] == "llm"
107 assert review["conflicts"] == [{"ids": ["CMT-1", "EV-1"], "text": "Concern against a pass."}]
108
109
110 def test_rule_review_spots_a_concern_contradicted_by_a_pass():
111 review = correlation_review.correlation_review_step(_bundle_state())["correlation_review"]
112 assert review["source"] == "rules"
113 assert review["conflicts"][0]["ids"] == ["CMT-1", "REQ-1", "EV-1"]
114
115
116 class CountingProvider(BaseLLMProvider):
117     """A remote provider that records every request instead of sending it."""
118
119     name = "counting"
120     is_mock = False
121     supports_remote_embeddings = False
122
123     def __init__(self) -> None:
124         self.requests: list[str] = []
125
126     def generate(self, prompt, *, system=None, max_tokens=None, temperature=0.2, fast=False):
127         self.requests.append("generate")
128         return "Model text."
129
130     def generate_structured(self, prompt, schema, *, system=None, fallback=None, temperature=0.0, fast=True
131 + ):
132         self.requests.append(schema.__name__)
133         data = {**(fallback or {}), "source": "llm"}
134         if schema.__name__ == "SummaryReply":
135             data.update(answer="Grounded answer from the model.", defect_summary="Model defect summary.")
136         return schema.model_validate(data)
137
138     def embed(self, texts):
139         self.requests.append("embed")
140         return super().embed(texts)
141
142 def test_an_analysis_makes_exactly_three_llm_requests_one_per_agent(rag, monkeypatch):
143     """Guardrail: the gateway quota pays for one request per agent and nothing else."""
144     counting = CountingProvider()
145     monkeypatch.setattr(provider_module, "build_provider", lambda name=None: counting)
146     get_llm.cache_clear()
147     try:
148         with session_scope() as db:
149             ecr = ECRRepository(db).get("ECR-2026-001").as_dict()

```

```

150     _, state = run_workflow(ecr, {})
151 finally:
152     get_llm.cache_clear()
153
154     assert sorted(counting.requests) == ["CorrelationReview", "RetrievalPlan", "SummaryReply"]
155     assert state["retrieval_plan"]["source"] == "llm"
156     assert state["correlation_review"]["summary"]
157     assert state["final_answer"] == "Grounded answer from the model."
158     assert state["defect_summary"] == "Model defect summary."
159     report = state["final_report"]
160     assert report["retrieval_plan"]["source"] == "llm"
161     assert "correlation_findings" in {section["key"] for section in report["sections"]}
162
163
164 def test_search_embeddings_stay_local_by_default(monkeypatch):
165     monkeypatch.setattr(openai_provider.settings, "openai_api_key", "test-key")
166     assert openai_provider.OpenAIProvider().supports_remote_embeddings is False

```

backend/app/tests/test_agents/test_export_ecrs.py

```

1  """ECRs imported from the change-tracking export (ECR-1, ECR-2) keep every exported value."""
2  from __future__ import annotations
3
4  from app.agents.orchestrator import run_workflow
5  from app.database import session_scope
6  from app.repositories import ECRRepository
7  from app.services.chat_service import detect_intent
8
9  ECR_1_TESTS = {"L2R26:TC1", "L2R26:TC2", "L2R26:TC3", "L2R35:TC1", "L2R35:TC2"}
10
11
12 def test_ecr_2_returns_the_exported_values(client):
13     ecr = client.get("/api/ecr/ECR-2").json()
14     assert ecr["record_type"] == "ECR"
15     assert ecr["title"] == (
16         "Port Build 7 ECR -2 to Build 8: Flrt3: Horn state doesn't change on brake/horn interface screen"
17     )
18     assert ecr["lifecycle_status"] == "System Verified"
19     assert ecr["severity"] == "Minor"
20     assert ecr["assigned_testers"] == ["Anirudh"]
21     assert ecr["actual_modified_objects"] == ["Brake Interface"]
22     assert ecr["planned_modified_objects"] == ["Brake Interface"]
23     assert ecr["build_resolved_in"] == "6.5.7.0.ENG1"
24     assert (ecr["estimate"], ecr["test_estimate"], ecr["test_actual_time"]) == ("8 hours", "4", "4 hours")
25     assert (ecr["creation_date"], ecr["modified_date"]) == ("Apr 22, 2026, 6:44 PM", "Jun 10, 2026, 6:54 AM")
26
27     assert ecr["affected_test_cases"] == [
28         "L2R01:TC1", "L2R01:TC2", "L2R01:TC3", "L2R05:TC1", "L2R05:TC2", "L2R05:TC3", "L2R05:TC4",
29     ]
30     assert ecr["description"].startswith('When the Horn key is pressed on the "brake/horn interface" screen')
31
32     assert ecr["steps_to_reproduce"].splitlines()[0] == "1. Set control type SR-FLRT3-DMU and bring OB to c
33     utout"
34     assert len(ecr["steps_to_reproduce"].splitlines()) == 6
35     assert ecr["observed_behavior"] == (
36         "On pressing the Horn key, OB keeps showing the Horn state field status as Inactive"
37     )
38     assert ecr["expected_behavior"] == "OB should update the horn state field as Active"
39     assert ecr["attachments"] == ["Logs.zip", "dev_logs.log", "test artifacts.zip"]
40
41
42 def test_ecr_1_returns_the_exported_values(client):
43     ecr = client.get("/api/ecr/ecr-1").json() # ids are case-insensitive
44     assert ecr["title"] == "Port Build 7 ECR -1 to Build 8: Depart test not performed well"
45     assert (ecr["lifecycle_status"], ecr["severity"]) == ("System Verified", "Medium")
46     assert ecr["assigned_testers"] == ["Deepak Yadav"]
47     assert ecr["actual_modified_objects"] == ["Depart Test"]
48     assert (ecr["estimate"], ecr["test_estimate"], ecr["test_actual_time"]) == ("16 hours", "4", "4 hours")
49     assert (ecr["creation_date"], ecr["modified_date"]) == ("May 6, 2026, 2:42 PM", "Jun 9, 2026, 3:27 PM")
50     assert set(ecr["affected_test_cases"]) == ECR_1_TESTS
51     assert '-send 1010 "Command locomotive system state to Non-controlling"' in ecr["description"]
52     assert ecr["attachments"] == ["test.log", "Fixed_ecr.rtf"]
53     assert ecr["steps_to_reproduce"] == ""
54
55
56 def test_analysis_recommends_every_listed_test_first(rag):
57     with session_scope() as db:
58         ecr = ECRRepository(db).get("ECR-1").as_dict()
59         _, state = run_workflow(ecr, {})
60         selected = state["test_selection"]["selected_tests"]
61         assert ECR_1_TESTS <= {t["test_case_id"] for t in selected}
62         assert all(t["domain"] == "rail" for t in selected)
63         ordered = [t["test_case_id"] for t in state["test_prioritization"]["prioritized_tests"]]
64         assert set(ordered[: len(ECR_1_TESTS)]) == ECR_1_TESTS

```

```

62     listed = [t for t in selected if t["test_case_id"] in ECR_1_TESTS]
63     assert all([t["reason"].startswith("Listed as an affected test case on the ECR") for t in listed])
64     assert {"L2R26", "L2R35"} <= {r["requirement_id"] for r in state["requirements"]}
65
66
67 def test_analysing_keeps_the_exported_status(client):
68     client.post("/api/ecr/ECR-2/analyze?wait=true", json={})
69     ecr = client.get("/api/ecr/ECR-2").json()
70     assert ecr["lifecycle_status"] == "System Verified" # the export's status is untouched
71     assert ecr["status"] == "ANALYSED" # the app's own analysis state
72
73
74 def test_chat_understands_short_ecr_ids_and_export_test_ids():
75     detection = detect_intent("why was L2R26: TC2 selected for ECR-1?")
76     assert detection["ecr_id"] == "ECR-1"
77     assert "L2R26:TC2" in detection["artefacts"]
78     assert detect_intent("analyse ECR-2026-001")["ecr_id"] == "ECR-2026-001"
79     assert detect_intent("what about ecr 2026 7")["ecr_id"] == "ECR-2026-7"
80
81
82 def test_every_ecr_shows_only_its_own_data(rag):
83     """ECR-1 gets its own requirements, tests, components, defects and evidence - nothing wider."""
84     with session_scope() as db:
85         ecr = ECRRepository(db).get("ECR-1").as_dict()
86         _, state = run_workflow(ecr, {})
87         assert {r["requirement_id"] for r in state["requirements"]} == {"L2R26", "L2R35"}
88         assert {t["test_case_id"] for t in state["test_selection"]["selected_tests"]} == ECR_1_TESTS
89         assert {e["title"] for e in state["evidence"]} == {"test.log", "Fixed_ecr.rtf"}
90         assert all(e["ecr_id"] == "ECR-1" for e in state["evidence"])
91         assert state["comments"] == []
92         assert state["defects"] == [] # no defect in the data is linked to L2R26 or L2R35
93         impact = state["impact_analysis"]
94         assert [c["component_id"] for c in impact["directly_impacted_components"]] == ["RCMP-002"]
95         assert {c["component_id"] for c in impact["indirectly_impacted_components"]} == {
96             "RCMP-001", "RCMP-003", "RCMP-005", "RCMP-009", # the other components L2R26/L2R35 name
97         }
98         assert state["defect_insights"]["component_history"] == []
99
100
101 def test_sample_ecrs_are_scoped_to_their_linked_requirements(rag):
102     with session_scope() as db:
103         ecr = ECRRepository(db).get("ECR-2026-002").as_dict()
104         _, state = run_workflow(ecr, {})
105         assert {r["requirement_id"] for r in state["requirements"]} == set(ecr["linked_requirements"])
106         selected = {t["test_case_id"] for t in state["test_selection"]["selected_tests"]}
107         assert selected == set(ecr["affected_test_cases"])
108         assert all(t["requirement_id"] in ecr["linked_requirements"] for t in state["test_selection"]["selected
+ _tests"])
109         assert all(c["ecr_id"] == "ECR-2026-002" for c in state["comments"])

```

backend/app/tests/test_agents/test_three_agents.py

```

1  """Guardrail: the platform has exactly three agents, and nothing else poses as one."""
2  from __future__ import annotations
3
4  import pkgutil
5
6  from app.agents import nodes
7  from app.agents.nodes.agents import AGENTS
8  from app.agents.orchestrator import get_compiled_graph
9  from app.agents.state import AGENT_CATALOG, AGENT_STEPS
10
11 AGENT_NAMES = {"Retrieval Agent", "Correlation Agent", "Summarization Agent"}
12
13
14 def test_catalogue_lists_exactly_three_agents():
15     assert [a.name for a in AGENT_CATALOG] == [
16         "Retrieval Agent",
17         "Correlation Agent",
18         "Summarization Agent",
19     ]
20     assert set(AGENT_STEPS) == {a.key for a in AGENT_CATALOG}
21
22
23 def test_graph_has_three_agent_nodes_plus_one_checkpoint():
24     graph_nodes = set(get_compiled_graph().get_graph().nodes) - {"__start__", "__end__"}
25     assert graph_nodes == {a.key for a in AGENT_CATALOG} | {"human_approval"}
26     assert len(AGENTS) == 3
27
28
29 def test_every_step_is_owned_by_one_of_the_three_agents():
30     import importlib
31     import inspect
32

```

```

33     for module_info in pkgutil.iter_modules(nodes.__path__):
34         module = importlib.import_module(f"app.agents.nodes.{module_info.name}")
35         for _, fn in inspect.getmembers(module, inspect.isfunction):
36             if hasattr(fn, "step_name"):
37                 assert fn.step_name in AGENT_NAMES, f"{fn.step_key} is labelled {fn.step_name!r}"
38                 assert not fn.step_key.endswith("_agent")
39
40
41 def test_no_step_module_is_named_as_an_agent():
42     names = {m.name for m in pkgutil.iter_modules(nodes.__path__)}
43     assert not [n for n in names if n.endswith("_agent") or n.endswith("_agents")]
44
45
46 def test_the_three_agents_run_five_steps():
47     assert [step["key"] for steps in AGENT_STEPS.values() for step in steps] == [
48         "understand_ecr",
49         "gather_evidence",
50         "assess_impact",
51         "select_tests",
52         "summarize",
53     ]

```

backend/app/tests/test_agents/test_workflow.py

```

1  """Agent nodes and the orchestrated workflow (offline / mock provider)."""
2  from __future__ import annotations
3
4  import pytest
5
6  from app.agents.nodes.ecr_understanding import classify_change, extract_entities
7  from app.agents.nodes.planning import build_plan
8  from app.agents.orchestrator import needs_approval, run_workflow
9  from app.database import session_scope
10 from app.repositories import ECRRepository
11 from app.schemas.common import ChangeType
12
13
14 @pytest.fixture(scope="module")
15 def demo_state(rag):
16     with session_scope() as db:
17         ecr = ECRRepository(db).get("ECR-2026-001").as_dict()
18         _, state = run_workflow(ecr, {"human_in_the_loop": False})
19     return state
20
21
22 def test_classifier_separates_ui_from_database_changes():
23     ui = classify_change(
24         "Modify the label of the save button on the user profile page", ["portal/profile_page.py"]
25     )
26     database = classify_change(
27         "Add a column to the transaction table and backfill historical rows",
28         ["schema/transaction_schema.py"],
29     )
30     assert max(ui, key=ui.get) == ChangeType.UI.value
31     assert max(database, key=database.get) == ChangeType.DATABASE.value
32
33
34 def test_entity_extraction_finds_features_and_components(database):
35     entities = extract_entities(
36         "Modify payment validation logic to support multi currency transactions",
37         ["payment/validator.py"],
38     )
39     assert entities["business_domain"] == "Payments"
40     assert entities["affected_features"]
41     assert "CMP-004" in entities["candidate_components"]
42     assert "financial_transaction" in entities["risk_indicators"]
43
44
45 def test_planner_skips_expensive_agents_for_a_ui_tweak():
46     plan = build_plan(
47         {"change_type": ChangeType.UI.value, "risk_indicators": [], "change_complexity": 0.1}, {}
48     )
49     assert "dependencies" in plan["skipped"]
50     assert "requirements" in plan["investigations"]
51     assert "historical_defects" in plan["investigations"] # defects are always analysed
52     assert plan["rationale"]
53
54
55 def test_planner_runs_everything_for_a_schema_change():
56     plan = build_plan(
57         {
58             "change_type": ChangeType.DATABASE.value,
59             "risk_indicators": ["schema_migration"],
60             "change_complexity": 0.8,

```

```

61         },
62         {},
63     )
64     assert "dependencies" in plan["investigations"]
65     assert "historical_defects" in plan["investigations"]
66     assert not plan["skipped"]
67
68
69 def test_force_full_analysis_overrides_skipping():
70     plan = build_plan(
71         {"change_type": ChangeType.UI.value, "risk_indicators": [], "change_complexity": 0.1},
72         {"force_full_analysis": True},
73     )
74     assert not plan["skipped"]
75
76
77 def test_workflow_produces_every_stage(demo_state):
78     for key in (
79         "ecr_analysis",
80         "requirements",
81         "defects",
82         "comments",
83         "evidence",
84         "code_impact",
85         "dependency_analysis",
86         "impact_analysis",
87         "test_selection",
88         "test_prioritization",
89         "correlation",
90         "final_answer",
91         "final_report",
92     ):
93         assert demo_state.get(key), f"missing {key}"
94
95
96 def test_workflow_identifies_the_expected_payment_impact(demo_state):
97     impact = demo_state["impact_analysis"]
98     direct = {c["component_id"] for c in impact["directly_impacted_components"]}
99     indirect = {c["component_id"] for c in impact["indirectly_impacted_components"]}
100     assert "CMP-004" in direct # payment validator
101     assert "CMP-005" in direct # currency service
102     assert "CMP-007" in indirect # refund service is downstream
103     assert "risk_score" not in impact
104
105
106 def test_workflow_selects_a_smaller_suite_than_the_baseline(demo_state):
107     selection = demo_state["test_selection"]
108     assert 0 < len(selection["selected_tests"]) < selection["total_available"]
109     assert selection["reduction_percentage"] > 0
110     assert selection["estimated_duration_minutes"] < selection["baseline_duration_minutes"]
111
112
113 def test_every_selected_test_is_explained(demo_state):
114     for test in demo_state["test_selection"]["selected_tests"]:
115         assert test["reason"]
116         assert test["priority"] in ("P0", "P1", "P2", "P3")
117
118
119 def test_correlation_links_artefacts_to_the_ecr(demo_state):
120     graph = demo_state["correlation"]["graph"]
121     types = {node["type"] for node in graph["nodes"]}
122     assert {"REQUIREMENT", "TEST_CASE", "DEFECT", "COMMENT", "EVIDENCE"} <= types
123     assert all(edge["reason"] is not None for edge in graph["edges"])
124
125
126 def test_defect_analysis_explains_every_related_defect(demo_state):
127     insights = demo_state["defect_insights"]
128     assert insights["total"] == len(demo_state["defects"]) > 0
129     selected = {t["test_case_id"] for t in demo_state["test_selection"]["selected_tests"]}
130     for row in insights["defects"]:
131         assert row["chance_of_recurrence"] in ("High", "Medium", "Low")
132         assert row["why_it_matters"] and row["what_to_check"]
133         assert set(row["covering_tests"]) <= selected
134         assert row["covered"] == bool(row["covering_tests"])
135     assert insights["high"] >= 1 # the payment change touches components with critical history
136     # Scoped to the ECR's own data: no component-wide defect history is added.
137     assert insights["scoped"] is True
138     assert insights["component_history"] == []
139     assert demo_state["defect_summary"]
140     section = next(s for s in demo_state["final_report"]["sections"] if s["key"] == "historical_defects")
141     assert section["body"] == demo_state["defect_summary"]
142
143
144 def test_a_saved_report_rebuilds_the_page_after_a_restart(demo_state):

```

```

145     from app.api.routes.analysis import page_fields_from_report
146
147     fields = page_fields_from_report(demo_state["final_report"])
148     assert (
149         fields["impact_analysis"]["directly_impacted_components"]
150         == demo_state["impact_analysis"]["directly_impacted_components"]
151     )
152     assert "component_history" in fields["defect_insights"]
153     assert fields["defect_summary"] == demo_state["defect_summary"]
154
155
156 def test_defect_summary_uses_component_history_when_nothing_matches():
157     from app.agents.nodes.summarization import _baseline_defect_summary
158
159     text = _baseline_defect_summary(
160         "ECR-X",
161         {
162             "defects": [],
163             "component_history": [
164                 {"component": "CMP-1", "component_name": "Search Service", "total": 2, "last_seen_days": 40
165 + ,
166 +         "top_cause": "Stale cache", "impact": "Direct", "severe": 1, "open": 0, "reached_productio
167 n": 1}
168         ],
169         "open_defects": [{"defect_id": "BUG-9", "title": "Index lag", "severity": "HIGH"}],
170     )
171     assert "No past defect matches ECR-X directly" in text
172     assert "Search Service" in text and "40 days ago" in text and "BUG-9" in text
173
174 def test_answer_cites_real_artefacts(demo_state):
175     citations = demo_state["answer_citations"]
176     assert citations
177     known = {r["requirement_id"] for r in demo_state["requirements"]} | {
178         d["defect_id"] for d in demo_state["defects"]
179     } | {t["test_case_id"] for t in demo_state["test_selection"]["selected_tests"]} | {
180         c["comment_id"] for c in demo_state["comments"]
181     } | {e["evidence_id"] for e in demo_state["evidence"]}
182     assert set(citations) <= known
183
184
185 def test_report_sees_every_agents_evidence(demo_state):
186     """Regression: an agent's partial state must not shadow earlier agents.
187
188     The report is assembled inside the last agent, so if the per-step view
189     replaced (rather than merged) the incoming trace and confidence, the report
190     would only ever contain the summarization agent's own contribution.
191     """
192     report = demo_state["final_report"]
193     # The report cannot contain the two entries written after it is assembled:
194     # its own step, and the summarization agent's node-level entry.
195     assert len(report["execution_trace"]) == len(demo_state["execution_trace"]) - 2
196     per_step = report["confidence"]["per_step"]
197     assert {"understand_ecr", "gather_evidence", "assess_impact", "select_tests"} <= set(per_step)
198     assert len(per_step) >= 5
199
200
201 def test_execution_trace_records_every_step(demo_state):
202     trace = demo_state["execution_trace"]
203     assert len(trace) == 8 # five steps + three agents
204     assert all(entry["agent"] and entry["action"] for entry in trace)
205     assert all(entry["status"] in ("completed", "failed") for entry in trace)
206
207
208 def test_ui_only_change_skips_dependency_analysis(rag):
209     with session_scope() as db:
210         ecr = ECRRepository(db).get("ECR-2026-002").as_dict()
211         _, state = run_workflow(ecr, {})
212         assert "dependencies" in state.get("skipped_steps", [])
213         assert "historical_defects" not in state.get("skipped_steps", [])
214         assert "defect_insights" in state and state["defect_summary"]
215         assert state["impact_analysis"]["directly_impacted_components"]
216
217
218 def test_rail_domain_change_stays_in_its_own_suite(rag):
219     with session_scope() as db:
220         ecr = ECRRepository(db).get("ECR-2026-012").as_dict()
221         _, state = run_workflow(ecr, {})
222         selected = state["test_selection"]["selected_tests"]
223         assert selected
224         assert all(test["domain"] == "rail" for test in selected)
225
226

```

```

227 def test_approval_gate_is_only_entered_when_requested():
228     on = {"options": {"human_in_the_loop": True}, "impact_analysis": {}}
229     off = {"options": {}, "impact_analysis": {}}
230     assert needs_approval(on) == "human_approval"
231     assert needs_approval(off) == "summarization_agent"

```

backend/app/tests/test_api/_init_.py

backend/app/tests/test_api/test_endpoints.py

```

1  """API contract tests (offline provider, seeded database)."""
2  from __future__ import annotations
3
4  import pytest
5
6
7  def test_health_reports_subsystems(client):
8      payload = client.get("/api/health").json()
9      assert payload["status"] == "ok"
10     assert payload["database"]["ok"] is True
11     assert payload["demo_mode"] is True # tests force the mock provider
12     assert payload["code_analysis"]["modules"] > 0
13
14
15 def test_data_health_counts_the_seeded_corpus(client):
16     counts = client.get("/api/health/data").json()
17     assert counts["components"] == 55
18     assert counts["requirements"] == 64
19     assert counts["test_cases"] == 251
20     assert counts["defects"] == 68 and counts["ecrs"] == 45
21     assert counts["comments"] > 0 and counts["evidence"] > 0
22
23
24 def test_list_and_search_ecrs(client):
25     all_ecrs = client.get("/api/ecr").json()
26     assert len(all_ecrs) == 45
27     matched = client.get("/api/ecr?search=currency").json()
28     assert any(item["ecr_id"] == "ECR-2026-001" for item in matched)
29
30
31 def test_get_ecr_and_missing_ecr(client):
32     assert client.get("/api/ecr/ECR-2026-001").json()["ecr_id"] == "ECR-2026-001"
33     assert client.get("/api/ecr/ECR-9999-999").status_code == 404
34
35
36 def test_ecr_id_is_case_insensitive(client):
37     assert client.get("/api/ecr/ecr-2026-001").json()["ecr_id"] == "ECR-2026-001"
38
39
40 def test_sources_endpoint_returns_the_raw_multisource_view(client):
41     payload = client.get("/api/ecr/ECR-2026-001/sources").json()
42     assert payload["ecr"]["ecr_id"] == "ECR-2026-001"
43     assert payload["comments"] and payload["evidence"]
44
45
46 def test_create_ecr_generates_an_id_and_rejects_duplicates(client):
47     created = client.post(
48         "/api/ecr",
49         json={
50             "title": "Adjust refund rounding for zero-decimal currencies",
51             "description": "Refund amounts for JPY must not be multiplied by 100.",
52             "changed_files": ["refund/service.py"],
53         },
54     )
55     assert created.status_code == 201
56     ecr_id = created.json()["ecr_id"]
57     assert ecr_id.startswith("ECR-")
58
59     duplicate = client.post(
60         "/api/ecr", json={"ecr_id": ecr_id, "title": "Duplicate", "description": "x"}
61     )
62     assert duplicate.status_code == 409
63
64
65 def test_create_ecr_validates_input(client):
66     assert client.post("/api/ecr", json={"title": "x"}).status_code == 422
67
68
69 @pytest.fixture(scope="module")
70 def analysed(client):
71     response = client.post("/api/ecr/ECR-2026-001/analyze?wait=true", json={})
72     assert response.status_code == 202
73     return response.json()

```

```

74
75
76 def test_analysis_returns_every_section(client, analysed):
77     payload = client.get("/api/ecr/ECR-2026-001/analysis").json()
78     assert payload["status"] == "COMPLETED"
79     assert payload["answer"]
80     assert payload["impact_analysis"]["directly_impacted_components"]
81     assert payload["test_selection"]["selected_tests"]
82     assert payload["correlation"]["graph"]["nodes"]
83
84
85 def test_impact_endpoint(client, analysed):
86     payload = client.get("/api/ecr/ECR-2026-001/impact").json()
87     assert "risk_level" not in payload["impact"]
88     assert payload["dependency"]["blast_radius"] >= 0
89
90
91 def test_tests_endpoint_filters_by_priority(client, analysed):
92     everything = client.get("/api/ecr/ECR-2026-001/tests").json()
93     p0 = client.get("/api/ecr/ECR-2026-001/tests?priority=P0").json()
94     assert everything["selected"] > 0
95     assert all(test["priority"] == "P0" for test in p0["tests"])
96     assert len(p0["tests"]) <= everything["selected"]
97
98
99 def test_workflow_endpoint_exposes_the_trace(client, analysed):
100     payload = client.get("/api/ecr/ECR-2026-001/workflow").json()
101     assert payload["status"] == "COMPLETED"
102     assert payload["events"]
103     assert payload["topology"]["nodes"]
104     assert len(payload["topology"]["nodes"]) == 3 # exactly three agents
105     assert [c["key"] for c in payload["topology"]["checkpoints"]] == ["human_approval"]
106
107
108 @pytest.mark.parametrize("fmt", ["json", "markdown", "html"])
109 def test_report_renders_in_every_format(client, analysed, fmt):
110     response = client.get(f"/api/ecr/ECR-2026-001/report?format={fmt}")
111     assert response.status_code == 200
112     assert len(response.text) > 500
113
114
115 def test_report_rejects_an_unknown_format(client, analysed):
116     assert client.get("/api/ecr/ECR-2026-001/report?format=pdf").status_code == 422
117
118
119 def test_dashboard_stats(client, analysed):
120     payload = client.get("/api/dashboard/stats").json()
121     assert payload["total_ecrs"] >= 13
122     assert payload["analysed_ecrs"] >= 1
123     assert "risk_distribution" not in payload
124
125
126 def test_agents_catalogue_lists_three_agents(client):
127     payload = client.get("/api/agents").json()
128     keys = [agent["key"] for agent in payload["agents"]]
129     assert keys == ["retrieval_agent", "correlation_agent", "summarization_agent"]
130     assert all(agent["steps"] for agent in payload["agents"][:3])
131
132
133 def test_tool_registry_is_populated(client):
134     tools = client.get("/api/agents/tools").json()["tools"]
135     names = {tool["name"] for tool in tools}
136     assert {"semantic_requirement_search", "analyze_changed_files", "comments_for_ecr"} <= names
137
138
139 def test_agent_runs_are_persisted(client, analysed):
140     runs = client.get(f"/api/agents/runs?workflow_id={analysed['workflow_id']}").json()
141     assert runs
142     assert all(run["agent"] for run in runs)
143
144
145 def test_component_graph(client):
146     payload = client.get("/api/components/graph").json()
147     assert len(payload["nodes"]) == 55
148     assert payload["stats"]["edges"] == 72
149
150
151 def test_chat_query_resolves_the_ecr_and_answers(client, analysed):
152     payload = client.post(
153         "/api/chat/query",
154         json={"query": "Analyze ECR-2026-001 and tell me which regression tests to run"},
155     ).json()
156     assert payload["ecr_id"] == "ECR-2026-001"
157     assert payload["intent"] == "TEST_RECOMMENDATION"

```

```

158     assert payload["answer"]
159
160
161 def test_chat_query_without_an_ecr_asks_for_one(client):
162     payload = client.post("/api/chat/query", json={"query": "hello there"}).json()
163     assert payload["ecr_id"] is None
164     assert "ECR" in payload["answer"]
165
166
167 def test_ask_explains_a_specific_test(client, analysed):
168     tests = client.get("/api/ecr/ECR-2026-001/tests?priority=P0").json()["tests"]
169     test_id = tests[0]["test_case_id"]
170     payload = client.post(
171         f"/api/ecr/ECR-2026-001/ask", json={"question": f"Why was {test_id} selected?"}
172     ).json()
173     assert test_id in payload["focus"]
174     assert payload["answer"]
175
176
177 def test_feedback_round_trip(client, analysed):
178     created = client.post(
179         "/api/feedback",
180         json={"ecr_id": "ECR-2026-001", "target_id": "TC-1020", "useful": True},
181     ).json()
182     assert created["useful"] is True
183     assert any(item["target_id"] == "TC-1020" for item in client.get("/api/feedback").json())
184
185
186 def test_approve_rejects_when_not_awaiting(client, analysed):
187     response = client.post("/api/ecr/ECR-2026-001/approve", json={"approved": True})
188     assert response.status_code == 409

```

backend/app/tests/test_services/__init__.py

backend/app/tests/test_services/test_confidence.py

```

1  """Overall confidence of a run."""
2  from __future__ import annotations
3
4  from app.services.confidence import compute_confidence
5
6
7  def test_confidence_penalises_failures():
8      clean, penalties = compute_confidence({"a": 0.9, "b": 0.9, "c": 0.9, "d": 0.9, "e": 0.9})
9      degraded, degraded_penalties = compute_confidence(
10         {"a": 0.9, "b": 0.9, "c": 0.9, "d": 0.9, "e": 0.9, errors=[{"agent": "Retrieval Agent"}]}
11     )
12     assert clean > degraded
13     assert not penalties
14     assert degraded_penalties
15
16
17 def test_confidence_stays_in_range():
18     low, _ = compute_confidence({}, errors=[{"agent": "x"}] * 20)
19     assert 0.05 <= low <= 1.0

```

backend/app/tests/test_services/test_csv_store.py

```

1  """CSV files are the system of record: load, validate and write back."""
2  from __future__ import annotations
3
4  import csv
5  from pathlib import Path
6
7  import pytest
8  from sqlalchemy import create_engine
9  from sqlalchemy.orm import sessionmaker
10 from sqlalchemy.pool import StaticPool
11
12 from app.config import settings
13 from app.data import csv_store
14 from app.database import Base
15 from app.models import ECR
16
17
18 def _ids(file_name: str, column: str) -> list[str]:
19     with open(Path(settings.data_dir) / file_name, newline="", encoding="utf-8") as handle:
20         return [row[column] for row in csv.DictReader(handle)]
21
22
23 def test_every_table_has_a_csv_file(database):
24     assert all(csv_store.files_status()["files"].values())
25

```

```

26
27 def test_the_csv_files_hold_old_and_new_sample_data(database):
28     ecrs = _ids("ecrs.csv", "ecr_id")
29     assert "ECR-2026-001" in ecrs and "ECR-2026-043" in ecrs
30     assert len(_ids("requirements.csv", "requirement_id")) == 64
31
32
33 def test_a_change_is_written_back_to_its_csv_file(db_session):
34     db_session.add(ECR(ecr_id="ECR-2099-001", title="Write-back check", description="Created by a test.))
35     db_session.commit()
36     try:
37         csv_store.writer.flush()
38         assert "ECR-2099-001" in _ids("ecrs.csv", "ecr_id")
39     finally:
40         db_session.delete(db_session.query(ECR).filter_by(ecr_id="ECR-2099-001").one())
41         db_session.commit()
42         csv_store.writer.flush()
43     assert "ECR-2099-001" not in _ids("ecrs.csv", "ecr_id")
44
45
46 def test_a_bad_value_names_the_file_line_and_column(tmp_path):
47     (tmp_path / "components.csv").write_text(
48         "component_id,name,type,criticality_score\nCMP-900,Broken,SERVICE,not-a-number\n",
49         encoding="utf-8",
50     )
51     engine = create_engine("sqlite://", poolclass=StaticPool, connect_args={"check_same_thread": False})
52     Base.metadata.create_all(engine)
53     with pytest.raises(csv_store.CsvDataError, match=r"components\.csv line 2, column 'criticality_score'")
54 + :
55         csv_store.load_all(sessionmaker(bind=engine)(), tmp_path)

```

backend/app/tests/test_services/test_graph_and_code.py

```

1  """Dependency graph traversal, code analysis and retrieval."""
2  from __future__ import annotations
3
4  from app.graph.dependency_graph import DependencyGraphService, build_dependency_graph
5  from app.services.code_analysis_service import CodeAnalysisService
6  from app.vectorstore import COLLECTION_DEFECTS, COLLECTION_REQUIREMENTS
7
8
9  def test_graph_loads_both_domains(db_session):
10     graph = build_dependency_graph(db_session)
11     stats = graph.stats()
12     assert stats["nodes"] == 55 # 35 commerce + 20 rail
13     assert stats["edges"] == 72
14
15
16 def test_downstream_finds_the_blast_radius(db_session):
17     graph = build_dependency_graph(db_session)
18     downstream = graph.downstream(["CMP-004"], max_depth=3) # payment validator
19     # payment and refund depend on the shared validator; order/gateway follow
20     assert "CMP-003" in downstream and downstream["CMP-003"] == 1
21     assert "CMP-007" in downstream and downstream["CMP-007"] == 1
22     assert "CMP-006" in downstream
23     assert "CMP-002" in downstream
24
25
26 def test_upstream_finds_what_the_change_relies_on(db_session):
27     graph = build_dependency_graph(db_session)
28     upstream = graph.upstream(["CMP-003"], max_depth=2)
29     assert "CMP-004" in upstream # payment -> validator
30     assert "CMP-005" in upstream # payment -> currency
31     assert "CMP-008" in upstream # payment -> transaction db
32
33
34 def test_propagation_decays_with_distance(db_session):
35     graph = build_dependency_graph(db_session)
36     scores = graph.propagation_scores(["CMP-004"], max_depth=3)
37     assert scores["CMP-003"] > scores["CMP-002"]
38     assert all(0.0 < value <= 1.0 for value in scores.values())
39
40
41 def test_edge_propagation_ranks_coupling_types():
42     library = DependencyGraphService.edge_propagation("LIBRARY", "CRITICAL")
43     event = DependencyGraphService.edge_propagation("ASYNC_EVENT", "CRITICAL")
44     assert library > event
45
46
47 def test_critical_paths_reach_the_change(db_session):
48     graph = build_dependency_graph(db_session)
49     paths = graph.critical_paths(["CMP-004"], max_paths=3)
50     assert paths
51     assert all(path[-1] == "CMP-004" for path in paths)

```

```

52
53
54 def test_python_ast_analyzer_resolves_direct_and_indirect_impact():
55     service = CodeAnalysisService()
56     result = service.analyze_change(["payment/validator.py", "payment/currency.py"])
57     assert set(result["resolved_modules"]) == {"payment.validator", "payment.currency"}
58     assert "CMP-004" in result["direct_components"]
59     # refund and payment services import the shared validator
60     assert "refund.service" in result["dependent_modules"]
61     assert "CMP-007" in result["indirect_components"]
62     assert 0.0 < result["change_surface"] <= 1.0
63
64
65 def test_code_analyzer_uses_keywords_when_no_files_are_declared():
66     service = CodeAnalysisService()
67     result = service.analyze_change([], keywords=["currency"])
68     assert result["resolved_modules"] # keyword discovery kicked in
69
70
71 def test_unknown_declared_files_do_not_pull_in_unrelated_modules():
72     """Regression: rail onboard files made keyword search report payment modules as changed."""
73     service = CodeAnalysisService()
74     result = service.analyze_change(["onboard/consist.py"], keywords=["validate", "currency"])
75     assert result["unresolved_files"] == ["onboard/consist.py"]
76     assert not result["resolved_modules"]
77     assert not result["direct_components"]
78
79
80 def test_component_mapping_prefers_the_most_specific_path():
81     service = CodeAnalysisService()
82     assert service.component_for_path("payment/validator.py") == "CMP-004"
83     assert service.component_for_path("payment/service.py") == "CMP-003"
84     assert service.component_for_path("schema/transaction_schema.py") == "CMP-008"
85
86
87 def test_rag_retrieves_the_expected_requirement(rag):
88     hits = rag.retrieve(
89         COLLECTION_REQUIREMENTS,
90         "multi currency payment validation for unsupported currency codes",
91         k=5,
92     )
93     ids = [hit.id for hit in hits]
94     assert "REQ-1003" in ids or "REQ-1004" in ids
95     assert hits[0].relevance > 0
96
97
98 def test_rag_separates_unrelated_domains(rag):
99     hits = rag.retrieve(COLLECTION_DEFECTS, "depart test key availability onboard state", k=5)
100     ids = [hit.id for hit in hits]
101     assert any(defect.startswith("PTC-") for defect in ids)

```

backend/app/tests/test_services/test_integrations.py

```

1  """Source-system providers: local ones work, enterprise ones fail loudly."""
2  from __future__ import annotations
3
4  import pytest
5
6  from app.integrations.providers import (
7      AzureDevOpsRequirementProvider,
8      GitHubRepositoryProvider,
9      LocalDefectProvider,
10     LocalRequirementProvider,
11     LocalTestProvider,
12     NotConfigured,
13     source_systems,
14 )
15
16
17 def test_local_requirement_provider_reads_the_corpus(database):
18     rows = LocalRequirementProvider().fetch()
19     assert len(rows) == 64
20     subset = LocalRequirementProvider().fetch(ids=["REQ-1003", "L2R367"])
21     assert {row["requirement_id"] for row in subset} == {"REQ-1003", "L2R367"}
22
23
24 def test_local_defect_provider_filters_by_component(database):
25     rows = LocalDefectProvider().fetch(components=["CMP-004"])
26     assert rows
27     assert all(
28         row["affected_component"] == "CMP-004" or "CMP-004" in row["related_components"]
29         for row in rows
30     )
31

```

```

32
33 def test_local_test_provider_filters_by_requirement(database):
34     rows = LocalTestProvider().fetch(requirement_ids=["L2R367"])
35     assert rows
36     assert all(row["requirement_id"] == "L2R367" for row in rows)
37
38
39 def test_enterprise_adapters_fail_loudly_until_configured():
40     with pytest.raises(NotConfigured):
41         AzureDevOpsRequirementProvider().fetch()
42     with pytest.raises(NotConfigured):
43         GitHubRepositoryProvider().changed_files("PR-1")
44
45
46 def test_source_system_inventory_marks_what_is_connected(database):
47     systems = source_systems()
48     # several local providers share the "local" key, one per artefact kind
49     local = [entry for entry in systems if entry["key"] == "local"]
50     assert len(local) == 4 and all(entry["connected"] for entry in local)
51     remote = [entry for entry in systems if entry["key"] != "local"]
52     assert remote and not any(entry["connected"] for entry in remote)
53     assert all(entry["kind"] and entry["name"] for entry in systems)

```

backend/app/tests/test_services/test_llm_rate_limit.py

```

1  """A throttled gateway (HTTP 429) must be retried, not treated as a dead endpoint."""
2  from __future__ import annotations
3
4  import httpx
5  import pytest
6
7  from app.llm import openai_provider
8  from app.llm.base import LLMError
9  from app.llm.openai_provider import OpenAIProvider
10 from app.llm.provider import ResilientProvider
11
12
13 def _response(status: int, body: dict | None = None, headers: dict | None = None) -> httpx.Response:
14     request = httpx.Request("POST", "https://gateway.test/v1/chat/completions")
15     return httpx.Response(status, json=body or {}, headers=headers or {}, request=request)
16
17
18 OK = {"choices": [{"message": {"content": "READY"}}]}
19
20
21 @pytest.fixture
22 def provider(monkeypatch):
23     monkeypatch.setattr(openai_provider.time, "sleep", lambda _seconds: None)
24     monkeypatch.setattr(openai_provider.settings, "openai_api_key", "test-key")
25     monkeypatch.setattr(openai_provider.settings, "openai_base_url", "https://gateway.test/v1")
26     return OpenAIProvider()
27
28
29 def test_429_is_retried_until_the_gateway_answers(provider, monkeypatch):
30     replies = [_response(429, headers={"retry-after": "2"}), _response(429), _response(200, OK)]
31     monkeypatch.setattr(openai_provider.httpx, "post", lambda *a, **k: replies.pop(0))
32     assert provider.generate("ping") == "READY"
33     assert not replies
34
35
36 def test_client_errors_other_than_throttling_are_not_retried(provider, monkeypatch):
37     calls = []
38
39     def post(*args, **kwargs):
40         calls.append(1)
41         return _response(400)
42
43     monkeypatch.setattr(openai_provider.httpx, "post", post)
44     with pytest.raises(LLMError) as raised:
45         provider.generate("ping")
46     assert raised.value.status_code == 400
47     assert len(calls) == 1
48
49
50 def test_rate_limited_probe_keeps_the_real_model_active(provider, monkeypatch):
51     monkeypatch.setattr(openai_provider.httpx, "post", lambda *a, **k: _response(429))
52     resilient = ResilientProvider(provider)
53     result = resilient.probe()
54     assert result["rate_limited"] is True
55     assert not resilient.is_open
56     assert resilient.active is provider
57
58
59 def test_a_refused_call_returns_the_callers_fallback_never_the_prompt(provider, monkeypatch):

```

```

60     """Regression: a throttled answer call used to show the raw prompt on the page."""
61     monkeypatch.setattr(openai_provider.httpx, "post", lambda *a, **k: _response(429))
62     resilient = ResilientProvider(provider)
63     assert resilient.generate("QUESTION: prompt text", fallback="baseline answer") == "baseline answer"
64     for _ in range(3):
65         resilient.generate("again", fallback="baseline answer")
66     assert resilient.is_open
67     assert resilient.generate("QUESTION: prompt text", fallback="baseline answer") == "baseline answer"
68
69
70 def test_a_timeout_is_not_retried_because_the_gateway_may_have_billed_it(provider, monkeypatch):
71     calls = []
72
73     def post(*_args, **_kwargs):
74         calls.append(1)
75         raise httpx.ReadTimeout("timed out")
76
77     monkeypatch.setattr(openai_provider.httpx, "post", post)
78     with pytest.raises(LLMError):
79         provider.generate("ping")
80     assert len(calls) == 1

```

backend/app/tests/test_services/test_prompts.py

```

1  """Every prompt lives in app/prompts.py, and every placeholder is still filled in."""
2  from __future__ import annotations
3
4  import re
5
6  import pytest
7
8  from app import prompts
9  from app.agents.nodes import correlation_review, retrieval_planning, summarization
10 from app.llm import base
11 from app.services import chat_service
12
13 # Template -> the placeholders the agent that owns it supplies. Renaming one on
14 # either side breaks this test instead of the analysis.
15 TEMPLATES = {
16     "RETRIEVAL_TASK": {
17         "ecr_id",
18         "title",
19         "description",
20         "changed_files",
21         "linked_requirements",
22         "classification",
23         "max_terms",
24         "max_behaviours",
25     },
26     "CORRELATION_TASK": {"bundle", "rule_findings", "max_findings"},
27     "SUMMARY_TASK": {"question", "bundle", "defect_instruction"},
28     "SUMMARY_DEFECT_INSTRUCTION": {"ecr_id"},
29     "NARRATION_TASK": {"task", "context", "fallback"},
30 }
31
32 SYSTEM_PROMPTS = [
33     "RETRIEVAL_SYSTEM_PROMPT",
34     "CORRELATION_SYSTEM_PROMPT",
35     "ANSWER_SYSTEM_PROMPT",
36     "NARRATION_SYSTEM_PROMPT",
37 ]
38
39
40 def _placeholders(template: str) -> set[str]:
41     """Single-brace fields only; a doubled brace is literal text, not a placeholder."""
42     return {match.group(1) for match in re.finditer(r"(?!\{)\{([a-z_]+\)}(?:\{!\})", template)}
43
44
45 @pytest.mark.parametrize("name, expected", sorted(TEMPLATES.items()))
46 def test_each_template_declares_the_placeholders_its_agent_fills(name, expected):
47     template = getattr(prompts, name)
48     assert _placeholders(template) == expected
49     filled = template.format(**{key: f"<{key}>" for key in expected})
50     assert all(f"<{key}>" in filled for key in expected)
51
52
53 @pytest.mark.parametrize("name", SYSTEM_PROMPTS)
54 def test_every_system_prompt_has_text(name):
55     assert getattr(prompts, name).strip()
56
57
58 def test_the_correlation_json_example_survives_formatting():
59     """Its braces are doubled in the file; the model must still see real JSON."""
60     filled = prompts.CORRELATION_TASK.format(bundle="{}", rule_findings="{}", max_findings=4)

```

```

61     assert '{"ids": [...], "text": "one sentence under 30 words"}' in filled
62
63
64 def test_no_agent_keeps_its_own_copy_of_a_prompt():
65     """Each caller must use the shared text, so editing prompts.py changes the run."""
66     assert retrieval_planning.RETRIEVAL_SYSTEM_PROMPT is prompts.RETRIEVAL_SYSTEM_PROMPT
67     assert correlation_review.CORRELATION_SYSTEM_PROMPT is prompts.CORRELATION_SYSTEM_PROMPT
68     assert summarization.ANSWER_SYSTEM_PROMPT is prompts.ANSWER_SYSTEM_PROMPT
69     assert summarization.SUMMARY_STYLE is prompts.SUMMARY_STYLE
70     assert chat_service.ANSWER_SYSTEM_PROMPT is prompts.ANSWER_SYSTEM_PROMPT
71     assert base.NARRATION_SYSTEM_PROMPT is prompts.NARRATION_SYSTEM_PROMPT
72
73
74 def test_the_mock_provider_markers_stay_in_the_narration_prompt():
75     """The offline provider splits on these two strings to echo the baseline text."""
76     assert "DETERMINISTIC BASELINE NARRATIVE:" in prompts.NARRATION_TASK
77     assert "\n\nRewrite" in prompts.NARRATION_TASK
78
79
80 def test_every_agent_prompt_shares_the_ground_rules_and_names_its_agent():
81     for name, agent in [
82         ("RETRIEVAL_SYSTEM_PROMPT", "Retrieval Agent"),
83         ("CORRELATION_SYSTEM_PROMPT", "Correlation Agent"),
84         ("ANSWER_SYSTEM_PROMPT", "Summarization Agent"),
85     ]:
86         text = getattr(prompts, name)
87         assert agent in text
88         assert prompts.GROUND_RULES in text and prompts.RAILWAY_CHECKS in text
89         # The two agents whose replies are parsed as JSON are told so; the answer prompt,
90         # also used for plain-text chat replies, is not.
91         assert prompts.RETRIEVAL_SYSTEM_PROMPT.endswith(prompts.JSON_ONLY)
92         assert prompts.CORRELATION_SYSTEM_PROMPT.endswith(prompts.JSON_ONLY)
93         assert "JSON" not in prompts.ANSWER_SYSTEM_PROMPT
94
95
96 def test_the_answer_reports_the_human_approval_decision():
97     from app.agents.nodes.summarization import _baseline_answer, build_answer_context
98
99     state = {
100         "ecr_id": "ECR-1",
101         "ecr_input": {"title": "Depart test"},
102         "approval": {"required": True, "status": "REJECTED", "decided_by": "qa.lead@enterprise.com"},
103     }
104     context = build_answer_context(state)
105     assert context["human_approval"]["status"] == "REJECTED"
106     assert "rejected this analysis, so the test recommendation is on hold" in _baseline_answer(context)
107     assert build_answer_context({"ecr_id": "ECR-2"})["human_approval"] == {"requested": False}

```

backend/app/tests/test_services/test_test_selection.py

```

1  """Regression selection: scoring, banding, filtering and ordering."""
2  from __future__ import annotations
3
4  from app.config import TestSelectionWeights
5  from app.schemas.common import Priority
6  from app.services.test_selection_service import (
7      SelectionContext,
8      TestSelectionEngine,
9      priority_distribution,
10 )
11
12
13 def make_test(**overrides):
14     base = {
15         "test_case_id": "TC-9001",
16         "title": "Validate payment with a supported currency",
17         "description": "",
18         "feature": "Multi Currency Processing",
19         "suite": "Payments Regression",
20         "component": "CMP-005",
21         "linked_components": ["CMP-005", "CMP-003"],
22         "requirement_id": "REQ-1004",
23         "automation_status": "AUTOMATED",
24         "average_execution_time": 60.0,
25         "historical_failure_rate": 0.2,
26         "tags": [],
27     }
28     base.update(overrides)
29     return base
30
31
32 def context(**overrides):
33     base = dict(
34         matched_requirements={"REQ-1004": 95.0},

```

```

35         direct_components={"CMP-005"},
36         indirect_components={"CMP-006"},
37         dependency_distance={"CMP-006": 1},
38         propagation={"CMP-006": 0.7},
39         component_criticality={"CMP-005": "HIGH", "CMP-006": "HIGH"},
40         defect_components={"CMP-005"},
41         semantic_scores={"TC-9001": 90.0},
42         affected_features={"Multi Currency Processing"},
43     )
44     base.update(overrides)
45     return SelectionContext(**base)
46
47
48 def test_directly_relevant_test_scores_high():
49     scored = TestSelectionEngine().score_test(make_test(), context())
50     assert scored.relevance_score >= 90
51     assert scored.priority is Priority.P0
52     assert scored.selected
53
54
55 def test_unrelated_test_is_dropped():
56     unrelated = make_test(
57         test_case_id="TC-9999",
58         requirement_id="REQ-9999",
59         component="CMP-013",
60         linked_components=["CMP-013"],
61         feature="Reporting",
62         historical_failure_rate=0.0,
63     )
64     scored = TestSelectionEngine().score_test(unrelated, context(semantic_scores={}))
65     assert scored.relevance_score < 50
66     assert not scored.selected
67     assert scored.priority is Priority.P3
68
69
70 def test_breakdown_sums_to_the_relevance_score():
71     scored = TestSelectionEngine().score_test(make_test(), context())
72     parts = sum(scored.breakdown.model_dump().values())
73     assert round(parts, 1) == round(scored.relevance_score, 1)
74
75
76 def test_every_selected_test_explains_itself():
77     scored = TestSelectionEngine().score_test(make_test(), context())
78     assert scored.reasons
79     assert "REQ-1004" in scored.reason or "relevance" in scored.reason.lower()
80
81
82 def test_selection_respects_the_threshold_and_the_cap():
83     tests = [make_test(test_case_id=f"TC-90{index:02d}") for index in range(10)]
84     engine = TestSelectionEngine(threshold=95.0)
85     selected, rejected = engine.select(tests, context(), max_tests=3)
86     assert len(selected) <= 3
87     assert all(test.relevance_score >= 95.0 for test in selected)
88     assert all(not test.selected for test in rejected)
89
90
91 def test_weights_are_configurable():
92     only_semantic = TestSelectionWeights(
93         requirement_match=0.0,
94         component_match=0.0,
95         dependency_relevance=0.0,
96         historical_failure=0.0,
97         semantic_similarity=1.0,
98     )
99     scored = TestSelectionEngine(weights=only_semantic).score_test(make_test(), context())
100     assert scored.breakdown.requirement_match == 0.0
101     assert round(scored.relevance_score) == round(scored.breakdown.semantic_similarity)
102
103
104 def test_prioritisation_orders_p0_first_then_by_runtime():
105     engine = TestSelectionEngine()
106     slow = engine.score_test(make_test(test_case_id="TC-SLOW", average_execution_time=300.0), context())
107     fast = engine.score_test(make_test(test_case_id="TC-FAST", average_execution_time=30.0), context())
108     weak = engine.score_test(
109         make_test(
110             test_case_id="TC-WEAK",
111             requirement_id="REQ-0000",
112             component="CMP-006",
113             linked_components=["CMP-006"],
114             historical_failure_rate=0.0,
115         ),
116         context(semantic_scores={}),
117     )
118     ordered = engine.prioritise([slow, weak, fast], context())

```

```

119     assert ordered[0].test_case_id == "TC-FAST" # same band, quickest feedback first
120     assert ordered[-1].test_case_id == "TC-WEAK"
121     assert [test.execution_order for test in ordered] == [1, 2, 3]
122
123
124 def test_priority_distribution_counts_every_band():
125     engine = TestSelectionEngine()
126     scored = [engine.score_test(make_test(), context())]
127     distribution = priority_distribution(scored)
128     assert set(distribution) == {"P0", "P1", "P2", "P3"}
129     assert sum(distribution.values()) == 1

```

Frontend - application

frontend/src/App.js

```

1 import React from "react";
2 import "./App.css";
3 import { BrowserRouter, Navigate, Route, Routes } from "react-router-dom";
4 import { Toaster } from "../components/ui/sonner";
5 import { AnalysisProvider } from "@context/AnalysisContext";
6 import AppShell from "@components/layout/AppShell";
7 import DashboardPage from "@pages/DashboardPage";
8 import AnalyzePage from "@pages/AnalyzePage";
9 import ReportPage from "@pages/ReportPage";
10
11 function App() {
12     return (
13         <div className="App dark">
14             <BrowserRouter>
15                 <AnalysisProvider>
16                     <AppShell>
17                         <Routes>
18                             <Route path="/" element={<DashboardPage />} />
19                             <Route path="/analyze" element={<AnalyzePage />} />
20                             <Route path="/report" element={<ReportPage />} />
21                             <Route path="*" element={<Navigate to="/" replace />} />
22                         </Routes>
23                     </AppShell>
24                     <Toaster position="top-right" richColors theme="dark" />
25                 </AnalysisProvider>
26             </BrowserRouter>
27         </div>
28     );
29 }
30
31 export default App;

```

frontend/src/index.js

```

1 import React from "react";
2 import ReactDOM from "react-dom/client";
3 import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
4 import "@index.css";
5 import App from "@App";
6
7 const queryClient = new QueryClient({
8     defaultOptions: {
9         queries: {
10             staleTime: 60_000,
11             refetchOnWindowFocus: false,
12         },
13     },
14 });
15
16 const root = ReactDOM.createRoot(document.getElementById("root"));
17 root.render(
18     <React.StrictMode>
19         <QueryClientProvider client={queryClient}>
20             <App />
21         </QueryClientProvider>
22     </React.StrictMode>,
23 );

```

frontend/src/App.css

```

1 /* App-level styling. Design tokens live in index.css; this file only holds
2    layout affordances that are awkward as utility classes. */
3 .App {
4     min-height: 100vh;
5 }
6
7 /* The report page is print-to-PDF: hide chrome and flatten surfaces. */

```

```

8  @media print {
9    aside,
10   header,
11   footer,
12   button {
13     display: none !important;
14   }
15   .surface {
16     border-color: #ccc !important;
17     box-shadow: none !important;
18     background: #fff !important;
19     color: #111 !important;
20   }
21   body {
22     background: #fff !important;
23     color: #111 !important;
24   }
25 }

```

frontend/src/index.css

```

1  @import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');
2
3  @tailwind base;
4  @tailwind components;
5  @tailwind utilities;
6
7  body {
8    margin: 0;
9    font-family:
10     "Inter", -apple-system, BlinkMacSystemFont, "Segoe UI", "Roboto", "Oxygen",
11     "Ubuntu", "Cantarell", "Fira Sans", "Droid Sans", "Helvetica Neue",
12     sans-serif;
13     -webkit-font-smoothing: antialiased;
14     -moz-osx-font-smoothing: grayscale;
15     background: #080c17;
16 }
17
18 code {
19   font-family:
20     source-code-pro, Menlo, Monaco, Consolas, "Courier New", monospace;
21 }
22
23 @layer base {
24   /*
25    * Single dark enterprise theme. The app is deliberately dark-only, so the
26    * tokens are defined once on :root and .dark alike - a light stamp must not
27    * produce a half-themed page.
28    */
29   :root,
30   .dark {
31     --background: 222 47% 6%; /* #080c17 page plane */
32     --foreground: 213 31% 95%;
33     --card: 220 44% 10%; /* #0f1626 chart/card surface */
34     --card-foreground: 213 31% 95%;
35     --popover: 220 44% 10%;
36     --popover-foreground: 213 31% 95%;
37     --primary: 213 76% 56%; /* #3987e5 */
38     --primary-foreground: 0 0% 100%;
39     --secondary: 220 33% 16%;
40     --secondary-foreground: 213 31% 95%;
41     --muted: 220 30% 15%;
42     --muted-foreground: 219 14% 60%;
43     --accent: 258 68% 62%; /* violet accent */
44     --accent-foreground: 0 0% 100%;
45     --destructive: 0 61% 53%; /* #d03b3b status critical */
46     --destructive-foreground: 0 0% 100%;
47     --border: 220 30% 18%;
48     --input: 220 30% 18%;
49     --ring: 213 76% 56%;
50     --radius: 0.65rem;
51   }
52 }
53
54 @layer base {
55   * {
56     @apply border-border;
57   }
58   body {
59     @apply bg-background text-foreground;
60   }
61 }
62
63 @layer components {

```

```

64     /* Elevated dark surface used by every panel in the app. */
65     .surface {
66         @apply rounded-xl border border-border/80 bg-card/80 backdrop-blur-sm;
67         box-shadow: 0 1px 0 0 rgba(255, 255, 255, 0.03) inset,
68             0 16px 40px -24px rgba(0, 0, 0, 0.9);
69     }
70
71     .eyebrow-label {
72         @apply text-[11px] font-semibold uppercase tracking-[0.14em] text-muted-foreground;
73     }
74
75     .stat-value {
76         @apply text-3xl font-semibold tracking-tight text-foreground;
77     }
78
79     /* Ambient AI-network glow behind the hero areas. */
80     .glow-grid {
81         background-image:
82             radial-gradient(60rem 30rem at 15% -10%, rgba(57, 135, 229, 0.16), transparent 60%),
83             radial-gradient(50rem 26rem at 85% 0%, rgba(124, 92, 232, 0.14), transparent 60%),
84             linear-gradient(rgba(255, 255, 255, 0.022) 1px, transparent 1px),
85             linear-gradient(90deg, rgba(255, 255, 255, 0.022) 1px, transparent 1px);
86         background-size: auto, auto, 44px 44px, 44px 44px;
87     }
88
89     .scroll-thin::-webkit-scrollbar {
90         width: 8px;
91         height: 8px;
92     }
93     .scroll-thin::-webkit-scrollbar-thumb {
94         background: hsl(var(--border));
95         border-radius: 999px;
96     }
97     .scroll-thin::-webkit-scrollbar-track {
98         background: transparent;
99     }
100 }
101
102 /* Pulsing dot for a running agent. */
103 @keyframes pulse-ring {
104     0% { box-shadow: 0 0 0 0 rgba(57, 135, 229, 0.55); }
105     70% { box-shadow: 0 0 0 10px rgba(57, 135, 229, 0); }
106     100% { box-shadow: 0 0 0 0 rgba(57, 135, 229, 0); }
107 }
108 .pulse-ring {
109     animation: pulse-ring 1.6s cubic-bezier(0.4, 0, 0.6, 1) infinite;
110 }
111
112 @keyframes dash-flow {
113     to { stroke-dashoffset: -24; }
114 }
115 .dash-flow {
116     stroke-dasharray: 6 6;
117     animation: dash-flow 1.1s linear infinite;
118 }

```

Frontend - pages

frontend/src/pages/AnalyzePage.jsx

```

1 import React, { useEffect, useMemo, useRef, useState } from "react";
2 import { Check, Copy, Play, Search, Send, ShieldCheck, XCircle } from "lucide-react";
3 import { toast } from "sonner";
4 import api from "@services/api";
5 import { useAnalysis } from "@context/AnalysisContext";
6 import AgentPipeline, { AGENT_NAMES, STEP_LABELS } from "@components/agents/AgentPipeline";
7 import { DataTable, Expander, KeyValues, titleCase } from "@components/common/Sections";
8 import { Button } from "@components/ui/button";
9 import { Input } from "@components/ui/input";
10 import { chanceColor, priorityColor } from "@theme/viz";
11
12 // Long lists show this many rows; the rest sit behind a "Show all" button.
13 const PREVIEW_ROWS = 5;
14
15 /** A table that shows the first few rows and hides the rest behind "Show all". */
16 function PreviewTable({ rows = [], noun = "items", limit = PREVIEW_ROWS, ...tableProps }) {
17     const [showAll, setShowAll] = useState(false);
18     const hidden = rows.length - limit;
19     return (
20         <div className="space-y-3">
21             <DataTable {...tableProps} rows={showAll ? rows : rows.slice(0, limit)} />
22             {hidden > 0 && (
23                 <Button size="sm" variant="outline" onClick={() => setShowAll((value) => !value)}>

```

```

24         {showAll ? "Show fewer" : `Show all ${rows.length} ${noun} (${hidden} more)`}
25     </Button>
26     })
27 </div>
28 );
29 }
30
31 /**
32  * ECR analysis as a readable answer: a lookup box, then a few collapsible
33  * sections of plain facts - no charts, no scores.
34  */
35 export default function AnalyzePage() {
36     const { ecrId, setEcrId, analyze, analysis, status, events, selectEcr, approve, startedAt } = useAnalysis
37     + ();
38     const [ecrs, setEcrs] = useState([]);
39     const [hitl, setHitl] = useState(false);
40     const loadedFor = useRef(null);
41
42     useEffect(() => {
43         api.listEcrs().then(setEcrs).catch(() => setEcrs([]));
44     }, []);
45
46     // Show an existing analysis as soon as a known ECR id is entered.
47     useEffect(() => {
48         if (status !== "idle" || !ecrs.some((item) => item.ecr_id === ecrId)) return;
49         if (loadedFor.current === ecrId) return;
50         loadedFor.current = ecrId;
51         selectEcr(ecrId);
52     }, [ecrId, ecrs, status, selectEcr]);
53
54     const running = status === "running" || status === "awaiting_approval";
55     const current = analysis && analysis.ecr_id === ecrId ? analysis : null;
56     const record = ecrs.find((item) => item.ecr_id === ecrId) || current?.report?.ecr_overview;
57
58     const impact = current?.impact_analysis;
59     const selection = current?.test_selection;
60     const tests = current?.test_prioritization?.prioritized_tests || selection?.selected_tests || [];
61     const requirements = current?.requirement_analysis?.matched_requirements || [];
62     const review = current?.correlation_review;
63     const findings = [...(review?.conflicts || []), ...(review?.gaps || [])];
64
65     return (
66         <div className="mx-auto max-w-5xl space-y-4">
67             <header className="pb-2">
68                 <h2 className="text-3xl font-semibold tracking-tight">AI ECR Analysis Assistant</h2>
69                 <p className="mt-1 text-sm text-muted-foreground">
70                     Trace an Engineering Change Request to its requirements, tests and past defects.
71                 </p>
72             </header>
73
74             { /* Lookup */ }
75             <div className="surface flex flex-col gap-3 p-4 sm:flex-row sm:items-end">
76                 <div className="flex-1">
77                     <label htmlFor="ecr-id" className="mb-1.5 block text-sm font-medium">
78                         ECR ID
79                     </label>
80                     <div className="relative">
81                         <Search className="pointer-events-none absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-mu
82 + ted-foreground" />
83                         <Input
84                             id="ecr-id"
85                             value={ecrId}
86                             onChange={(event) => setEcrId(event.target.value.toUpperCase())}
87                             onKeyDown={(event) => event.key === "Enter" && !running && analyze(ecrId, { human_in_the_loop
88 + : hitl })}
89                             placeholder="ECR-2026-001"
90                             className="h-10 pl-9"
91                             list="ecr-options"
92                         />
93                         <datalist id="ecr-options">
94                             {ecrs.map((item) => (
95                                 <option key={item.ecr_id} value={item.ecr_id}>
96                                     {item.title}
97                                 </option>
98                             ))}
99                         </datalist>
100                     </div>
101                 </div>
102                 <div>
103                     <label className="flex h-10 cursor-pointer items-center gap-2 text-sm text-muted-foreground">
104                         <input type="checkbox" checked={hitl} onChange={(event) => setHitl(event.target.checked)} />
105                         Require approval
106                     </label>
107                     <Button
108                         onClick={() => analyze(ecrId, { human_in_the_loop: hitl })}

```

```

105         disabled={running || !ecrId}
106         className="h-10 gap-2 px-6"
107     >
108         <Play className="h-4 w-4" />
109         {running ? "Analyzing..." : "Analyze"}
110     </Button>
111 </div>
112 <p className="px-1 text-xs text-muted-foreground">
113     Data source: CSV files{ecrs.length > 0 && ` - ${ecrs.length} ECRs available`}
114 </p>
115
116 {!record && !running && (
117     <p className="px-1 text-sm text-muted-foreground">Enter an ECR ID and press Analyze.</p>
118 )}
119
120 {running && (
121     <Expander title="Agents running">
122         <AgentPipeline events={events} skipped={current?.skipped_steps || []} startedAt={startedAt} />
123         {status === "awaiting_approval" && <ApprovalPanel selection={selection} onDecide={approve} />}
124     </Expander>
125 )}
126
127 {current && impact && (
128     <>
129         <Expander title="Execution Trace">
130             <ExecutionTrace trace={current.execution_trace || []} duration={current.duration} />
131         </Expander>
132
133         <Expander title="ECR Summary">
134             <SummaryCard ecr={record} analysis={current} />
135         </Expander>
136     </>
137 )}
138
139 {record && (
140     <Expander title="ECR Details" defaultOpen={!current}>
141         <KeyValues rows={ecrDetailRows(record)} />
142     </Expander>
143 )}
144
145 {current && impact && (
146     <>
147         <Expander title="Requirements">
148             <PreviewTable
149                 noun="requirements"
150                 rows={requirements}
151                 rowKey={row => row.requirement_id}
152                 empty="No requirement could be traced to this ECR."
153                 columns={[
154                     { key: "requirement_id", label: "Requirement ID" },
155                     { key: "title", label: "Title" },
156                     { key: "reason", label: "Why it is linked", className: "text-muted-foreground" },
157                 ]}
158             />
159         </Expander>
160
161         <Expander title="Recommended Test Cases">
162             <TestsSection tests={tests} selection={selection} />
163         </Expander>
164
165         <Expander title="Impacted Components">
166             <div className="space-y-4 text-sm">
167                 <ComponentList title="Changed directly" items={impact.directly_impacted_components} />
168                 <ComponentList title="Reached through dependencies" items={impact.indirectly_impacted_compone
169 + nts} />
170                 {impact.mitigations?.length > 0 && (
171                     <div>
172                         <div className="font-semibold">Recommended actions</div>
173                         <ol className="mt-1 list-decimal space-y-0.5 pl-5 text-foreground/90">
174                             {impact.mitigations.map((item) => (
175                                 <li key={item}>{item}</li>
176                             ))}
177                         </ol>
178                     </div>
179                 )}
180             </div>
181         </Expander>
182
183         <Expander title="Defect Analysis">
184             <DefectSection
185                 insights={current.defect_insights}
186                 summary={current.defect_summary}
187                 similar={current.defect_analysis?.similar_defects}
188             />

```

```

188     </Expander>
189
190     {findings.length > 0 && (
191       <Expander title="Conflicts and Gaps">
192         <ul className="list-disc space-y-1 pl-5 text-sm text-foreground/90">
193           {findings.map((item) => (
194             <li key={item.text}>{item.text}</li>
195           ))}
196         </ul>
197       </Expander>
198     )}
199
200     <Expander title="Review Comments and Evidence" defaultOpen={false}>
201       <div className="space-y-4">
202         <DataTable
203           rows={current.comments}
204           rowKey={({row) => row.comment_id}
205           empty="No review comments are attached to this ECR."
206           columns={[
207             { key: "author", label: "Author" },
208             { key: "body", label: "Comment", className: "text-muted-foreground" },
209           ]}
210         />
211         <DataTable
212           rows={current.evidence}
213           rowKey={({row) => row.evidence_id}
214           empty="No evidence is attached to this ECR."
215           columns={[
216             { key: "title", label: "Evidence" },
217             { key: "outcome", label: "Outcome", render: (row) => titleCase(row.outcome) },
218           ]}
219         />
220       </div>
221     </Expander>
222
223     <Expander title="Ask a Question" defaultOpen={false}>
224       <FollowUp ecrId={ecrId} />
225     </Expander>
226   </>
227 }
228 </div>
229 );
230 }
231
232 const joined = (values) => (values || []).join(", ");
233
234 /**
235  * ECR fields in display order. Records imported from the change-tracking export
236  * (ECR-1, ECR-2 ...) carry extra columns; a row only appears when it has a value.
237  */
238 function ecrDetailRows(record) {
239   const exported = Boolean(record.lifecycle_status || record.severity || record.affected_test_cases?.length
240   + );
241   const rows = [
242     ["Type", exported ? record.record_type : ""],
243     ["Summary", record.title],
244     ["Status", record.lifecycle_status],
245     ["Severity", record.severity],
246     ["Priority", exported ? "" : titleCase(record.priority)],
247     ["Assigned Testers", joined(record.assigned_testers)],
248     ["Actual Modified Software Objects", joined(record.actual_modified_objects)],
249     ["Planned Modified Software Objects", joined(record.planned_modified_objects)],
250     ["Build Resolved In", record.build_resolved_in],
251     ["Test Estimate (Archived)", record.test_estimate_archived],
252     ["Estimate", record.estimate],
253     ["Test Estimate", record.test_estimate],
254     ["Creation Date", record.creation_date],
255     ["Modified Date", record.modified_date],
256     ["Software Integration Test Assigned Testers", joined(record.sit_assigned_testers)],
257     ["Test Actual Time", record.test_actual_time],
258     ["Description", record.description],
259     ["Affected Test Cases", joined(record.affected_test_cases)],
260     ["Steps to Reproduce", record.steps_to_reproduce],
261     ["Observed Behavior", record.observed_behavior],
262     ["Expected Behavior", record.expected_behavior],
263     ["Attachments", joined(record.attachments)],
264     ["Business Domain", record.business_domain],
265     ["Requested By", exported ? "" : record.requested_by],
266     ["Changed Files", joined(record.changed_files)],
267     ["Linked Requirements", joined(record.linked_requirements)],
268   ];
269   return rows.filter(([, value]) => value);
270 }

```

```

271 /** What ran, in order, and how long it took. */
272 function ExecutionTrace({ trace, duration }) {
273   const steps = trace.filter((row) => !String(row.agent_key || "").endsWith("_agent"));
274   const agents = AGENT_NAMES.filter((name) =>
275     trace.some((entry) => entry.agent === name && entry.status === "completed")
276   );
277   return (
278     <div className="space-y-3 text-sm">
279       <DataTable
280         minWidth={480}
281         rows={steps}
282         empty="No steps were recorded for this run."
283         columns={[
284           { key: "step", label: "Step", render: (row) => STEP_LABELS[row.agent_key] || row.action },
285           { key: "agent", label: "Agent", className: "text-muted-foreground" },
286           { key: "status", label: "Status", render: (row) => titleCase(row.status) },
287           {
288             key: "duration",
289             label: "Time",
290             render: (row) => `${Number(row.duration || 0).toFixed(1)}s`,
291             className: "tabular-nums text-muted-foreground",
292           },
293         ]}
294       />
295       <p className="text-muted-foreground">
296         {agents.length ? `${agents.join(", ")} completed` : "No agent completed"}
297         {duration !== null && ` in ${Number(duration).toFixed(1)}s`}.
298       </p>
299     </div>
300   );
301 }
302
303 function SummaryCard({ ecr, analysis }) {
304   const [copied, setCopied] = useState(false);
305   const impact = analysis.impact_analysis;
306   const selection = analysis.test_selection || {};
307   const distribution = selection.priority_distribution || {};
308   const concerns = analysis.collaboration_analysis?.open_concerns || [];
309   const direct = (impact.directly_impacted_components || []).map((c) => c.name);
310   const indirect = (impact.indirectly_impacted_components || []).map((c) => c.name);
311   const selected = selection.selected_tests?.length || 0;
312   const defectInfo = analysis.defect_insights;
313
314   const lines = [
315     ["ECR", `${ecr?.ecr_id || analysis.ecr_id} - ${ecr?.title || ""}`],
316     ["Change Type", titleCase(analysis.ecr_analysis?.change_type)],
317     ["AI Summary", analysis.answer],
318     [
319       "Impacted",
320       `${direct.join(", ") || "None identified"}` +
321       (indirect.length ? ` (also reaches ${indirect.join(", ")})` : ""),
322     ],
323     [
324       "Recommended Tests",
325       `${selected} of ${selection.total_available || 0} (${Math.round(selection.reduction_percentage || 0)}
326       % fewer),` +
327       `${distribution.P0 || 0} P0 and ${distribution.P1 || 0} P1`,
328     ],
329     [
330       "Past Defects",
331       defectInfo?.total
332       ? `${defectInfo.total} related, ${defectInfo.high} could happen again`
333       : "None related",
334     ],
335   ];
336
337   const markdown = useMemo(() => {
338     const out = lines.map(([label, value]) => `**${label}** ${value || "-"}`);
339     if (concerns.length) {
340       out.push("", "Open Concerns:", ...concerns.map((c) => `- ${c.author}: ${c.text}`));
341     }
342     if (impact.mitigations?.length) {
343       out.push("", "Recommended Actions:", ...impact.mitigations.map((m, i) => `${i + 1}. ${m}`));
344     }
345     return out.join("\n");
346   }, [analysis, ecr]);
347
348   const copy = async () => {
349     try {
350       await navigator.clipboard.writeText(markdown);
351       setCopied(true);
352       toast.success("Copied - paste it straight into Teams");
353       setTimeout(() => setCopied(false), 1800);
354     }
355   };
356 }

```

```

354     } catch {
355       toast.error("Clipboard unavailable in this browser");
356     }
357   };
358
359   return (
360     <div className="space-y-3 text-sm leading-relaxed">
361       {lines.map(([label, value]) => (
362         <p key={label}>
363           <span className="font-semibold">{label}</span> {value || "-"}
364         </p>
365       ))}
366       {concerns.length > 0 && (
367         <div>
368           <div className="font-semibold">Open Concerns:</div>
369           <ul className="mt-1 list-disc space-y-1 pl-5">
370             {concerns.map((concern) => (
371               <li key={concern.comment_id}>
372                 {concern.author}: {concern.text}
373               </li>
374             ))}
375           </ul>
376         </div>
377       )}
378       <div className="flex justify-end pt-1">
379         <Button size="sm" variant="outline" className="gap-2" onClick={copy}>
380           {copied ? <Check className="h-3.5 w-3.5" /> : <Copy className="h-3.5 w-3.5" />}
381           Copy for Teams
382         </Button>
383       </div>
384     </div>
385   );
386 }
387
388 function TestsSection({ tests, selection }) {
389   return (
390     <div className="space-y-3">
391       {selection && (
392         <p className="text-sm">
393           {selection.selected_tests?.length || 0} of {selection.total_available} tests recommended - about{
394             + " "
395             {Math.round(selection.estimated_duration_minutes || 0)} min instead of{" "}
396             {Math.round(selection.baseline_duration_minutes || 0)} min.
397           </p>
398         <PreviewTable
399           noun="tests"
400           minWidth={640}
401           rows={tests}
402           rowKey={(row) => row.test_case_id}
403           empty="No test cases were selected."
404           columns={[
405             { key: "test_case_id", label: "Test Case ID" },
406             { key: "title", label: "Title" },
407             {
408               key: "priority",
409               label: "Priority",
410               render: (row) => (
411                 <span className="font-medium" style={{ color: priorityColor(row.priority) }}>
412                   {row.priority}
413                 </span>
414               ),
415             },
416             { key: "reason", label: "Why", className: "text-muted-foreground" },
417           ]}
418         </PreviewTable>
419       )}
420     </div>
421   );
422 }
423
424 function DefectSection({ insights, summary, similar = [] }) {
425   const rows = insights?.defects || [];
426   const history = insights?.component_history || [];
427   const openNow = insights?.open_defects || [];
428   // Analyses saved before defect analysis existed only carry the similar-defect list.
429   const legacy = insights ? [] : similar || [];
430
431   if (!summary && !rows.length && !history.length && !openNow.length && !legacy.length) {
432     return <p className="text-sm text-muted-foreground">No defects are recorded for this ECR or the compone
433       + nts it touches.</p>
434   }
435
436   return (
437     <div className="space-y-5">

```

```

436 {summary && <p className="text-sm leading-relaxed">{summary}</p>}
437
438 {rows.length > 0 && (
439   <PreviewTable
440     noun="defects"
441     minWidth={680}
442     rows={rows}
443     rowKey={(row) => row.defect_id}
444     columns={[
445       { key: "defect_id", label: "Defect ID" },
446       { key: "title", label: "What went wrong" },
447       {
448         key: "chance_of_recurrence",
449         label: "Can it happen again?",
450         render: (row) => (
451           <span className="font-medium" style={{ color: chanceColor(row.chance_of_recurrence) }}>
452             {row.chance_of_recurrence}
453           </span>
454         ),
455       },
456       { key: "what_to_check", label: "What to check", className: "text-muted-foreground" },
457     ]}
458   />
459 )}
460
461 {legacy.length > 0 && (
462   <DataTable
463     minWidth={520}
464     rows={legacy}
465     rowKey={(row) => row.defect_id}
466     columns={[
467       { key: "defect_id", label: "Defect ID" },
468       { key: "title", label: "What went wrong" },
469       { key: "root_cause", label: "Cause", className: "text-muted-foreground" },
470     ]}
471   />
472 )}
473
474 {openNow.length > 0 && (
475   <Block title="Still open in the affected area">
476     <DataTable
477       minWidth={520}
478       rows={openNow}
479       rowKey={(row) => row.defect_id}
480       columns={[
481         { key: "defect_id", label: "Defect ID" },
482         { key: "title", label: "What is wrong" },
483         { key: "component_name", label: "Component" },
484         { key: "status", label: "Status", render: (row) => titleCase(row.status) },
485       ]}
486     />
487   </Block>
488 )}
489
490 {/* Component history is only needed when no past defect matched the change itself. */}
491 {!rows.length && history.length > 0 && (
492   <Block title="Defect history of the affected components">
493     <DataTable
494       minWidth={520}
495       rows={history}
496       rowKey={(row) => row.component}
497       columns={[
498         { key: "component_name", label: "Component" },
499         { key: "total", label: "Past defects", className: "tabular-nums" },
500         { key: "open", label: "Still open", className: "tabular-nums" },
501         { key: "top_cause", label: "Usual cause", className: "text-muted-foreground" },
502       ]}
503     />
504   </Block>
505 )}
506 </div>
507 );
508 }
509
510 function ComponentList({ title, items = [] }) {
511   return (
512     <div>
513       <div className="font-semibold">{title}</div>
514       {items.length ? (
515         <ul className="mt-1 list-disc space-y-0.5 pl-5 text-foreground/90">
516           {items.map((component) => (
517             <li key={component.component_id}>
518               {component.name}
519               {component.reasons?.length ? ` - ${component.reasons.join("; ")}` : ""}

```

```

520         </li>
521     )}}
522 </ul>
523 ) : (
524     <p className="mt-1 text-muted-foreground">None identified.</p>
525 )}
526 </div>
527 );
528 }
529
530 function FollowUp({ ecrId }) {
531     const [question, setQuestion] = useState("");
532     const [asking, setAsking] = useState(false);
533     const [history, setHistory] = useState([]);
534
535     const ask = async (event) => {
536         event.preventDefault();
537         const text = question.trim();
538         if (!text) return;
539         setAsking(true);
540         setQuestion("");
541         try {
542             const response = await api.ask(ecrId, text);
543             setHistory((prev) => [...prev, { question: text, answer: response.answer, citations: response.citations }]);
544         } catch (error) {
545             toast.error(error.message);
546         } finally {
547             setAsking(false);
548         }
549     };
550
551     return (
552         <div className="space-y-3 text-sm">
553             {history.map((entry, index) => (
554                 <div key={index}>
555                     <p className="font-semibold">Q: {entry.question}</p>
556                     <p className="mt-1 whitespace-pre-line">{entry.answer}</p>
557                     {entry.citations?.length > 0 && (
558                         <p className="mt-1 text-xs text-muted-foreground">Sources: {entry.citations.join(", ")}</p>
559                     )}
560                 </div>
561             ))}
562             <form onSubmit={ask} className="flex gap-2">
563                 <Input
564                     value={question}
565                     onChange={(event) => setQuestion(event.target.value)}
566                     placeholder={`Ask about ${ecrId} - e.g. why was the top test selected?`}
567                     disabled={asking}
568                 />
569                 <Button type="submit" disabled={asking || !question.trim()} className="gap-2">
570                     <Send className="h-4 w-4" />
571                     {asking ? "Thinking..." : "Ask"}
572                 </Button>
573             </form>
574         </div>
575     );
576 }
577
578 function ApprovalPanel({ selection, onDecide }) {
579     const [busy, setBusy] = useState(false);
580     const decide = async (approved) => {
581         setBusy(true);
582         try {
583             await onDecide({ approved, decided_by: "qa.lead@enterprise.com", comment: "" });
584         } finally {
585             setBusy(false);
586         }
587     };
588     return (
589         <div className="mt-4 flex flex-wrap items-center justify-between gap-3 border-t border-border/60 pt-4 text-sm">
590             <p>
591                 {selection?.selected_tests?.length ?? "The"} recommended tests are ready. Approve to publish the report, or
592                 reject to withhold the recommendation.
593             </p>
594             <div className="flex gap-2">
595                 <Button size="sm" disabled={busy} onClick={() => decide(true)} className="gap-2">
596                     <ShieldCheck className="h-4 w-4" /> Approve
597                 </Button>
598                 <Button size="sm" variant="outline" disabled={busy} onClick={() => decide(false)} className="gap-2">
599                     <XCircle className="h-4 w-4" /> Reject

```

```

600         </Button>
601     </div>
602 </div>
603 );
604 }
605
606 function Block({ title, children }) {
607     return (
608         <div>
609             <div className="mb-1 font-semibold">{title}</div>
610             <div className="text-foreground/90">{children}</div>
611         </div>
612     );
613 }

```

frontend/src/pages/DashboardPage.jsx

```

1  import React, { useEffect, useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import api from "@services/api";
4  import { useAnalysis } from "@context/AnalysisContext";
5  import { DataTable, Expander, KeyValues, titleCase } from "@components/common/Sections";
6
7  /** Every ECR and the latest analysis of each, as plain tables. */
8  export default function DashboardPage() {
9      const [stats, setStats] = useState(null);
10     const [error, setError] = useState(null);
11     const { selectEcr } = useAnalysis();
12     const navigate = useNavigate();
13
14     useEffect(() => {
15         api.dashboardStats().then(setStats).catch(setError);
16     }, []);
17
18     const openEcr = async (ecrId) => {
19         await selectEcr(ecrId);
20         navigate("/analyze");
21     };
22
23     return (
24         <div className="mx-auto max-w-5xl space-y-4">
25             <header className="pb-2">
26                 <h2 className="text-3xl font-semibold tracking-tight">ECR Dashboard</h2>
27                 <p className="mt-1 text-sm text-muted-foreground">
28                     Every Engineering Change Request and the latest analysis of each.
29                 </p>
30             </header>
31
32             {error && <p className="surface p-4 text-sm text-[#d03b3b]">{error.message}</p>}
33             {!stats && !error && <p className="px-1 text-sm text-muted-foreground">Loading...</p>}
34
35             {stats && (
36                 <>
37                     <Expander title="Overview">
38                         <KeyValues
39                             rows={[
40                                 ["Total ECRs", stats.total_ecrs],
41                                 ["Analysed ECRs", stats.analysed_ecrs],
42                                 [
43                                     "Average regression reduction",
44                                     stats.analysed_ecrs ? `${Math.round(stats.average_regression_reduction)}%` : "-",
45                                 ],
46                                 [
47                                     "Average analysis time",
48                                     stats.analysed_ecrs ? `${Number(stats.average_analysis_seconds).toFixed(1)}s` : "-",
49                                 ],
50                                 ["Test cases in the catalogue", stats.catalogue?.test_cases],
51                             ]
52                         </>
53                     </Expander>
54
55                     <Expander title="Recent Analyses">
56                         <DataTable
57                             rows={stats.recent_analyses}
58                             rowKey={(row) => row.workflow_id}
59                             onRowClick={(row) => openEcr(row.ecr_id)}
60                             empty="No analyses yet. Open ECR Analysis and analyse an ECR."
61                             columns={[
62                                 { key: "ecr_id", label: "ECR ID" },
63                                 {
64                                     key: "tests",
65                                     label: "Tests",
66                                     render: (row) => `${row.selected_tests} of ${row.total_tests}`,
67                                     className: "tabular-nums",

```

```

68         },
69         {
70             key: "reduction",
71             label: "Reduction",
72             render: (row) => `${Math.round(row.reduction_percentage)}%`,
73             className: "tabular-nums",
74         },
75         {
76             key: "time",
77             label: "Time",
78             render: (row) => `${Number(row.execution_time).toFixed(1)}s`,
79             className: "tabular-nums text-muted-foreground",
80         },
81     ]],
82     />
83 </Expander>
84
85 <Expander title="ECRs">
86     <DataTable
87       rows={stats.recent_ecrs}
88       rowKey={(row) => row.ecr_id}
89       onRowClick={(row) => openEcr(row.ecr_id)}
90       columns={[
91         { key: "ecr_id", label: "ECR ID" },
92         { key: "title", label: "Title" },
93         { key: "business_domain", label: "Domain" },
94         { key: "status", label: "Status", render: (row) => row.lifecycle_status || titleCase(row.st
95 +       atus) },
96     ]}
97     />
98 </Expander>
99 </>
100 </div>
101 );
102 }

```

frontend/src/pages/ReportPage.jsx

```

1 import React from "react";
2 import { Download, ExternalLink, FileJson, FileText } from "lucide-react";
3 import api from "@services/api";
4 import { useAnalysis } from "@context/AnalysisContext";
5 import EmptyState from "@components/common/EmptyState";
6 import { Button } from "@components/ui/button";
7
8 export default function ReportPage() {
9   const { analysis, ecrId } = useAnalysis();
10   const report = analysis?.report;
11
12   if (!report) {
13     return (
14       <EmptyState
15         title="No report yet"
16         hint={`Analyse ${ecrId} || "an ECR"} to generate its intelligence report.`
17       />
18     );
19   }
20
21   const confidence = report.confidence || {};
22   const metrics = report.metrics || {};
23
24   return (
25     <div className="space-y-6">
26       <section className="surface p-5">
27         <div className="flex flex-wrap items-start justify-between gap-4">
28           <div>
29             <div className="eyebrow-label">ECR Intelligence Report</div>
30             <h2 className="mt-1 text-xl font-semibold tracking-tight">
31               {report.ecr_id} - {report.title}
32             </h2>
33             <div className="mt-1 text-xs text-muted-foreground">
34               Generated {new Date(report.generated_at).toLocaleString()} - workflow{" "}
35               <span className="font-mono">{report.workflow_id}</span>
36             </div>
37           </div>
38           <div className="flex flex-wrap gap-2">
39             <Button asChild size="sm" variant="outline" className="gap-2">
40               <a href={api.reportUrl(report.ecr_id, "html")} target="_blank" rel="noreferrer">
41                 <ExternalLink className="h-3.5 w-3.5" /> HTML
42               </a>
43             </Button>
44             <Button asChild size="sm" variant="outline" className="gap-2">
45               <a href={api.reportUrl(report.ecr_id, "markdown")} target="_blank" rel="noreferrer">

```

```

46         <FileText className="h-3.5 w-3.5" /> Markdown
47     </a>
48 </Button>
49 <Button asChild size="sm" variant="outline" className="gap-2">
50     <a href={api.reportUrl(report.ecr_id, "json")} target="_blank" rel="noreferrer">
51         <FileJson className="h-3.5 w-3.5" /> JSON
52     </a>
53 </Button>
54 <Button
55     size="sm"
56     className="gap-2"
57     onClick={() => window.print()}
58     title="Print or save as PDF"
59 >
60     <Download className="h-3.5 w-3.5" /> PDF
61 </Button>
62 </div>
63 </div>
64
65 <div className="mt-5 grid gap-3 sm:grid-cols-3">
66     <Metric
67         label="Confidence"
68         value={`\${Math.round((confidence.overall || 0) * 100)}%`}
69         hint={confidence.band}
70     />
71     <Metric
72         label="Recommended tests"
73         value={`\${metrics.selected_tests || 0} / \${metrics.total_tests || 0}`}
74         hint={`\${Number(metrics.reduction_percentage || 0).toFixed(0)}% reduction`}
75     />
76     <Metric
77         label="Correlated artefacts"
78         value={metrics.correlated_artefacts || 0}
79         hint={`\${metrics.correlated_links || 0} typed links`}
80     />
81 </div>
82
83 {confidence.penalties?.length > 0 && (
84     <div className="mt-4 rounded-lg border border-[#fab219]/30 bg-[#fab219]/5 p-3 text-xs text-[#fab2
85 + 19]">
86         Confidence reduced: {confidence.penalties.join("; ")}
87     </div>
88 )}
89 </section>
90
91 <section className="space-y-4">
92     {report.sections
93     ?.filter((section) => section.key !== "execution_trace")
94     .map((section) => (
95         <div key={section.key} className="surface p-5">
96             <h3 className="text-sm font-semibold">{section.title}</h3>
97             {section.body && (
98                 <p className="mt-2 whitespace-pre-line text-sm leading-relaxed text-muted-foreground">
99                     {section.body}
100                 </p>
101             )}
102             <SectionData section={section} />
103         </div>
104     )}
105 </section>
106 </div>
107 );
108 }
109
110 function Metric({ label, value, hint, color }) {
111     return (
112         <div className="rounded-lg border border-border/70 bg-background/40 p-3">
113             <div className="eyebrow-label">{label}</div>
114             <div className="mt-1 text-xl font-semibold" style={color ? { color } : undefined}>
115                 {value}
116             </div>
117             {hint && <div className="text-[11px] text-muted-foreground">{hint}</div>}
118         </div>
119     );
120 }
121
122 function SectionData({ section }) {
123     const { key, data } = section;
124     if (key === "affected_requirements" && data.requirements?.length) {
125         return (
126             <ul className="mt-3 space-y-1.5 text-xs">
127                 {data.requirements.map((requirement) => (
128                     <li key={requirement.requirement_id} className="flex flex-wrap gap-2">
129                         <span className="font-mono text-primary">{requirement.requirement_id}</span>

```

```

129         <span className="text-foreground">{requirement.title}</span>
130         <span className="text-muted-foreground">
131           {Math.round(requirement.relevance)}% - {requirement.reason}
132         </span>
133       </li>
134     )}
135   </ul>
136 );
137 }
138 if (key === "recommended_tests" && data.selected?.length) {
139   return (
140     <div className="scroll-thin mt-3 max-h-72 overflow-y-auto">
141       <table className="w-full text-xs">
142         <thead>
143           <tr className="text-left text-[10px] uppercase tracking-wide text-muted-foreground">
144             <th className="pb-1.5 font-medium">Test</th>
145             <th className="pb-1.5 font-medium">Priority</th>
146             <th className="pb-1.5 font-medium">Relevance</th>
147             <th className="pb-1.5 font-medium">Why</th>
148           </tr>
149         </thead>
150         <tbody>
151           {data.selected.slice(0, 40).map((test) => (
152             <tr key={test.test_case_id} className="border-t border-border/50 align-top">
153               <td className="py-1.5 font-mono">{test.test_case_id}</td>
154               <td className="py-1.5">{test.priority}</td>
155               <td className="py-1.5 tabular-nums">{Math.round(test.relevance_score)}%</td>
156               <td className="py-1.5 text-muted-foreground">{test.reason}</td>
157             </tr>
158           ))}
159         </tbody>
160       </table>
161     </div>
162   );
163 }
164 if (key === "historical_defects" && data.defects?.length) {
165   return (
166     <ul className="mt-3 space-y-2 text-xs">
167       {data.defects.map((defect) => (
168         <li key={defect.defect_id}>
169           <span className="font-mono text-primary">{defect.defect_id}</span>{" "}
170           <span className="text-foreground">{defect.title}</span>
171           {defect.chance_of_recurrence && (
172             <span className="text-muted-foreground"> - can it happen again: {defect.chance_of_recurrence}
173           )}
174           {defect.what_to_check && <div className="mt-0.5 text-muted-foreground">{defect.what_to_check}</div>}
175         </li>
176       ))}
177     </ul>
178   );
179 }
180 if (key === "mitigations" && data.mitigations?.length) {
181   return (
182     <ul className="mt-2 space-y-1 text-xs text-muted-foreground">
183       {data.mitigations.map((item, index) => (
184         <li key={index}>- {item}</li>
185       ))}
186     </ul>
187   );
188 }
189 if (key === "team_signals") {
190   return (
191     <div className="mt-3 grid gap-3 md:grid-cols-2">
192       {["decisions", "risks"].map((bucket) =>
193         data[bucket]?.length ? (
194           <div key={bucket}>
195             <div className="eyebrow-label">{bucket}</div>
196             <ul className="mt-1.5 space-y-1.5 text-xs text-muted-foreground">
197               {data[bucket].slice(0, 4).map((item) => (
198                 <li key={item.comment_id}>
199                   <span className="text-foreground">{item.author}</span>: {item.text}
200                 </li>
201               ))}
202             </ul>
203           </div>
204         ) : null
205       )}
206     </div>
207   );
208 }
209 return null;
210 }

```

Frontend - components

frontend/src/components/agents/AgentPipeline.jsx

```
1 import React, { useEffect, useMemo, useState } from "react";
2 import { Check, Loader2, Minus, X } from "lucide-react";
3
4 /**
5  * The three agents and the steps each one runs. Keys match the backend step
6  * keys, so live events mark the right rows. There are no other agents.
7  */
8 export const AGENTS = [
9   {
10     key: "retrieval_agent",
11     name: "Retrieval Agent",
12     steps: [
13       ["understand_ecr", "Understand the ECR"],
14       ["gather_evidence", "Gather evidence"],
15     ],
16   },
17   {
18     key: "correlation_agent",
19     name: "Correlation Agent",
20     steps: [
21       ["assess_impact", "Assess impact"],
22       ["select_tests", "Select regression tests"],
23     ],
24   },
25   {
26     key: "summarization_agent",
27     name: "Summarization Agent",
28     steps: ["summarize", "Summarize and report"],
29   },
30 ];
31
32 /** Evidence the Gather step may collect; the planner can skip the optional ones. */
33 export const INVESTIGATION_LABELS = {
34   requirements: "Impacted requirements",
35   code_impact: "Code impact",
36   historical_defects: "Similar historical defects",
37   dependencies: "Component dependencies",
38   collaboration_retrieval: "Review comments and evidence",
39 };
40
41 export const AGENT_NAMES = AGENTS.map((agent) => agent.name);
42
43 export const STEP_LABELS = Object.fromEntries(AGENTS.flatMap((agent) => agent.steps));
44
45 export const STEP_OWNER = Object.fromEntries(
46   AGENTS.flatMap((agent) => agent.steps.map(([key]) => [key, agent.name]))
47 );
48
49 function useStatuses(events) {
50   return useMemo(() => {
51     const statuses = {};
52     events.forEach((event) => {
53       if (!event.agent_key) return;
54       statuses[event.agent_key] = {
55         status: String(event.status || "").toLowerCase(),
56         duration: event.duration ?? statuses[event.agent_key]?.duration,
57       };
58     });
59     return statuses;
60   }, [events]);
61 }
62
63 /** Seconds since ``startedAt``, ticking while the run is in progress. */
64 function useElapsed(startedAt) {
65   const [now, setNow] = useState(() => Date.now());
66   useEffect(() => {
67     if (!startedAt) return undefined;
68     const timer = setInterval(() => setNow(Date.now()), 200);
69     return () => clearInterval(timer);
70   }, [startedAt]);
71   return startedAt ? Math.max(0, (now - startedAt) / 1000) : null;
72 }
73
74 const LABEL = { running: "Running", completed: "Done", failed: "Failed", skipped: "Not needed" };
75
76 function Mark({ status }) {
77   if (status === "completed") return <Check className="h-3.5 w-3.5 text-[#0ca30c]" />;
78   if (status === "running") return <Loader2 className="h-3.5 w-3.5 animate-spin text-[#3987e5]" />;
```

```

79   if (status === "failed") return <X className="h-3.5 w-3.5 text-[#d03b3b]" />;
80   return <Minus className="h-3.5 w-3.5 text-muted-foreground/60" />;
81 }
82
83 /** Live progress as a plain checklist: three agents, their steps, a status each. */
84 export default function AgentPipeline({ events = [], skipped = [], startedAt = null }) {
85   const statuses = useStatuses(events);
86   const awaitingApproval = statuses.human_approval?.status === "awaiting_approval";
87   const elapsed = useElapsed(startedAt);
88
89   return (
90     <div className="space-y-4 text-sm">
91       {elapsed != null && (
92         <p className="text-xs tabular-nums text-muted-foreground">Running for {elapsed.toFixed(1)}s</p>
93       )}
94       {AGENTS.map((agent, index) => {
95         const agentStatus = statuses[agent.key]?.status || "waiting";
96         return (
97           <div key={agent.key}>
98             <div className="flex items-center justify-between font-semibold">
99               <span>
100                 {index + 1}. {agent.name}
101               </span>
102               <span className="text-xs font-normal text-muted-foreground">
103                 {LABEL[agentStatus] || "Waiting"}
104               </span>
105             </div>
106             <ul className="mt-1.5 space-y-1 pl-4">
107               {agent.steps.map(([key, label]) => {
108                 const status = skipped.includes(key) ? "skipped" : statuses[key]?.status || "waiting";
109                 return (
110                   <li key={key} className="flex items-center gap-2">
111                     <Mark status={status} />
112                     <span className={status === "waiting" || status === "skipped" ? "text-muted-foreground"
113 + : ""}>
114                       {label}
115                     </span>
116                     <span className="ml-auto text-xs tabular-nums text-muted-foreground">
117                       {status === "skipped"
118                         ? "Not needed"
119                         : statuses[key]?.duration != null
120                           ? `${Number(statuses[key].duration).toFixed(1)}s`
121                           : ""}
122                     </span>
123                   </li>
124                 );
125               })}
126             </ul>
127           </div>
128         );
129       })}
130       {awaitingApproval && (
131         <p className="text-[#fab219]">
132           Approval requested - waiting for a human decision before the report is published.
133         </p>
134       )}
135     </div>
136   );
137 }

```

frontend/src/components/common/EmptyState.jsx

```

1 import React from "react";
2 import { useNavigate } from "react-router-dom";
3 import { Compass } from "lucide-react";
4 import { Button } from "@components/ui/button";
5
6 export default function EmptyState({ title, hint, actionLabel = "Go to ECR analysis", to = "/analyze" }) {
7   const navigate = useNavigate();
8   return (
9     <div className="surface flex flex-col items-center justify-center gap-3 p-12 text-center">
10       <div className="grid h-11 w-11 place-items-center rounded-xl bg-secondary/70">
11         <Compass className="h-5 w-5 text-muted-foreground" />
12       </div>
13       <div>
14         <div className="text-sm font-semibold">{title}</div>
15         {hint && <p className="mt-1 text-xs text-muted-foreground">{hint}</p>}
16       </div>
17       <Button size="sm" variant="outline" onClick={() => navigate(to)}>
18         {actionLabel}
19       </Button>
20     </div>
21   );
22 }

```

frontend/src/components/common/Sections.jsx

```
1 import React from "react";
2 import { ChevronRight } from "lucide-react";
3
4 /**
5  * Plain building blocks for every page: a collapsible section, a label/value
6  * table and a simple data table. Text and tables only - no charts or graphs.
7  */
8 export function Expander({ title, note, defaultOpen = true, children }) {
9   return (
10     <details open={defaultOpen} className="surface group">
11       <summary className="flex cursor-pointer select-none list-none items-center gap-2 px-5 py-3 text-sm font-medium [&::-webkit-details-marker]:hidden">
12         <ChevronRight className="h-4 w-4 text-muted-foreground transition-transform group-open:rotate-90" />
13         {title}
14       </summary>
15       <div className="border-t border-border/60 px-5 py-4">
16         {note ? <p className="text-sm italic text-muted-foreground">{note}</p> : children}
17       </div>
18     </details>
19   );
20 }
21
22 export function KeyValues({ rows }) {
23   return (
24     <table className="mb-3 w-full text-sm last:mb-0">
25       <tbody>
26         {rows.map(([label, value]) => (
27           <tr key={label} className="border-t border-border/50 align-top first:border-t-0">
28             <td className="w-48 py-2 pr-4 text-muted-foreground">{label}</td>
29             <td className="whitespace-pre-line py-2">{value ?? "-"}</td>
30           </tr>
31         ))}
32       </tbody>
33     </table>
34   );
35 }
36
37 /**
38  * columns: [{ key, label, render?, className? }]
39  */
40 export function DataTable({ columns, rows, rowKey, onClick, empty = "Nothing to show.", minWidth = 640 }) {
41   if (!rows?.length) return <p className="text-sm text-muted-foreground">{empty}</p>;
42   return (
43     <div className="overflow-x-auto">
44       <table className="w-full text-sm" style={{ minWidth }}>
45         <thead>
46           <tr className="text-left text-xs text-muted-foreground">
47             {columns.map((column) => (
48               <th key={column.key} className="pb-2 pr-3 font-medium">
49                 {column.label}
50             </th>
51             ))}
52           </tr>
53         </thead>
54         <tbody>
55           {rows.map((row, index) => (
56             <tr>
57               key={rowKey ? rowKey(row) : index}
58               onClick={onClick ? () => onClick(row) : undefined}
59               className={`border-t border-border/50 align-top ${onClick ? "cursor-pointer hover:bg-secondary/40" : ""}`}
60             >
61               {columns.map((column) => (
62                 <td key={column.key} className={`py-2 pr-3 ${column.className || ""}`}>
63                   {column.render ? column.render(row, index) : row[column.key] ?? "-"}
64                 </td>
65               ))}
66             </tr>
67           ))}
68         </tbody>
69       </table>
70     </div>
71   );
72 }
73
74 export const titleCase = (value) =>
75   value
76     ? String(value)
77       .replace(/_/g, " ")
```

```

78         .toLowerCase()
79         .replace(/^\w/, (c) => c.toUpperCase())
80         : "";

```

frontend/src/components/layout/AppShell.jsx

```

1 import React from "react";
2 import { NavLink } from "react-router-dom";
3 import { FileText, LayoutDashboard, Search } from "lucide-react";
4 import { useAnalysis } from "@context/AnalysisContext";
5 import { cn } from "@lib/utlis";
6
7 const NAV = [
8   { to: "/", label: "Dashboard", icon: LayoutDashboard, end: true },
9   { to: "/analyze", label: "ECR Analysis", icon: Search },
10  { to: "/report", label: "Report", icon: FileText },
11 ];
12
13 const STATUS_TEXT = {
14   idle: "Idle",
15   running: "Agents running",
16   awaiting_approval: "Waiting for approval",
17   done: "Analysis complete",
18   error: "Analysis failed",
19 };
20
21 export default function AppShell({ children }) {
22   const { ecrId, status, health, isDemoMode } = useAnalysis();
23
24   return (
25     <div className="min-h-screen bg-background text-foreground">
26       <div className="flex min-h-screen">
27         <aside className="hidden w-56 shrink-0 border-r border-border/70 bg-card/40 lg:flex lg:flex-col">
28           <div className="px-5 py-5">
29             <div className="text-sm font-semibold">ECR Assistant</div>
30             <div className="text-xs text-muted-foreground">3 agents</div>
31           </div>
32
33           <nav className="flex-1 space-y-1 px-3 py-2">
34             {NAV.map(({ to, label, icon, end }) => (
35               <NavLink
36                 key={to}
37                 to={to}
38                 end={end}
39                 className={({ isActive }) =>
40                   cn(
41                     "flex items-center gap-3 rounded-lg px-3 py-2 text-sm transition-colors",
42                     isActive
43                       ? "bg-secondary text-foreground"
44                       : "text-muted-foreground hover:bg-secondary/60 hover:text-foreground"
45                   )
46                 }
47             >
48               <Icon className="h-4 w-4" />
49               {label}
50             </NavLink>
51             )))
52           </nav>
53
54           <div className="space-y-1.5 border-t border-border/70 px-4 py-4 text-xs text-muted-foreground">
55             <div>Backend: {health ? "connected" : "offline"}</div>
56             <div>Reasoning: {isDemoMode ? "rule-based (offline)" : health?.llm?.effective_provider || "-"}</div>
57           </div>
58         </aside>
59
60         <div className="flex min-w-0 flex-1 flex-col">
61           <header className="sticky top-0 z-20 border-b border-border/70 bg-background/90 backdrop-blur">
62             <div className="flex items-center justify-between gap-4 px-5 py-3 text-sm">
63               <span className="truncate font-medium">{ecrId || "No ECR selected"}</span>
64               <span className="text-muted-foreground">{STATUS_TEXT[status] || STATUS_TEXT.idle}</span>
65             </div>
66           </header>
67           <main className="flex-1 px-5 py-6">{children}</main>
68         </div>
69       </div>
70     </div>
71   );
72 }

```

Frontend - context, services, theme

frontend/src/context/AnalysisContext.jsx

```
1 import React, { createContext, useCallback, useContext, useEffect, useMemo, useRef, useState } from "react"
2 ;
3 import { toast } from "sonner";
4 import api from "@services/api";
5 /**
6  * Holds the ECR currently under analysis, the live agent event stream and the
7  * finished analysis payload, so every page reads one source of truth.
8  */
9 const AnalysisContext = createContext(null);
10
11 const STORAGE_KEY = "ecr.lastEcrId";
12
13 export function AnalysisProvider({ children }) {
14   const [ecrId, setEcrId] = useState(() => localStorage.getItem(STORAGE_KEY) || "");
15   const [status, setStatus] = useState("idle"); // idle | running | awaiting_approval | done | error
16   const [workflowId, setWorkflowId] = useState(null);
17   const [events, setEvents] = useState([]);
18   const [analysis, setAnalysis] = useState(null);
19   const [error, setError] = useState(null);
20   const [health, setHealth] = useState(null);
21   const [startedAt, setStartedAt] = useState(null); // when the current run began (ms)
22   const unsubscribeRef = useRef(null);
23
24   useEffect(() => {
25     api.health().then(setHealth).catch(() => setHealth(null));
26     return () => unsubscribeRef.current?.();
27   }, []);
28
29   useEffect(() => {
30     if (ecrId) localStorage.setItem(STORAGE_KEY, ecrId);
31   }, [ecrId]);
32
33   const loadAnalysis = useCallback(async (id) => {
34     try {
35       const payload = await api.analysis(id);
36       setAnalysis(payload);
37       return payload;
38     } catch (err) {
39       if (err.status !== 404) setError(err);
40       return null;
41     }
42   }, []);
43
44   const analyze = useCallback(
45     async (id, options = {}) => {
46       const target = (id || "").trim().toUpperCase();
47       if (!target) {
48         toast.error("Enter an ECR number first");
49         return;
50       }
51       unsubscribeRef.current?.();
52       setEcrId(target);
53       setEvents([]);
54       setAnalysis(null);
55       setError(null);
56       setStatus("running");
57       setStartedAt(Date.now());
58       try {
59         const started = await api.analyze(target, options);
60         setWorkflowId(started.workflow_id);
61         unsubscribeRef.current = api.streamWorkflow(started.workflow_id, {
62           onAgent: (event) => setEvents((prev) => [...prev, event]),
63           onDone: async (done) => {
64             await loadAnalysis(target);
65             setStatus(done.status === "FAILED" ? "error" : "done");
66             if (done.status === "FAILED") {
67               toast.error(`Analysis of ${target} failed`);
68             } else {
69               toast.success(`${target} analysed in ${Number(done.duration).toFixed(1)}s`);
70             }
71           },
72           onError: async () => {
73             // The stream can drop behind a proxy; the result is still on the API.
74             const payload = await loadAnalysis(target);
75             setStatus(payload ? "done" : "error");
76           },
77         });
78       } catch (err) {
```

```

79         setError(err);
80         setStatus("error");
81         toast.error(err.message);
82     }
83 },
84 [loadAnalysis]
85 );
86
87 const selectEcr = useCallback(
88   async (id) => {
89     const target = (id || "").trim().toUpperCase();
90     setEcrId(target);
91     setEvents([]);
92     setStatus("idle");
93     const payload = await loadAnalysis(target);
94     if (payload) setStatus("done");
95     else setAnalysis(null);
96   },
97   [loadAnalysis]
98 );
99
100 const approve = useCallback(
101   async (decision) => {
102     if (!ecrId) return;
103     await api.approve(ecrId, decision);
104     toast.success(decision.approved ? "Test suite approved" : "Recommendation rejected");
105   },
106   [ecrId]
107 );
108
109 // The approval checkpoint announces itself on the event stream.
110 useEffect(() => {
111   const last = events[events.length - 1];
112   if (last?.status === "AWAITING_APPROVAL") setStatus("awaiting_approval");
113   else if (last && status === "awaiting_approval" && last.agent_key === "human_approval") {
114     setStatus("running");
115   }
116 }, [events, status]);
117
118 const value = useMemo(
119   () => ({
120     ecrId,
121     setEcrId,
122     status,
123     workflowId,
124     events,
125     analysis,
126     error,
127     health,
128     startedAt,
129     analyze,
130     selectEcr,
131     approve,
132     loadAnalysis,
133     isDemoMode: Boolean(health?.demo_mode),
134   }),
135   [ecrId, status, workflowId, events, analysis, error, health, startedAt, analyze, selectEcr, approve, loadAnalysis]
136 );
137
138 return <AnalysisContext.Provider value={value}>{children}</AnalysisContext.Provider>;
139 }
140
141 export function useAnalysis() {
142   const context = useContext(AnalysisContext);
143   if (!context) throw new Error("useAnalysis must be used inside <AnalysisProvider>");
144   return context;
145 }
146
147 export default AnalysisContext;

```

frontend/src/services/api.js

```

1  /**
2   * Backend API client.
3   *
4   * Every call goes through `request()` so failures surface as a single Error
5   * shape the UI can render, and the base URL is resolved once from the
6   * environment (REACT_APP_BACKEND_URL) with a localhost fallback.
7   */
8   const BASE_URL = (process.env.REACT_APP_BACKEND_URL || "http://localhost:8000").replace(/\/$/, "");
9   const API = `${BASE_URL}/api`;
10
11   export class ApiError extends Error {

```

```

12   constructor(message, status, detail) {
13     super(message);
14     this.name = "ApiError";
15     this.status = status;
16     this.detail = detail;
17   }
18 }
19
20 async function request(path, { method = "GET", body, signal } = {}) {
21   let response;
22   try {
23     response = await fetch(`${API}${path}`, {
24       method,
25       headers: body ? { "Content-Type": "application/json" } : undefined,
26       body: body ? JSON.stringify(body) : undefined,
27       signal,
28     });
29   } catch (error) {
30     if (error.name === "AbortError") throw error;
31     throw new ApiError(
32       "Cannot reach the ECR Assistant backend. Is it running on " + BASE_URL + "?",
33       0,
34       error.message
35     );
36   }
37   const isJson = (response.headers.get("content-type") || "").includes("application/json");
38   const payload = isJson ? await response.json() : await response.text();
39   if (!response.ok) {
40     const detail = isJson ? payload?.detail : payload;
41     throw new ApiError(
42       typeof detail === "string" ? detail : `Request failed (${response.status})`,
43       response.status,
44       detail
45     );
46   }
47   return payload;
48 }
49
50 const id = (ecrId) => encodeURIComponent(ecrId);
51
52 export const api = {
53   baseUrl: BASE_URL,
54
55   health: () => request("/health"),
56   dashboardStats: () => request("/dashboard/stats"),
57
58   listEcrs: (search) => request(`/ecr${search ? `?search=${encodeURIComponent(search)}` : ""}`),
59   getEcr: (ecrId) => request(`/ecr/${id(ecrId)}`),
60   createEcr: (payload) => request("/ecr", { method: "POST", body: payload }),
61
62   analyze: (ecrId, options = {}) => request(`/ecr/${id(ecrId)}/analyze`, { method: "POST", body: options })
63 + ,
64   analysis: (ecrId) => request(`/ecr/${id(ecrId)}/analysis`),
65   approve: (ecrId, decision) => request(`/ecr/${id(ecrId)}/approve`, { method: "POST", body: decision }),
66   reportUrl: (ecrId, format) => `${API}/ecr/${id(ecrId)}/report?format=${format}`,
67
68   ask: (ecrId, question) => request(`/ecr/${id(ecrId)}/ask`, { method: "POST", body: { question } }),
69   feedback: (payload) => request("/feedback", { method: "POST", body: payload }),
70
71   /**
72    * Subscribe to the live agent event stream for a workflow.
73    * Returns an unsubscribe function.
74    */
75   streamWorkflow(workflowId, { onAgent, onDone, onError } = {}) {
76     const source = new EventSource(`${API}/workflows/${workflowId}/stream`);
77     source.addEventListener("agent", (event) => onAgent?.(JSON.parse(event.data)));
78     source.addEventListener("done", (event) => {
79       onDone?.(JSON.parse(event.data));
80       source.close();
81     });
82     source.onerror = () => {
83       onError?.(new ApiError("Lost the live connection to the workflow", 0));
84       source.close();
85     };
86     return () => source.close();
87   },
88 };
89 export default api;

```

frontend/src/theme/viz.js

```

1  /**
2  * Visualization palette.

```

```

3  *
4  * Validated with the data-viz palette validator against this app's own dark
5  * chart surface (#0f1626):
6  *   - categorical trio -> ALL CHECKS PASS (--pairs all)
7  *   - ordinal blue ramp -> ALL CHECKS PASS (--ordinal)
8  *   - status palette -> clears 3:1 contrast; warning/serious sit close in
9  *     CVD terms, so every status mark ships with a direct label (never colour
10 *     alone), which is the documented mitigation.
11 */
12 export const SURFACE = "#0f1626";
13 export const PAGE = "#080c17";
14
15 export const INK = {
16   primary: "#ffffff",
17   secondary: "#c3c2b7",
18   muted: "#8b93a7",
19   grid: "#1c2536",
20   axis: "#334155",
21 };
22
23 /** Categorical - assign in fixed order, never cycled. Cap at 3 for all-pairs forms. */
24 export const CATEGORICAL = ["#3987e5", "#d95926", "#199e70"];
25
26 /** Ordinal single-hue ramp, light -> dark. */
27 export const ORDINAL_BLUE = ["#86b6ef", "#5598e7", "#3987e5", "#2a78d6", "#184f95"];
28
29 /** Status - reserved for state, never reused as a series colour. */
30 export const STATUS = {
31   good: "#0ca30c",
32   warning: "#fab219",
33   serious: "#ec835a",
34   critical: "#d03b3b",
35   neutral: "#64748b",
36 };
37
38 /** "Can this past defect happen again?" */
39 export const CHANCE_COLOR = {
40   Low: STATUS.good,
41   Medium: STATUS.warning,
42   High: STATUS.critical,
43 };
44
45 export const PRIORITY_COLOR = {
46   P0: STATUS.critical,
47   P1: STATUS.serious,
48   P2: STATUS.warning,
49   P3: STATUS.neutral,
50 };
51
52 export const PRIORITIES = ["P0", "P1", "P2", "P3"];
53
54 export function chanceColor(chance) {
55   return CHANCE_COLOR[chance] || STATUS.neutral;
56 }
57
58 export function priorityColor(priority) {
59   return PRIORITY_COLOR[priority] || STATUS.neutral;
60 }

```

frontend/src/lib/utils.js

```

1  import { clsx } from "clsx";
2  import { twMerge } from "tailwind-merge"
3
4  export function cn(...inputs) {
5    return twMerge(clsx(inputs));
6  }

```

frontend/src/hooks/use-toast.js

```

1  "use client";
2  // Inspired by react-hot-toast library
3  import * as React from "react"
4
5  const TOAST_LIMIT = 1
6  const TOAST_REMOVE_DELAY = 1000000
7
8  const actionTypes = {
9    ADD_TOAST: "ADD_TOAST",
10   UPDATE_TOAST: "UPDATE_TOAST",
11   DISMISS_TOAST: "DISMISS_TOAST",
12   REMOVE_TOAST: "REMOVE_TOAST"
13 }
14

```

```

15 let count = 0
16
17 function genId() {
18   count = (count + 1) % Number.MAX_SAFE_INTEGER
19   return count.toString();
20 }
21
22 const toastTimeouts = new Map()
23
24 const addToRemoveQueue = (toastId) => {
25   if (toastTimeouts.has(toastId)) {
26     return
27   }
28
29   const timeout = setTimeout(() => {
30     toastTimeouts.delete(toastId)
31     dispatch({
32       type: "REMOVE_TOAST",
33       toastId: toastId,
34     })
35   }, TOAST_REMOVE_DELAY)
36
37   toastTimeouts.set(toastId, timeout)
38 }
39
40 export const reducer = (state, action) => {
41   switch (action.type) {
42     case "ADD_TOAST":
43       return {
44         ...state,
45         toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
46       };
47
48     case "UPDATE_TOAST":
49       return {
50         ...state,
51         toasts: state.toasts.map((t) =>
52           t.id === action.toast.id ? { ...t, ...action.toast } : t),
53       };
54
55     case "DISMISS_TOAST": {
56       const { toastId } = action
57
58       // ! Side effects ! - This could be extracted into a dismissToast() action,
59       // but I'll keep it here for simplicity
60       if (toastId) {
61         addToRemoveQueue(toastId)
62       } else {
63         state.toasts.forEach((toast) => {
64           addToRemoveQueue(toast.id)
65         })
66       }
67
68       return {
69         ...state,
70         toasts: state.toasts.map((t) =>
71           t.id === toastId || toastId === undefined
72             ? {
73                 ...t,
74                 open: false,
75               }
76             : t),
77       };
78     }
79     case "REMOVE_TOAST":
80       if (action.toastId === undefined) {
81         return {
82           ...state,
83           toasts: [],
84         }
85       }
86       return {
87         ...state,
88         toasts: state.toasts.filter((t) => t.id !== action.toastId),
89       };
90   }
91 }
92
93 const listeners = []
94
95 let memoryState = { toasts: [] }
96
97 function dispatch(action) {
98   memoryState = reducer(memoryState, action)

```

```

99   listeners.forEach((listener) => {
100     listener(memoryState)
101   })
102 }
103
104 function toast({
105   ...props
106 }) {
107   const id = genId()
108
109   const update = (props) =>
110     dispatch({
111       type: "UPDATE_TOAST",
112       toast: { ...props, id },
113     })
114   const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id })
115
116   dispatch({
117     type: "ADD_TOAST",
118     toast: {
119       ...props,
120       id,
121       open: true,
122       onOpenChange: (open) => {
123         if (!open) dismiss()
124       },
125     },
126   })
127
128   return {
129     id: id,
130     dismiss,
131     update,
132   }
133 }
134
135 function useToast() {
136   const [state, setState] = React.useState(memoryState)
137
138   React.useEffect(() => {
139     listeners.push(setState)
140     return () => {
141       const index = listeners.indexOf(setState)
142       if (index > -1) {
143         listeners.splice(index, 1)
144       }
145     };
146   }, [state])
147
148   return {
149     ...state,
150     toast,
151     dismiss: (toastId) => dispatch({ type: "DISMISS_TOAST", toastId }),
152   };
153 }
154
155 export { useToast, toast }

```

Frontend - shadcn/ui primitives

frontend/src/components/ui/accordion.jsx

```

1 import * as React from "react"
2 import * as AccordionPrimitive from "@radix-ui/react-accordion"
3 import { ChevronDown } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6
7 const Accordion = AccordionPrimitive.Root
8
9 const AccordionItem = React.forwardRef(({ className, ...props }, ref) => (
10   <AccordionPrimitive.Item ref={ref} className={cn("border-b", className)} {...props} />
11 ))
12 AccordionItem.displayName = "AccordionItem"
13
14 const AccordionTrigger = React.forwardRef(({ className, children, ...props }, ref) => (
15   <AccordionPrimitive.Header className="flex">
16     <AccordionPrimitive.Trigger
17       ref={ref}
18       className={cn(
19         "flex flex-1 items-center justify-between py-4 text-sm font-medium transition-all hover:underline t
20       + ext-left [&[data-state=open]>svg]:rotate-180",
21       className
22     )}

```

```

21     })
22     {...props}>
23     {children}
24     <ChevronDown
25       className="h-4 w-4 shrink-0 text-muted-foreground transition-transform duration-200" />
26   </AccordionPrimitive.Trigger>
27 </AccordionPrimitive.Header>
28 ))
29 AccordionTrigger.displayName = AccordionPrimitive.Trigger.displayName
30
31 const AccordionContent = React.forwardRef(({ className, children, ...props }, ref) => (
32   <AccordionPrimitive.Content
33     ref={ref}
34     className="overflow-hidden text-sm data-[state=closed]:animate-accordion-up data-[state=open]:animate-accordion-down"
35     {...props}>
36     <div className={cn("pb-4 pt-0", className)}>{children}</div>
37   </AccordionPrimitive.Content>
38 ))
39 AccordionContent.displayName = AccordionPrimitive.Content.displayName
40
41 export { Accordion, AccordionItem, AccordionTrigger, AccordionContent }

```

frontend/src/components/ui/alert-dialog.jsx

```

1 import * as React from "react"
2 import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog"
3
4 import { cn } from "@lib/Utils"
5 import { buttonVariants } from "@components/ui/button"
6
7 const AlertDialog = AlertDialogPrimitive.Root
8
9 const AlertDialogTrigger = AlertDialogPrimitive.Trigger
10
11 const AlertDialogPortal = AlertDialogPrimitive.Portal
12
13 const AlertDialogOverlay = React.forwardRef(({ className, ...props }, ref) => (
14   <AlertDialogPrimitive.Overlay
15     className={cn(
16       "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
17       className
18     )}
19     {...props}
20     ref={ref} />
21 ))
22 AlertDialogOverlay.displayName = AlertDialogPrimitive.Overlay.displayName
23
24 const AlertDialogContent = React.forwardRef(({ className, ...props }, ref) => (
25   <AlertDialogPortal>
26     <AlertDialogOverlay />
27     <AlertDialogPrimitive.Content
28       ref={ref}
29       className={cn(
30         "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border bg-background p-6 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2 data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
31         className
32       )}
33       {...props} />
34   </AlertDialogPortal>
35 ))
36 AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName
37
38 const AlertDialogHeader = ({
39   className,
40   ...props
41 }) => (
42   <div
43     className={cn("flex flex-col space-y-2 text-center sm:text-left", className)}
44     {...props} />
45 )
46 AlertDialogHeader.displayName = "AlertDialogHeader"
47
48 const AlertDialogFooter = ({
49   className,
50   ...props
51 }) => (
52   <div
53     className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
54     {...props} />

```

```

55 )
56 AlertDialogFooter.displayName = "AlertDialogFooter"
57
58 const AlertDialogTitle = React.forwardRef(({ className, ...props }, ref) => (
59   <AlertDialogPrimitive.Title ref={ref} className={cn("text-lg font-semibold", className)} {...props} />
60 ))
61 AlertDialogTitle.displayName = AlertDialogPrimitive.Title.displayName
62
63 const AlertDialogDescription = React.forwardRef(({ className, ...props }, ref) => (
64   <AlertDialogPrimitive.Description
65     ref={ref}
66     className={cn("text-sm text-muted-foreground", className)}
67     {...props} />
68 ))
69 AlertDialogDescription.displayName =
70   AlertDialogPrimitive.Description.displayName
71
72 const AlertDialogAction = React.forwardRef(({ className, ...props }, ref) => (
73   <AlertDialogPrimitive.Action ref={ref} className={cn(buttonVariants(), className)} {...props} />
74 ))
75 AlertDialogAction.displayName = AlertDialogPrimitive.Action.displayName
76
77 const AlertDialogCancel = React.forwardRef(({ className, ...props }, ref) => (
78   <AlertDialogPrimitive.Cancel
79     ref={ref}
80     className={cn(buttonVariants({ variant: "outline" }), "mt-2 sm:mt-0", className)}
81     {...props} />
82 ))
83 AlertDialogCancel.displayName = AlertDialogPrimitive.Cancel.displayName
84
85 export {
86   AlertDialog,
87   AlertDialogPortal,
88   AlertDialogOverlay,
89   AlertDialogTrigger,
90   AlertDialogContent,
91   AlertDialogHeader,
92   AlertDialogFooter,
93   AlertDialogTitle,
94   AlertDialogDescription,
95   AlertDialogAction,
96   AlertDialogCancel,
97 }

```

frontend/src/components/ui/alert.jsx

```

1 import * as React from "react"
2 import { cva } from "class-variance-authority";
3
4 import { cn } from "@lib/utils"
5
6 const alertVariants = cva(
7   "relative w-full rounded-lg border px-4 py-3 text-sm [&svg+div]:translate-y-[-3px] [&svg]:absolute [&svg]:left-4 [&svg]:top-4 [&svg]:text-foreground [&svg~*]:pl-7",
8   {
9     variants: {
10       variant: {
11         default: "bg-background text-foreground",
12         destructive:
13           "border-destructive/50 text-destructive dark:border-destructive [&svg]:text-destructive",
14       },
15       defaultVariants: {
16         variant: "default",
17       },
18     },
19   }
20 )
21
22 const Alert = React.forwardRef(({ className, variant, ...props }, ref) => (
23   <div
24     ref={ref}
25     role="alert"
26     className={cn(alertVariants({ variant }), className)}
27     {...props} />
28 ))
29 Alert.displayName = "Alert"
30
31 const AlertTitle = React.forwardRef(({ className, ...props }, ref) => (
32   <h5
33     ref={ref}
34     className={cn("mb-1 font-medium leading-none tracking-tight", className)}
35     {...props} />
36 ))
37 AlertTitle.displayName = "AlertTitle"

```

```

38
39 const AlertDescription = React.forwardRef(({ className, ...props }, ref) => (
40   <div
41     ref={ref}
42     className={cn("text-sm [&_p]:leading-relaxed", className)}
43     {...props} />
44   ))
45 AlertDescription.displayName = "AlertDescription"
46
47 export { Alert, AlertTitle, AlertDescription }

```

frontend/src/components/ui/aspect-ratio.jsx

```

1 import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio"
2
3 const AspectRatio = AspectRatioPrimitive.Root
4
5 export { AspectRatio }

```

frontend/src/components/ui/avatar.jsx

```

1 import * as React from "react"
2 import * as AvatarPrimitive from "@radix-ui/react-avatar"
3
4 import { cn } from "@lib/utils"
5
6 const Avatar = React.forwardRef(({ className, ...props }, ref) => (
7   <AvatarPrimitive.Root
8     ref={ref}
9     className={cn("relative flex h-10 w-10 shrink-0 overflow-hidden rounded-full", className)}
10    {...props} />
11 ))
12 Avatar.displayName = AvatarPrimitive.Root.displayName
13
14 const AvatarImage = React.forwardRef(({ className, ...props }, ref) => (
15   <AvatarPrimitive.Image
16     ref={ref}
17     className={cn("aspect-square h-full w-full", className)}
18     {...props} />
19 ))
20 AvatarImage.displayName = AvatarPrimitive.Image.displayName
21
22 const AvatarFallback = React.forwardRef(({ className, ...props }, ref) => (
23   <AvatarPrimitive.Fallback
24     ref={ref}
25     className={cn(
26       "flex h-full w-full items-center justify-center rounded-full bg-muted",
27       className
28     )}
29     {...props} />
30 ))
31 AvatarFallback.displayName = AvatarPrimitive.Fallback.displayName
32
33 export { Avatar, AvatarImage, AvatarFallback }

```

frontend/src/components/ui/badge.jsx

```

1 import * as React from "react"
2 import { cva } from "class-variance-authority";
3
4 import { cn } from "@lib/utils"
5
6 const badgeVariants = cva(
7   "inline-flex items-center rounded-md border px-2.5 py-0.5 text-xs font-semibold transition-colors focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2",
8   {
9     variants: {
10       variant: {
11         default:
12           "border-transparent bg-primary text-primary-foreground shadow hover:bg-primary/80",
13         secondary:
14           "border-transparent bg-secondary text-secondary-foreground hover:bg-secondary/80",
15         destructive:
16           "border-transparent bg-destructive text-destructive-foreground shadow hover:bg-destructive/80",
17         outline: "text-foreground",
18       },
19     },
20     defaultVariants: {
21       variant: "default",
22     },
23   }
24 )
25
26 function Badge({

```

```

27   className,
28   variant,
29   ...props
30 }) {
31   return (<div className={cn(badgeVariants({ variant }), className)} {...props} />);
32 }
33
34 export { Badge, badgeVariants }

```

frontend/src/components/ui/breadcrumb.jsx

```

1 import * as React from "react"
2 import { Slot } from "@radix-ui/react-slot"
3 import { ChevronRight, MoreHorizontal } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6
7 const Breadcrumb = React.forwardRef(
8   ({ ...props }, ref) => <nav ref={ref} aria-label="breadcrumb" {...props} />
9 )
10 Breadcrumb.displayName = "Breadcrumb"
11
12 const BreadcrumbList = React.forwardRef(({ className, ...props }, ref) => (
13   <ol
14     ref={ref}
15     className={cn(
16       "flex flex-wrap items-center gap-1.5 break-words text-sm text-muted-foreground sm:gap-2.5",
17       className
18     )}
19     {...props} />
20 ))
21 BreadcrumbList.displayName = "BreadcrumbList"
22
23 const BreadcrumbItem = React.forwardRef(({ className, ...props }, ref) => (
24   <li
25     ref={ref}
26     className={cn("inline-flex items-center gap-1.5", className)}
27     {...props} />
28 ))
29 BreadcrumbItem.displayName = "BreadcrumbItem"
30
31 const BreadcrumbLink = React.forwardRef(({ asChild, className, ...props }, ref) => {
32   const Comp = asChild ? Slot : "a"
33
34   return (
35     <Comp
36       ref={ref}
37       className={cn("transition-colors hover:text-foreground", className)}
38       {...props} />
39   );
40 })
41 BreadcrumbLink.displayName = "BreadcrumbLink"
42
43 const BreadcrumbPage = React.forwardRef(({ className, ...props }, ref) => (
44   <span
45     ref={ref}
46     role="link"
47     aria-disabled="true"
48     aria-current="page"
49     className={cn("font-normal text-foreground", className)}
50     {...props} />
51 ))
52 BreadcrumbPage.displayName = "BreadcrumbPage"
53
54 const BreadcrumbSeparator = ({
55   children,
56   className,
57   ...props
58 }) => (
59   <li
60     role="presentation"
61     aria-hidden="true"
62     className={cn("[&svg]:w-3.5 [&svg]:h-3.5", className)}
63     {...props}>
64     {children ?? <ChevronRight />}
65   </li>
66 )
67 BreadcrumbSeparator.displayName = "BreadcrumbSeparator"
68
69 const BreadcrumbEllipsis = ({
70   className,
71   ...props
72 }) => (
73   <span

```

```

74     role="presentation"
75     aria-hidden="true"
76     className={cn("flex h-9 w-9 items-center justify-center", className)}
77     {...props}>
78     <MoreHorizontal className="h-4 w-4" />
79     <span className="sr-only">More</span>
80   </span>
81 )
82 BreadcrumbEllipsis.displayName = "BreadcrumbEllipsis"
83
84 export {
85   Breadcrumb,
86   BreadcrumbList,
87   BreadcrumbItem,
88   BreadcrumbLink,
89   BreadcrumbPage,
90   BreadcrumbSeparator,
91   BreadcrumbEllipsis,
92 }

```

frontend/src/components/ui/button.jsx

```

1 import * as React from "react"
2 import { Slot } from "@radix-ui/react-slot"
3 import { cva } from "class-variance-authority";
4
5 import { cn } from "@lib/utils"
6
7 const buttonVariants = cva(
8   "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium transiti
+   n-colors focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-no
+   ne disabled:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
9   {
10     variants: {
11       variant: {
12         default:
13           "bg-primary text-primary-foreground shadow hover:bg-primary/90",
14         destructive:
15           "bg-destructive text-destructive-foreground shadow-sm hover:bg-destructive/90",
16         outline:
17           "border border-input shadow-sm hover:bg-accent hover:text-accent-foreground",
18         secondary:
19           "bg-secondary text-secondary-foreground shadow-sm hover:bg-secondary/80",
20         ghost: "hover:bg-accent hover:text-accent-foreground",
21         link: "text-primary underline-offset-4 hover:underline",
22       },
23       size: {
24         default: "h-9 px-4 py-2",
25         sm: "h-8 rounded-md px-3 text-xs",
26         lg: "h-10 rounded-md px-8",
27         icon: "h-9 w-9",
28       },
29     },
30     defaultVariants: {
31       variant: "default",
32       size: "default",
33     },
34   }
35 )
36
37 const Button = React.forwardRef(({ className, variant, size, asChild = false, ...props }, ref) => {
38   const Comp = asChild ? Slot : "button"
39   return (
40     <Comp
41       className={cn(buttonVariants({ variant, size, className }))}
42       ref={ref}
43       {...props} />
44   );
45 })
46 Button.displayName = "Button"
47
48 export { Button, buttonVariants }

```

frontend/src/components/ui/calendar.jsx

```

1 import * as React from "react"
2 import { ChevronLeft, ChevronRight } from "lucide-react"
3 import { DayPicker } from "react-day-picker"
4
5 import { cn } from "@lib/utils"
6 import { buttonVariants } from "@components/ui/button"
7
8 function Calendar({
9   className,

```

```

10   classNames,
11   showOutsideDays = true,
12   ...props
13 } {
14   return (
15     <DatePicker
16       showOutsideDays={showOutsideDays}
17       className={cn("p-3", className)}
18       classNames={{
19         months: "flex flex-col sm:flex-row space-y-4 sm:space-x-4 sm:space-y-0",
20         month: "space-y-4",
21         caption: "flex justify-center pt-1 relative items-center",
22         caption_label: "text-sm font-medium",
23         nav: "space-x-1 flex items-center",
24         nav_button: cn(
25           buttonVariants({ variant: "outline" }),
26           "h-7 w-7 bg-transparent p-0 opacity-50 hover:opacity-100"
27         ),
28         nav_button_previous: "absolute left-1",
29         nav_button_next: "absolute right-1",
30         table: "w-full border-collapse space-y-1",
31         head_row: "flex",
32         head_cell:
33           "text-muted-foreground rounded-md w-8 font-normal text-[0.8rem]",
34         row: "flex w-full mt-2",
35         cell: cn(
36           "relative p-0 text-center text-sm focus-within:relative focus-within:z-20 [&:has([aria-selected])
37 +       ]:bg-accent [&:has([aria-selected].day-outside):bg-accent/50 [&:has([aria-selected].day-range-end)]:roun
38 +       d-r-md",
39         props.mode === "range"
40         ? "[&:has(>.day-range-end)]:rounded-r-md [&:has(>.day-range-start)]:rounded-l-md first:[&:has([
41 +       aria-selected])]:rounded-l-md last:[&:has([aria-selected])]:rounded-r-md"
42         : "[&:has([aria-selected])]:rounded-md"
43       ),
44         day: cn(
45           buttonVariants({ variant: "ghost" }),
46           "h-8 w-8 p-0 font-normal aria-selected:opacity-100"
47         ),
48         day_range_start: "day-range-start",
49         day_range_end: "day-range-end",
50         day_selected:
51           "bg-primary text-primary-foreground hover:bg-primary hover:text-primary-foreground focus:bg-prima
52 +       ry focus:text-primary-foreground",
53         day_today: "bg-accent text-accent-foreground",
54         day_outside:
55           "day-outside text-muted-foreground aria-selected:bg-accent/50 aria-selected:text-muted-foreground
56 +       ",
57         day_disabled: "text-muted-foreground opacity-50",
58         day_range_middle:
59           "aria-selected:bg-accent aria-selected:text-accent-foreground",
60         day_hidden: "invisible",
61         ...classNames,
62       }}
63     components={{
64       IconLeft: ({ className, ...props }) => (
65         <ChevronLeft className={cn("h-4 w-4", className)} {...props} />
66       ),
67       IconRight: ({ className, ...props }) => (
68         <ChevronRight className={cn("h-4 w-4", className)} {...props} />
69       ),
70     }}
71     {...props} />
72   );
73 }
74 Calendar.displayName = "Calendar"
75
76 export { Calendar }

```

frontend/src/components/ui/card.jsx

```

1  import * as React from "react"
2
3  import { cn } from "@lib/utils"
4
5  const Card = React.forwardRef(({ className, ...props }, ref) => (
6    <div
7      ref={ref}
8      className={cn("rounded-xl border bg-card text-card-foreground shadow", className)}
9      {...props} />
10  ))
11  Card.displayName = "Card"
12
13  const CardHeader = React.forwardRef(({ className, ...props }, ref) => (
14    <div

```

```

15     ref={ref}
16     className={cn("flex flex-col space-y-1.5 p-6", className)}
17     {...props} />
18   ))
19   CardHeader.displayName = "CardHeader"
20
21   const CardTitle = React.forwardRef(({ className, ...props }, ref) => (
22     <div
23       ref={ref}
24       className={cn("font-semibold leading-none tracking-tight", className)}
25       {...props} />
26     ))
27   CardTitle.displayName = "CardTitle"
28
29   const CardDescription = React.forwardRef(({ className, ...props }, ref) => (
30     <div
31       ref={ref}
32       className={cn("text-sm text-muted-foreground", className)}
33       {...props} />
34     ))
35   CardDescription.displayName = "CardDescription"
36
37   const CardContent = React.forwardRef(({ className, ...props }, ref) => (
38     <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
39   ))
40   CardContent.displayName = "CardContent"
41
42   const CardFooter = React.forwardRef(({ className, ...props }, ref) => (
43     <div
44       ref={ref}
45       className={cn("flex items-center p-6 pt-0", className)}
46       {...props} />
47     ))
48   CardFooter.displayName = "CardFooter"
49
50   export { Card, CardHeader, CardFooter, CardTitle, CardDescription, CardContent }

```

frontend/src/components/ui/carousel.jsx

```

1  import * as React from "react"
2  import useEmblaCarousel from "embla-carousel-react";
3  import { ArrowLeft, ArrowRight } from "lucide-react"
4
5  import { cn } from "@lib/Utils"
6  import { Button } from "@components/ui/button"
7
8  const CarouselContext = React.createContext(null)
9
10 function useCarousel() {
11   const context = React.useContext(CarouselContext)
12
13   if (!context) {
14     throw new Error("useCarousel must be used within a <Carousel />")
15   }
16
17   return context
18 }
19
20 const Carousel = React.forwardRef((
21   {
22     orientation = "horizontal",
23     opts,
24     setApi,
25     plugins,
26     className,
27     children,
28     ...props
29   },
30   ref
31 ) => {
32   const [carouselRef, api] = useEmblaCarousel({
33     ...opts,
34     axis: orientation === "horizontal" ? "x" : "y",
35   }, plugins)
36   const [canScrollPrev, setCanScrollPrev] = React.useState(false)
37   const [canScrollNext, setCanScrollNext] = React.useState(false)
38
39   const onSelect = React.useCallback((api) => {
40     if (!api) {
41       return
42     }
43
44     setCanScrollPrev(api.canScrollPrev())
45     setCanScrollNext(api.canScrollNext())

```

```

46   }, [])
47
48   const scrollPrev = React.useCallback(() => {
49     api?.scrollPrev()
50   }, [api])
51
52   const scrollNext = React.useCallback(() => {
53     api?.scrollNext()
54   }, [api])
55
56   const handleKeyDown = React.useCallback((event) => {
57     if (event.key === "ArrowLeft") {
58       event.preventDefault()
59       scrollPrev()
60     } else if (event.key === "ArrowRight") {
61       event.preventDefault()
62       scrollNext()
63     }
64   }, [scrollPrev, scrollNext])
65
66   React.useEffect(() => {
67     if (!api || !setApi) {
68       return
69     }
70
71     setApi(api)
72   }, [api, setApi])
73
74   React.useEffect(() => {
75     if (!api) {
76       return
77     }
78
79     onSelect(api)
80     api.on("reInit", onSelect)
81     api.on("select", onSelect)
82
83     return () => {
84       api?.off("select", onSelect)
85     };
86   }, [api, onSelect])
87
88   return (
89     <CarouselContext.Provider
90       value={{
91         carouselRef,
92         api: api,
93         opts,
94         orientation:
95           orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
96         scrollPrev,
97         scrollNext,
98         canScrollPrev,
99         canScrollNext,
100       }}>
101       <div
102         ref={ref}
103         onKeyDownCapture={handleKeyDown}
104         className={cn("relative", className)}
105         role="region"
106         aria-roledescription="carousel"
107         {...props}>
108         {children}
109       </div>
110     </CarouselContext.Provider>
111   );
112 })
113 Carousel.displayName = "Carousel"
114
115 const CarouselContent = React.forwardRef(({ className, ...props }, ref) => {
116   const { carouselRef, orientation } = useCarousel()
117
118   return (
119     <div ref={carouselRef} className="overflow-hidden">
120       <div
121         ref={ref}
122         className={cn(
123           "flex",
124           orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
125           className
126         )}
127         {...props} />
128     </div>
129   );

```

```

130 })
131 CarouselContent.displayName = "CarouselContent"
132
133 const CarouselItem = React.forwardRef(({ className, ...props }, ref) => {
134   const { orientation } = useCarousel()
135
136   return (
137     <div
138       ref={ref}
139       role="group"
140       aria-roledescription="slide"
141       className={cn(
142         "min-w-0 shrink-0 grow-0 basis-full",
143         orientation === "horizontal" ? "pl-4" : "pt-4",
144         className
145       )}
146       {...props} />
147   );
148 })
149 CarouselItem.displayName = "CarouselItem"
150
151 const CarouselPrevious = React.forwardRef(({ className, variant = "outline", size = "icon", ...props }, ref
+
152 ) => {
153   const { orientation, scrollPrev, canScrollPrev } = useCarousel()
154
155   return (
156     <Button
157       ref={ref}
158       variant={variant}
159       size={size}
160       className={cn("absolute h-8 w-8 rounded-full", orientation === "horizontal"
161         ? "-left-12 top-1/2 -translate-y-1/2"
162         : "-top-12 left-1/2 -translate-x-1/2 rotate-90", className)}
163       disabled={!canScrollPrev}
164       onClick={scrollPrev}
165       {...props}>
166       <ArrowLeft className="h-4 w-4" />
167       <span className="sr-only">Previous slide</span>
168     </Button>
169   );
170 })
171 CarouselPrevious.displayName = "CarouselPrevious"
172
173 const CarouselNext = React.forwardRef(({ className, variant = "outline", size = "icon", ...props }, ref) =>
+
174 {
175   const { orientation, scrollNext, canScrollNext } = useCarousel()
176
177   return (
178     <Button
179       ref={ref}
180       variant={variant}
181       size={size}
182       className={cn("absolute h-8 w-8 rounded-full", orientation === "horizontal"
183         ? "-right-12 top-1/2 -translate-y-1/2"
184         : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90", className)}
185       disabled={!canScrollNext}
186       onClick={scrollNext}
187       {...props}>
188       <ArrowRight className="h-4 w-4" />
189       <span className="sr-only">Next slide</span>
190     </Button>
191   );
192 })
193 CarouselNext.displayName = "CarouselNext"
194
195 export { Carousel, CarouselContent, CarouselItem, CarouselPrevious, CarouselNext };

```

frontend/src/components/ui/checkbox.jsx

```

1 import * as React from "react"
2 import * as CheckboxPrimitive from "@radix-ui/react-checkbox"
3 import { Check } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6
7 const Checkbox = React.forwardRef(({ className, ...props }, ref) => (
8   <CheckboxPrimitive.Root
9     ref={ref}
10     className={cn(
11       "peer h-4 w-4 shrink-0 rounded-sm border border-primary shadow focus-visible:outline-none focus-visib
+
12       le:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50 data-[state=checked]:bg-p
+
13       rimary data-[state=checked]:text-primary-foreground",
14       className
15     )}
16   >
17     <CheckboxPrimitive.Input type="checkbox"/>
18     <Check/>
19   </CheckboxPrimitive.Root>
20 )

```

```

14     {...props}>
15     <CheckboxPrimitive.Indicator className={cn("flex items-center justify-center text-current")}>
16       <Check className="h-4 w-4" />
17     </CheckboxPrimitive.Indicator>
18   </CheckboxPrimitive.Root>
19 )}
20 Checkbox.displayName = CheckboxPrimitive.Root.displayName
21
22 export { Checkbox }

```

frontend/src/components/ui/collapsible.jsx

```

1 import * as CollapsiblePrimitive from "@radix-ui/react-collapsible"
2
3 const Collapsible = CollapsiblePrimitive.Root
4
5 const CollapsibleTrigger = CollapsiblePrimitive.CollapsibleTrigger
6
7 const CollapsibleContent = CollapsiblePrimitive.CollapsibleContent
8
9 export { Collapsible, CollapsibleTrigger, CollapsibleContent }

```

frontend/src/components/ui/command.jsx

```

1 import * as React from "react"
2 import { Command as CommandPrimitive } from "cmdk"
3 import { Search } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6 import { Dialog, DialogContent } from "@components/ui/dialog"
7
8 const Command = React.forwardRef(({ className, ...props }, ref) => (
9   <CommandPrimitive
10     ref={ref}
11     className={cn(
12       "flex h-full w-full flex-col overflow-hidden rounded-md bg-popover text-popover-foreground",
13       className
14     )}
15     {...props} />
16 ))
17 Command.displayName = CommandPrimitive.displayName
18
19 const CommandDialog = ({
20   children,
21   ...props
22 }) => {
23   return (
24     <Dialog {...props}>
25       <DialogContent className="overflow-hidden p-0">
26         <Command
27           className="[&_cmdk-group-heading]:px-2 [&_cmdk-group-heading]:font-medium [&_cmdk-group-head
28 + ing]:text-muted-foreground [&_cmdk-group]:not([hidden])~[cmdk-group]:pt-0 [&_cmdk-group]:px-2 [&_cmd
29 + k-input-wrapper]:h-5 [&_cmdk-input-wrapper]:w-5 [&_cmdk-input]:h-12 [&_cmdk-item]:px-2 [&_c
30 + mdk-item]:py-3 [&_cmdk-item]:h-5 [&_cmdk-item]:w-5"
31           {children}
32         </Command>
33       </DialogContent>
34     </Dialog>
35   );
36 }
37
38 const CommandInput = React.forwardRef(({ className, ...props }, ref) => (
39   <div className="flex items-center border-b px-3 cmdk-input-wrapper">
40     <Search className="mr-2 h-4 w-4 shrink-0 opacity-50" />
41     <CommandPrimitive.Input
42       ref={ref}
43       className={cn(
44         "flex h-10 w-full rounded-md bg-transparent py-3 text-sm outline-none placeholder:text-muted-foregr
45 + ound disabled:cursor-not-allowed disabled:opacity-50",
46         className
47       )}
48       {...props} />
49   </div>
50 ))
51 CommandInput.displayName = CommandPrimitive.Input.displayName
52
53 const CommandList = React.forwardRef(({ className, ...props }, ref) => (
54   <CommandPrimitive.List
55     ref={ref}
56     className={cn("max-h-[300px] overflow-y-auto overflow-x-hidden", className)}
57     {...props} />
58 ))
59
60

```

```

57 CommandList.displayName = CommandPrimitive.List.displayName
58
59 const CommandEmpty = React.forwardRef((props, ref) => (
60   <CommandPrimitive.Empty ref={ref} className="py-6 text-center text-sm" {...props} />
61 ))
62
63 CommandEmpty.displayName = CommandPrimitive.Empty.displayName
64
65 const CommandGroup = React.forwardRef(({ className, ...props }, ref) => (
66   <CommandPrimitive.Group
67     ref={ref}
68     className={cn(
69       "overflow-hidden p-1 text-foreground [&_cmdk-group-heading]:px-2 [&_cmdk-group-heading]:py-1.5 [&
70 + _cmdk-group-heading]:text-xs [&_cmdk-group-heading]:font-medium [&_cmdk-group-heading]:text-muted-for
71 + eground",
72     className
73   )}
74   {...props} />
75 ))
76
77 CommandGroup.displayName = CommandPrimitive.Group.displayName
78
79 const CommandSeparator = React.forwardRef(({ className, ...props }, ref) => (
80   <CommandPrimitive.Separator ref={ref} className={cn("-mx-1 h-px bg-border", className)} {...props} />
81 ))
82
83 CommandSeparator.displayName = CommandPrimitive.Separator.displayName
84
85 const CommandItem = React.forwardRef(({ className, ...props }, ref) => (
86   <CommandPrimitive.Item
87     ref={ref}
88     className={cn(
89       "relative flex cursor-default gap-2 select-none items-center rounded-sm px-2 py-1.5 text-sm outline-n
90 + one data-[disabled=true]:pointer-events-none data-[selected=true]:bg-accent data-[selected=true]:text-accen
91 + t-foreground data-[disabled=true]:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
92     className
93   )}
94   {...props} />
95 ))
96
97 CommandItem.displayName = CommandPrimitive.Item.displayName
98
99 const CommandShortcut = ({
100   className,
101   ...props
102 }) => {
103   return (
104     <span
105       className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
106       {...props} />
107   );
108 }
109
110 CommandShortcut.displayName = "CommandShortcut"
111
112 export {
113   Command,
114   CommandDialog,
115   CommandInput,
116   CommandList,
117   CommandEmpty,
118   CommandGroup,
119   CommandItem,
120   CommandShortcut,
121   CommandSeparator,
122 }

```

frontend/src/components/ui/context-menu.jsx

```

1 import * as React from "react"
2 import * as ContextMenuPrimitive from "@radix-ui/react-context-menu"
3 import { Check, ChevronRight, Circle } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6
7 const ContextMenu = ContextMenuPrimitive.Root
8
9 const ContextMenuTrigger = ContextMenuPrimitive.Trigger
10
11 const ContextMenuGroup = ContextMenuPrimitive.Group
12
13 const ContextMenuPortal = ContextMenuPrimitive.Portal
14
15 const ContextMenuSub = ContextMenuPrimitive.Sub
16
17 const ContextMenuRadioGroup = ContextMenuPrimitive.RadioGroup

```

```

18
19 const ContextMenuSubTrigger = React.forwardRef(({ className, inset, children, ...props }, ref) => (
20   <ContextMenuPrimitive.SubTrigger
21     ref={ref}
22     className={cn(
23       "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-ac
+ cent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
24       inset && "pl-8",
25       className
26     )}
27     {...props}>
28     {children}
29     <ChevronRight className="ml-auto h-4 w-4" />
30   </ContextMenuPrimitive.SubTrigger>
31 )
32 ContextMenuSubTrigger.displayName = ContextMenuPrimitive.SubTrigger.displayName
33
34 const ContextMenuSubContent = React.forwardRef(({ className, ...props }, ref) => (
35   <ContextMenuPrimitive.SubContent
36     ref={ref}
37     className={cn(
38       "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg
+ data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=op
+ en]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from
+ -top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide
+ in-from-bottom-2 origin-[--radix-context-menu-content-transform-origin]",
39       className
40     )}
41     {...props} />
42 )
43 ContextMenuSubContent.displayName = ContextMenuPrimitive.SubContent.displayName
44
45 const ContextMenuContent = React.forwardRef(({ className, ...props }, ref) => (
46   <ContextMenuPrimitive.Portal>
47     <ContextMenuPrimitive.Content
48       ref={ref}
49       className={cn(
50         "z-50 max-h-[--radix-context-menu-content-available-height] min-w-[8rem] overflow-y-auto overflow-x
+ -hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md data-[state=open]:animate-in dat
+ a-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]
+ :zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-
+ from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-
+ context-menu-content-transform-origin]",
51         className
52       )}
53       {...props} />
54   </ContextMenuPrimitive.Portal>
55 )
56 ContextMenuContent.displayName = ContextMenuPrimitive.Content.displayName
57
58 const ContextMenuItem = React.forwardRef(({ className, inset, ...props }, ref) => (
59   <ContextMenuPrimitive.Item
60     ref={ref}
61     className={cn(
62       "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none fo
+ cus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
63       inset && "pl-8",
64       className
65     )}
66     {...props} />
67 )
68 ContextMenuItem.displayName = ContextMenuPrimitive.Item.displayName
69
70 const ContextMenuCheckboxItem = React.forwardRef(({ className, children, checked, ...props }, ref) => (
71   <ContextMenuPrimitive.CheckboxItem
72     ref={ref}
73     className={cn(
74       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-no
+ ne focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity
+ -50",
75       className
76     )}
77     checked={checked}
78     {...props}>
79     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
80       <ContextMenuPrimitive.ItemIndicator>
81         <Check className="h-4 w-4" />
82       </ContextMenuPrimitive.ItemIndicator>
83     </span>
84     {children}
85   </ContextMenuPrimitive.CheckboxItem>
86 )
87 ContextMenuCheckboxItem.displayName =
88   ContextMenuPrimitive.CheckboxItem.displayName

```

```

89
90 const ContextMenuRadioItem = React.forwardRef(({ className, children, ...props }, ref) => (
91   <ContextMenuPrimitive.RadioItem
92     ref={ref}
93     className={cn(
94       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-no
+ ne focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity
+ -50",
95     className
96   )}
97   {...props}>
98     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
99       <ContextMenuPrimitive.ItemIndicator>
100         <Circle className="h-4 w-4 fill-current" />
101       </ContextMenuPrimitive.ItemIndicator>
102     </span>
103     {children}
104   </ContextMenuPrimitive.RadioItem>
105 ))
106 ContextMenuRadioItem.displayName = ContextMenuPrimitive.RadioItem.displayName
107
108 const ContextMenuLabel = React.forwardRef(({ className, inset, ...props }, ref) => (
109   <ContextMenuPrimitive.Label
110     ref={ref}
111     className={cn(
112       "px-2 py-1.5 text-sm font-semibold text-foreground",
113       inset && "pl-8",
114       className
115     )}
116     {...props} />
117 ))
118 ContextMenuLabel.displayName = ContextMenuPrimitive.Label.displayName
119
120 const ContextMenuSeparator = React.forwardRef(({ className, ...props }, ref) => (
121   <ContextMenuPrimitive.Separator
122     ref={ref}
123     className={cn("-mx-1 my-1 h-px bg-border", className)}
124     {...props} />
125 ))
126 ContextMenuSeparator.displayName = ContextMenuPrimitive.Separator.displayName
127
128 const ContextMenuShortcut = ({
129   className,
130   ...props
131 }) => {
132   return (
133     <span
134       className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
135       {...props} />
136   );
137 }
138 ContextMenuShortcut.displayName = "ContextMenuShortcut"
139
140 export {
141   ContextMenu,
142   ContextMenuTrigger,
143   ContextMenuContent,
144   ContextMenuItem,
145   ContextMenuCheckboxItem,
146   ContextMenuRadioItem,
147   ContextMenuLabel,
148   ContextMenuSeparator,
149   ContextMenuShortcut,
150   ContextMenuGroup,
151   ContextMenuPortal,
152   ContextMenuSub,
153   ContextMenuSubContent,
154   ContextMenuSubTrigger,
155   ContextMenuRadioGroup,
156 }

```

frontend/src/components/ui/dialog.jsx

```

1 import * as React from "react"
2 import * as DialogPrimitive from "@radix-ui/react-dialog"
3 import { X } from "lucide-react"
4
5 import { cn } from "@/lib/utils"
6
7 const Dialog = DialogPrimitive.Root
8
9 const DialogTrigger = DialogPrimitive.Trigger
10
11 const DialogPortal = DialogPrimitive.Portal

```

```

12
13 const DialogClose = DialogPrimitive.Close
14
15 const DialogOverlay = React.forwardRef(({ className, ...props }, ref) => (
16   <DialogPrimitive.Overlay
17     ref={ref}
18     className={cn(
19       "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[s
20 + tate=closed]:fade-out-0 data-[state=open]:fade-in-0",
21     className
22   )}
23   {...props} />
24 ))
25 DialogOverlay.displayName = DialogPrimitive.Overlay.displayName
26
27 const DialogContent = React.forwardRef(({ className, children, ...props }, ref) => (
28   <DialogPortal>
29     <DialogOverlay />
30     <DialogPrimitive.Content
31       ref={ref}
32       className={cn(
33         "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 b
34 + order bg-background p-6 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out
35 + data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=ope
36 + n]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-[st
37 + ate=open]:slide-in-from-left-1/2 data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
38       className
39     )}
40     {...props}>
41       <children>
42     </DialogPrimitive.Content>
43   </DialogPortal>
44 ))
45 DialogContent.displayName = DialogPrimitive.Content.displayName
46
47 const DialogHeader = ({
48   className,
49   ...props
50 }) => (
51   <div
52     className={cn("flex flex-col space-y-1.5 text-center sm:text-left", className)}
53     {...props} />
54 )
55 DialogHeader.displayName = "DialogHeader"
56
57 const DialogFooter = ({
58   className,
59   ...props
60 }) => (
61   <div
62     className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
63     {...props} />
64 )
65 DialogFooter.displayName = "DialogFooter"
66
67 const DialogTitle = React.forwardRef(({ className, ...props }, ref) => (
68   <DialogPrimitive.Title
69     ref={ref}
70     className={cn("text-lg font-semibold leading-none tracking-tight", className)}
71     {...props} />
72 ))
73 DialogTitle.displayName = DialogPrimitive.Title.displayName
74
75 const DialogDescription = React.forwardRef(({ className, ...props }, ref) => (
76   <DialogPrimitive.Description
77     ref={ref}
78     className={cn("text-sm text-muted-foreground", className)}
79     {...props} />
80 ))
81 DialogDescription.displayName = DialogPrimitive.Description.displayName
82
83 export {
84   Dialog,
85   DialogPortal,
86   DialogOverlay,
87   DialogTrigger,
88   DialogClose,

```

```

89   DialogContent,
90   DialogHeader,
91   DialogFooter,
92   DialogTitle,
93   DialogDescription,
94 }

```

frontend/src/components/ui/drawer.jsx

```

1  import * as React from "react"
2  import { Drawer as DrawerPrimitive } from "vaul"
3
4  import { cn } from "@lib/utils"
5
6  const Drawer = ({
7    shouldScaleBackground = true,
8    ...props
9  }) => (
10    <DrawerPrimitive.Root shouldScaleBackground={shouldScaleBackground} {...props} />
11  )
12  Drawer.displayName = "Drawer"
13
14  const DrawerTrigger = DrawerPrimitive.Trigger
15
16  const DrawerPortal = DrawerPrimitive.Portal
17
18  const DrawerClose = DrawerPrimitive.Close
19
20  const DrawerOverlay = React.forwardRef(({ className, ...props }, ref) => (
21    <DrawerPrimitive.Overlay
22      ref={ref}
23      className={cn("fixed inset-0 z-50 bg-black/80", className)}
24      {...props} />
25  ))
26  DrawerOverlay.displayName = DrawerPrimitive.Overlay.displayName
27
28  const DrawerContent = React.forwardRef(({ className, children, ...props }, ref) => (
29    <DrawerPortal>
30      <DrawerOverlay />
31      <DrawerPrimitive.Content
32        ref={ref}
33        className={cn(
34          "fixed inset-x-0 bottom-0 z-50 mt-24 flex h-auto flex-col rounded-t-[10px] border bg-background",
35          className
36        )}
37        {...props}>
38        <div className="mx-auto mt-4 h-2 w-[100px] rounded-full bg-muted" />
39        {children}
40      </DrawerPrimitive.Content>
41    </DrawerPortal>
42  ))
43  DrawerContent.displayName = "DrawerContent"
44
45  const DrawerHeader = ({
46    className,
47    ...props
48  }) => (
49    <div
50      className={cn("grid gap-1.5 p-4 text-center sm:text-left", className)}
51      {...props} />
52  )
53  DrawerHeader.displayName = "DrawerHeader"
54
55  const DrawerFooter = ({
56    className,
57    ...props
58  }) => (
59    <div className={cn("mt-auto flex flex-col gap-2 p-4", className)} {...props} />
60  )
61  DrawerFooter.displayName = "DrawerFooter"
62
63  const DrawerTitle = React.forwardRef(({ className, ...props }, ref) => (
64    <DrawerPrimitive.Title
65      ref={ref}
66      className={cn("text-lg font-semibold leading-none tracking-tight", className)}
67      {...props} />
68  ))
69  DrawerTitle.displayName = DrawerPrimitive.Title.displayName
70
71  const DrawerDescription = React.forwardRef(({ className, ...props }, ref) => (
72    <DrawerPrimitive.Description
73      ref={ref}
74      className={cn("text-sm text-muted-foreground", className)}
75      {...props} />

```

```

76 ))
77 DrawerDescription.displayName = DrawerPrimitive.Description.displayName
78
79 export {
80   Drawer,
81   DrawerPortal,
82   DrawerOverlay,
83   DrawerTrigger,
84   DrawerClose,
85   DrawerContent,
86   DrawerHeader,
87   DrawerFooter,
88   DrawerTitle,
89   DrawerDescription,
90 }

```

frontend/src/components/ui/dropdown-menu.jsx

```

1 import * as React from "react"
2 import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu"
3 import { Check, ChevronRight, Circle } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6
7 const DropdownMenu = DropdownMenuPrimitive.Root
8
9 const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger
10
11 const DropdownMenuGroup = DropdownMenuPrimitive.Group
12
13 const DropdownMenuPortal = DropdownMenuPrimitive.Portal
14
15 const DropdownMenuSub = DropdownMenuPrimitive.Sub
16
17 const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup
18
19 const DropdownMenuSubTrigger = React.forwardRef(({ className, inset, children, ...props }, ref) => (
20   <DropdownMenuPrimitive.SubTrigger
21     ref={ref}
22     className={cn(
23       "flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-none focus
24       + :bg-accent data-[state=open]:bg-accent [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
25       inset && "pl-8",
26       className
27     )}
28     {...props}>
29     {children}
30     <ChevronRight className="ml-auto" />
31   </DropdownMenuPrimitive.SubTrigger>
32 ))
33 DropdownMenuSubTrigger.displayName =
34   DropdownMenuPrimitive.SubTrigger.displayName
35
36 const DropdownMenuSubContent = React.forwardRef(({ className, ...props }, ref) => (
37   <DropdownMenuPrimitive.SubContent
38     ref={ref}
39     className={cn(
40       "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg
41       + data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-dropdown-menu-content-transform-origin]",
42       className
43     )}
44     {...props} />
45 ))
46 DropdownMenuSubContent.displayName =
47   DropdownMenuPrimitive.SubContent.displayName
48
49 const DropdownMenuContent = React.forwardRef(({ className, sideOffset = 4, ...props }, ref) => (
50   <DropdownMenuPrimitive.Portal>
51     <DropdownMenuPrimitive.Content
52       ref={ref}
53       sideOffset={sideOffset}
54       className={cn(
55         "z-50 max-h-[var(--radix-dropdown-menu-content-available-height)] min-w-[8rem] overflow-y-auto over
56         + flow-x-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md",
57         "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-dropdown-menu-content-transform-origin]",
58         className
59       )}
60     </DropdownMenuPrimitive.Content>
61   </DropdownMenuPrimitive.Portal>
62 ))
63 DropdownMenuContent.displayName =
64   DropdownMenuPrimitive.Content.displayName

```

```

58     </DropdownMenuPrimitive.Portal>
59   ))
60   DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName
61
62   const DropdownMenuItem = React.forwardRef(({ className, inset, ...props }, ref) => (
63     <DropdownMenuPrimitive.Item
64       ref={ref}
65       className={cn(
66         "relative flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-n
+ one transition-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data
+ -[disabled]:opacity-50 [&svg]:size-4 [&svg]:shrink-0",
67       inset && "pl-8",
68       className
69     )}
70     {...props} />
71   ))
72   DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName
73
74   const DropdownMenuCheckboxItem = React.forwardRef(({ className, children, checked, ...props }, ref) => (
75     <DropdownMenuPrimitive.CheckboxItem
76       ref={ref}
77       className={cn(
78         "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-no
+ ne transition-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-
+ [disabled]:opacity-50",
79       className
80     )}
81     checked={checked}
82     {...props}>
83     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
84       <DropdownMenuPrimitive.ItemIndicator>
85         <Check className="h-4 w-4" />
86       </DropdownMenuPrimitive.ItemIndicator>
87     </span>
88     {children}
89   </DropdownMenuPrimitive.CheckboxItem>
90   ))
91   DropdownMenuCheckboxItem.displayName =
92     DropdownMenuPrimitive.CheckboxItem.displayName
93
94   const DropdownMenuRadioItem = React.forwardRef(({ className, children, ...props }, ref) => (
95     <DropdownMenuPrimitive.RadioItem
96       ref={ref}
97       className={cn(
98         "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-no
+ ne transition-colors focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-
+ [disabled]:opacity-50",
99       className
100     )}
101     {...props}>
102     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
103       <DropdownMenuPrimitive.ItemIndicator>
104         <Circle className="h-2 w-2 fill-current" />
105       </DropdownMenuPrimitive.ItemIndicator>
106     </span>
107     {children}
108   </DropdownMenuPrimitive.RadioItem>
109   ))
110   DropdownMenuRadioItem.displayName = DropdownMenuPrimitive.RadioItem.displayName
111
112   const DropdownMenuLabel = React.forwardRef(({ className, inset, ...props }, ref) => (
113     <DropdownMenuPrimitive.Label
114       ref={ref}
115       className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8", className)}
116       {...props} />
117   ))
118   DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName
119
120   const DropdownMenuSeparator = React.forwardRef(({ className, ...props }, ref) => (
121     <DropdownMenuPrimitive.Separator
122       ref={ref}
123       className={cn("-mx-1 my-1 h-px bg-muted", className)}
124       {...props} />
125   ))
126   DropdownMenuSeparator.displayName = DropdownMenuPrimitive.Separator.displayName
127
128   const DropdownMenuShortcut = ({
129     className,
130     ...props
131   }) => {
132     return (
133       <span
134         className={cn("ml-auto text-xs tracking-widest opacity-60", className)}
135         {...props} />

```

```

136   });
137 }
138 DropdownMenuShortcut.displayName = "DropdownMenuShortcut"
139
140 export {
141   DropdownMenu,
142   DropdownMenuTrigger,
143   DropdownMenuContent,
144   DropdownMenuItem,
145   DropdownMenuCheckboxItem,
146   DropdownMenuRadioItem,
147   DropdownMenuLabel,
148   DropdownMenuSeparator,
149   DropdownMenuShortcut,
150   DropdownMenuGroup,
151   DropdownMenuPortal,
152   DropdownMenuSub,
153   DropdownMenuSubContent,
154   DropdownMenuSubTrigger,
155   DropdownMenuRadioGroup,
156 }

```

frontend/src/components/ui/form.jsx

```

1 import * as React from "react"
2 import { Slot } from "@radix-ui/react-slot"
3 import { Controller, FormProvider, useFormContext } from "react-hook-form";
4
5 import { cn } from "@lib/utils"
6 import { Label } from "@components/ui/label"
7
8 const Form = FormProvider
9
10 const FormFieldContext = React.createContext({})
11
12 const FormField = (
13   {
14     ...props
15   }
16 ) => {
17   return (
18     <FormFieldContext.Provider value={{ name: props.name }}>
19       <Controller {...props} />
20     </FormFieldContext.Provider>
21   );
22 }
23
24 const useFormField = () => {
25   const fieldContext = React.useContext(FormFieldContext)
26   const itemContext = React.useContext(FormItemContext)
27   const { getFieldState, formState } = useFormContext()
28
29   const fieldState = getFieldState(fieldContext.name, formState)
30
31   if (!fieldContext) {
32     throw new Error("useFormField should be used within <FormField>")
33   }
34
35   const { id } = itemContext
36
37   return {
38     id,
39     name: fieldContext.name,
40     formItemId: `${id}-form-item`,
41     formDescriptionId: `${id}-form-item-description`,
42     formMessageId: `${id}-form-item-message`,
43     ...fieldState,
44   }
45 }
46
47 const FormItemContext = React.createContext({})
48
49 const FormItem = React.forwardRef(({ className, ...props }, ref) => {
50   const id = React.useId()
51
52   return (
53     <FormItemContext.Provider value={{ id }}>
54       <div ref={ref} className={cn("space-y-2", className)} {...props} />
55     </FormItemContext.Provider>
56   );
57 })
58 FormItem.displayName = "FormItem"
59
60 const FormLabel = React.forwardRef(({ className, ...props }, ref) => {

```

```

61   const { error, formItemId } = useFormField()
62
63   return (
64     <Label
65       ref={ref}
66       className={cn(error && "text-destructive", className)}
67       htmlFor={formItemId}
68       {...props} />
69   );
70 })
71 FormLabel.displayName = "FormLabel"
72
73 const FormControl = React.forwardRef(({ ...props }, ref) => {
74   const { error, formItemId, formDescriptionId, formMessageId } = useFormField()
75
76   return (
77     <Slot
78       ref={ref}
79       id={formItemId}
80       aria-describedby={
81         !error
82           ? `${formDescriptionId}`
83           : `${formDescriptionId} ${formMessageId}`
84       }
85       aria-invalid={!!error}
86       {...props} />
87   );
88 })
89 FormControl.displayName = "FormControl"
90
91 const FormDescription = React.forwardRef(({ className, ...props }, ref) => {
92   const { formDescriptionId } = useFormField()
93
94   return (
95     <p
96       ref={ref}
97       id={formDescriptionId}
98       className={cn("text-[0.8rem] text-muted-foreground", className)}
99       {...props} />
100   );
101 })
102 FormDescription.displayName = "FormDescription"
103
104 const FormMessage = React.forwardRef(({ className, children, ...props }, ref) => {
105   const { error, formMessageId } = useFormField()
106   const body = error ? String(error?.message ?? "") : children
107
108   if (!body) {
109     return null
110   }
111
112   return (
113     <p
114       ref={ref}
115       id={formMessageId}
116       className={cn("text-[0.8rem] font-medium text-destructive", className)}
117       {...props}>
118       {body}
119     </p>
120   );
121 })
122 FormMessage.displayName = "FormMessage"
123
124 export {
125   useFormField,
126   Form,
127   FormItem,
128   FormLabel,
129   FormControl,
130   FormDescription,
131   FormMessage,
132   FormField,
133 }

```

frontend/src/components/ui/hover-card.jsx

```

1 import * as React from "react"
2 import * as HoverCardPrimitive from "@radix-ui/react-hover-card"
3
4 import { cn } from "@lib/Utils"
5
6 const HoverCard = HoverCardPrimitive.Root
7
8 const HoverCardTrigger = HoverCardPrimitive.Trigger

```

```

9
10 const HoverCardContent = React.forwardRef(({ className, align = "center", sideOffset = 4, ...props }, ref)
+ => (
11   <HoverCardPrimitive.Content
12     ref={ref}
13     align={align}
14     sideOffset={sideOffset}
15     className={cn(
16       "z-50 w-64 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[stat
+ e=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in
+ -0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data
+ -[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bot
+ tom-2 origin-[--radix-hover-card-content-transform-origin]",
17     className
18   )}
19   {...props} />
20 ))
21 HoverCardContent.displayName = HoverCardPrimitive.Content.displayName
22
23 export { HoverCard, HoverCardTrigger, HoverCardContent }

```

frontend/src/components/ui/input-otp.jsx

```

1 import * as React from "react"
2 import { OTPInput, OTPInputContext } from "input-otp"
3 import { Minus } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6
7 const InputOTP = React.forwardRef(({ className, containerClassName, ...props }, ref) => (
8   <OTPInput
9     ref={ref}
10    containerClassName={cn("flex items-center gap-2 has-[:disabled]:opacity-50", containerClassName)}
11    className={cn("disabled:cursor-not-allowed", className)}
12    {...props} />
13 )
14 InputOTP.displayName = "InputOTP"
15
16 const InputOTPGroup = React.forwardRef(({ className, ...props }, ref) => (
17   <div ref={ref} className={cn("flex items-center", className)} {...props} />
18 )
19 InputOTPGroup.displayName = "InputOTPGroup"
20
21 const InputOTPSlot = React.forwardRef(({ index, className, ...props }, ref) => {
22   const inputOTPContext = React.useContext(OTPInputContext)
23   const { char, hasFakeCaret, isActive } = inputOTPContext.slots[index]
24
25   return (
26     <div
27       ref={ref}
28       className={cn(
29         "relative flex h-9 w-9 items-center justify-center border-y border-r border-input text-sm shadow-sm
+ transition-all first:rounded-l-md first:border-l last:rounded-r-md",
30         isActive && "z-10 ring-1 ring-ring",
31         className
32       )}
33       {...props}>
34       {char}
35       {hasFakeCaret && (
36         <div
37           className="pointer-events-none absolute inset-0 flex items-center justify-center">
38             <div className="h-4 w-px animate-caret-blink bg-foreground duration-1000" />
39           </div>
40         )}
41     </div>
42   );
43 )
44 InputOTPSlot.displayName = "InputOTPSlot"
45
46 const InputOTPSeparator = React.forwardRef(({ ...props }, ref) => (
47   <div ref={ref} role="separator" {...props}>
48     <Minus />
49   </div>
50 )
51 InputOTPSeparator.displayName = "InputOTPSeparator"
52
53 export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator }

```

frontend/src/components/ui/input.jsx

```

1 import * as React from "react"
2
3 import { cn } from "@lib/utils"
4

```

```

5 const Input = React.forwardRef(({ className, type, ...props }, ref) => {
6   return (
7     <input
8       type={type}
9       className={cn(
10        "flex h-9 w-full rounded-md border border-input bg-transparent px-3 py-1 text-base shadow-sm transi
+       tion-colors file:border-0 file:bg-transparent file:text-sm file:font-medium file:text-foreground placeholde
+       r:text-muted-foreground focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:cu
+       rsor-not-allowed disabled:opacity-50 md:text-sm",
11        className
12      )}
13      ref={ref}
14      {...props} />
15   );
16 })
17 Input.displayName = "Input"
18
19 export { Input }

```

frontend/src/components/ui/label.jsx

```

1 import * as React from "react"
2 import * as LabelPrimitive from "@radix-ui/react-label"
3 import { cva } from "class-variance-authority";
4
5 import { cn } from "@lib/utils"
6
7 const labelVariants = cva(
8   "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
9 )
10
11 const Label = React.forwardRef(({ className, ...props }, ref) => (
12   <LabelPrimitive.Root ref={ref} className={cn(labelVariants(), className)} {...props} />
13 ))
14 Label.displayName = LabelPrimitive.Root.displayName
15
16 export { Label }

```

frontend/src/components/ui/menubar.jsx

```

1 import * as React from "react"
2 import * as MenubarPrimitive from "@radix-ui/react-menubar"
3 import { Check, ChevronRight, Circle } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6
7 function MenubarMenu({
8   ...props
9 }) {
10   return <MenubarPrimitive.Menu {...props} />;
11 }
12
13 function MenubarGroup({
14   ...props
15 }) {
16   return <MenubarPrimitive.Group {...props} />;
17 }
18
19 function MenubarPortal({
20   ...props
21 }) {
22   return <MenubarPrimitive.Portal {...props} />;
23 }
24
25 function MenubarRadioGroup({
26   ...props
27 }) {
28   return <MenubarPrimitive.RadioGroup {...props} />;
29 }
30
31 function MenubarSub({
32   ...props
33 }) {
34   return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />;
35 }
36
37 const Menubar = React.forwardRef(({ className, ...props }, ref) => (
38   <MenubarPrimitive.Root
39     ref={ref}
40     className={cn(
41       "flex h-9 items-center space-x-1 rounded-md border bg-background p-1 shadow-sm",
42       className
43     )}
44     {...props} />

```

```

45 ))
46 Menubar.displayName = MenubarPrimitive.Root.displayName
47
48 const MenubarTrigger = React.forwardRef(({ className, ...props }, ref) => (
49   <MenubarPrimitive.Trigger
50     ref={ref}
51     className={cn(
52       "flex cursor-default select-none items-center rounded-sm px-3 py-1 text-sm font-medium outline-none f
+   ocus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreg
+   round",
53       className
54     )}
55     {...props} />
56 ))
57 MenubarTrigger.displayName = MenubarPrimitive.Trigger.displayName
58
59 const MenubarSubTrigger = React.forwardRef(({ className, inset, children, ...props }, ref) => (
60   <MenubarPrimitive.SubTrigger
61     ref={ref}
62     className={cn(
63       "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-ac
+   cent focus:text-accent-foreground data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
64       inset && "pl-8",
65       className
66     )}
67     {...props}>
68     {children}
69     <ChevronRight className="ml-auto h-4 w-4" />
70   </MenubarPrimitive.SubTrigger>
71 ))
72 MenubarSubTrigger.displayName = MenubarPrimitive.SubTrigger.displayName
73
74 const MenubarSubContent = React.forwardRef(({ className, ...props }, ref) => (
75   <MenubarPrimitive.SubContent
76     ref={ref}
77     className={cn(
78       "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-lg
+   data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from
+   -top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-menubar-content-transform-origin]",
79       className
80     )}
81     {...props} />
82 ))
83 MenubarSubContent.displayName = MenubarPrimitive.SubContent.displayName
84
85 const MenubarContent = React.forwardRef((
86   { className, align = "start", alignOffset = -4, sideOffset = 8, ...props },
87   ref
88 ) => (
89   <MenubarPrimitive.Portal>
90     <MenubarPrimitive.Content
91       ref={ref}
92       align={align}
93       alignOffset={alignOffset}
94       sideOffset={sideOffset}
95       className={cn(
96         "z-50 min-w-[12rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow
+   -md data-[state=open]:animate-in data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=clo
+   sed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide
+   -in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--ra
+   dix-menubar-content-transform-origin]",
97         className
98       )}
99       {...props} />
100   </MenubarPrimitive.Portal>
101 ))
102 MenubarContent.displayName = MenubarPrimitive.Content.displayName
103
104 const MenubarItem = React.forwardRef(({ className, inset, ...props }, ref) => (
105   <MenubarPrimitive.Item
106     ref={ref}
107     className={cn(
108       "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none fo
+   cus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
109       inset && "pl-8",
110       className
111     )}
112     {...props} />
113 ))
114 MenubarItem.displayName = MenubarPrimitive.Item.displayName
115
116 const MenubarCheckboxItem = React.forwardRef(({ className, children, checked, ...props }, ref) => (

```

```

117 <MenubarPrimitive.CheckboxItem
118   ref={ref}
119   className={cn(
120     "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-no
+ ne focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity
+ -50",
121     className
122   )}
123   checked={checked}
124   {...props}>
125   <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
126     <MenubarPrimitive.ItemIndicator>
127       <Check className="h-4 w-4" />
128     </MenubarPrimitive.ItemIndicator>
129   </span>
130   {children}
131 </MenubarPrimitive.CheckboxItem>
132 ))
133 MenubarCheckboxItem.displayName = MenubarPrimitive.CheckboxItem.displayName
134
135 const MenubarRadioItem = React.forwardRef(({ className, children, ...props }, ref) => (
136   <MenubarPrimitive.RadioItem
137     ref={ref}
138     className={cn(
139       "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-no
+ ne focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity
+ -50",
140       className
141     )}
142     {...props}>
143     <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
144       <MenubarPrimitive.ItemIndicator>
145         <Circle className="h-4 w-4 fill-current" />
146       </MenubarPrimitive.ItemIndicator>
147     </span>
148     {children}
149   </MenubarPrimitive.RadioItem>
150 ))
151 MenubarRadioItem.displayName = MenubarPrimitive.RadioItem.displayName
152
153 const MenubarLabel = React.forwardRef(({ className, inset, ...props }, ref) => (
154   <MenubarPrimitive.Label
155     ref={ref}
156     className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8", className)}
157     {...props} />
158 ))
159 MenubarLabel.displayName = MenubarPrimitive.Label.displayName
160
161 const MenubarSeparator = React.forwardRef(({ className, ...props }, ref) => (
162   <MenubarPrimitive.Separator
163     ref={ref}
164     className={cn("-mx-1 my-1 h-px bg-muted", className)}
165     {...props} />
166 ))
167 MenubarSeparator.displayName = MenubarPrimitive.Separator.displayName
168
169 const MenubarShortcut = ({
170   className,
171   ...props
172 }) => {
173   return (
174     <span
175       className={cn("ml-auto text-xs tracking-widest text-muted-foreground", className)}
176       {...props} />
177   );
178 }
179 MenubarShortcut.displayName = "MenubarShortcut"
180
181 export {
182   Menubar,
183   MenubarMenu,
184   MenubarTrigger,
185   MenubarContent,
186   MenubarItem,
187   MenubarSeparator,
188   MenubarLabel,
189   MenubarCheckboxItem,
190   MenubarRadioGroup,
191   MenubarRadioItem,
192   MenubarPortal,
193   MenubarSubContent,
194   MenubarSubTrigger,
195   MenubarGroup,
196   MenubarSub,

```

```
197   MenubarShortcut,  
198 }
```

frontend/src/components/ui/navigation-menu.jsx

```
1 import * as React from "react"  
2 import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu"  
3 import { cva } from "class-variance-authority"  
4 import { ChevronDown } from "lucide-react"  
5  
6 import { cn } from "@lib/utils"  
7  
8 const NavigationMenu = React.forwardRef(({ className, children, ...props }, ref) => (  
9   <NavigationMenuPrimitive.Root  
10     ref={ref}  
11     className={cn(  
12       "relative z-10 flex max-w-max flex-1 items-center justify-center",  
13       className  
14     )}  
15     {...props}>  
16     {children}  
17     <NavigationMenuViewport />  
18   </NavigationMenuPrimitive.Root>  
19 ))  
20 NavigationMenu.displayName = NavigationMenuPrimitive.Root.displayName  
21  
22 const NavigationMenuList = React.forwardRef(({ className, ...props }, ref) => (  
23   <NavigationMenuPrimitive.List  
24     ref={ref}  
25     className={cn(  
26       "group flex flex-1 list-none items-center justify-center space-x-1",  
27       className  
28     )}  
29     {...props} />  
30 ))  
31 NavigationMenuList.displayName = NavigationMenuPrimitive.List.displayName  
32  
33 const NavigationMenuItem = NavigationMenuPrimitive.Item  
34  
35 const navigationMenuTriggerStyle = cva(  
36   "group inline-flex h-9 w-max items-center justify-center rounded-md bg-background px-4 py-2 text-sm font-  
+ medium transition-colors hover:bg-accent hover:text-accent-foreground focus:bg-accent focus:text-accent-fo-  
+ reground focus:outline-none disabled:pointer-events-none disabled:opacity-50 data-[state=open]:text-accent-f-  
+ oreground data-[state=open]:bg-accent/50 data-[state=open]:hover:bg-accent data-[state=open]:focus:bg-accen-  
+ t"  
37 )  
38  
39 const NavigationMenuTrigger = React.forwardRef(({ className, children, ...props }, ref) => (  
40   <NavigationMenuPrimitive.Trigger  
41     ref={ref}  
42     className={cn(navigationMenuTriggerStyle(), "group", className)}  
43     {...props}>  
44     {children}{" "  
45     <ChevronDown  
46       className="relative top-[1px] ml-1 h-3 w-3 transition duration-300 group-data-[state=open]:rotate-180  
+ "  
47       aria-hidden="true" />  
48   </NavigationMenuPrimitive.Trigger>  
49 ))  
50 NavigationMenuTrigger.displayName = NavigationMenuPrimitive.Trigger.displayName  
51  
52 const NavigationMenuContent = React.forwardRef(({ className, ...props }, ref) => (  
53   <NavigationMenuPrimitive.Content  
54     ref={ref}  
55     className={cn(  
56       "left-0 top-0 w-full data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out data-[motion^=fro-  
+ m-]:fade-in data-[motion^=to-]:fade-out data-[motion=from-end]:slide-in-from-right-52 data-[motion=from-sta-  
+ rt]:slide-in-from-left-52 data-[motion=to-end]:slide-out-to-right-52 data-[motion=to-start]:slide-out-to-le-  
+ ft-52 md:absolute md:w-auto ",  
57       className  
58     )}  
59     {...props} />  
60 ))  
61 NavigationMenuContent.displayName = NavigationMenuPrimitive.Content.displayName  
62  
63 const NavigationMenuLink = NavigationMenuPrimitive.Link  
64  
65 const NavigationMenuViewport = React.forwardRef(({ className, ...props }, ref) => (  
66   <div className={cn("absolute left-0 top-full flex justify-center")}>  
67     <NavigationMenuPrimitive.Viewport  
68       className={cn(  
69         "origin-top-center relative mt-1.5 h-[var(--radix-navigation-menu-viewport-height)] w-full overflow  
+ -hidden rounded-md border bg-popover text-popover-foreground data-[state=open]:animate-in data-[stat-  
+ e=closed]:animate-out data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-90 md:w-[var(--radix-naviga
```

```

+   tion-menu-viewport-width))",
70     className
71   })
72   ref={ref}
73   {...props} />
74 </div>
75 ))
76 NavigationMenuViewport.displayName =
77   NavigationMenuPrimitive.Viewport.displayName
78
79 const NavigationMenuIndicator = React.forwardRef(({ className, ...props }, ref) => (
80   <NavigationMenuPrimitive.Indicator
81     ref={ref}
82     className={cn(
83       "top-full z-[1] flex h-1.5 items-end justify-center overflow-hidden data-[state=visible]:animate-in d
+   ata-[state=hidden]:animate-out data-[state=hidden]:fade-out data-[state=visible]:fade-in",
84     className
85   )}
86   {...props}>
87     <div
88       className="relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm bg-border shadow-md" />
89   </NavigationMenuPrimitive.Indicator>
90 )>
91 NavigationMenuIndicator.displayName =
92   NavigationMenuPrimitive.Indicator.displayName
93
94 export {
95   navigationMenuTriggerStyle,
96   NavigationMenu,
97   NavigationMenuList,
98   NavigationMenuItem,
99   NavigationMenuContent,
100  NavigationMenuTrigger,
101  NavigationMenuLink,
102  NavigationMenuIndicator,
103  NavigationMenuViewport,
104 }

```

frontend/src/components/ui/pagination.jsx

```

1 import * as React from "react"
2 import { ChevronLeft, ChevronRight, MoreHorizontal } from "lucide-react"
3
4 import { cn } from "@lib/Utils"
5 import { buttonVariants } from "@components/ui/button";
6
7 const Pagination = ({
8   className,
9   ...props
10 }) => (
11   <nav
12     role="navigation"
13     aria-label="pagination"
14     className={cn("mx-auto flex w-full justify-center", className)}
15     {...props} />
16 )
17 Pagination.displayName = "Pagination"
18
19 const PaginationContent = React.forwardRef(({ className, ...props }, ref) => (
20   <ul
21     ref={ref}
22     className={cn("flex flex-row items-center gap-1", className)}
23     {...props} />
24 )>
25   PaginationContent.displayName = "PaginationContent"
26
27   const PaginationItem = React.forwardRef(({ className, ...props }, ref) => (
28     <li ref={ref} className={cn("", className)} {...props} />
29   ))
30   PaginationItem.displayName = "PaginationItem"
31
32   const PaginationLink = ({
33     className,
34     isActive,
35     size = "icon",
36     ...props
37   }) => (
38     <a
39       aria-current={isActive ? "page" : undefined}
40       className={cn(buttonVariants({
41         variant: isActive ? "outline" : "ghost",
42         size,
43       }), className)}
44       {...props} />

```

```

45 )
46 PaginationLink.displayName = "PaginationLink"
47
48 const PaginationPrevious = ({
49   className,
50   ...props
51 }) => (
52   <PaginationLink
53     aria-label="Go to previous page"
54     size="default"
55     className={cn("gap-1 pl-2.5", className)}
56     {...props}>
57     <ChevronLeft className="h-4 w-4" />
58     <span>Previous</span>
59   </PaginationLink>
60 )
61 PaginationPrevious.displayName = "PaginationPrevious"
62
63 const PaginationNext = ({
64   className,
65   ...props
66 }) => (
67   <PaginationLink
68     aria-label="Go to next page"
69     size="default"
70     className={cn("gap-1 pr-2.5", className)}
71     {...props}>
72     <span>Next</span>
73     <ChevronRight className="h-4 w-4" />
74   </PaginationLink>
75 )
76 PaginationNext.displayName = "PaginationNext"
77
78 const PaginationEllipsis = ({
79   className,
80   ...props
81 }) => (
82   <span
83     aria-hidden
84     className={cn("flex h-9 w-9 items-center justify-center", className)}
85     {...props}>
86     <MoreHorizontal className="h-4 w-4" />
87     <span className="sr-only">More pages</span>
88   </span>
89 )
90 PaginationEllipsis.displayName = "PaginationEllipsis"
91
92 export {
93   Pagination,
94   PaginationContent,
95   PaginationLink,
96   PaginationItem,
97   PaginationPrevious,
98   PaginationNext,
99   PaginationEllipsis,
100 }

```

frontend/src/components/ui/popover.jsx

```

1 import * as React from "react"
2 import * as PopoverPrimitive from "@radix-ui/react-popover"
3
4 import { cn } from "@lib/utils"
5
6 const Popover = PopoverPrimitive.Root
7
8 const PopoverTrigger = PopoverPrimitive.Trigger
9
10 const PopoverAnchor = PopoverPrimitive.Anchor
11
12 const PopoverContent = React.forwardRef(({ className, align = "center", sideOffset = 4, ...props }, ref) =>
13   (
14     <PopoverPrimitive.Portal>
15       <PopoverPrimitive.Content
16         ref={ref}
17         align={align}
18         sideOffset={sideOffset}
19         className={cn(
20           "z-50 w-72 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-popover-content-transform-origin]",
21           className
22         )}
23       </PopoverPrimitive.Content>
24     </PopoverPrimitive.Portal>
25   )
26 )

```

```

21     })
22     {...props} />
23   </PopoverPrimitive.Portal>
24 )}
25 PopoverContent.displayName = PopoverPrimitive.Content.displayName
26
27 export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor }

```

frontend/src/components/ui/progress.jsx

```

1 import * as React from "react"
2 import * as ProgressPrimitive from "@radix-ui/react-progress"
3
4 import { cn } from "@lib/utils"
5
6 const Progress = React.forwardRef(({ className, value, ...props }, ref) => (
7   <ProgressPrimitive.Root
8     ref={ref}
9     className={cn(
10       "relative h-2 w-full overflow-hidden rounded-full bg-primary/20",
11       className
12     )}
13     {...props}>
14     <ProgressPrimitive.Indicator
15       className="h-full w-full flex-1 bg-primary transition-all"
16       style={{ transform: `translateX(-${100 - (value || 0)}%)` }} />
17   </ProgressPrimitive.Root>
18 ))
19 Progress.displayName = ProgressPrimitive.Root.displayName
20
21 export { Progress }

```

frontend/src/components/ui/radio-group.jsx

```

1 import * as React from "react"
2 import * as RadioGroupPrimitive from "@radix-ui/react-radio-group"
3 import { Circle } from "lucide-react"
4
5 import { cn } from "@lib/utils"
6
7 const RadioGroup = React.forwardRef(({ className, ...props }, ref) => {
8   return (<RadioGroupPrimitive.Root className={cn("grid gap-2", className)} {...props} ref={ref} />);
9 })
10 RadioGroup.displayName = RadioGroupPrimitive.Root.displayName
11
12 const RadioGroupItem = React.forwardRef(({ className, ...props }, ref) => {
13   return (
14     <RadioGroupPrimitive.Item
15       ref={ref}
16       className={cn(
17         "aspect-square h-4 w-4 rounded-full border border-primary text-primary shadow focus:outline-none fo
18 + cus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50",
19         className
20       )}
21       {...props}>
22       <RadioGroupPrimitive.Indicator className="flex items-center justify-center">
23         <Circle className="h-3.5 w-3.5 fill-primary" />
24       </RadioGroupPrimitive.Indicator>
25     </RadioGroupPrimitive.Item>
26   );
27 })
28 RadioGroupItem.displayName = RadioGroupPrimitive.Item.displayName
29 export { RadioGroup, RadioGroupItem }

```

frontend/src/components/ui/resizable.jsx

```

1 import { GripVertical } from "lucide-react"
2 import * as ResizablePrimitive from "react-resizable-panels"
3
4 import { cn } from "@lib/utils"
5
6 const ResizablePanelGroup = ({
7   className,
8   ...props
9 }) => (
10   <ResizablePrimitive.PanelGroup
11     className={cn(
12       "flex h-full w-full data-[panel-group-direction=vertical]:flex-col",
13       className
14     )}
15     {...props} />
16 )
17

```

```

18 const ResizablePanel = ResizablePrimitive.Panel
19
20 const ResizableHandle = ({
21   withHandle,
22   className,
23   ...props
24 }) => (
25   <ResizablePrimitive.PanelResizeHandle
26     className={cn(
27       "relative flex w-px items-center justify-center bg-border after:absolute after:inset-y-0 after:left-1
28       /2 after:w-1 after:-translate-x-1/2 focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring
29       focus-visible:ring-offset-1 data-[panel-group-direction=vertical]:h-px data-[panel-group-direction=vertical]
30       l:w-full data-[panel-group-direction=vertical]:after:left-0 data-[panel-group-direction=vertical]:after:h-
31       1 data-[panel-group-direction=vertical]:after:w-full data-[panel-group-direction=vertical]:after:-translate
32       -y-1/2 data-[panel-group-direction=vertical]:after:translate-x-0 [&[data-panel-group-direction=vertical]]>di
33       v]:rotate-90",
34       className
35     )}
36     {...props}>
37     {withHandle && (
38       <div
39         className="z-10 flex h-4 w-3 items-center justify-center rounded-sm border bg-border">
40         <GripVertical className="h-2.5 w-2.5" />
41       </div>
42     )}
43   </ResizablePrimitive.PanelResizeHandle>
44 )
45
46 export { ResizablePanelGroup, ResizablePanel, ResizableHandle }

```

frontend/src/components/ui/scroll-area.jsx

```

1 import * as React from "react"
2 import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area"
3
4 import { cn } from "@lib/Utils"
5
6 const ScrollArea = React.forwardRef(({ className, children, ...props }, ref) => (
7   <ScrollAreaPrimitive.Root
8     ref={ref}
9     className={cn("relative overflow-hidden", className)}
10    {...props}>
11    <ScrollAreaPrimitive.Viewport className="h-full w-full rounded-[inherit]">
12      {children}
13    </ScrollAreaPrimitive.Viewport>
14    <ScrollBar />
15    <ScrollAreaPrimitive.Corner />
16  </ScrollAreaPrimitive.Root>
17 ))
18 ScrollArea.displayName = ScrollAreaPrimitive.Root.displayName
19
20 const ScrollBar = React.forwardRef(({ className, orientation = "vertical", ...props }, ref) => (
21   <ScrollAreaPrimitive.ScrollAreaScrollbar
22     ref={ref}
23     orientation={orientation}
24     className={cn(
25       "flex touch-none select-none transition-colors",
26       orientation === "vertical" &&
27       "h-full w-2.5 border-l border-l-transparent p-[1px]",
28       orientation === "horizontal" &&
29       "h-2.5 flex-col border-t border-t-transparent p-[1px]",
30       className
31     )}
32     {...props}>
33     <ScrollAreaPrimitive.ScrollAreaThumb className="relative flex-1 rounded-full bg-border" />
34   </ScrollAreaPrimitive.ScrollAreaScrollbar>
35 ))
36 ScrollBar.displayName = ScrollAreaPrimitive.ScrollAreaScrollbar.displayName
37
38 export { ScrollArea, ScrollBar }

```

frontend/src/components/ui/select.jsx

```

1 import * as React from "react"
2 import * as SelectPrimitive from "@radix-ui/react-select"
3 import { Check, ChevronDown, ChevronUp } from "lucide-react"
4
5 import { cn } from "@lib/Utils"
6
7 const Select = SelectPrimitive.Root
8
9 const SelectGroup = SelectPrimitive.Group
10
11 const SelectValue = SelectPrimitive.Value

```

```

12
13 const SelectTrigger = React.forwardRef(({ className, children, ...props }, ref) => (
14   <SelectPrimitive.Trigger
15     ref={ref}
16     className={cn(
17       "flex h-9 w-full items-center justify-between whitespace-nowrap rounded-md border border-input bg-tra
18       + nsparent px-3 py-2 text-sm shadow-sm ring-offset-background data-[placeholder]:text-muted-foreground focus:
19       + outline-none focus:ring-1 focus:ring-ring disabled:cursor-not-allowed disabled:opacity-50 [&span]:line-cla
20       mp-1",
21       className
22     )}
23     {...props}>
24     {children}
25     <SelectPrimitive.Icon asChild>
26       <ChevronDown className="h-4 w-4 opacity-50" />
27     </SelectPrimitive.Icon>
28   </SelectPrimitive.Trigger>
29 ))
30 SelectTrigger.displayName = SelectPrimitive.Trigger.displayName
31
32 const SelectScrollUpButton = React.forwardRef(({ className, ...props }, ref) => (
33   <SelectPrimitive.ScrollUpButton
34     ref={ref}
35     className={cn("flex cursor-default items-center justify-center py-1", className)}
36     {...props}>
37     <ChevronUp className="h-4 w-4" />
38   </SelectPrimitive.ScrollUpButton>
39 ))
40 SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName
41
42 const SelectScrollDownButton = React.forwardRef(({ className, ...props }, ref) => (
43   <SelectPrimitive.ScrollDownButton
44     ref={ref}
45     className={cn("flex cursor-default items-center justify-center py-1", className)}
46     {...props}>
47     <ChevronDown className="h-4 w-4" />
48   </SelectPrimitive.ScrollDownButton>
49 ))
50 SelectScrollDownButton.displayName =
51   SelectPrimitive.ScrollDownButton.displayName
52
53 const SelectContent = React.forwardRef(({ className, children, position = "popper", ...props }, ref) => (
54   <SelectPrimitive.Portal>
55     <SelectPrimitive.Content
56       ref={ref}
57       className={cn(
58         "relative z-50 max-h-[--radix-select-content-available-height] min-w-[8rem] overflow-y-auto overflo
59         + w-x-hidden rounded-md border bg-popover text-popover-foreground shadow-md data-[state=open]:animate-in data
60         + -[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:
61         + zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-f
62         + rom-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-s
63         + elect-content-transform-origin]",
64         position === "popper" &&
65         "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1
66         + data-[side=top]:-translate-y-1",
67         className
68       )}
69       position={position}
70       {...props}>
71       <SelectScrollUpButton />
72       <SelectPrimitive.Viewport
73         className={cn("p-1", position === "popper" &&
74           "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-width)]")}>
75         {children}
76       </SelectPrimitive.Viewport>
77       <SelectScrollDownButton />
78     </SelectPrimitive.Content>
79   </SelectPrimitive.Portal>
80 ))
81 SelectContent.displayName = SelectPrimitive.Content.displayName
82
83 const SelectLabel = React.forwardRef(({ className, ...props }, ref) => (
84   <SelectPrimitive.Label
85     ref={ref}
86     className={cn("px-2 py-1.5 text-sm font-semibold", className)}
87     {...props} />
88 ))
89 SelectLabel.displayName = SelectPrimitive.Label.displayName
90
91 const SelectItem = React.forwardRef(({ className, children, ...props }, ref) => (
92   <SelectPrimitive.Item
93     ref={ref}
94     className={cn(
95       "relative flex w-full cursor-default select-none items-center rounded-sm py-1.5 pl-2 pr-8 text-sm out

```

```

+   line-none focus:bg-accent focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:
+   opacity-50",
87     className
88   )}
89   {...props}>
90   <span className="absolute right-2 flex h-3.5 w-3.5 items-center justify-center">
91     <SelectPrimitive.ItemIndicator>
92       <Check className="h-4 w-4" />
93     </SelectPrimitive.ItemIndicator>
94   </span>
95   <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
96 </SelectPrimitive.Item>
97 ))
98 SelectItem.displayName = SelectPrimitive.Item.displayName
99
100 const SelectSeparator = React.forwardRef(({ className, ...props }, ref) => (
101   <SelectPrimitive.Separator
102     ref={ref}
103     className={cn("-mx-1 my-1 h-px bg-muted", className)}
104     {...props} />
105 ))
106 SelectSeparator.displayName = SelectPrimitive.Separator.displayName
107
108 export {
109   Select,
110   SelectGroup,
111   SelectValue,
112   SelectTrigger,
113   SelectContent,
114   SelectLabel,
115   SelectItem,
116   SelectSeparator,
117   SelectScrollUpButton,
118   SelectScrollDownButton,
119 }

```

frontend/src/components/ui/separator.jsx

```

1 import * as React from "react"
2 import * as SeparatorPrimitive from "@radix-ui/react-separator"
3
4 import { cn } from "@lib/utils"
5
6 const Separator = React.forwardRef((
7   { className, orientation = "horizontal", decorative = true, ...props },
8   ref
9 ) => (
10   <SeparatorPrimitive.Root
11     ref={ref}
12     decorative={decorative}
13     orientation={orientation}
14     className={cn(
15       "shrink-0 bg-border",
16       orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",
17       className
18     )}
19     {...props} />
20 ))
21 Separator.displayName = SeparatorPrimitive.Root.displayName
22
23 export { Separator }

```

frontend/src/components/ui/sheet.jsx

```

1 import * as React from "react"
2 import * as SheetPrimitive from "@radix-ui/react-dialog"
3 import { cva } from "class-variance-authority";
4 import { X } from "lucide-react"
5
6 import { cn } from "@lib/utils"
7
8 const Sheet = SheetPrimitive.Root
9
10 const SheetTrigger = SheetPrimitive.Trigger
11
12 const SheetClose = SheetPrimitive.Close
13
14 const SheetPortal = SheetPrimitive.Portal
15
16 const SheetOverlay = React.forwardRef(({ className, ...props }, ref) => (
17   <SheetPrimitive.Overlay
18     className={cn(
19       "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[s
20     tate=closed]:fade-out-0 data-[state=open]:fade-in-0",

```

```

20     className
21   })
22   {...props}
23   ref={ref} />
24 ))
25 SheetOverlay.displayName = SheetPrimitive.Overlay.displayName
26
27 const sheetVariants = cva(
28   "fixed z-50 gap-4 bg-background p-6 shadow-lg transition ease-in-out data-[state=closed]:duration-300 dat
29   + a-[state=open]:duration-500 data-[state=open]:animate-in data-[state=closed]:animate-out",
30   {
31     variants: {
32       side: {
33         top: "inset-x-0 top-0 border-b data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from
34         + -top",
35         bottom:
36         + "inset-x-0 bottom-0 border-t data-[state=closed]:slide-out-to-bottom data-[state=open]:slide-in-f
37         + rom-bottom",
38         left: "inset-y-0 left-0 h-full w-3/4 border-r data-[state=closed]:slide-out-to-left data-[state=ope
39         + n]:slide-in-from-left sm:max-w-sm",
40         right:
41         + "inset-y-0 right-0 h-full w-3/4 border-l data-[state=closed]:slide-out-to-right data-[state=open]
42         + :slide-in-from-right sm:max-w-sm",
43       },
44     },
45     defaultVariants: {
46       side: "right",
47     },
48   }
49 )
50
51 const SheetContent = React.forwardRef(({ side = "right", className, children, ...props }, ref) => (
52   <SheetPortal>
53     <SheetOverlay />
54     <SheetPrimitive.Content ref={ref} className={cn(sheetVariants({ side }), className)} {...props}>
55       <SheetPrimitive.Close
56       + className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background transition-opacity h
57       + over:opacity-100 focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:pointer-event
58       + s-none data-[state=open]:bg-secondary">
59         <X className="h-4 w-4" />
60         <span className="sr-only">Close</span>
61       </SheetPrimitive.Close>
62       {children}
63     </SheetPrimitive.Content>
64   </SheetPortal>
65 ))
66 SheetContent.displayName = SheetPrimitive.Content.displayName
67
68 const SheetHeader = ({
69   className,
70   ...props
71 }) => (
72   <div
73     + className={cn("flex flex-col space-y-2 text-center sm:text-left", className)}
74     + {...props} />
75 )
76 SheetHeader.displayName = "SheetHeader"
77
78 const SheetFooter = ({
79   className,
80   ...props
81 }) => (
82   <div
83     + className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
84     + {...props} />
85 )
86 SheetFooter.displayName = "SheetFooter"
87
88 const SheetTitle = React.forwardRef(({ className, ...props }, ref) => (
89   <SheetPrimitive.Title
90     + ref={ref}
91     + className={cn("text-lg font-semibold text-foreground", className)}
92     + {...props} />
93 ))
94 SheetTitle.displayName = SheetPrimitive.Title.displayName
95
96 const SheetDescription = React.forwardRef(({ className, ...props }, ref) => (
97   <SheetPrimitive.Description
98     + ref={ref}
99     + className={cn("text-sm text-muted-foreground", className)}
100    + {...props} />
101 ))
102 SheetDescription.displayName = SheetPrimitive.Description.displayName

```

```

97 export {
98   Sheet,
99   SheetPortal,
100  SheetOverlay,
101  SheetTrigger,
102  SheetClose,
103  SheetContent,
104  SheetHeader,
105  SheetFooter,
106  SheetTitle,
107  SheetDescription,
108 }

```

frontend/src/components/ui/skeleton.jsx

```

1 import { cn } from "@lib/Utils"
2
3 function Skeleton({
4   className,
5   ...props
6 }) {
7   return (
8     <div
9       className={cn("animate-pulse rounded-md bg-primary/10", className)}
10      {...props} />
11   );
12 }
13
14 export { Skeleton }

```

frontend/src/components/ui/slider.jsx

```

1 import * as React from "react"
2 import * as SliderPrimitive from "@radix-ui/react-slider"
3
4 import { cn } from "@lib/Utils"
5
6 const Slider = React.forwardRef(({ className, ...props }, ref) => (
7   <SliderPrimitive.Root
8     ref={ref}
9     className={cn("relative flex w-full touch-none select-none items-center", className)}
10    {...props}>
11     <SliderPrimitive.Track
12       className="relative h-1.5 w-full grow overflow-hidden rounded-full bg-primary/20">
13       <SliderPrimitive.Range className="absolute h-full bg-primary" />
14     </SliderPrimitive.Track>
15     <SliderPrimitive.Thumb
16       className="block h-4 w-4 rounded-full border border-primary/50 bg-background shadow transition-colors
17       + focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none disab
18       + led:opacity-50" />
19     </SliderPrimitive.Root>
20   ))
21   Slider.displayName = SliderPrimitive.Root.displayName
22
23 export { Slider }

```

frontend/src/components/ui/sonner.jsx

```

1 import { useTheme } from "next-themes"
2 import { Toaster as Sonner, toast } from "sonner"
3
4 const Toaster = ({
5   ...props
6 }) => {
7   const { theme = "system" } = useTheme()
8
9   return (
10     <Sonner
11       theme={theme}
12       className="toaster group"
13       toastOptions={{
14         classNames: {
15           toast:
16             "group toast group-[.toaster]:bg-background group-[.toaster]:text-foreground group-[.toaster]:b
17           + order-border group-[.toaster]:shadow-lg",
18           description: "group-[.toast]:text-muted-foreground",
19           actionButton:
20             "group-[.toast]:bg-primary group-[.toast]:text-primary-foreground",
21           cancelButton:
22             "group-[.toast]:bg-muted group-[.toast]:text-muted-foreground",
23         },
24       }}
25       {...props} />
26   );
27 }

```

```

26 }
27
28 export { Toaster, toast }

```

frontend/src/components/ui/switch.jsx

```

1 import * as React from "react"
2 import * as SwitchPrimitives from "@radix-ui/react-switch"
3
4 import { cn } from "@lib/utils"
5
6 const Switch = React.forwardRef(({ className, ...props }, ref) => (
7   <SwitchPrimitives.Root
8     className={cn(
9       "peer inline-flex h-5 w-9 shrink-0 cursor-pointer items-center rounded-full border-2 border-transpare
+ nt shadow-sm transition-colors focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focu
+ s-visible:ring-offset-2 focus-visible:ring-offset-background disabled:cursor-not-allowed disabled:opacity-5
+ 0 data-[state=checked]:bg-primary data-[state=unchecked]:bg-input",
10     className
11   )}
12   {...props}
13   ref={ref}>
14     <SwitchPrimitives.Thumb
15       className={cn(
16         "pointer-events-none block h-4 w-4 rounded-full bg-background shadow-lg ring-0 transition-transform
+ data-[state=checked]:translate-x-4 data-[state=unchecked]:translate-x-0"
17       )} />
18   </SwitchPrimitives.Root>
19 )
20 Switch.displayName = SwitchPrimitives.Root.displayName
21
22 export { Switch }

```

frontend/src/components/ui/table.jsx

```

1 import * as React from "react"
2
3 import { cn } from "@lib/utils"
4
5 const Table = React.forwardRef(({ className, ...props }, ref) => (
6   <div className="relative w-full overflow-auto">
7     <table
8       ref={ref}
9       className={cn("w-full caption-bottom text-sm", className)}
10      {...props} />
11   </div>
12 )
13 Table.displayName = "Table"
14
15 const TableHeader = React.forwardRef(({ className, ...props }, ref) => (
16   <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
17 )
18 TableHeader.displayName = "TableHeader"
19
20 const TableBody = React.forwardRef(({ className, ...props }, ref) => (
21   <tbody
22     ref={ref}
23     className={cn("[&_tr:last-child]:border-0", className)}
24     {...props} />
25 )
26 TableBody.displayName = "TableBody"
27
28 const TableFooter = React.forwardRef(({ className, ...props }, ref) => (
29   <tfoot
30     ref={ref}
31     className={cn("border-t bg-muted/50 font-medium [&_tr]:last:border-b-0", className)}
32     {...props} />
33 )
34 TableFooter.displayName = "TableFooter"
35
36 const TableRow = React.forwardRef(({ className, ...props }, ref) => (
37   <tr
38     ref={ref}
39     className={cn(
40       "border-b transition-colors hover:bg-muted/50 data-[state=selected]:bg-muted",
41       className
42     )}
43     {...props} />
44 )
45 TableRow.displayName = "TableRow"
46
47 const TableHead = React.forwardRef(({ className, ...props }, ref) => (
48   <th
49     ref={ref}

```

```

50     className={cn(
51       "h-10 px-2 text-left align-middle font-medium text-muted-foreground [&:has([role=checkbox])]:pr-0 [&
+ [role=checkbox]]:translate-y-[2px]",
52       className
53     )}
54     {...props} />
55   ))
56   TableHead.displayName = "TableHead"
57
58   const TableCell = React.forwardRef(({ className, ...props }, ref) => (
59     <td
60       ref={ref}
61       className={cn(
62         "p-2 align-middle [&:has([role=checkbox])]:pr-0 [&[role=checkbox]]:translate-y-[2px]",
63         className
64       )}
65       {...props} />
66   ))
67   TableCell.displayName = "TableCell"
68
69   const TableCaption = React.forwardRef(({ className, ...props }, ref) => (
70     <caption
71       ref={ref}
72       className={cn("mt-4 text-sm text-muted-foreground", className)}
73       {...props} />
74   ))
75   TableCaption.displayName = "TableCaption"
76
77   export {
78     Table,
79     TableHeader,
80     TableBody,
81     TableFooter,
82     TableHead,
83     TableRow,
84     TableCell,
85     TableCaption,
86   }

```

frontend/src/components/ui/tabs.jsx

```

1  import * as React from "react"
2  import * as TabsPrimitive from "@radix-ui/react-tabs"
3
4  import { cn } from "@lib/utils"
5
6  const Tabs = TabsPrimitive.Root
7
8  const TabsList = React.forwardRef(({ className, ...props }, ref) => (
9    <TabsPrimitive.List
10      ref={ref}
11      className={cn(
12        "inline-flex h-9 items-center justify-center rounded-lg bg-muted p-1 text-muted-foreground",
13        className
14      )}
15      {...props} />
16  ))
17  TabsList.displayName = TabsPrimitive.List.displayName
18
19  const TabsTrigger = React.forwardRef(({ className, ...props }, ref) => (
20    <TabsPrimitive.Trigger
21      ref={ref}
22      className={cn(
23        "inline-flex items-center justify-center whitespace-nowrap rounded-md px-3 py-1 text-sm font-medium r
+ ing-offset-background transition-all focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-rin
+ g focus-visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50 data-[state=active]:bg-backg
+ round data-[state=active]:text-foreground data-[state=active]:shadow",
24        className
25      )}
26      {...props} />
27  ))
28  TabsTrigger.displayName = TabsPrimitive.Trigger.displayName
29
30  const TabsContent = React.forwardRef(({ className, ...props }, ref) => (
31    <TabsPrimitive.Content
32      ref={ref}
33      className={cn(
34        "mt-2 ring-offset-background focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
+ focus-visible:ring-offset-2",
35        className
36      )}
37      {...props} />
38  ))
39  TabsContent.displayName = TabsPrimitive.Content.displayName

```

```

40
41 export { Tabs, TabsList, TabsTrigger, TabsContent }

```

frontend/src/components/ui/textarea.jsx

```

1 import * as React from "react"
2
3 import { cn } from "@lib/utils"
4
5 const Textarea = React.forwardRef(({ className, ...props }, ref) => {
6   return (
7     <textarea
8       className={cn(
9         "flex min-h-[60px] w-full rounded-md border border-input bg-transparent px-3 py-2 text-base shadow-
+ sm placeholder:text-muted-foreground focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-rin
+ g disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
10       className
11     )}
12     ref={ref}
13     {...props} />
14   );
15 })
16 Textarea.displayName = "Textarea"
17
18 export { Textarea }

```

frontend/src/components/ui/toast.jsx

```

1 import * as React from "react"
2 import * as ToastPrimitives from "@radix-ui/react-toast"
3 import { cva } from "class-variance-authority";
4 import { X } from "lucide-react"
5
6 import { cn } from "@lib/utils"
7
8 const ToastProvider = ToastPrimitives.Provider
9
10 const ToastViewport = React.forwardRef(({ className, ...props }, ref) => (
11   <ToastPrimitives.Viewport
12     ref={ref}
13     className={cn(
14       "fixed top-0 z-[100] flex max-h-screen w-full flex-col-reverse p-4 sm:bottom-0 sm:right-0 sm:top-auto
+ sm:flex-col md:max-w-[420px]",
15     className
16   )}
17   {...props} />
18 ))
19 ToastViewport.displayName = ToastPrimitives.Viewport.displayName
20
21 const toastVariants = cva(
22   "group pointer-events-auto relative flex w-full items-center justify-between space-x-2 overflow-hidden ro
+ unded-md border p-4 pr-6 shadow-lg transition-all data-[swipe=cancel]:translate-x-0 data-[swipe=end]:transl
+ ate-x-[var(--radix-toast-swipe-end-x)] data-[swipe=move]:translate-x-[var(--radix-toast-swipe-move-x)] data
+ -[swipe=move]:transition-none data-[state=open]:animate-in data-[state=closed]:animate-out data-[swipe=end]
+ :animate-out data-[state=closed]:fade-out-80 data-[state=closed]:slide-out-to-right-full data-[state=open]:
+ slide-in-from-top-full data-[state=open]:sm:slide-in-from-bottom-full",
23   {
24     variants: {
25       variant: {
26         default: "border bg-background text-foreground",
27         destructive:
28           "destructive group border-destructive bg-destructive text-destructive-foreground",
29       },
30     },
31     defaultVariants: {
32       variant: "default",
33     },
34   }
35 )
36
37 const Toast = React.forwardRef(({ className, variant, ...props }, ref) => {
38   return (
39     <ToastPrimitives.Root
40       ref={ref}
41       className={cn(toastVariants({ variant }), className)}
42       {...props} />
43   );
44 })
45 Toast.displayName = ToastPrimitives.Root.displayName
46
47 const ToastAction = React.forwardRef(({ className, ...props }, ref) => (
48   <ToastPrimitives.Action
49     ref={ref}
50     className={cn(

```

```

51     "inline-flex h-8 shrink-0 items-center justify-center rounded-md border bg-transparent px-3 text-sm font-medium transition-colors hover:bg-secondary focus:outline-none focus:ring-1 focus:ring-ring disabled:opacity-50 group-[.destructive]:border-muted/40 group-[.destructive]:hover:border-destructive/30 group-[.destructive]:hover:bg-destructive group-[.destructive]:hover:text-destructive-foreground group-[.destructive]:focus:ring-destructive",
52     className
53   )}
54   {...props} />
55 ))
56 ToastAction.displayName = ToastPrimitives.Action.displayName
57
58 const ToastClose = React.forwardRef(({ className, ...props }, ref) => (
59   <ToastPrimitives.Close
60     ref={ref}
61     className={cn(
62       "absolute right-1 top-1 rounded-md p-1 text-foreground/50 opacity-0 transition-opacity hover:text-foreground focus:opacity-100 focus:outline-none focus:ring-1 group-hover:opacity-100 group-[.destructive]:text-red-300 group-[.destructive]:hover:text-red-50 group-[.destructive]:focus:ring-red-400 group-[.destructive]:focus:ring-offset-red-600",
63       className
64     )}
65     toast-close=""
66     {...props}>
67     <X className="h-4 w-4" />
68   </ToastPrimitives.Close>
69 )
70 ToastClose.displayName = ToastPrimitives.Close.displayName
71
72 const ToastTitle = React.forwardRef(({ className, ...props }, ref) => (
73   <ToastPrimitives.Title
74     ref={ref}
75     className={cn("text-sm font-semibold [&div]:text-xs", className)}
76     {...props} />
77 )
78 ToastTitle.displayName = ToastPrimitives.Title.displayName
79
80 const ToastDescription = React.forwardRef(({ className, ...props }, ref) => (
81   <ToastPrimitives.Description ref={ref} className={cn("text-sm opacity-90", className)} {...props} />
82 )
83 ToastDescription.displayName = ToastPrimitives.Description.displayName
84
85 export { ToastProvider, ToastViewport, Toast, ToastTitle, ToastDescription, ToastClose, ToastAction };

```

frontend/src/components/ui/toaster.jsx

```

1 import { useToast } from "@hooks/use-toast"
2 import {
3   Toast,
4   ToastClose,
5   ToastDescription,
6   ToastProvider,
7   ToastTitle,
8   ToastViewport,
9 } from "@components/ui/toast"
10
11 export function Toaster() {
12   const { toasts } = useToast()
13
14   return (
15     <ToastProvider>
16       {toasts.map(function ({ id, title, description, action, ...props }) {
17         return (
18           <Toast key={id} {...props}>
19             <div className="grid gap-1">
20               {title && <ToastTitle>{title}</ToastTitle>}
21               {description && (
22                 <ToastDescription>{description}</ToastDescription>
23               )}
24             </div>
25             {action}
26           <ToastClose />
27         </Toast>
28       );
29     )}
30   <ToastViewport />
31 </ToastProvider>
32 );
33 }

```

frontend/src/components/ui/toggle-group.jsx

```

1 import * as React from "react"
2 import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group"
3

```

```

4 import { cn } from "@lib/utils"
5 import { toggleVariants } from "@components/ui/toggle"
6
7 const ToggleGroupContext = React.createContext({
8   size: "default",
9   variant: "default",
10 })
11
12 const ToggleGroup = React.forwardRef(({ className, variant, size, children, ...props }, ref) => (
13   <ToggleGroupPrimitive.Root
14     ref={ref}
15     className={cn("flex items-center justify-center gap-1", className)}
16     {...props}>
17     <ToggleGroupContext.Provider value={{ variant, size }}>
18       {children}
19     </ToggleGroupContext.Provider>
20   </ToggleGroupPrimitive.Root>
21 ))
22
23 ToggleGroup.displayName = ToggleGroupPrimitive.Root.displayName
24
25 const ToggleGroupItem = React.forwardRef(({ className, children, variant, size, ...props }, ref) => {
26   const context = React.useContext(ToggleGroupContext)
27
28   return (
29     <ToggleGroupPrimitive.Item
30       ref={ref}
31       className={cn(toggleVariants({
32         variant: context.variant || variant,
33         size: context.size || size,
34       }), className)}
35       {...props}>
36       {children}
37     </ToggleGroupPrimitive.Item>
38   );
39 })
40
41 ToggleGroupItem.displayName = ToggleGroupPrimitive.Item.displayName
42
43 export { ToggleGroup, ToggleGroupItem }

```

frontend/src/components/ui/toggle.jsx

```

1 "use client"
2
3 import * as React from "react"
4 import * as TogglePrimitive from "@radix-ui/react-toggle"
5 import { cva } from "class-variance-authority";
6
7 import { cn } from "@lib/utils"
8
9 const toggleVariants = cva(
10   "inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium transition-colors hover:bg-
11   + muted hover:text-muted-foreground focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring d
12   + isabled:pointer-events-none disabled:opacity-50 data-[state=on]:bg-accent data-[state=on]:text-accent-foreg
13   + round [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
14   {
15     variants: {
16       variant: {
17         default: "bg-transparent",
18         outline:
19           "border border-input bg-transparent shadow-sm hover:bg-accent hover:text-accent-foreground",
20       },
21       size: {
22         default: "h-9 px-2 min-w-9",
23         sm: "h-8 px-1.5 min-w-8",
24         lg: "h-10 px-2.5 min-w-10",
25       },
26     },
27     defaultVariants: {
28       variant: "default",
29       size: "default",
30     },
31   },
32 )
33
34 const Toggle = React.forwardRef(({ className, variant, size, ...props }, ref) => (
35   <TogglePrimitive.Root
36     ref={ref}
37     className={cn(toggleVariants({ variant, size, className }))}
38     {...props} />
39 ))
40
41 Toggle.displayName = TogglePrimitive.Root.displayName

```

```
39
40 export { Toggle, toggleVariants }
```

frontend/src/components/ui/tooltip.jsx

```
1 import * as React from "react"
2 import * as TooltipPrimitive from "@radix-ui/react-tooltip"
3
4 import { cn } from "@lib/utils"
5
6 const TooltipProvider = TooltipPrimitive.Provider
7
8 const Tooltip = TooltipPrimitive.Root
9
10 const TooltipTrigger = TooltipPrimitive.Trigger
11
12 const TooltipContent = React.forwardRef(({ className, sideOffset = 4, ...props }, ref) => (
13   <TooltipPrimitive.Portal>
14     <TooltipPrimitive.Content
15       ref={ref}
16       sideOffset={sideOffset}
17       className={cn(
18         "z-50 overflow-hidden rounded-md bg-primary px-3 py-1.5 text-xs text-primary-foreground animate-in
19 + fade-in-0 zoom-in-95 data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=closed]:zoom
20 + m-out-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:sl
21 + ide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-tooltip-content-transform-origin]
22 + ",
19         className
20       )}
21       {...props} />
22     </TooltipPrimitive.Portal>
23   ))
24 TooltipContent.displayName = TooltipPrimitive.Content.displayName
25
26 export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider }
```

Project configuration

backend/requirements.txt

```
1 # -----
2 # ECR Intelligence Agent - backend dependencies
3 # -----
4 # Core API
5 fastapi>=0.115.0
6 uvicorn[standard]>=0.30.0
7 python-multipart>=0.0.9
8 sse-starlette>=2.1.0
9
10 # Validation / settings
11 pydantic>=2.9.0
12 pydantic-settings>=2.5.0
13 python-dotenv>=1.0.1
14
15 # Persistence
16 SQLAlchemy>=2.0.30
17 alembic>=1.13.0
18 # Data lives in CSV files (backend/data). SQLAlchemy queries a working copy
19 # loaded from them. The PostgreSQL driver is only needed when DATABASE_URL points
20 # that working copy at PostgreSQL (e.g. for VECTOR_STORE=pgvector).
21 psycopg[binary]>=3.2.0 ; platform_python_implementation == "CPython"
22
23 # Agentic AI
24 langgraph>=0.2.0
25 langchain-core>=0.3.0
26
27 # Analysis
28 networkx>=3.3
29 numpy>=1.26.0
30
31 # HTTP / LLM providers (providers are called over plain HTTP - no vendor SDK required)
32 httpx>=0.27.0
33
34 # Logging
35 structlog>=24.1.0
36
37 # Testing
38 pytest>=8.0.0
39 pytest-xdist>=3.6.0
```

backend/pytest.ini

```
1 [pytest]
```

```

2 # Fixed 2 xdist workers (deterministic, not reliant on the agent passing -n). loadscope pins each test
3 # class/module to one worker - generated suites share one preview backend and assume sequential shared
4 # state - so it parallelizes across classes/modules without cross-test races.
5 # AGENT: do NOT modify addopts; keep exactly -n 2 --dist loadscope and run only what is configured here.
6 # Serial = '-n 0' (NOT '-p no:xdist', which errors because addopts still passes -n/--dist). A custom '-n'
7 # option in your own pytest setup collides with xdist's -n - rename it.
8 required_plugins = pytest-xdist
9 addopts = -n 2 --dist loadscope
10 testpaths = app/tests
11 # The app has legitimate classes named Test* (TestSelectionEngine); only collect
12 # classes ending in Tests so pytest does not warn about them.
13 python_classes = *Tests

```

backend/Dockerfile

```

1 # syntax=docker/dockerfile:1
2 FROM python:3.12-slim AS base
3
4 ENV PYTHONDONTWRITEBYTECODE=1 \
5     PYTHONUNBUFFERED=1 \
6     PIP_NO_CACHE_DIR=1
7
8 WORKDIR /app
9
10 # Build tooling is only needed while wheels are resolved.
11 RUN apt-get update \
12     && apt-get install -y --no-install-recommends build-essential curl \
13     && rm -rf /var/lib/apt/lists/*
14
15 COPY requirements.txt .
16 RUN pip install --upgrade pip && pip install -r requirements.txt
17
18 COPY . .
19
20 # Non-root runtime user.
21 RUN useradd --create-home --uid 10001 appuser \
22     && chown -R appuser:appuser /app
23 USER appuser
24
25 EXPOSE 8000
26
27 HEALTHCHECK --interval=20s --timeout=5s --start-period=30s --retries=5 \
28     CMD curl -fsS http://localhost:8000/api/health || exit 1
29
30 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

docker-compose.yml

```

1 # ECR Intelligence Agent -- full local stack.
2 # docker compose up --build
3 # frontend http://localhost:3000
4 # backend http://localhost:8000 (Swagger at /docs)
5 #
6 # Data lives in the CSV files under backend/data. They are mounted into the
7 # container, so changes made by the app (new ECRs, reports, feedback) land in
8 # your checkout.
9 services:
10     backend:
11         build:
12             context: ./backend
13         environment:
14             APP_ENV: ${APP_ENV:-development}
15             LOG_LEVEL: ${LOG_LEVEL:-INFO}
16             DATA_DIR: /data
17             VECTOR_STORE: ${VECTOR_STORE:-memory}
18             EMBEDDING_DIM: ${EMBEDDING_DIM:-1024}
19             # LLM - leave unset to run fully offline in demo mode.
20             LLM_PROVIDER: ${LLM_PROVIDER:-auto}
21             LLM_NARRATION: ${LLM_NARRATION:-false}
22             OPENAI_API_KEY: ${OPENAI_API_KEY:-}
23             OPENAI_BASE_URL: ${OPENAI_BASE_URL:-https://api.openai.com/v1}
24             OPENAI_MODEL: ${OPENAI_MODEL:-gpt-4o}
25             OPENAI_FAST_MODEL: ${OPENAI_FAST_MODEL:-}
26             OPENAI_EMBEDDING_MODEL: ${OPENAI_EMBEDDING_MODEL:-text-embedding-3-small}
27             ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY:-}
28             AZURE_OPENAI_ENDPOINT: ${AZURE_OPENAI_ENDPOINT:-}
29             AZURE_OPENAI_API_KEY: ${AZURE_OPENAI_API_KEY:-}
30             CORS_ORIGINS: ${CORS_ORIGINS:-*}
31         volumes:
32             - ./backend/data:/data
33         ports:
34             - "8000:8000"
35         restart: unless-stopped
36

```

```

37 frontend:
38   build:
39     context: ../frontend
40     args:
41       REACT_APP_BACKEND_URL: ${REACT_APP_BACKEND_URL:-http://localhost:8000}
42   ports:
43     - "3000:3000"
44   depends_on:
45     - backend
46   restart: unless-stopped
47
48 # Optional: enable with `docker compose --profile cache up`
49 redis:
50   image: redis:7-alpine
51   profiles: ["cache"]
52   ports:
53     - "6379:6379"

```

Makefile

```

1 # ECR Intelligence Agent -- developer entry points.
2 .DEFAULT_GOAL := help
3 PY := .venv/Scripts/python.exe
4 ifeq ($(OS),)
5 PY := .venv/bin/python
6 endif
7
8 .PHONY: help
9 help: ## Show the available targets
10    @grep -hE '^[a-zA-Z_-]+:.*?## ' $(MAKEFILE_LIST) | awk 'BEGIN {FS = ":.*?## ";} {printf " \033[36m%-16s\033[0m %s\n", $$1, $$2}'
11
12 .PHONY: venv
13 venv: ## Create the virtualenv and install backend dependencies
14     python -m venv .venv
15     $(PY) -m pip install --upgrade pip
16     $(PY) -m pip install -r backend/requirements.txt
17
18 .PHONY: seed
19 seed: ## Create the schema and load the sample corpus
20     cd backend && ../$(PY) -m app.seed
21
22 .PHONY: backend
23 backend: ## Run the API on :8000 with reload
24     cd backend && ../$(PY) -m uvicorn app.main:app --reload --port 8000
25
26 .PHONY: frontend
27 frontend: ## Run the dashboard on :3000
28     cd frontend && yarn start
29
30 .PHONY: install-frontend
31 install-frontend: ## Install frontend dependencies
32     cd frontend && yarn install
33
34 .PHONY: test
35 test: ## Run the backend test suite
36     cd backend && ../$(PY) -m pytest -n 0 -q
37
38 .PHONY: demo
39 demo: ## Run the demo analysis against a running backend
40     $(PY) scripts/demo.py
41
42 .PHONY: docker
43 docker: ## Build and start the whole stack
44     docker compose up --build
45
46 .PHONY: docker-down
47 docker-down: ## Stop the stack and drop volumes
48     docker compose down -v
49
50 .PHONY: lint
51 lint: ## Byte-compile every backend module (fast syntax gate)
52     $(PY) -m compileall -q backend/app

```

.env.example

```

1 # -----
2 # ECR Intelligence Agent - environment template
3 # Copy to .env (repo root) or backend/.env and fill in what you need.
4 # The platform runs fully in DEMO MODE with none of these set.
5 # -----
6 APP_ENV=development
7 LOG_LEVEL=INFO
8

```

```

9 # --- Data -----
10 # All data lives in CSV files, one per table (backend/data by default). The app
11 # loads them at startup and writes every change back to them.
12 # DATA_DIR=C:/path/to/data
13
14 # --- LLM provider -----
15 # mock | openai | anthropic | azure_openai | auto ("auto" picks by key presence)
16 LLM_PROVIDER=auto
17 # false (default): the LLM writes only the answer and the defect summary, so an
18 # analysis takes seconds. true: it also rewords every step (about 7 more calls).
19 LLM_NARRATION=false
20
21 # OpenAI (or any OpenAI-compatible gateway, e.g. an enterprise LLM gateway)
22 OPENAI_API_KEY=
23 OPENAI_BASE_URL=https://api.openai.com/v1
24 OPENAI_MODEL=gpt-4o
25 OPENAI_EMBEDDING_MODEL=text-embedding-3-small
26
27 # Anthropic
28 ANTHROPIC_API_KEY=
29 ANTHROPIC_MODEL=claude-sonnet-5
30
31 # Azure OpenAI
32 AZURE_OPENAI_ENDPOINT=
33 AZURE_OPENAI_API_KEY=
34 AZURE_OPENAI_DEPLOYMENT=gpt-4o
35
36 # --- Vector search -----
37 # memory | pgvector | chroma
38 VECTOR_STORE=memory
39 # Must match the embedding model in use (1024 = Titan v2 / local hashed default)
40 EMBEDDING_DIM=1024
41
42 # --- Agent runtime -----
43 AGENT_STEP_DELAY_MS=0
44 HUMAN_IN_THE_LOOP=false
45
46 # --- Test selection weights (must sum to 1.0) -----
47 TEST_WEIGHT_REQUIREMENT_MATCH=0.30
48 TEST_WEIGHT_COMPONENT_MATCH=0.25
49 TEST_WEIGHT_DEPENDENCY_RELEVANCE=0.15
50 TEST_WEIGHT_HISTORICAL_FAILURE=0.15
51 TEST_WEIGHT_SEMANTIC_SIMILARITY=0.15
52 TEST_SELECTION_THRESHOLD=50
53
54 # --- Security -----
55 # When set, every /api call must send X-API-Key
56 API_KEY=
57 CORS_ORIGINS=*
58
59 # --- Frontend -----
60 REACT_APP_BACKEND_URL=http://localhost:8000

```

frontend/package.json

```

1 {
2   "name": "frontend",
3   "version": "0.1.0",
4   "private": true,
5   "dependencies": {
6     "@hookform/resolvers": "5.0.1",
7     "@radix-ui/react-accordion": "1.2.8",
8     "@radix-ui/react-alert-dialog": "1.1.11",
9     "@radix-ui/react-aspect-ratio": "1.1.4",
10    "@radix-ui/react-avatar": "1.1.7",
11    "@radix-ui/react-checkbox": "1.2.3",
12    "@radix-ui/react-collapsible": "1.1.8",
13    "@radix-ui/react-context-menu": "2.2.12",
14    "@radix-ui/react-dialog": "1.1.11",
15    "@radix-ui/react-dropdown-menu": "2.1.12",
16    "@radix-ui/react-hover-card": "1.1.11",
17    "@radix-ui/react-label": "2.1.4",
18    "@radix-ui/react-menubar": "1.1.12",
19    "@radix-ui/react-navigation-menu": "1.2.10",
20    "@radix-ui/react-popover": "1.1.11",
21    "@radix-ui/react-progress": "1.1.4",
22    "@radix-ui/react-radio-group": "1.3.4",
23    "@radix-ui/react-scroll-area": "1.2.6",
24    "@radix-ui/react-select": "2.2.2",
25    "@radix-ui/react-separator": "1.1.4",
26    "@radix-ui/react-slider": "1.3.2",
27    "@radix-ui/react-slot": "1.2.0",
28    "@radix-ui/react-switch": "1.2.2",
29    "@radix-ui/react-tabs": "1.1.9",

```

```

30     "@radix-ui/react-toast": "1.2.11",
31     "@radix-ui/react-toggle": "1.1.6",
32     "@radix-ui/react-toggle-group": "1.1.7",
33     "@radix-ui/react-tooltip": "1.2.4",
34     "@tanstack/react-query": "5.56.2",
35     "axios": "1.18.0",
36     "class-variance-authority": "0.7.1",
37     "clsx": "2.1.1",
38     "cmdk": "1.1.1",
39     "cra-template": "1.2.0",
40     "date-fns": "4.1.0",
41     "dayjs": "1.11.13",
42     "embla-carousel-react": "8.6.0",
43     "framer-motion": "11.18.0",
44     "input-otp": "1.4.2",
45     "lodash": "4.18.1",
46     "lucide-react": "0.516.0",
47     "next-themes": "0.4.6",
48     "react": "19.0.0",
49     "react-day-picker": "8.10.1",
50     "react-dom": "19.0.0",
51     "react-hook-form": "7.56.2",
52     "react-resizable-panels": "3.0.1",
53     "react-router-dom": "7.15.0",
54     "react-scripts": "5.0.1",
55     "recharts": "3.6.0",
56     "sonner": "2.0.3",
57     "swr": "2.3.8",
58     "tailwind-merge": "3.2.0",
59     "tailwindcss-animate": "1.0.7",
60     "vaul": "1.1.2",
61     "zod": "3.24.4"
62   },
63   "scripts": {
64     "start": "craco start",
65     "build": "craco build",
66     "test": "craco test"
67   },
68   "browserslist": {
69     "production": [
70       ">0.2%",
71       "not dead",
72       "not op_mini all"
73     ],
74     "development": [
75       "last 1 chrome version",
76       "last 1 firefox version",
77       "last 1 safari version"
78     ]
79   },
80   "devDependencies": {
81     "@babel/plugin-proposal-private-property-in-object": "7.21.11",
82     "@craco/craco": "7.1.0",
83     "@emergentbase/visual-edits": "https://assets.emergent.sh/npm/emergentbase-visual-edits-1.0.13.tgz",
84     "@eslint/js": "9.23.0",
85     "@types/lodash": "4.17.24",
86     "autoprefixer": "10.4.20",
87     "dotenv": "16.4.5",
88     "eslint": "9.23.0",
89     "eslint-plugin-import": "2.31.0",
90     "eslint-plugin-jsx-a11y": "6.10.2",
91     "eslint-plugin-react": "7.37.4",
92     "eslint-plugin-react-hooks": "5.2.0",
93     "globals": "15.15.0",
94     "postcss": "8.5.10",
95     "tailwindcss": "3.4.17"
96   },
97   "resolutions": {
98     "react-router": "7.15.1",
99     "node-forge": "1.4.0",
100    "fast-uri": "3.1.2",
101    "flatted": "3.4.2",
102    "qs": "6.15.2",
103    "diff": "4.0.4",
104    "follow-redirects": "1.16.0",
105    "path-to-regexp": "0.1.13",
106    "rollup": "2.80.0",
107    "underscore": "1.13.8",
108    "@babel/plugin-transform-modules-systemjs": "7.29.4",
109    "@eslint/plugin-kit": "0.3.4",
110    "shell-quote": "1.9.0",
111    "jsonpath": "1.3.0",
112    "nth-check": "2.0.1",
113    "serialize-javascript": "7.0.5",

```

```

114     "uuid": "11.1.1",
115     "@tootallnate/once": "2.0.1",
116     "webpack-dev-server": "5.2.6",
117     "resolve-url-loader": "5.0.0",
118     "**/resolve-url-loader/postcss": "8.5.10",
119     "**/axios/form-data": "4.0.6",
120     "**/jsdom/form-data": "3.0.5",
121     "**/postcss-svgo/svgo": "2.8.1",
122     "**/webpack-dev-server/ws": "8.21.0",
123     "**/postcss-load-config/yaml": "2.8.3",
124     "**/cosmiconfig/yaml": "1.10.3",
125     "**/cssnano/yaml": "1.10.3",
126     "**/eslint/js-yaml": "4.3.0",
127     "**/@eslint/eslintrc/js-yaml": "4.3.0",
128     "**/svgo/js-yaml": "3.15.0",
129     "**/@istanbuljs/load-nyc-config/js-yaml": "3.15.0",
130     "**/css-loader/postcss": "8.5.10",
131     "**/css-minimizer-webpack-plugin/postcss": "8.5.10",
132     "**/react-scripts/postcss": "8.5.10",
133     "**/filelist/minimatch": "5.1.8",
134     "**/anymatch/picomatch": "2.3.2",
135     "**/micromatch/picomatch": "2.3.2",
136     "**/readdirp/picomatch": "2.3.2",
137     "**/jest-util/picomatch": "2.3.2",
138     "**/tinyglobby/picomatch": "4.0.4",
139     "http-proxy-middleware": "2.0.10"
140   },
141   "packageManager": "yarn@1.22.22+sha512.a6b2f7906b721bba3d67d4aff083df04dad64c399707841b7acf00f6b133b7ac24
+ 255f2652fa22ae3534329dc6180534e98d17432037ff6fd140556e2bb3137e"
142 }

```

frontend/craco.config.js

```

1 // craco.config.js
2 const path = require("path");
3 require("dotenv").config();
4
5 // Check if we're in development/preview mode (not production build)
6 // Craco sets NODE_ENV=development for start, NODE_ENV=production for build
7 const isDevServer = process.env.NODE_ENV !== "production";
8
9 // Environment variable overrides
10 const config = {
11   enableHealthCheck: process.env.ENABLE_HEALTH_CHECK === "true",
12 };
13
14 function makeDevServerV5Compatible(devServerConfig) {
15   const {
16     https,
17     onAfterSetupMiddleware,
18     onBeforeSetupMiddleware,
19     onListening,
20     setupMiddlewares,
21     ...compatibleConfig
22   } = devServerConfig;
23
24   compatibleConfig.server =
25     typeof https === "object"
26       ? { type: "https", options: https }
27       : https
28       ? "https"
29       : "http";
30   compatibleConfig.headers = {
31     ...compatibleConfig.headers,
32     "Cross-Origin-Resource-Policy": "same-origin",
33   };
34
35   if (onBeforeSetupMiddleware || setupMiddlewares) {
36     compatibleConfig.setupMiddlewares = (middlewares, devServer) => {
37       if (onBeforeSetupMiddleware) {
38         onBeforeSetupMiddleware(devServer);
39       }
40
41       return setupMiddlewares
42         ? setupMiddlewares(middlewares, devServer)
43         : middlewares;
44     };
45   }
46
47   compatibleConfig.onListening = (devServer) => {
48     devServer.close ??= (callback) => devServer.stopCallback(callback);
49
50     if (onListening) {
51       onListening(devServer);

```

```

52     }
53     if (onAfterSetupMiddleware) {
54         onAfterSetupMiddleware(devServer);
55     }
56 };
57
58 return compatibleConfig;
59 }
60
61 // Conditionally load health check modules only if enabled
62 let WebpackHealthPlugin;
63 let setupHealthEndpoints;
64 let healthPluginInstance;
65
66 if (config.enableHealthCheck) {
67     WebpackHealthPlugin = require("./plugins/health-check/webpack-health-plugin");
68     setupHealthEndpoints = require("./plugins/health-check/health-endpoints");
69     healthPluginInstance = new WebpackHealthPlugin();
70 }
71
72 let webpackConfig = {
73     eslint: {
74         configure: {
75             extends: ["plugin:react-hooks/recommended"],
76             rules: {
77                 "react-hooks/rules-of-hooks": "error",
78                 "react-hooks/exhaustive-deps": "warn",
79             },
80         },
81     },
82     webpack: {
83         alias: {
84             '@': path.resolve(__dirname, 'src'),
85         },
86         configure: (webpackConfig) => {
87
88             // Add ignored patterns to reduce watched directories
89             webpackConfig.watchOptions = {
90                 ...webpackConfig.watchOptions,
91                 ignored: [
92                     '**/node_modules/**',
93                     '**/.git/**',
94                     '**/build/**',
95                     '**/dist/**',
96                     '**/coverage/**',
97                     '**/public/**',
98                 ],
99             };
100
101             // Add health check plugin to webpack if enabled
102             if (config.enableHealthCheck && healthPluginInstance) {
103                 webpackConfig.plugins.push(healthPluginInstance);
104             }
105             return webpackConfig;
106         },
107     },
108 };
109
110 webpackConfig.devServer = (devServerConfig) => {
111     // Add health check endpoints if enabled
112     if (config.enableHealthCheck && setupHealthEndpoints && healthPluginInstance) {
113         const originalSetupMiddlewares = devServerConfig.setupMiddlewares;
114
115         devServerConfig.setupMiddlewares = (middlewares, devServer) => {
116             // Call original setup if exists
117             if (originalSetupMiddlewares) {
118                 middlewares = originalSetupMiddlewares(middlewares, devServer);
119             }
120
121             // Setup health endpoints
122             setupHealthEndpoints(devServer, healthPluginInstance);
123
124             return middlewares;
125         };
126     }
127
128     return devServerConfig;
129 };
130
131 // Wrap with visual edits (automatically adds babel plugin, dev server, and overlay in dev mode)
132 if (isDevServer) {
133     try {
134         const { withVisualEdits } = require("@emergentbase/visual-edits/craco");
135         webpackConfig = withVisualEdits(webpackConfig);

```

```

136   } catch (err) {
137     if (err.code === 'MODULE_NOT_FOUND' && err.message.includes('@emergentbase/visual-edits/craco')) {
138       console.warn(
139         "[visual-edits] @emergentbase/visual-edits not installed - visual editing disabled."
140       );
141     } else {
142       throw err;
143     }
144   }
145 }
146
147 const configureDevServer = webpackConfig.devServer;
148 webpackConfig.devServer = (devServerConfig) =>
149   makeDevServerV5Compatible(configureDevServer(devServerConfig));
150
151 module.exports = webpackConfig;

```

frontend/tailwind.config.js

```

1  /** @type {import('tailwindcss').Config} */
2  module.exports = {
3    // `overline` is a Tailwind utility; without this an app's own eyebrow-label class draws a line above t
+  he text.
4    blocklist: ["overline"],
5    darkMode: ["class"],
6    content: [
7      "./src/**/*.jsx",
8      "./public/index.html"
9    ],
10   theme: {
11     extend: {
12       borderRadius: {
13         lg: 'var(--radius)',
14         md: 'calc(var(--radius) - 2px)',
15         sm: 'calc(var(--radius) - 4px)'
16       },
17       colors: {
18         background: 'hsl(var(--background))',
19         foreground: 'hsl(var(--foreground))',
20         card: {
21           DEFAULT: 'hsl(var(--card))',
22           foreground: 'hsl(var(--card-foreground))'
23         },
24         popover: {
25           DEFAULT: 'hsl(var(--popover))',
26           foreground: 'hsl(var(--popover-foreground))'
27         },
28         primary: {
29           DEFAULT: 'hsl(var(--primary))',
30           foreground: 'hsl(var(--primary-foreground))'
31         },
32         secondary: {
33           DEFAULT: 'hsl(var(--secondary))',
34           foreground: 'hsl(var(--secondary-foreground))'
35         },
36         muted: {
37           DEFAULT: 'hsl(var(--muted))',
38           foreground: 'hsl(var(--muted-foreground))'
39         },
40         accent: {
41           DEFAULT: 'hsl(var(--accent))',
42           foreground: 'hsl(var(--accent-foreground))'
43         },
44         destructive: {
45           DEFAULT: 'hsl(var(--destructive))',
46           foreground: 'hsl(var(--destructive-foreground))'
47         },
48         border: 'hsl(var(--border))',
49         input: 'hsl(var(--input))',
50         ring: 'hsl(var(--ring))',
51         chart: {
52           '1': 'hsl(var(--chart-1))',
53           '2': 'hsl(var(--chart-2))',
54           '3': 'hsl(var(--chart-3))',
55           '4': 'hsl(var(--chart-4))',
56           '5': 'hsl(var(--chart-5))'
57       },
58     },
59     keyframes: {
60       'accordion-down': {
61         from: {
62           height: '0'
63         },
64         to: {

```

```

65         height: 'var(--radix-accordion-content-height)'
66     }
67 },
68     'accordion-up': {
69         from: {
70             height: 'var(--radix-accordion-content-height)'
71         },
72         to: {
73             height: '0'
74         }
75     },
76 },
77     animation: {
78         'accordion-down': 'accordion-down 0.2s ease-out',
79         'accordion-up': 'accordion-up 0.2s ease-out'
80     }
81 },
82 },
83     plugins: [require("tailwindcss-animate")],
84 };

```

Source totals

196 files, 18,177 lines of code.

Data file samples (header and first rows)

The full data is in backend/data. Each file below shows its header row and the first two data rows, so the column layout and value format are visible without reproducing thousands of rows.

agent_runs.csv (456 rows)

```

workflow_id,ecr_id,agent_name,agent_key,status,action,input,output,reasoning,confidence,execution_time,error,created_at
WF-22fe1f93230b,ECR-2026-002,Retrieval Agent,ecr_understanding,COMPLETED,Extracting change entities and classifying the change,{},{,"{"change_type": "UI_CHANGE", "business_domain": "Identity", "affected_features": [{"User Profile"}, {"risk_indicators": [{"customer_facing", "validation_logic_change"}]}, "Classified as UI_CHANGE (confidence 97%) from 3 signal groups. Extracted 12 technical keywords, 1 affected feature(s) and 2 risk indicator(s).",0.97,4.812,,2026-09-13T12:47:14.994008
WF-22fe1f93230b,ECR-2026-002,Retrieval Agent,planning,COMPLETED,Planning which sources to investigate,{},{,"{"plan": [{"ecr_understanding", "planning", "requirements", "code_impact", "historical_defects", "collaboration_retrieval", "impact_assessment", "test_discovery", "test_selection", "test_prioritization", "correlation", "summarization", "report"}, {"skipped": [{"dependencies"}, {"routing_reason": "Ui change with 2 risk indicator(s) detected. Skipping dependencies."}], "Ui change with 2 risk indicator(s) detected. Skipping dependencies.",0.95,0.0,,2026-09-13T12:4

```

comments.csv (55 rows)

```

comment_id,ecr_id,target_type,target_id,author,author_role,body,sentiment,category,source_system,references,created_at
CMT-1001,ECR-2026-001,ECR,ECR-2026-001,priya.raman@enterprise.com,Payments Architect,"The validator is a shared library. Refund Service links the same package, so any new mandatory field breaks refunds exactly like BUG-332 did. Please keep the new currency check optional on the refund path or version the contract.",CONCERN,RISK,AZURE_DEVOPS,"[""BUG-332"", ""REQ-1013"", ""CMP-004""]",2026-09-01T12:45:40.612432
CMT-1002,ECR-2026-001,ECR,ECR-2026-001,daniel.okafor@enterprise.com,QA Lead,Agreed scope with product: full currency matrix for the top 6 settlement currencies plus the zero-decimal (JPY) and three-decimal (BHD) edge cases. Manual exploratory pass on the merchant config screen is not needed for this change.,NEUTRAL,TEST_SCOPE,AZURE_DEVOPS,"[""REQ-1004"", ""TC-1035"", ""TC-1036""]",2026-09-03T12:45:40.612432

```

components.csv (55 rows)

```

component_id,name,type,business_criticality,criticality_score,owner,repository,description,business_domain,domain,created_at,updated_at
CMP-001,Frontend Portal,UI,MEDIUM,0.5,Digital Experience,sample-repo/portal,React storefront and customer self-service portal.,Customer Experience,commerce,2026-09-13T12:45:40.313169,2026-09-13T12:45:40.313174
CMP-002,API Gateway,GATEWAY,HIGH,0.8,Platform Engineering,sample-repo/gateway,"Edge routing, authentication propagation and rate limiting.",Platform,commerce,2026-09-13T12:45:40.313176,2026-09-13T12:45:40.313177

```

defects.csv (68 rows)

```

defect_id,title,description,severity,status,root_cause,root_cause_category,affected_component,related_components,linked_requirements,escaped_to_production,reopen_count,detected_at,severity_weight,created_at,updated_at
BUG-101,Currency conversion failed for unsupported currencies,Payments in CHF returned HTTP 500 because the conversion helper assumed every currency code existed in the rate cache.,CRITICAL,CLOSED,Missing guard clause for unknown currency codes in convert().,Missing validation,CMP-005,"[""CMP-003"", ""CMP-004""]",[""REQ-1004"", ""REQ-1005""],true,1,2026-02-15T12:45:40.351164,1.0,2026-09-13T12:45:40.354744,2026-09-13T16:26:18.326101
BUG-201,Payment validation allowed invalid currency codes,"The validator accepted three-letter codes that were not ISO-4217, so authorisation failed downstream at the acquirer.",HIGH,CLOSED,"Validation rule checked length only, not the currency catalogue.",Incomplete validation rule,CMP-004,"[""CMP-003"", ""CMP-005""],[""REQ-1002"", ""REQ-1003""],true,2,2026-04-01T12:45:40.351164,0.75,2026-09-13T12:45:40.354752,2026-09-13T16:26:18.326105

```

dependencies.csv (72 rows)

source_component,target_component,dependency_type,criticality,weight,description,created_at,updated_at
CMP-001,CMP-002,SYNC_API,CRITICAL,1.0,Portal calls every backend through the gateway,2026-09-13T12:45:40.325825,2026-09-13T12:45:40.325830
CMP-002,CMP-003,SYNC_API,CRITICAL,1.0,Checkout payment routes,2026-09-13T12:45:40.325831,2026-09-13T12:45:40.325832

ecrs.csv (62 rows)

ecr_id,title,description,status,change_type,business_domain,requested_by,priority,target_release,linked_requirements,changed_files,change_metadata,record_type,lifecycle_status,severity,assigned_testers,actual_modified_objects,planned_modified_objects,build_resolved_in,test_estimate_archived,estimate,test_estimate,creation_date,modified_date,sit_assigned_testers,test_actual_time,affected_test_cases,steps_to_reproduce,observed_behavior,expected_behavior,attachments,created_at,updated_at
ECR-2026-001,Multi-currency support in payment validation,"Modify payment validation logic to support multi-currency transactions and introduce validation for unsupported currencies. The validator must read the merchant supported currency list from the Currency Service, reject unsupported currency codes with UNSUPPORTED_CURRENCY before funds are reserved, and store both the original and the converted settlement amount on the transaction record.",ANALYSED,UNCLASSIFIED,Payments,payments.product@enterprise.com,HIGH,R2026.03,"[""REQ-1002"", ""REQ-1003"", ""REQ-1004"""],"[""payment/validator.py""],
ECR-2026-002,Update button text on the user profile page,"Modify the label of the save button on the user profile page from 'Submit' to 'Save changes' and adjust the confirmation toast wording. No API contract, no persistence and no validation logic changes.",ANALYSED,UNCLASSIFIED,Identity,ux.team@enterprise.com,LOW,R2026.02,"[""REQ-1016"""],"[""portal/profile_page.py"""],"{"scenario": "SCENARIO_2", "expected_profile": "Low impact - mostly UI tests", "pull_request": "PR-4830"}", ECR,,[],[],[],,,,,,[""TC-1117"", ""TC-1118"", ""TC-1119"", ""TC-1120"", ""TC-1121"", ""TC-1122""]

evidence.csv (51 rows)

evidence_id,ecr_id,target_type,target_id,title,evidence_type,summary,outcome,uri,source_system,confidence,captured_at
EV-3001,ECR-2026-001,TEST_CASE,TC-1035,Currency matrix run - build 2649,TEST_RUN,Multi-currency matrix executed on build 2649. 38/40 passed. TC-1035 failed on CHF with a stale rate; defect raised against the rate cache.,FAIL,ado://testruns/2649,ADO,1.0,2026-09-08T12:45:40.612432
EV-3002,ECR-2026-001,ECR,ECR-2026-001,Payments design review minutes,REVIEW,Design review approved with two conditions: version the shared validator contract and add a timeout on the currency catalogue lookup.,APPROVED,confluence://payments/design-reviews/ecr-2026-001,CONFLUENCE,1.0,2026-09-04T12:45:40.612432

feedback.csv (0 rows)

ecr_id,workflow_id,target_type,target_id,useful,comment,created_at

impact_reports.csv (47 rows)

workflow_id,ecr_id,confidence,total_tests,selected_tests,reduction_percentage,execution_time,approval_status,report_json,created_at
WF-22fe1f93230b,ECR-2026-002,0.923,150,7,95.3,5.96,NOT_REQUIRED,reports/WF-22fe1f93230b.json,2026-09-13T12:47:16.116593
WF-814b4e899e4c,ECR-2026-001,0.947,150,66,56.0,5.55,NOT_REQUIRED,reports/WF-814b4e899e4c.json,2026-09-13T12:50:27.358136

requirements.csv (64 rows)

requirement_id,title,description,business_domain,priority,status,component,linked_components,features,source_system,domain,created_at,updated_at
REQ-1001,Card Payment Authorization,"The Payment Service shall authorise card payments against the acquirer, capture an authorisation code and persist the transaction with its currency, amount and merchant reference.",Payments,CRITICAL,APPROVED,CMP-003,"[""CMP-003"", ""CMP-008"", ""CMP-014"""],"[""Payment Authorization"", ""Transaction Processing""],LOCAL,commerce,2026-09-13T12:45:40.340370,2026-09-13T16:26:06.890822
REQ-1002,Payment Validation Rules Engine,"The Payment Validator shall validate amount, currency code, merchant configuration and payment instrument before any authorisation call, returning a deterministic list of validation errors.",Payments,CRITICAL,APPROVED,CMP-004,"[""CMP-004"", ""CMP-003"", ""CMP-007""],"[""Payment Validation"", ""Validation Rules""],LOCAL,commerce,2026-09-13T12:45:40.340392,2026-09-13T16:26:06.890826

test_cases.csv (251 rows)

test_case_id,title,description,feature,suite,component,linked_components,requirement_id,test_type,test_level,automation_status,folder,optimization_technique,test_polarity,test_technique,retired,scorable,domain,average_execution_time,historical_failure_rate,last_executed_at,tags,source_system,created_at,updated_at
TC-1001,Authorise a card payment with a valid card and sufficient funds,Authorise a card payment with a valid card and sufficient funds. Traced to REQ-1001 (Payment Authorization) on component CMP-003.,Payment Authorization,Payments Regression,CMP-003,"[""CMP-003"", ""CMP-008"", ""CMP-014""]",REQ-1001,FUNCTIONAL,SYSTEM,AUTOMATED,,Default,Positive,,FALSE,TRUE,commerce,37,0,2026-09-06T12:45:40.403221,"[""payment-authorization"", ""regression"", ""req-1001""],LOCAL,2026-09-13T12:45:40.380848,2026-09-13T16:27:21.374016
TC-1002,Reject authorisation when the acquirer declines the card,Reject authorisation when the acquirer declines the card. Traced to REQ-1001 (Payment Authorization) on component CMP-003.,Payment Authorization,Payments Regression,CMP-003,"[""CMP-003"", ""CMP-008"", ""CMP-014""]",REQ-1001,FUNCTIONAL,SYSTEM,AUTOMATED,,Default,Positive,,FALSE,TRUE,commerce,171,0.02,2026-09-06T12:45:40.403221,"[""negative"", ""payment-authorization"", ""regression"", ""req-1001""],LOCAL,2026-09-13T12:45:40.380855,2026-09-13T16:27:21.374019

test_executions.csv (2510 rows)

test_case_id	result	duration_seconds	build_id	defect_id	executed_at
TC-1001	PASS	35.8	BUILD-2600		2026-07-05T12:45:40.403221
TC-1001	PASS	38.3	BUILD-2601		2026-07-12T12:45:40.403221